从第一步讲起

计算机系统概论

超级纲要 · 教材式初稿

概念 · 推导 · 原图 · 例题 · 自测

例题保留数据的中间状态与公式的推导步骤。
原始课件图标明出处，补充示意按正文模型解释。

2026-10-25 · sTBook
Wiki 与 Typst 共用正文 · 原图均在图注中注明出处

章节目录

从一行 C 代码开始，看懂计算机在忙什么 3
1 位、字节、地址：先把内存格子画对 5
2 整数：位串没有变，为什么 255 会成为 −1 15
3 浮点数：怎样用有限的格子存下很大和很小的数 20
4 汇编寻址：先执行一次，再看地址表达式 26
5 分支与循环：CPU 怎样决定下一条执行哪里 31
6 函数调用：参数怎么进去，返回时怎么找回原来的位置 38
7 数组、结构体与内存安全：把对象边界画在地址旁边 44
8 编译与链接：函数写在另一个文件里，机器怎么找到它 49
9 异常与进程：程序为什么能停下来，又从原处接着跑 54
10 信号与竞态：通知来得太早，也会让程序等不到 60
11 虚拟内存：同一个地址，为什么能装着两份不同数据 67
12 动态分配：malloc 给了你一块内存，还给了你哪些责任 72
13 输入输出：文件描述符、偏移和缓冲到底分几层 79
14 线程与并发：同一个变量被两个人动过以后 88
15 同步：什么时候锁门，什么时候等人，怎样避免大家都等着 98
16 性能与缓存：同样读一个数组，为什么顺序不同就差很多 105
17 综合走读：沿着一个小程序，把前面的知识连起来 110
18 机器表示工坊：从题面条件一步步算到答案 116
19 故障定位：先找第一步错在哪里，再修最后看到的症状 123
本课程资料、章节范围与图源 128
看内存、指令和系统状态时怎样查术语 130

从一行 C 代码开始，看懂计算机在忙什么

你写下 `int x = 1; x += 2;`，电脑给出 3。前一门程序设计课可能已经教过这种代码。计算机系统概论接着问：1 放在哪里？加法由谁执行？为什么同样的加法，有时会得到负数？程序明明没有写“同时执行”，为什么窗口、播放器和终端还能一起动？

这些问题都能落到具体状态上。内存某个地址有几个字节，寄存器里有哪个位串，下一条指令是哪条，操作系统允许哪个线程继续执行。读本书时，手边留一张纸。遇到程序，就画几个格子，把格子里的值随执行过程改掉。许多“玄学错误”只是某个格子比你想象中早改了，或者根本没有改。

先认识四个每天会见面的东西

CPU 执行机器指令，寄存器是 CPU 内部少量可直接操作的存储位置。内存可以先看成按字节编号的连续格子。地址是位置编号，内容是那个位置里的位串。例如“地址 100 里的字节为 7”，100 和 7 各有用途，不能相互代换。第 01 章从这里开始，不要求先会指针。

源程序是我们写的 C 文本；编译和链接把它变成机器可执行的文件。程序运行后，操作系统安排处理器和内存等资源。文件怎样变成一次运行、一次运行怎样暂停和恢复，会在 08、09 章用实际文件和状态展开，不需要现在把各阶段的名称全部记住。

用三个小问题确定阅读起点

本书假定你见过 C 的变量、赋值、if、循环和普通函数。例如能跟出下面几行中的 x 依次为 1、3、6：

```
int x = 1;
x = x + 2;
if (x > 2) x = x * 2;
```

这里的等号是把右侧计算结果存入左侧变量，不是说 x 与 x+2 在数学上相等。若对这类代码还不熟，先用纸逐行改变 x 的值；本书在第一次用到指针、数组布局和系统接口时另作解释，不要求你靠一串声明猜出内存关系。

下面三个问题分别对应本书的三个入口。它们是阅读定位，不是进入下一章前的考试。

- 每个 int 占 4 字节时，数组从 0x1000 开始，a[2]为什么从 0x1008 开始？01 与 07 章会把一个元素占几格、下标要跨过几个元素画在一起。
- 八位 11111111 为什么能读成 255，也能读成−1？01 先建立位权，02 再定义无符号与补码两种解释；规则不同，不是处理器临时改变答案。
- 两个执行活动各做一次加 1，为什么需要同步？09 先区分进程，14 再创建线程、分别追踪局部值和共享值。第一次读可以按这个顺序来，不必先猜并发结果。

怎么用每一章

每章先交代一个实际问题，再介绍解决它所需的名词。算例会写出中间状态。你第一次读时可以跟着算；第二次读时遮住答案，只看题意，自己重画地址、寄存器或时间顺序。两种读法带来的感觉不同：看懂别人怎么算，比自己从空白纸算出来容易得多。

练习答案紧跟在题后，方便即时核对。不要只核对最后一个数。假如最终数一样，但你把小端顺序反过来又误用另一条公式，两个错误可能刚好抵消。应检查每一步使用的位宽、数据类型、地址单位和执行顺序。

遇到不认识的名词，可以把问题写得具体一点：“这里说的页，是磁盘上的块还是地址空间的一段？”“这个锁现在由哪个线程持有？”“这一位为什么补 1？”这种问题能直接指出下一段需要补的知识。单纯觉得“好抽象”时，也可以先找出句子里第一个不能画出来的名词。

本书采用什么机器与规则

机器表示和汇编主线使用教师课件中的 x86-64、AT&T 语法。多数例题采用 Linux 上的 System V AMD64 调用约定，以及常见 LP64 数据模型：`char` 1 字节、`short` 2 字节、`int` 4 字节、`long` 和指针 8 字节。这里

的“常见”是环境假设，不能拿去断言所有 C 实现。Windows x64 的 `long` 通常为 4 字节，调用约定也不同；苹果芯片的本机指令是 ARM64，不能直接运行书中的 x86-64 汇编。

C 语言边界主要按 C17 讨论。无符号算术按模运算，普通有符号溢出属于未定义行为；负有符号数右移在这个版本的 C 里由实现规定。书中若模拟“截断后的位串”，会明确那是固定宽度机器算术，不能把它冒充 C 对所有表达式的承诺。汇编例子中的地址是便于手算的假地址，不是你电脑上的实际地址。

课程范围怎样对应

01 至 03 章处理数据表示；04 至 07 章把 C 操作落实为指令、调用和对象布局；08 章把多个文件接成程序；09 至 10 章处理进程、异常和信号；11 至 13 章讲地址翻译、分配和 I/O；14 至 15 章讲线程和同步；16 章是缓存与性能补充；17 至 19 章用完整任务与故障练习连接这些内容。

每一部分都会沿用已经学过的对象和规则。遇到一处没有解释过的接口，先看该处说明和对应章节，不需要到处查一整套操作系统名词。96 术语页用于回查，95 列课程资料与扩展范围。

第一次读时，先确认输入、环境和本步改动；读完一段，再换一个小输入独立走一次。能说明某个值从哪里来、下一步为什么允许这样做，才算掌握了这段机制。

1 位、字节、地址：先把内存格子画对

终端输出 `78 56 34 12`，代码里写的却是 `0x12345678`。如果你第一次接触内存查看器，很容易怀疑程序把数据写反了。实际发生的是：一个整数占了四个字节，查看器按地址从低到高显示，而这台机器把低权重字节放在低地址。数值和存放次序是两个问题。本章会把它们拆开，然后教你从内存重新读回整数。

1.1 一个格子到底能装多少

位，英文 bit，只有 0 和 1 两种状态。两个位有 `00`、`01`、`10`、`11` 四种组合。再添一个位时，原来的每种组合后面都能添 0 或 1，所以总数从 4 变成 8。用n表示位数，2^n表示把n个 2 相乘，例如$2^3 = 2 \times 2 \times 2 = 8$。据此，$n$个位共有$2^n$种位串；约定$2^0 = 1$，表示没有位需要选择时只有一种空串。

8 位组成一个字节，英文 byte。因此一个字节共有$2^8 = 256$种位串。若把它们解释成非负整数，从 0 开始编号，得到 0 到 255；最后是 255，因为 0 也占一种。一个位串怎样解释，还取决于格式：它可以是整数的一部分、字符编码的一部分，也可以是指令的一部分。下面先学非负整数的位权，补码会在下一章另讲。

十进制 307 中的 3 表示 3 个百，0 表示 0 个十，7 表示 7 个一，因而$307 = 3 \times 10^2 + 0 \times 10 + 7$。二进制沿用同一方法，只把每向左一位乘 10 改成乘 2：

从左到右的位	1	0	1	1
这一位的权重	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$
位值乘权重	8	0	2	1

把最后一行相加，得到$8 + 0 + 2 + 1 = 11$。下标或指数从右侧的 0 开始，是因为最右位本来就表示“几个一”；向左走一次才乘 2。这里的 `1011` 是二进制写法，等号右边的 11 是十进制，不能把两个字符串直接按外形比较大小。

四个二进制位共有 16 种组合，所以可用一个十六进制数字替代一组四位。十六进制的 0 至 9 仍表示 0 至 9；A、B、C、D、E、F 依次表示 10、11、12、13、14、15。前缀 `0x` 只标记进制，不占存储，也不是乘法。例如 `0xB6` 可分成 B 和 6，B 对应 `1011`，6 对应 `0110`，连起来是 `10110110`。它的无符号数值是 $11 \times 16 + 6 = 182$。不要把十六进制 B 当成变量，也不要把 `0x10` 读成十进制 10；它是十进制 16。

存储容量的单位也要看定义。KiB 是 2^{10} 字节，MiB 是 2^{20} 字节；十进制 kB 是 10^3 字节。教师课件用 K、M 讲二进制数量级时，要按课件约定读。硬盘标称容量、网络速率和程序显示值可能采用不同单位，计算前先统一成字节。网络中的 bit/s 还是位每秒，转成 byte/s 需要除以 8，且实际协议还会消耗额外带宽。

1.2 先存入一个两字节整数，再读回来

先保存一个明确规定为 16 位的无符号字段，数值是 `0x0123`。16 位就是两个字节；每个十六进制数字占 4 位，所以高字节是 01，低字节是 23。高、低说的是数值权重：01 乘 256，23 乘 1。23 在这里按十六进制读，为十进制 35，整个数是$1 \times 256 + 35 = 291$。

给两个字节安排相邻地址 0x100 和 0x101。地址是位置编号，每增加 1 向后跨一个字节；它本身不说明这一格应该装高字节还是低字节。两种常用约定如下：

位置，按地址递增	0x100	0x101	读回时怎样组合
大端	01	23	先读的 01 乘 256，再加 35，得 291
小端	23	01	后读的 01 乘 256，再加 35，仍得 291

两种存法没有改变数值，只改变哪一字节占低地址。假如把低权重字节 23 改为 24，十六进制 24 为 36，读回就变为 292；高字节仍为 01。可以这样直接检查一次字节修改的结果。

这道题的 16 位来自字段定义，不是声称 C 的 int 在所有平台都占 16 位。下面把字段扩为 32 位，沿用同一拆法；等端序路线完整后，再把 C 指针接到这些地址上。

1.3 小端、大端各自怎么摆

把 0x12345678 从右向左每两位分组，得到 78、56、34、12。两位十六进制正好 8 位，恰好一个字节。先明确“高”“低”指位权：最右字节 0x78 乘 1，再往左 0x56 乘 256，接着 0x34 乘256^2，最左 0x12 乘256^3。

为什么是 256？因为跨过一个字节就是跨过 8 个二进制位，位权相差$2^8 = 256$倍。把四个十六进制字节换成十进制，0x12=18、0x34=52、0x56=86、0x78=120，原数等于

$$18 \times 256^3 + 52 \times 256^2 + 86 \times 256 + 120.$$

小端规定把最低权重字节放在最低地址，因此从 0x1000 起依次放 78、56、34、12。大端规定最高权重字节放在最低地址，因此同一数值依次放 12、34、56、78。下表始终按地址递增书写，才有可比较的意义：

地址	小端存法	大端存法
0x1000	78，权重 1	12，权重256^3
0x1001	56，权重 256	34，权重256^2
0x1002	34，权重256^2	56，权重 256
0x1003	12，权重256^3	78，权重 1

这项规则安排的是字节之间的次序。同一字节 0x12 仍写作二进制 00010010，不会改成 01001000。把位也倒排，会改变每个字节的值，已不是端序转换。

先看怎样逐字节写出小端存法。演示中的“按位与 0xFF”是逐位筛选：0xFF 的低 8 位全为 1，其他位为 0；两个对应位都为 1，按位与的结果才为 1，所以它只保留原数的低 8 位。每保存一格，再把未保存部分无符号右移 8 位；对这里的非负数，这等于整除 256，丢掉刚保存的低字节。

小端：一次写入一个字节

1. 要保存数值0x12345678

尚未取出的部分：0x12345678



格子下方0、1、2、3表示相对起始地址的字节偏移。

规定对象4字节，从0x1000起，采用小端。
原因：小端规定最低权重字节先存入最低地址；没有改变数值。

图 1　逐步演示：小端：一次写入一个字节

小端：一次写入一个字节

2. 取最低8位并写入一格

尚未取出的部分：0x12345678



格子下方0、1、2、3表示相对起始地址的字节偏移。

第1次取出0x78，写到地址0x1000。
原因：一个字节8位；与0xFF按位与保留低8位，其余位变成0。

图 2　小端：一次写入一个字节，第 2 步

小端：一次写入一个字节

3. 丢掉已经保存的低字节

尚未取出的部分：0x00123456



格子下方0、1、2、3表示相对起始地址的字节偏移。

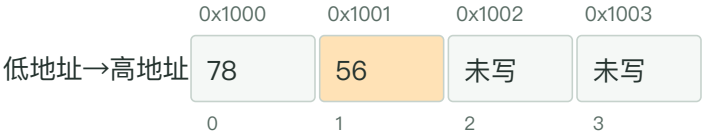
无符号右移8位，让下一字节来到最低位置。
原因：右移8位相当于整数除以256并舍去余数，已经保存的余数不再重复处理。

图 3　小端：一次写入一个字节，第 3 步

小端：一次写入一个字节

4. 取最低8位并写入一格

尚未取出的部分：0x00123456



格子下方0、1、2、3表示相对起始地址的字节偏移。

第2次取出0x56，写到地址0x1001。
原因：一个字节8位；与0xFF按位与保留低8位，其余位变成0。

图 4　小端：一次写入一个字节，第 4 步

小端：一次写入一个字节

5. 丢掉已经保存的低字节

尚未取出的部分：0x00001234



格子下方0、1、2、3表示相对起始地址的字节偏移。

无符号右移8位，让下一字节来到最低位置。
原因：右移8位相当于整数除以256并舍去余数，已经保存的余数不再重复处理。

图 5　小端：一次写入一个字节，第 5 步

小端：一次写入一个字节

6. 取最低8位并写入一格

尚未取出的部分：0x00001234



格子下方0、1、2、3表示相对起始地址的字节偏移。

第3次取出0x34，写到地址0x1002。
原因：一个字节8位；与0xFF按位与保留低8位，其余位变成0。

图 6　小端：一次写入一个字节，第 6 步

小端：一次写入一个字节

7. 丢掉已经保存的低字节

尚未取出的部分：0x00000012



格子下方0、1、2、3表示相对起始地址的字节偏移。

无符号右移8位，让下一字节来到最低位置。
原因：右移8位相当于整数除以256并舍去余数，已经保存的余数不再重复处理。

图 7　小端：一次写入一个字节，第 7 步

小端：一次写入一个字节

8. 取最低8位并写入一格

尚未取出的部分：0x00000012



格子下方0、1、2、3表示相对起始地址的字节偏移。

第4次取出0x12，写到地址0x1003。
原因：一个字节8位；与0xFF按位与保留低8位，其余位变成0。

图 8　小端：一次写入一个字节，第 8 步

小端：一次写入一个字节

9. 丢掉已经保存的低字节

尚未取出的部分：0x00000000



格子下方0、1、2、3表示相对起始地址的字节偏移。

无符号右移8位，让下一字节来到最低位置。
原因：右移8位相当于整数除以256并舍去余数，已经保存的余数不再重复处理。

图 9　小端：一次写入一个字节，第 9 步

小端：一次写入一个字节

10. 读回验证

尚未取出的部分：0x00000000

	0x1000	0x1001	0x1002	0x1003
低地址→高地址	78	56	34	12
	0	1	2	3

格子下方0、1、2、3表示相对起始地址的字节偏移。

从最高权重字节起拼回12、34、56、78，得到原数。
原因：按地址看到的78、56、34、12，是存储次序；通常书写的12345678，是数值位权次序。

图 10　小端：一次写入一个字节，第 10 步

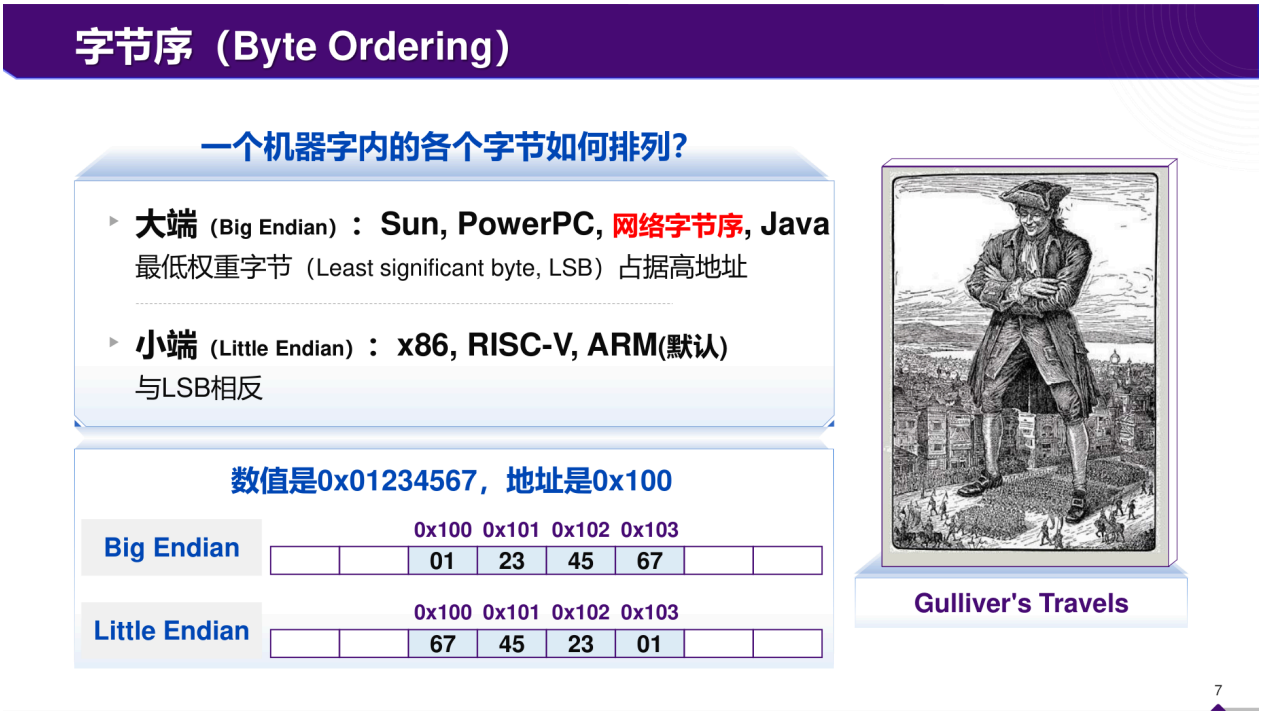


图 11　教师原页：字节序。原 PDF 第 7 页。

原图下半部的列标题从 `0x100` 增加到 `0x103`，因此从左往右走是在增加地址。第一行 Big Endian 的 `01` 最先出现，说明高权重字节在低地址。第二行 Little Endian 的 `67` 最先出现，说明低权重字节在低地址。图里的值是 `0x01234567`，它只有写法和本章例子不同，拆法相同。图上“与 LSB 相反”这一简写不够完整：实际需要记住的是最低权重字节占哪个地址，不要把 LSB 当成一种端序名字。

1.4 算例一：从内存还原数值

题目：小端机器上，一个 32 位无符号整数从 `0x2000` 开始，四个连续字节为 `EF BE AD DE`，整数是多少？第一步，确认长度为四个字节，不能只读第一格。第二步，小端下最低地址的 `EF` 权重为 256^0，其后依次是 256^1、256^2、256^3。第三步，按通常高位在左的写法重组为 `0xDEADBEEF`。若要验证，可以再从该数每次取最低 8 位并右移 8 位，依次取到 EF、BE、AD、DE，与内存表一致。

如果把同一组字节按大端格式解释，则得到 `0xEFBEADDE`。这不是两台机器计算能力不同，而是它们遵循的格式不同。文件格式或网络包应规定端序，不能指望接收者猜。

1.5 地址、内容、对象大小分开写

C 里 `&x` 表示取得对象 x 的地址。把这个地址保存在变量 p 里，就说“p 指向 x”。星号在声明和表达式中的用途要分开读：

```
int x = 7;        // 建立int对象x，里面先存7
int *p = &x;      // 声明p是指向int的指针，里面保存x的地址
int y = *p;       // 沿p保存的地址，读取一个int，y得到7
*p = 11;         // 沿同一个地址写入一个int，x因此变成11
```

这是一份新建的独立状态：假设 x 从地址 `0x1000` 开始，p 这个指针变量自己放在 `0x2000`。第二行改变的是 p 自身保存的内容，使它成为 `0x1000`；第三行读取这个地址处的 x；第四行写这个地址处的 x，p 保存的地址仍为 `0x1000`。所以 `p=...`和`*p=...`修改的是两个不同对象。

指针自身也要占空间。在本书常见 LP64 环境中，指针对象占 8 字节，int 占 4 字节；这是所选平台约定，不是 C 语言保证所有机器都如此。对上面的例子，`sizeof p` 为 8，`sizeof *p` 为 4，最后 x 的值为 11。这三个数字分别回答“地址对象有多大”“目标对象有多大”“目标里存多少”，不能互换。此处地址为便于说明的假设值；实际程序中由运行环境安排。

字这个词有两种使用场景。口头说“64 位机器字长”，通常指整数通路、寄存器等方面的宽度；x86 指令手册中的 word 却固定指 16 位，doubleword 指 32 位，quadword 指 64 位。教师课件已经提醒不同架构的 word 定义不同。做题时写出“多少位”，比只写“一个字”更不容易误会。

执行到哪一行	x 对象的内容	p 对象的内容	y 对象的内容
建立 x 后	7	尚未建立	尚未建立
建立 p 后	7	0x1000	尚未建立
建立 y 后	7	0x1000	7
执行*p=11 后	11	0x1000	7

写`*p`改变的是 0x1000 处的 int 对象；p 自身位于 0x2000，它保存的地址没有改变。y 先前取得的是数值 7 的副本，后续改变 x 不会让 y 自动变成 11。表中的地址只是用于演算。

1.6 算例二：数组下标为什么要乘四

题目：`int a[4]={7,11,19,23};`，每个元素 4 字节，从 `0x3000` 开始。求 `a[3]` 的地址以及 `a+1` 指向哪里。

a 的四个元素起点依次是 `0x3000`、`0x3004`、`0x3008`、`0x300C`。因此 `&a[3]=0x300C`。表达式 `a+1` 的类型为 `int *`，移动一个 int，得到 `0x3004`。这里的 `+1` 不是给裸地址加 1 个字节；C 已按所指类型缩放。

若先令 `unsigned char *q=(unsigned char *)a`，那么 `q+1` 才是 `0x3001`。通过字符类型观察对象表示是 C 允许的做法。假设小端，元素 7 的四字节表示为 `07 00 00 00`，`q[1]` 是 0，不是第二个数组元素 11。原来的类型信息决定一次跳多远、一次读多少。

还要分清 `a` 与 `&a`。a 在多数表达式中转换成指向首元素的指针；`&a` 是指向整个数组的指针。它们的起始数值可相同，但 `&a+1` 跨过整个 16 字节数组，和 `a+1` 的步长不同。

1.7 算例三：显式拼接一个协议字段

题目：文件中四个字节 `01 02 03 04` 按大端存储，怎样在任何端序的主机上取出数值？

可以逐字节读取，再用无符号移位明确赋予位权：

```c
#include <stdint.h>
uint32_t read_be32(const unsigned char p[4]) {
    return ((uint32_t)p[0] << 24)
         | ((uint32_t)p[1] << 16)
         | ((uint32_t)p[2] << 8)
         |  (uint32_t)p[3];
}
```

`uint32_t` 是恰好 32 位的无符号整数类型，前提是实现提供这个类型。`p[0]` 读第一个字节，依次到 `p[3]`。运算 `x<<s` 表示把 x 的二进制位向左移动 s 位，低端补 0；本例移位后都未超出 32 位。左移 8 位相当于乘 $2^8 = 256$，所以大端首字节要左移 24 位、次字节 16 位、第三字节 8 位、最后字节不移。括号 `(uint32_t)` 先把每个字节转成足够宽的无符号类型，再进行移位。

处理哪一字节	读出的数值	移位结果	累计按位或结果
p[0]	`01`	`01000000`	`01000000`
p[1]	`02`	`00020000`	`01020000`
p[2]	`03`	`00000300`	`01020300`
p[3]	`04`	`00000004`	`01020304`

表内均用十六进制。符号 | 为按位或：对齐的每一位只要有一边为 1，结果就是 1。四项的非零区域分别占不同字节，因此拼接时不会互相覆盖，得到 `0x01020304`。这个程序逐字节取数、明确指定各字节的位权，所以主机自己的端序不会改变返回的数值。

若直接写 `*(uint32_t *)p`，除了端序问题，还可能遇到对齐和 C 对象访问规则的问题。简单的强制类型转换并不会把外部字节自动转换成协议格式。需要高效批量读取时可研究 `memcpy` 与专用转换接口；先把格式解释正确，再考虑速度。

网络协议通常规定使用大端的网络字节序；主机内部采用什么端序，并不改变协议约定。因此发包之前要做主机到网络的转换，收包之后再转换回来。`htonl` 中的 l 在接口名里沿用历史称呼，它处理的是 32 位值，不应该因某台机器的 C long 为 64 位就推断它处理 64 位。

1.8 十六进制查看器怎样读

一次显示十六个字节的工具，通常左侧是行首地址，中间是十六进制字节，右侧是可打印字符。右边的字母只是同一字节按某种字符编码作的辅助显示。二进制文件某个位置恰好显示字母 A，不能据此认定那里保存着一个 C 字符串。

字符串 `"AB"` 在常见 ASCII 兼容编码下占 `41 42 00` 三个字节，末尾 00 是字符串终止符。将数值 `0x4142` 保存为 16 位无符号整数时，小端下占 `42 41` 两个字节；若保存为本书平台的 32 位 int，则占 `42 41 00 00` 四字节。数值写法本身没有规定对象只占两个字节。字符编码决定一个字符对应哪些字节；端序处理多字节数值的字节次序。这两个概念会在 UTF-16 等格式中相遇，但仍需要分别说明。

32 位地址能形成 2^{32} 个不同位串。若按字节寻址，理论上覆盖 4 GiB 地址范围。这个计算讨论的是地址编码能力，不等于某个进程可用 4 GiB 用户内存，更不等于机器必须装 4 GiB 内存。保留区域、操作系统布局、实际硬件和地址翻译都会影响可用空间。x86-64 也不能仅凭寄存器是 64 位就断言所有 2^{64} 字节均为当前可用地址，实际地址位数与规范地址规则要看具体处理器和系统。

1.9 再算一个有长度字段的二进制记录

一条记录规定：前两个字节是大端长度 L，接着 L 字节是正文。收到字节 `00 03 41 42 43 7F`。首先读长度 `0x0003=3`，正文范围从偏移 2 开始，占偏移 2、3、4，内容为 `41 42 43`；偏移 5 的 `7F` 已经是下一字段或多余数据，不能因为“缓冲还有字节”就算入正文。

如果接收缓冲总长为 n，要验证 `L<=n-2`，但前提先确认 `n>=2`。直接写 `L+2<=n`，在更一般的大类型长度场景中可能先发生加法溢出；先验证最小头长，再用减法比较剩余空间，是容易审查的结构。这个例子把格式、地址和长度联系起来：字段的数值决定后续读取范围，而范围必须受实际缓冲长度约束。

若正文声明为 UTF-8 文本，字节数不等于汉字数。一个汉字在 UTF-8 中常用三个字节，某些字符用更多；按字节偏移截断，可能恰好切在一个编码序列中间。于是“给我前 10 个字符”和“给我前 10 字节”是不同请求。C 的 char 是一个字节单位，不自动代表一个用户看到的完整字符。

再比较两个输出：打印指针用 `%p` 并传`(void *)p`，打印 size_t 用 `%zu`。printf 不会在运行时根据任意实参猜测你本来想用的格式；格式告诉它应怎样取与解释参数。地址数字用错格式，可能打印截断或形成未定义行为。调试内存问题时，先确保调试输出本身类型正确。

1.10 位编号和字节编号怎样同时标

一个 32 位数的位编号通常从最低位 0 数到 31。字节编号若按对象内地址递增，则从字节 0 到字节 3。小端机器的字节 0 承载数值位 0 至 7，大端机器的字节 0 承载数值位 24 至 31。两种编号都可用，但“第一个字节”必须说清是最低地址字节还是最高权重字节。

例如数值 `0x80000001` 有位 31 和位 0 为 1。小端字节为 `01 00 00 80`，大端为 `80 00 00 01`。最高数值位没有因存放而变成最低数值位，它只是落在不同地址。这也是为什么对数值做 `x&1` 检查奇偶不依赖主机端序：运算作用于数值位，而不是按内存显示顺序机械挑第一个字符。

反过来，如果你直接读`((unsigned char *)&x)[0]`，得到的是最低地址字节，它就受端序影响。两个写法观察的对象不同：前者问数值最低位，后者问存储起点上的字节。把问题写完整，答案自然不再打架。

练习：数值 `0x0100` 按小端存放，两字节为 `00 01`。若有人通过第一个字节是否非零判断原整数是否非零，错在哪里？答案是他只检查低 8 位，数值 256 的低 8 位恰为零，高 8 位仍有 1。正确判断整数非零直接比较 x 与 0，或检查全部相关字节，不能只看某一端。

1.11 自己做，再看答案

练习一：小端机器上，16 位无符号数 `0x12A0` 从地址 `0x80` 开始。写出两个字节。若只读取地址 `0x81` 的一个无符号字节，得到多少？

答案：先拆成高字节 `12`、低字节 `A0`。小端低字节在低地址，所以 `0x80:A0`, `0x81:12`。只读 `0x81` 得到 `0x12=18`。不能把它读成原整数的全部，也不能把 `12` 当十进制 12。

练习二：`double b[5]` 从 `0x5000` 开始，每个元素 8 字节。`&b[4]`、整个数组最后一个字节、`b+5` 分别在哪里？能解引用 `b+5` 吗？

答案：`&b[4]=0x5000+4×8=0x5020`；数组总长 40 字节，最后一字节为 `0x5027`；`b+5=0x5028`，是合法形成的尾后指针。尾后指针可用于结束比较等操作，但不能当作现存元素解引用。把“能形成地址”和“能访问该地址中的对象”分开，后面的越界问题会清楚很多。

练习三：内存依次是 `00 01`。你知道这是两个字节，但不知道字段类型与端序，能断言数值为 1 吗？

答案：不能。若是大端 16 位无符号字段，它是 1；若是小端 16 位无符号字段，它是 256；若是两个独立字节，它们是 0 和 1。解码需要格式信息，字节本身不会告诉你该套哪种解释。

2 整数：位串没有变，为什么 255 会成为 −1

假设只有八盏灯，每盏灯只能亮或灭。你想用它们表示整数，总共有 256 种灯光组合。若全给非负数，可以表示 0 至 255；若希望同时表示负数，就要把其中一部分组合分给负数。补码是一套具体的分配方法。它还让正数和负数尽量使用同一套加法电路，这正是它适合计算机的原因。

本章要同时处理两层规则：固定宽度位串的运算，以及 C 源代码是否允许你这样运算。八位加法电路把结果截成八位，是可以精确模拟的；C 中普通有符号加法发生溢出时，标准并没有要求编译器忠实模拟这套截断。两层规则在大部分正常输入上相同，在边界处必须分开。

溢出的一瞬间，先看逐步演示：

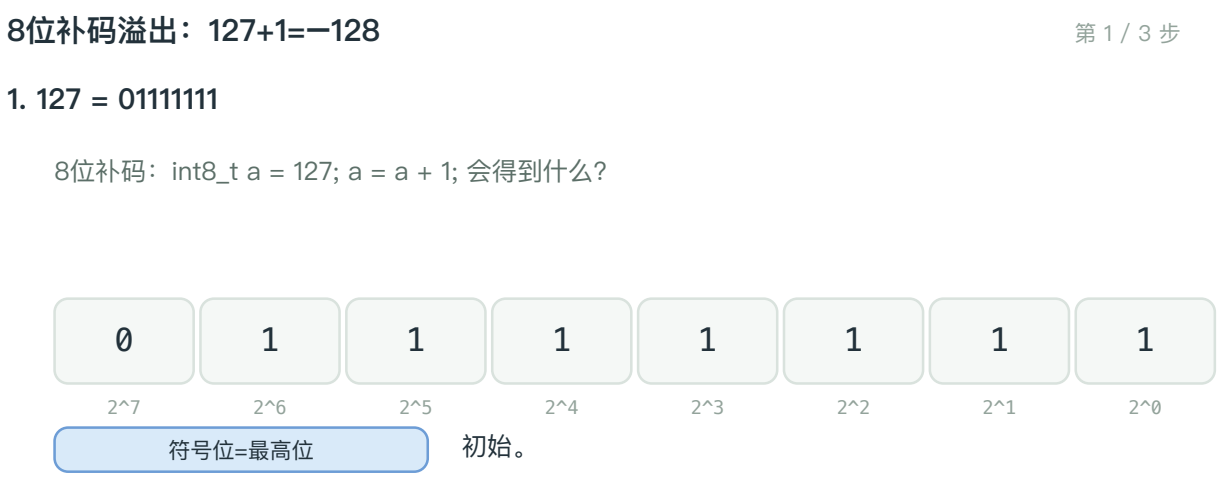


图 12　逐步演示：8 位补码溢出：127+1=−128

2.1 ## 无符号数逐位读，补码只改最高位的权

先沿用 01 的八位格子，读 11111111。无符号规则从右到左给权重 1、2、4、8、16、32、64、128，八项相加为 255。补码规则保留右边七项，只把最左一项的权重改为−128：

位，从左到右	1	1	1	1	1	1	1	1
无符号权重	128	64	32	16	8	4	2	1
补码权重	−128	64	32	16	8	4	2	1

因此同一位串按补码读为$-128+(64+32+16+8+4+2+1)=-1$。对 10000000，右边七项都为 0，结果为−128；对 01111111，最左位不贡献负权，结果为 127。这给出八位补码范围−128 至 127。最高位 0 时与无符号解释一致，最高位 1 时两种解释相差$128-(-128)=256$。

再把这条规则写成任意宽度。用 w 表示位数，b_i表示从右侧数第 i 位，i 从 0 到 w−1，每个b_i只能为 0 或 1。符号$\sum$表示把指定下标的各项相加，无符号值为

$$U=\sum_{i=0}^{w-1} b_i 2^i.$$

补码值则把最高项换成负权：

$$T=-b_{w-1}2^{w-1}+\sum_{i=0}^{w-2} b_i 2^i.$$

两式只在最高项不同，所以最高位为 1 时相差2^w。最小值是最高位 1、其余全 0，最大值是最高位 0、其余全 1，范围为-2^{w-1}至$2^{w-1}-1$。负数多一个，是因为非负一侧还要给 0 留一个位串。

2.2 怎样把一个负整数写成补码

对八位的 −37，先写 +37：`00100101`。逐位取反得到 `11011010`。现在从最右位加 1：最右位原为 0，0+1=1，不需向左进位，其余位不动，所以得到 `11011011`。如果最右位原为 1，就会有 1+1=二进制 10，本位写 0、向左进 1；进位遇到连续的 1 要继续传播。例如 `00101111+1=00110000`，不能只把末位改成 0 就结束。为什么“取反加一”有效？八位全 1 的无符号值是 255，取反把 37 变成 `255−37`，再加 1 就成为 `256−37`。这个数截在八位里，正是 −37 所对应的补码位串。

验证时不要再重复同一个口诀，改用位权：`11011011` 是 $-128+64+16+8+2+1=-37$。使用不同方式核对，才容易发现抄错。

最小负数是个特殊边界。八位 −128 取反加一后仍为 `10000000`。这不表示数学中 −(−128)=−128，而是 +128 无法装进八位有符号范围。求绝对值、取负、除以 −1 时都要想到这个边界。C 中 `-INT_MIN` 通常不能在 int 中表示，不能先算出溢出结果再试图补救。

2.3 固定位宽的加法怎样丢掉进位

将八位无符号 250 与 10 相加：`11111010+00001010=1 00000100`。九位数学结果是 260，容器只有八位，留下 `00000100`，即 4。无符号 C 算术正是按对应类型的模数约化：w 位时结果模 2^w。

八位加法可以逐位复演，不必把整串心算当黑箱。下面计算 250+10，从最低位 `i=0` 开始。每列加“两个输入位和上一列进位”，本列和为 0 或 1 就直接写；为 2 时写 0、向左进 1；为 3 时写 1、向左进 1。

先用原位串核对每列输入，再算传入与传出。

位 i（从低到高）	0	1	2	3	4	5	6	7
传入进位	0	0	1	0	1	1	1	1
本列和	0	2	1	2	2	2	2	2
结果位	0	0	1	0	0	0	0	0
传出进位	0	1	0	1	1	1	1	1

从最高结果位倒着读是 `00000100`，最左还多一个进位 1，即九位 `1 00000100`。八位容器只保留 4。进位必须沿列传播，不能因为前面一列暂时没有进位，就把所有后续进位固定为 0。

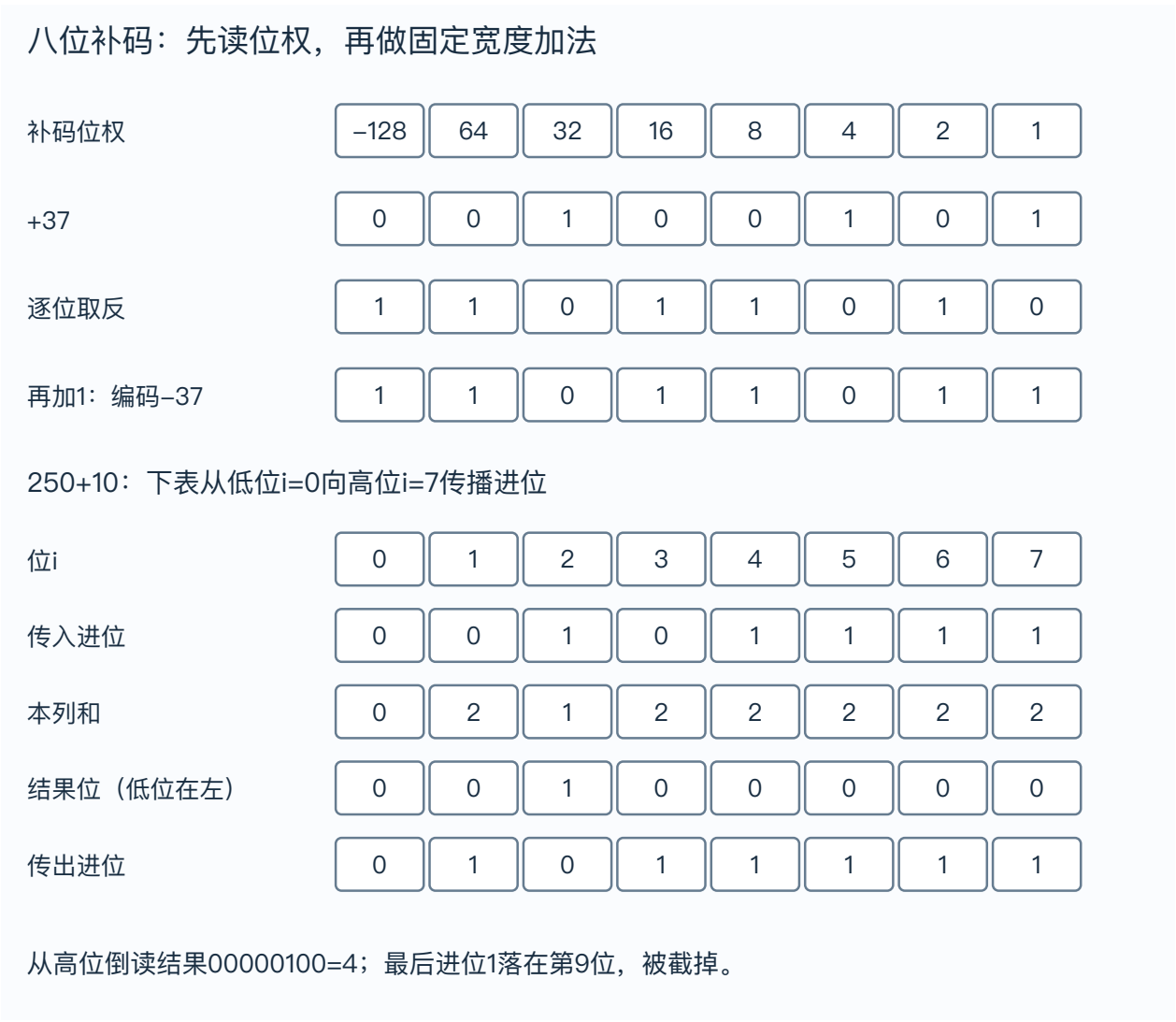


图 13　补码位权、取反加一与逐列进位

有符号补码的硬件加法可使用同样的位运算，但“是否溢出”要按补码范围判断。八位 120+20 的数学结果 140 超出 127，截断后是 `10001100`，按补码读为 −116。两个正数得到负位串，说明有符号溢出。两个负数得到非负位串也说明溢出。一个正一个负相加不会超出补码范围。

无符号进位和有符号溢出不是同一件事。八位 `255+1` 无符号有进位；若把同两组位串读成 −1 与 1，有符号和却是 0，没有溢出。后面汇编中的 CF、OF 分别反映这两种不同条件。

2.4 算例一：十六位加减完整走一遍

题目：两个 16 位补码数位串是 `X=0x2026`，`Y=0xFACE`，求和与差的结果位串，并判断是否有符号溢出。

先确定解释。X 最高位为 0，是十进制 8230。Y 最高位为 1，先当无符号数算得 64206，再减 65536，得到 −1330。于是数学和为 6900，数学差为 9560，均在 −32768 至 32767 内。

按十六进制位串计算，`0x2026+0xFACE=0x11AF4`，只留下低 16 位 `0x1AF4`。差则是 `0x2026−0xFACE` 模 65536，得到 `0x2558`。分别转换回十进制是 6900、9560，与前面的有符号数学计算一致，两次都没有有符号溢出。加法出现了第 17 位进位，但这并不意味着有符号溢出。

做这类题时写上“16 位”非常重要。同一算式若按 32 位零扩展，结果的解释就不同。题目里的方括号补码表示法描述的是位串，运算之前先补齐宽度。

2.5 为什么机器有一条加法，C 却不保证有符号绕回

C 给优化器一种约定：正确程序中的普通有符号运算不能超出其可表示范围。编译器因而可以基于这个前提推理。例如对定义良好的 int x，表达式 x+1 若发生，结果应大于 x；边界 x=INT_MAX 不是一个可要求它按绕回语义处理的合法反例，因为那次加法本身溢出。

这不是说编译器每次溢出都会主动报警，也不是说运行时硬件无法产生截断位串。它说的是源语言没有给那条路径一个可依赖结果。若你确实需要模2^w算术，就选恰当无符号类型；若需要数学有符号结果，就检查范围或用更宽/任意精度。把意图写进类型与检查，胜过依赖某个优化等级下碰巧的输出。

2.6 C 里的类型转换实际会改变哪一步

宽化是把短类型放进较宽类型。无符号值扩展时高位补 0；有符号值扩展到能容纳原值的更宽有符号类型时，需要保持数值，常见补码机器采用符号扩展，即复制原最高位。八位 `11111011` 表示 −5，扩展为 16 位 `1111111111111011`，仍表示 −5。若高位补 0，则变成 +251，数值已经改变。

截断是只留下目标宽度能保留的位。对无符号转换，C 明确按模数给出结果。例如 `(uint8_t)300` 得到 44，前提是该实现提供 uint8_t。对超出目标有符号范围的转换，C17 的规则是实现定义的结果或实现定义的信号，不能把常见机器上的截断读法冒充所有实现的要求。

还要注意整数提升。`unsigned char`、`short` 参与多数算术时，先提升到 int，或在 int 不足以表示全部值时提升到 unsigned int。两个八位变量相加，并不自动意味着每一步都在八位内计算。`unsigned char a=250,b=10; int c=a+b;` 在常见 32 位 int 环境里 c 为 260；将和再赋给 unsigned char 才得到 4。

混合有符号与无符号时，要做通常算术转换。例如常见环境中的 `-1 < 1u`，两边会转到 unsigned int，−1 变成 UINT_MAX，因此比较为假。这里并没有先按普通整数比较再转换布尔结果，转换发生在比较之前。`size_t` 是无符号类型，数组长度与负下标混算尤其容易出现这种情况。

2.7 用补码推导符号扩展，而不是死记补 1

w 位补码负数的无符号编码是$(T + 2^w)$。扩成 W 位后想保持同一负值 T，目标编码应为$(T + 2^W)$。两者之差为$2^W - 2^w$，正好对应从原符号位以上直到新最高位全补 1。比如八位−5 的编码 251，16 位−5 编码 65531，差 65280 即 `0xFF00`，因此得到 `0xFFFB`。

这个推导也解释了正数为何补 0：T 本来非负，编码就是 T，不需要加任何高位。扩展选择取决于源值的有符号解释，不是看目标寄存器名字。源类型为 unsigned char 的 0xFB 是 251，扩到 int 应保持 251；源类型为 signed char 的同位串在常见补码环境是−5，扩到 int 应保持−5。

将有符号短数先转成无符号短数，再扩宽，与直接符号扩宽可能不同。例如八位模型中，−5 先转 uint8_t 得到 251，再转 uint32_t 仍 251；若先作为 int32_t 的−5，再转 uint32_t，则得到$2^{32} - 5$。强制转换的顺序会改变每一步所要求保留的数值或模数，不能把所有括号简单删掉。

2.8 算例二：长度检查为什么能被绕过

题目：`size_t n=10; int i=-1;`，判断 `i<n` 是否为真，能否据此访问 `a[i]`？

在常见 64 位 size_t 的 LP64 环境，i 转成 size_t，−1 对应非常大的无符号数，因而 `i<n` 为假。若代码反过来写 `if (i >= n) reject(); else use(a[i]);`，这个特定负数也会被拒绝，但这不能成为通用设计：混合转换难读，而且若比较换成别的类型或先做溢出运算，条件可能失效。

清楚的检查是先确认 `i>=0`，再把已知非负的 i 转成 size_t 与 n 比较：`i >= 0 && (size_t)i < n`。`&&` 从左往右短路；左边为假时，不执行右边及访问。这样条件直接表达两个要求：下标非负，且小于长度。若下标本来就来自 size_t，则只需检查上界，并避免写永远为真的 `i>=0`。

2.9 怎样在计算前检查有符号加法

对 int 的 `a+b`，若 `b>0`，需要 `a<=INT_MAX-b`；若 `b<0`，需要 `a>=INT_MIN-b`。这两个减法在相应分支中是安全的，检查通过后才真正相加。不能先求 `c=a+b`，再用 `c<a` 判断有没有有符号溢出；溢出那一步已经使标准不再保证程序行为。

无符号加法可先检查 `a > UINT_MAX-b`，也可以利用定义明确的模结果检查进位。选择哪种方式，要先说清变量类型。编译器内建的溢出检测接口也很实用，但它们属于具体工具的扩展，不能当作普通 C 运算符的语义。

2.10 边界检查再练：乘法和减法会先于判断发生

假设输入给出元素数 n，每项 8 字节，程序写 `if(n*8<=capacity) copy(...)`。如果 n 是无符号类型，乘法可能按模数绕回较小值，使检查通过。应改成在单位一致时比较 `n<=capacity/8`，或先做 `n<=SIZE_MAX/8` 再乘。关键是检查应在可能溢出的运算之前，而不是算出坏结果再拿它判断。

以 8 位无符号模型为例，n=33，乘 8 的数学结果 264，模 256 后成为 8。capacity=16 时，错误检查觉得 8<=16，于是允许写 264 字节。除法检查 33<=16/8 为假，直接拒绝。这不是稀有的进位细节，而是长度字段变成缓冲区越界的常见路径。

减法同样有边界。size_t remaining=total-used，若 used>total，结果不是负数，而是模数下的巨大值。先检查 used<=total，再求 remaining。无符号并不意味着自动安全；它让溢出行为有定义，仍可能不符合你的业务含义。

2.11 位运算到底在改什么

按位与 `&` 保留两边都为 1 的位置，按位或 `|` 保留至少一边为 1 的位置，异或 `^` 在两位不同处产生 1，按位取反 `~` 翻转每一位。逻辑运算 `&&`、`||`、`!` 则把操作数视为真假，结果为 0 或 1。`6&3=2`，而 `6&&3=1`，两者不能互换。

取最低四位可写 `x & 0xFu`。设置第 k 位可写 `x | (1u<<k)`，清除第 k 位可写 `x & ~(1u<<k)`。这些写法要求移位位数在类型范围内；对 32 位 unsigned int，k 要在 0 至 31。写 `1<<31` 会把有符号 1 移向不可表示位置，应使用合适宽度的无符号常量，例如 `UINT32_C(1)<<31`。

左移在无符号类型内等于乘 2^k 再模位宽，但负有符号左移等情况在 C17 中有未定义行为。有符号负数右移在 C17 中由实现规定；本书明确假设算术右移时，复制最高位。无符号右移则高位补零。算术右移常相当于向负无穷取整的除法，C 的有符号整数除法向零截断，所以负数时二者不总相同。

2.12 算例三：−13 除以 4 与右移两位

数学商是 −3.25。C 中 `-13/4` 向零截断，得 −3，余数为 −1，使 $-13 = (-3) \times 4 + (-1)$ 成立。若用算术右移，八位 −13 是 `11110011`，右移两位为 `11111100`，即 −4。

若确实需要用算术右移实现向零截断，可在负数上加偏置 $2^k - 1$ 再移。例如 $(-13 + 3) >> 2$，算术右移得到 −3。偏置为什么为 3？它补偿丢弃的最多三单位，使负数向零靠近。这个技巧需要检查类型、溢出及移位语义；实际 C 源码直接写 `/4`，编译器往往能生成正确优化，读者不必为了“底层”主动制造不可靠表达式。

2.13 算例四：乘 25 的四步写法

题目允许移位与加减，不考虑溢出。先令 `t=x+(x<<1)`，得到 3x；再令 `y=(t<<3)+x`，得到 $24x + x = 25x$。总共两次移位、两次加法。另一种 $32x - 8x + x$ 也只需两次移位和两次加减。

这里“不考虑溢出”是题目给定的数学模型。如果写真实 C 程序，必须保证每个有符号中间结果都可表示，负数左移的规则也要处理。可以使用满足位宽要求的无符号类型做模运算，或在更宽有符号类型里用乘法并检查范围。不要因为代数恒等式正确，就直接认定任意位级改写符合 C。

2.14 自己做，再看答案

练习一：八位补码 `10000101` 是多少？加 5 后有没有有符号溢出？

答案：$-128 + 4 + 1 = -123$。加 5 得 −118，仍在范围内，位串是 `10001010`。最高位仍为 1，这只说明结果为负；负结果本身不代表溢出。这里一负一正相加，也可直接排除有符号溢出。

练习二：常见 32 位 int 环境中，`unsigned char a=255;`，`a+1` 与 `(unsigned char)(a+1)` 分别是多少？

答案：a 先提升为 int，255+1=256，所以 `a+1` 是 int 的 256。再转换回八位 unsigned char，按模 256 得 0。这里发生绕回的时刻是最后的窄化转换，不是前面的 int 加法。

练习三：为什么 `abs(INT_MIN)` 不能普遍当作普通正整数使用？

答案：最小负数的绝对值比最大正数大 1，同类型装不下。先把值转换到确实更宽、能够表示其绝对值的有符号类型，再取负，或者采用明确的无符号幅值算法。只是在外面强制转换 `((long)abs(x))` 无济于事，abs 的计算早已在 int 返回类型上遇到边界。

3 浮点数：怎样用有限的格子存下很大和很小的数

假设你需要同时保存地球尺度的距离和一根头发的直径。整数能精确表示一个固定范围内的每个整数，却不能直接保存 0.1。固定小数点可以留几位给小数，但范围和小数精度很快互相挤占。浮点数采用类似科学计数法的办法：分别保存正负号、有效数字和小数点移动量。它能覆盖很宽的范围，代价是许多实数只能取一个附近的近似值。

所以 `0.1f` 连加一千次，不一定恰好得到 `100.0f`。错误可能在第一次把十进制 0.1 写入二进制浮点数时就出现；后面的每次加法还可能再次舍入。先理解“哪些数能表示”，才能判断一个差异是正常舍入，还是程序真的写错了。

3.1 二进制也有小数点

二进制小数点右边的位权依次是 $2^{-1}=1/2$、$2^{-2}=1/4$、$2^{-3}=1/8$。因此 0.101_2 是 $1/2+1/8=0.625$。带下标 2 是为了提醒你按二进制读，它不是十进制 0.101。

判断一个十进制分数能不能用有限二进制小数表示，可以把它约成最简分数。如果分母只有 2 的因子，就有机会有限表示，例如 3/8。1/10 的分母还有 5，所以二进制展开会无限循环。十进制中 1/3 需要无限循环的 0.333...，你很可能已经接受；二进制中的 0.1 面临相似情况。

3.2 算例一：把 5.25 编码成 binary32

先完整保存一次 5.25。本例采用常见的 binary32 格式，共 32 位：最左 1 位记正负，接下来的 8 位记指数编码，最后 23 位记小数部分。下面先完成这个数，再把具体步骤整理成通式。

十进制 5 的二进制是 101，小数 0.25 为 1/4，对应二进制小数第二位，因此原数为 101.01_2。将小数点向左移动两格，数值先变为原来的四分之一；为保持原值，要乘2^2，得到 $1.0101_2\times2^2$。这一步让开头只剩一个 1，称为规格化。正数的符号位为 0，实际指数为 2。

binary32 规定正常数的指数编码要在实际指数上加 127，所以保存的整数是 129，即二进制 `10000001`。它不是把实际指数 2 直接写成 `00000010`。最后的小数域只存小数点后的 0101，因为这一类数开头的 1 已由格式规定，不再占一格；在 0101 后补 0 至 23 位。

完整分段为 `0 | 10000001 | 01010000000000000000000`。按四位分组可得到十六进制 `0x40A80000`。核对时反向解码：q=129，所以 E=2；有效数为 $1+1/4+1/16=1.3125$；乘 4 得 5.25，原值被精确表示，无须舍入。

如果要看小端内存中的四个字节，它们依次为 `00 00 A8 40`。这是上章端序问题，不会改变符号、指数和小数域在抽象位串中的定义。先解码整个位串，再讨论字节怎样存，步骤会更清楚。

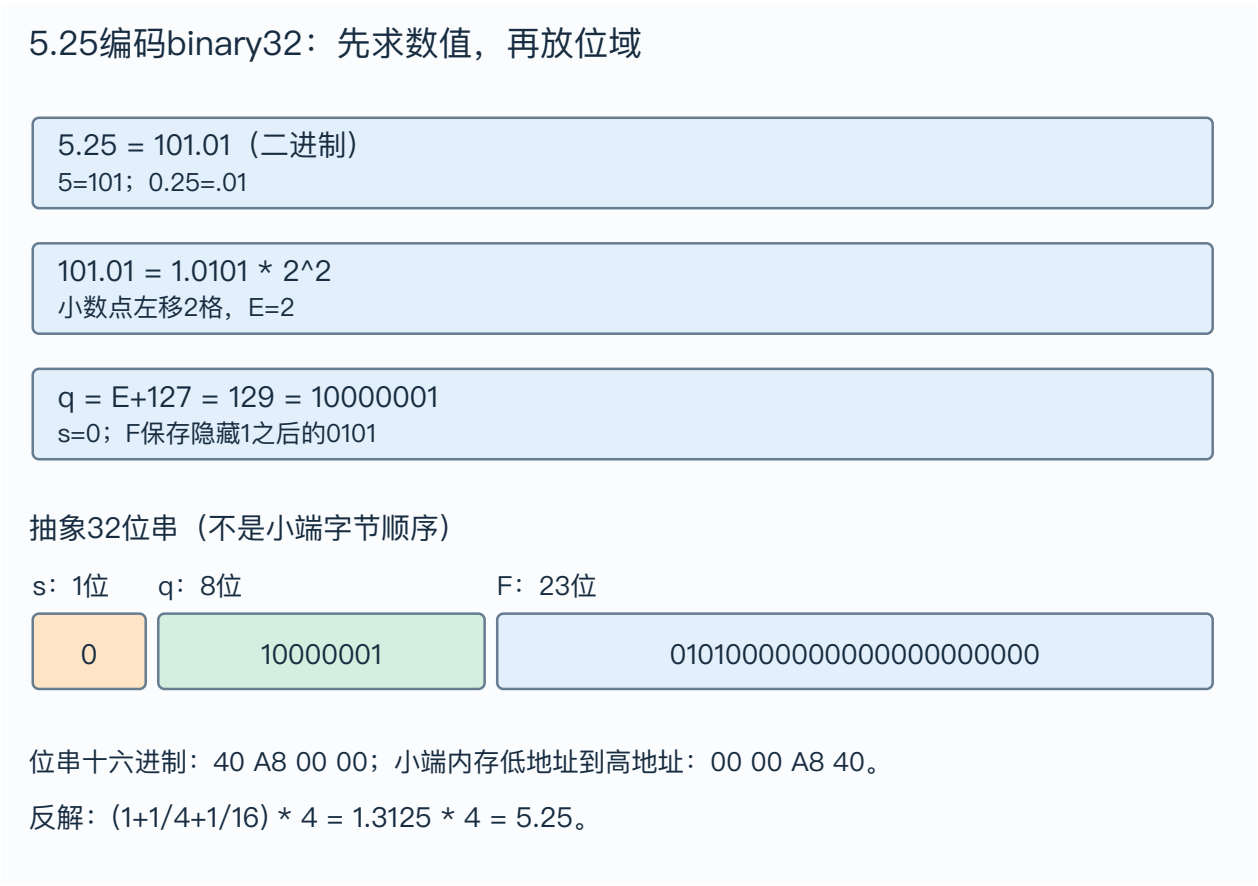


图 14　5.25 从二进制小数到三个 binary32 位域

从上往下读：原小数点左移两格，所以实际指数是 2；不是把小数点的纸面位置直接写进指数域。指数先加 127 才成为 8 位 `10000001`，有效数开头的 1 则被省略，23 位域从 `0101` 开始。最后一行按小端列出的字节只是存储顺序；不能把内存首字节 00 误当整个数的符号与指数都为零。

3.3 三段位域各装什么

规格化二进制科学计数法把非零数写成 $(-1)^s(1.f)2^E$。s 为符号位，0 表示正，1 表示负；`1.f` 是有效数，f 是小数部分；E 是实际指数。例如 $\texttt{101.01}_2=\texttt{1.0101}_2\times 2^2$，它等于十进制 5.25。有效数开头一定是 1，就可以不专门存这一位，把节省的格子用于小数。

以 IEEE 风格二进制格式为例，位串分为一个符号位、e 个指数位、f 个小数位。注意这里 f 有时表示位数，有时原资料把 frac 写成小数位串；本章谈数量时写“小数位数 f”，谈值时写“小数域 F”，避免把二者混用。

指数域当无符号整数读，记为 q。偏置量 bias 通常为 $2^{e-1}-1$。对非全零、非全一的指数域，实际指数为 $E=q-bias$，有效数为 $1+F/2^f$。数值为

$$(-1)^s(1+F/2^f)2^{q-bias}.$$

为什么指数需要偏置？指数可能是负数，用加上固定数的编码方式，可把常用指数装进无符号域，而且便于组织数值顺序。符号、特殊值等因素仍使“把整个浮点位串直接按无符号整数比较”不能无条件成立。

binary32，也就是常见 float，有 1 个符号位、8 个指数位、23 个小数位。正常数实际带 24 位有效二进制位，因为开头那一位 1 不必存。binary64，也就是常见 double，有 11 个指数位、52 个小数位，正常数有 53 位有效精度。C 的名字本身并不保证所有平台都采用这些格式，本书算例明确采用它们。

3.4 指数全零为什么要另设一条规则

指数域全零时，如果还套 `1.f`，最小正数和零之间会突然空出一大段。非规格化数，也常叫次正规数，改用有效数 `0.f`，并固定实际指数为 $1-bias$。于是数值变为

$$(-1)^s(F/2^f)2^{1-bias}.$$

当 F 从 1、2、3 逐渐增加，这些数以固定步长从零附近向上走。最小正次正规数为 $2^{1-bias-f}$，最大次正规数为 $(1-2^{-f})2^{1-bias}$。最小正常数为 2^{1-bias}。最大次正规数再增加一个最小步长，恰好接到最小正常数，空隙不会突然扩大。

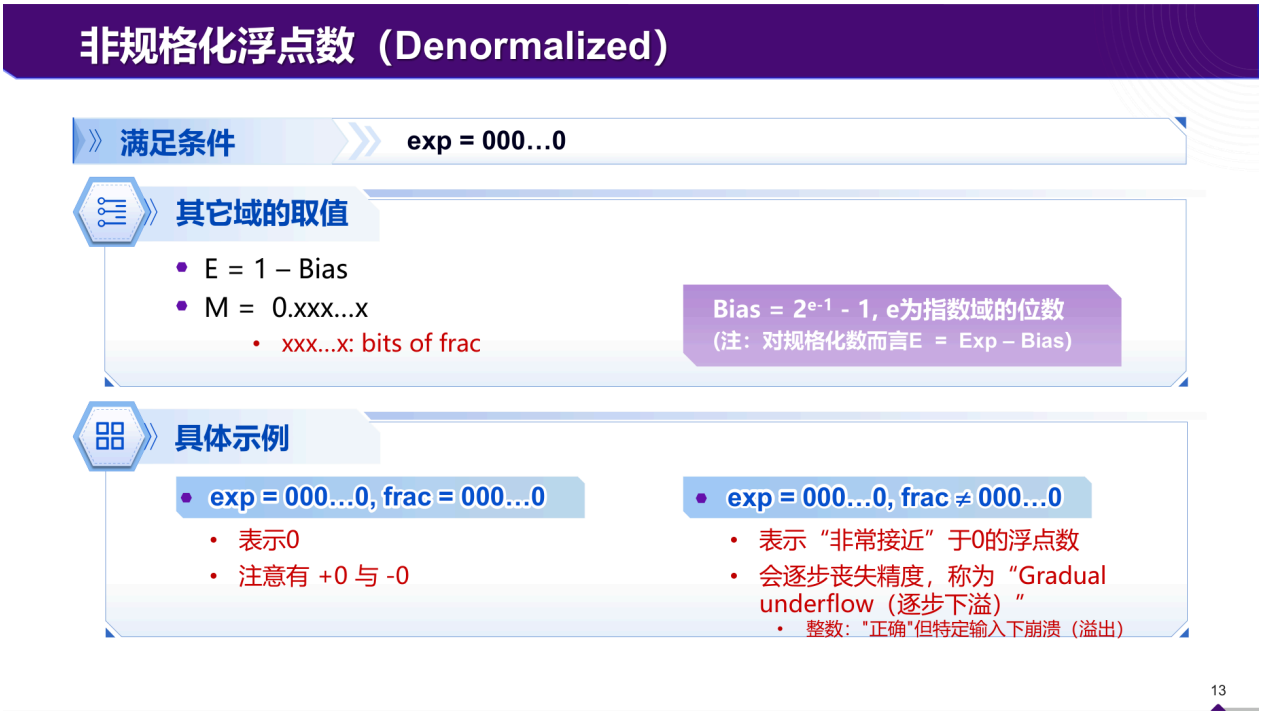


图 15　教师原页：非规格化数，PDF 第 13 页。

图顶部 `exp=000…0` 是触发条件，不是最终数值。中间的 `E=1-Bias` 与 `M=0.xxx…x` 必须一起使用。图左下小数域也全零时得到零，符号位仍可保留，因此有 +0 与 −0。右下“逐渐丧失精度”指接近零时可用的有效二进制位数减少；这一范围内相邻数的绝对间距固定，相对误差则可能变大。不能把整页概括成“指数为零，所以数值为零”。

指数域全一时，又有另一组解释：小数域为零表示正负无穷；小数域非零表示 NaN，即“不是一个数”的特殊结果。NaN 可由某些无效运算产生，例如 0/0；在通常 IEEE 比较语义中，NaN 连与自己比较相等也是假。检测它应使用 `isnan`。+0 与 −0 在相等比较中相等，但某些运算可区分，例如 IEEE 语义下 1/+0 与 1/−0 的无穷符号不同。

3.5 算例二：自己造一个八位格式

题目：1 个符号位、4 个指数位、3 个小数位，按本题的 IEEE 风格规则保留全一指数用于特殊值。求最小正常数、最小正次正规数、最大有限数。

e=4，所以 bias=$2^3-1=7$。最小正常数的 q=1、F=0，故实际指数为 −6，有效数为 1，得到 2^{-6}。最小次正规数的 q=0、F=1，得到 $2^{-3}\times 2^{-6}=2^{-9}$。最大有限数的指数域不能为 15，只能为 14，实际指数 7；小数域取 7，有效数为 $1+7/8=15/8$，所以最大有限数为 $(15/8)\times 128=240$。

最大次正规数的位串是 `0|0000|111`，数值为 7×2^{-9}。最小正常数是 `0|0001|000`，数值为 8×2^{-9}。两者差恰好为一个次正规步长，这也核对了规则交界处的连续间距。

现实里名叫 E4M3 的 FP8 还存在不同变体，特殊值约定不必和此题完全相同。作业题已经明确按 IEEE 风格处理，就按题目计算；不能从产品格式表拿一个不同约定的最大值替换答案。

3.6 舍入：结果落在两格之间怎么办

真实数轴上有无穷多个数，给定格式只有有限个可表示数。算出精确数学结果后，需要选一个可表示数，叫舍入。常见默认方式是“就近舍入，恰好中点时取偶数”。这里的偶数不是要求十进制数值为偶数，而是要求保留下来的有效位最后一位为 0。

用只保留两位二进制小数的简单格点说明。1.001_2 恰好处在 1.00_2 与 1.01_2 中间，后者最低保留位为 1，前者为 0，于是选 1.00_2。1.011_2 在 1.01_2 和 1.10_2 中间，选最低位为 0 的 1.10_2。长期反复处理正负中点时，这能避免“所有中点总向上”带来的方向性偏差，但不保证任何一组实际数据的误差都抵消。

若被丢弃部分小于半个间距，取较近的下邻；大于半个间距，取上邻。只有恰好中点才查看保留位的奇偶。单看第一个被丢弃位为 1，还不能认定是恰好中点；后面若仍有 1，就已经超过中点。

保留三位二进制小数时，把 1.0101011_2 明确拆成 `1.010 | 1011`。左侧是暂存的保留部分；右侧首个丢弃位为 1，后面还出现 1，所以超过半格，向上到 1.011_2。不是只见一个 1 就认定“恰好中点”。

精确输入的拆分	丢弃部分与半格比较	保留末位	舍入后
1.010 | 0111	小于 1000	0	1.010
1.010 | 1000	恰等半格	0（偶）	1.010
1.011 | 1000	恰等半格	1（奇）	1.100
1.010 | 1001	大于半格	0	1.011

表中每串都按二进制读，竖线只是人为分隔，不是运算符。第三行的末尾 `011+1` 先变 `100`，所以连带改变了多位。若保留部分已是 `1.111`，再进一个最低单位会变成 `10.000`，必须重新规格化成$1.000_2 \times 2^{E+1}$，实际指数也加 1；指数到了特殊值边界时还要检查溢出规则。舍入不是只改最后一位而永不影响指数。

3.7 有效精度怎样决定相邻数的间距

先用只有 3 位有效精度的正常二进制格式：开头 1 算一位，小数点后还剩两位。固定实际指数为 0，可表示 1.00_2、1.01_2、1.10_2、1.11_2，十进制为 1、1.25、1.5、1.75。最后一格增加 1，就是增加$2^{-2} = 1/4$。

把实际指数改为 2，整组数都乘 4，得到 4、5、6、7，间距也从 1/4 乘 4 变成 1。这不是保存位数增加了，而是同一最后一位被更大的指数放大。

现在用 p 表示有效位总数，E 表示实际指数。开头的 1 占一位，小数部分还剩 p−1 位，最低保留位的权为$2^{-(p-1)}$；乘指数因子2^E后，相邻格点的距离是

$$2^E \times 2^{-(p-1)} = 2^{E-p+1}.$$

这描述正常数同一指数区间$[2^E, 2^{E+1})$内的间距，不把它直接套到次正规区域。binary32 的 p=24，所以在[1,2)间距为2^{-23}；在[1024,2048)中 E=10，间距为2^{-13}，绝对间距是前者 1024 倍。数值本身也约放大 1024 倍，正常区域的相对精度保持同一数量级。

若精确结果不在上下溢边界，并按就近舍入，绝对误差通常至多附近半个间距。不能直接把这个局部误差界理解为“做一亿次运算后仍只差半格”；每一步的输入已经含前面误差，算法还可能放大它。数值稳定性研究的是误差如何随算法传播，精度研究的是格式本身能装多细。

例子：只保留三位有效二进制位，1 附近格点是 1.00、1.01、1.10、1.11。精确数 1.001_2为 1.125，离 1.00 与 1.01 各 0.125，就近取偶选 1.00。绝对误差 0.125，相对误差 0.125/1.125=1/9。把所有量乘 8，格点和误差也乘 8，相对比例不变。这一玩具格式精度很低，便于直接看见误差。

3.8 算例三：为什么 16777216 加 1 没变化

binary32 有 24 位有效精度，$16777216 = 2^{24}$。在这个数量级区间，相邻正常浮点数间距为 $2^{24-23} = 2$。可表示的相邻数包括 16777216 和 16777218，数学结果 16777217 恰在中间。

就近取偶看有效位末尾：16777216 的尾位为 0，16777218 的尾位为 1，因此选择 16777216。写在常见 binary32 计算里的 `16777216.0f+1.0f`，可能就得到原数。这里没有“加法器忘记加一”，而是精确结果没有可用格子可放。

继续加 2 则能到 16777218。把同样的问题改用 binary64，精度更高，边界会移到更大数量级；有限精度的问题仍然存在。不要把 double 当作任意精度实数。

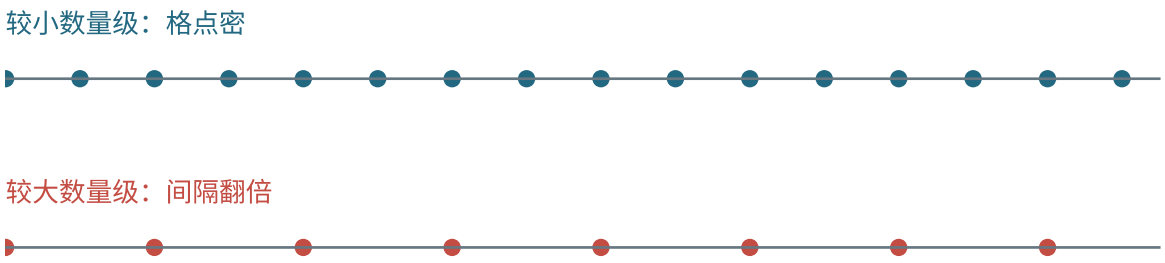


图 16　原创示意：数值越大，正常浮点数的间距越大。

图中上下两行是不同数量级的间距示意，不共用一条定量数值轴。上行点较密，下行把间隔画成约两倍，用来比较指数增大后的格距；不能把两行同一水平位置读成同一个真实浮点数。真实正常数跨过二的整数幂边界时，间距通常翻倍。float 大致提供固定有效数字数目，具体小数点后能留几位取决于数的大小，并非统一保留七位小数。

3.9 算例四：BF16 与 FP16 的小梯度

题目：$g = 2^{-130}$，BF16 有 8 个指数位、7 个小数位、bias=127；FP16 有 5 个指数位、10 个小数位、bias=15，均支持渐进下溢。就近取偶后分别是什么？

BF16 最小正常数是 2^{-126}，次正规步长为 $2^{-126-7} = 2^{-133}$。g 恰好等于 8 个步长，因此指数域为 0，小数域 F=8，符号位 0。整个位串的低位是十六进制 0008，结果为 `0x0008`，无须近似。

FP16 次正规步长为 $2^{-14-10} = 2^{-24}$。g 比这个步长的一半 2^{-25} 还小许多，最近的可表示值为 +0，位串 `0x0000`。指数位更多通常给出更宽范围，小数位更多通常给出更细精度；这道题主要受范围限制，不能只比较小数位数就选 FP16。

3.10 运算顺序为什么会改变结果

实数加法满足结合律，有限浮点加法每一步都可能舍入。例如在 binary32 中，令 a=2^{25}、b=-2^{25}、c=1。`(a+b)+c` 先精确相消为 0，再得到 1。`a+(b+c)`中，b+c 的精确结果−33554431 处在两个相邻浮点数−33554432 与−33554430 之间；就近取偶选择−33554432，随后与 a 相加得到 0。

如果把 a 改为2^{24}、b 改为-2^{24}，b+c=−16777215 却仍可精确表示，因为它落在接近零一侧、间距更细的区间。这说明反例的具体数量级必须核对，不能只看到“大正数、大负数、小数”三个角色就断言必然丢失。选择反例也要实际检查相邻格点，不能只背“大数加小数会丢”。真实数值计算可以采用更高精度、改变求和顺序、补偿求和等方法减小误差。它们各有适用条件，不会把浮点数变成实数。

比较浮点结果时，固定 `a==b` 是否适合，要看问题。计数、明确复制来的值、某些精确可表示运算可以直接比较；数值算法通常需要结合绝对误差和相对误差，例如 $|a-b| \le \epsilon_{\text{abs}} + \epsilon_{\text{rel}} \max(|a|,|b|)$。阈值来自任务容许误差，不能随意套一个“万能 1e-6”；NaN、无穷等情况也需单独处理。

3.11 相减可能暴露输入早已丢掉的信息

设测得两个长度各约 1000 米，它们各有毫米量级误差，想相减得到约 1 毫米差值。即使减法执行得非常准确，输入自身的误差已与差值同量级，相对误差会很大。计算机不能从两个近似输入里自动恢复真实的更细差异。

在浮点中，大而接近的数相减可能使许多高位抵消，剩下的结果突出原有舍入误差。遇到这种情形，应研究问题能否改写成更稳定公式，而不是只把输出多打印十位就认为精度提高了。打印位数改变展示，不能创造存储时已经丢掉的信息。

反例也要讲清：相减并不必然产生很大的绝对误差，有些相近浮点数相减还能精确得到差值。问题是相对于很小真实结果，输入近似误差可能占很大比例。把“相消必定导致减法器算错”作为解释，会混淆误差来源。

3.12 转成整数前要检查什么

C 中浮点转整数会丢弃小数部分，向零截断，但只有目标整数类型能表示转换后的值时才有相应保证。不能把 NaN、无穷或远超范围的 double 随手强转 int，再指望得到固定的 INT_MAX 或 0。不同指令或环境观测到某个值，也不是 C 层面承诺。

如果来自外部输入，先用 isfinite 等检查有限性，再按目标范围设计验证，注意边界本身是否能被浮点精确表示。尤其 64 位整数最大值不一定能被 double 精确区分：double 只有 53 位有效精度，附近相邻整数会合并。若需求是精确解析身份证号、文件长度或 64 位计数，优先直接按整数解析，不要先过一遍 double。

练习：十进制 9007199254740993 是$2^{53}+1$。用 binary64 保存时为何可能变成 9007199254740992？答案是该数量级间距为 2，原值是中点，就近取偶选择低邻。后来再转回整数，丢掉的 1 不会恢复。往返转换不保证恒等，必须先证明中间格式足以容纳原信息。

3.13 自己做，再看答案

练习一：上述 1+4+3 位的八位格式中，`1|1000|100` 表示多少？

答案：指数域 8 既非全零也非全一，是正常数；E=8−7=1。小数域 100_2=4，故有效数为 $1+4/8=1.5$。符号为负，得到 $-1.5\times 2^1=-3$。不能把指数域直接当 8 次方，也不能忘记隐藏的 1。

练习二：保留三位二进制小数，按就近取偶舍入 $\texttt{1.0101000}_2$ 与 $\texttt{1.0111000}_2$。

答案：第一个位于 `1.010` 与 `1.011` 中间，保留前者，因为其末位 0；第二个位于 `1.011` 与 `1.100` 中间，保留后者，因为其末位 0。中点规则有时不进位，有时进位，依据是最后保留位，不是“逢 1 必进”。

练习三：为什么次正规数接近零时相对精度下降？

答案：设固定步长为 δ，附近数值从 100δ 降为 2δ。半步长舍入误差仍约为 δ/2，但与数值相比，比例从 1/200 增至 1/4。绝对格距没变，分母变小，误差比例就变大。这个现象可直接从均匀格点图看出来。

4 汇编寻址：先执行一次，再看地址表达式

C 里读取一个变量时，源数据可能已经在寄存器里，也可能仍在内存。汇编把这两件事写成不同操作数。本章先用一张确定的状态表执行几条指令，分清地址数字和地址里的内容，再加入数组下标、位移和数据宽度。

全章采用教师使用的 x86-64、AT&T 写法，源在逗号前，目的在后；C 类型大小沿用 00 的 LP64 假设。某个寄存器里的位串此刻代表整数还是地址，要由正在做的操作判断。

一条 mov 指令的地址计算：

基址加偏移：0x8(%rbx)指到哪里 第 1 / 2 步

1. 读寄存器基址

mov %rax, 0x8(%rbx)：目标地址 = (%rbx) + 0x8

%rbx = 0x1000 + 0x8 目标 0x1100

待计算。

%rbx保存数组首地址0x1000。

为什么：编译器把数组名翻译成基址寄存器。

先从同一张底图执行三条指令

设 rcx 寄存器中保存 `0x100`；内存从 `0x100` 开始的八字节表示整数 7，从 `0x108` 开始的八字节表示整数 19。rax 初值未知。这些是本题给定、允许读写的普通内存，地址不是本机实测值。

`%rcx` 表示取寄存器 rcx 的内容，`(%rcx)`则把该内容当作地址去读内存。指令名 `movq` 表示复制 64 位，也就是 8 字节。以下三次执行各自从原底图开始，彼此独立：

执行的指令	先取得什么	是否读取内存	执行后 rax
`movq %rcx,%rax`	rcx 里的地址数字 0x100	否	0x100
`movq (%rcx),%rax`	rcx 给出地址 0x100，再取那里八字节	是	7
`leaq 8(%rcx),%rax`	计算 0x100+8	否	0x108

第一条复制后 rcx 仍为 0x100，复制不会清空来源。第二条多了一次沿地址取内容，得到 7 而不是 0x100。第三条的 `lea` 只计算位置：即使 0x108 处恰好存 19，也不会读取 19。换成 `movq 8(%rcx),%rax` 才会读取那里的 19。

这张表将“位置在哪里”和“那里是什么”分开。下面增加宽度和下标时，仍要分别记算出的地址和真正读到的数据。

4.1 后缀告诉你操作宽度

常见整数指令后缀 b、w、l、q 分别表示 1、2、4、8 字节。这里 l 是 32 位 longword 的历史命名，不等于本机 C long 的大小。`movl` 搬 4 字节，`movq` 搬 8 字节。你看到 `movl (%rdi),%eax`，需要知道它只从内存读四个字节，再因写 eax 规则把 rax 高半部清零。

一条普通 mov 通常不能同时把内存源和内存目的直接写在同一条指令里。把 `*q=*p` 实现为普通标量搬运，可先从 p 所指内存读到寄存器，再从寄存器写到 q 所指内存。源是地址、目的也是地址，并不表示 CPU 会自动沿着两个指针拷贝对象。

立即数使用 $ 前缀，例如 $8 就是数值 8。寄存器使用 % 前缀，例如 %rax。内存操作数通常带括号，例如 (%rax) 表示把 rax 的内容当地址再访问内存。三个写法 $8、%rax、(%rax) 对应常数、寄存器内容、该内容指向的内存。它们差一对符号，含义却完全不同。

4.2 有效地址公式怎样读

AT&T 的通用形式是 D(Rb,Ri,S)。D 是位移常数，Rb 是基址寄存器，Ri 是索引寄存器，S 是比例系数，可用 1、2、4、8。有效地址为

$$A = D + R_b + R_i S.$$

有效地址是指令计算出来用于访存的位置。它通常是虚拟地址，之后由硬件和操作系统规定的页表进行翻译。现在可以先把翻译视为下一层，专注于上式。

(%rdi) 只有基址；12(%rdi) 加了 12 字节；(%rdi,%rsi,4) 加了一个乘四的索引；20(,%rsi,8) 省掉基址。比例系数不是任意整数，不能写 3；如果要乘三，可能先用其他指令算 i+2i。位移单位是字节，不会因指令读八字节就自动乘八。

4.3 mov 和 lea 为什么不能混

movq 8(%rdi),%rax 计算地址 rdi+8，再从这个地址读取八字节。leaq 8(%rdi),%rax 只计算 rdi+8，把这个数字交给 rax，不读取那里。lea 的全称提示“装入有效地址”，它也可被编译器用来做普通整数加法与乘 2、4、8 的组合。

所以 leaq (%rdi,%rdi,2),%rax 可以计算 3×rdi。即使 rdi 的值不是合法内存地址，只要我们只讨论普通整数有效地址算术且不访存，这条指令也不需要去读那个位置。相反，mov 所需的地址必须允许访问，否则可能触发异常。

lea 通常不修改条件码，add 通常会修改。优化器选择哪条指令，也可能考虑后面还要不要使用某次比较留下的条件码。你不能只看它们都能加法，就认定它们对程序状态的影响完全一样。

4.4 算例一：三种操作数的区别

题目：rdi=0x1000，rsi=3，内存 0x1014 起八字节表示十进制 99。依次独立执行下面两条指令，各自得到什么？

```
movq  8(%rdi,%rsi,4), %rax
leaq  8(%rdi,%rsi,4), %rax
```

先求公共地址：0x1000+3×4+8=0x1014。mov 读取地址中的内容，所以 rax=99；lea 保存计算出的地址数字，所以 rax=0x1014。99 的十六进制是 0x63，和地址 0x1014 并非同一个值。

再考虑 movq %rdi,%rax，结果是 0x1000；movq (%rdi),%rax 的结果却无法从题目推出，因为我们只知道 0x1014 的内存，没给 0x1000。推题时遇到未知状态就写未知，不要随手把“未给出”当成零。

4.5 算例二：从 C 的数组访问读出汇编

函数 long get(long *a,long i){ return a[i+2]; }。在 System V AMD64 下 a 在 rdi，i 在 rsi；long 占 8 字节。第 i+2 个元素地址为 a+8×(i+2)=a+8i+16，因此可写：

```
get:
    movq 16(%rdi,%rsi,8), %rax
    ret
```

若 a=0x2000、i=1，则访问 0x2018，即 a[3]，不是 a[1]。如果元素类型改成 int，比例系数应改为 4，位移应改为 8，读取宽度为 4；如果函数仍返回 long，还要按原 int 的有符号性选择扩展方式。

例如 int 值 −1 的四字节全 1。若用 movl 读入 eax，则 rax 变成 0x00000000FFFFFFFF，按 64 位有符号数是 4294967295，不是 −1。要将有符号 int 转成 long，常见写法是 movslq，把 32 位符号扩展到 64 位。无符号 int 转 unsigned long 则可利用写 eax 的零扩展规则。

4.6 寄存器的几个名字其实可能共用一个位置

rax 是 64 位寄存器；eax 是它的低 32 位，ax 是低 16 位，al 是低 8 位。它们并不是四个独立盒子。读取 eax 是读取 rax 的一部分；写 eax 有一条容易漏掉的 x86-64 规则：高 32 位会被清零。写 ax 或 al 通常只改那部分位，保留其余位。

例如 rax 初始 `0x1122334455667788`。执行 `movb $0xAA,%al`，结果为 `0x11223344556677AA`。执行 `movw $0xBBBB,%ax`，结果为 `0x112233445566BBBB`。若再执行 `movl $0xCCCCDDDD,%eax`，结果成为 `0x00000000CCCCDDDD`。最后一次不是保留原来高 32 位。这条规则能解释为什么编译器常用 `xorl %eax,%eax` 清整个 rax。

rdi、rsi、rdx、rcx、r8、r9 常用来传前几个整数或指针参数，但“常用来传参”属于调用约定，不是寄存器硬件只能这样用。某个函数执行过程中可以把它们用于别的临时数据。rsp 则用于跟踪栈顶，后面调用章节会专门解释。

4.7 算例三：交换两个 long

C 语义是先保存旧 `*p`，再把旧 `*q` 给 p，最后把旧值给 q。rdi=p，rsi=q：

```
swap:
    movq (%rdi), %rax
    movq (%rsi), %rdx
    movq %rdx, (%rdi)
    movq %rax, (%rsi)
    ret
```

令 p 指向 0x1000 处的 long7，q 指向 0x2000 处的 long19。每一行只执行一条指令，未被写的值沿用上一行：

执行完哪一步	rax	rdx	0x1000 处对象	0x2000 处对象
尚未开始	未知	未知	7	19
从 p 加载	7	未知	7	19
从 q 加载	7	19	7	19
把 rdx 写给 p	7	19	19	19
把 rax 写给 q	7	19	19	7

中间两处都为 19，是因为旧 7 已经留在 rax 中。复制一个值不会清空源对象，但覆盖目标会丢掉目标原值；所以要在覆盖前保存后来还需要的数据。若先写 *p=q，再从*p 寻找“旧 7”，得到的已经是 19。

若 p 与 q 恰好相同，上面的交换仍保持原值。但另一些“异或交换”写法在两指针别名时会把值清零，不能把所有交换技巧当作等价。编译器是否能调整加载与存储顺序，也常取决于它能否排除两个指针指向同一对象。

普通内存复制也要确认源与目标是否重叠。memcpy 与 memmove 的重叠保证不同，不能因函数名相近就互换；这里逐步列出访问顺序，是在题目给定的两个 long 对象上证明这段交换的效果。

4.8 算例四：三条指令实现加权和

题目是 `32*x+4*y+2*z+w`，x、y、z、w 在 rdi、rsi、rdx、rcx。先把表达式分组为$(w+2z)+4(y+8x)$。两条 lea 分别算括号，最后一条把它们组合：

```
leaq (%rsi,%rdi,8), %rax
leaq (%rcx,%rdx,2), %rdx
leaq (%rdx,%rax,4), %rax
```

第一条 rax=y+8x；第二条 rdx=w+2z；第三条得到 `(w+2z)+4(y+8x)`，恰是目标。检查 x=1、y=2、z=3、w=4，三步 rax=10、rdx=10、rax=50，数学式也为 32+8+6+4=50。

这是机器整数代数，结果按寄存器宽度截断。若对应 C 的有符号表达式，需要另行保证输入与各中间运算不溢出。你可以看到，检查每项系数能比看“好像用了三个 lea”更可靠地验证答案。

对同一组 `x=1,y=2,z=3,w=4`，逐条记录“旧输入被谁覆盖”，未写的寄存器沿用前一行：

执行完哪条	rdi	rsi	rdx	rcx	rax	是否读写内存
初始	1	2	3	4	未知	否
lea (rsi,rdi,8),rax	1	2	3	4	2+8=10	否
lea (rcx,rdx,2),rdx	1	2	4+2×3=10	4	10	否
lea (rdx,rax,4),rax	1	2	10	4	10+4×10=50	否

第二行之后 `rdx` 不再是原来的 `z`，不能在第三行又把它代回 3。lea 只算数，三个结果都不需要去访问地址 10 或 50。对普通 mov 交换，则必须把内存列出来：



图 17　寄存器部分写入与交换对象的连续状态

4.9 反汇编时先做一张状态表

状态表每一行对应一条已执行指令，列出被改的寄存器和内存。没有改的列沿用上一行。若某寄存器只有低 32 位被写，顺手在高 32 位列记零。这样能避免把指令执行前的值与执行后的值混在同一算式里。

处理 RIP 相对地址时，例如 `leaq symbol(%rip),%rax`，目标由当前位置附近的位移确定。反汇编通常会给出目标符号或注释。不要把 `%rip` 当成任意通用寄存器随意代入；编码中的相对位移是相对于相关指令结束位置计算。第 08 章会把它和重定位联系起来。

本章暂不把全部 x86 指令列成字典。遇到 `imul`、`idiv` 等特殊操作数约定时，应单独查规则，尤其不要假定每条指令都恰有两个显式操作数。`idivq` 会同时使用 rdx:rax 作为被除数，并分别放置商和余数；下一章的教师 switch 原页会给出一个实际例子。

4.10 符号扩展指令与零扩展指令逐项看

假设内存地址 B 存一个字节 0x80。`movzbl (B),%eax` 先把该字节作为无符号数扩成 32 位，eax=128，写 eax 又清 rax 高 32 位，所以 rax=128。`movsbl (B),%eax` 把该字节作为有符号八位−128 扩成 32 位，eax 位串为 0xFFFFFF80；但写 eax 仍会把 rax 高 32 位清零，所以整个 rax 不是 64 位−128，而是 0x00000000FFFFFF80。

若目标是 64 位有符号−128，可用将字节直接符号扩展到 64 位的适当指令，或再把 eax 用 movslq 符号扩成 rax。这个例子同时用到两条规则：源宽度怎样解释，目的子寄存器怎样影响全寄存器。只记住“movs 会补 1”而不看最终目的宽度，仍会算错。

再看 `movl (%rdi),%eax` 与 `movslq (%rdi),%rax`，前者适合读取一个 32 位无符号值到 64 位零扩展结果；后者适合有符号 32 位值转 64 位。位串高位 0 时两者结果一样，因此测试必须含最高位 1 的输入，才有辨别力。

4.11 指令地址、数据地址、立即数地址常量

反汇编左边每行出现的地址，是那条机器指令在代码区的位置；操作数里算出的地址，则可能是数据位置。二者都是地址数字，但用途不同。比如一条位于 0x401000 的 mov，读取 0x600100 的全局变量，不能把左侧代码地址误当这次加载的源地址。

`movq $symbol,%rax` 在适当编码与重定位条件下可把符号地址这个常量放入寄存器；`movq symbol(%rip),%rax` 通常去读符号位置的内容；`leaq symbol(%rip),%rax` 求出符号地址。符号是函数还是对象、目标平台代码模型怎样，都会影响合法编码。初学时先区分“地址值”与“该地址内容”，再查具体汇编器接受哪种写法。

一个常见错误是把“函数指针”和“函数返回值”混淆。函数入口地址用于间接 call，调用完成后 rax 才可能装返回值。把入口地址当整数结果相加，或把返回整数当地址跳转，都不是类型转换自动能修正的问题。

4.12 内存操作的顺序与可见副作用

对普通用户内存，加载一项再算一项是本章教学主线；但地址若代表设备寄存器或受特殊接口管理的区域，读本身可能有副作用，不能随意重复或删除。volatile 等机制在这类场景中可能参与访问约束，但并不自动解决多线程同步。

在普通 C 里，优化器也会利用“不存在未定义行为、遵守对象访问规则”等前提重排或消除访问。你从一份未优化汇编学到的逐句对应，在优化代码里可能变成一条 lea 同时实现多句代数，或一个变量完全消失。反汇编还原应追踪数据依赖和可观察效果，不要求每个源码分号都有一条指令。

练习：源代码里先读 a[0]到临时 x，随后只是返回 x+1；生成汇编没有出现名为 x 的内存位置，是否正常？答案是正常，x 可直接留在寄存器。若 a[0]编译时已知，连那次读取都可能被常量替代。变量名帮助人组织代码，不强迫机器保留同名格子。

4.13 自己做，再看答案

练习一：rdi=100，rsi=5，求 `leaq -4(%rdi,%rsi,8),%rax` 的结果。这条指令读了多少字节内存？

答案：100+5×8−4=136。没有读取内存；q 描述结果寄存器宽度，不意味着 lea 会从 136 读取八字节。若换成 movq，就需要知道地址 136 是否有效以及其中的内容。

练习二：rax 初始全 1，执行 `movl $1,%eax` 后 rax 是多少？若执行的是 `movb $1,%al` 呢？

答案：前者 rax=1，高 32 位清零；后者 rax=`0xFFFFFFFFFFFFFF01`，只改最低八位。两条指令都写入数值 1，但写入位置与宽度不同，结果不能只报“寄存器是 1”。

练习三：`long *p` 从 `0x4000` 开始，`leaq 24(%rdi),%rax` 后能否说返回 `p[3]`？

答案：只返回地址 p+3，即 `0x4018`，可以对应 `&p[3]`；返回元素值需要再次加载。地址和元素值可能恰巧数字一样，但类型与操作过程仍不同。

5 分支与循环：CPU 怎样决定下一条执行哪里

C 中一句 `if (x<y)` 看起来只做了一次判断。处理器通常把它拆成两件事：先计算比较所需的状态，再根据状态决定是否跳到另一个指令地址。这个状态就包括条件码。读懂它以后，if、while、for 和 switch 都可以还原成“到哪个位置继续”的具体路线。

不要仅凭跳转名字里有 greater 就把它翻成数学里的“大于”。你还需要知道：比较指令的两个操作数谁减谁，数值按有符号还是无符号解释，跳转前是否有别的指令覆盖过条件码。三个问题少一个，都可能把条件写反。

cmp 设置标志位的瞬间：

条件码：cmp之后ZF/SF/OF 第 1 / 3 步

1. cmp后ZF=1：相等

cmp %rax, %rbx：做减法rbx−rax，只写条件码不改寄存器

ZF	SF	OF
1	0	0

结果为0。 跳转判断：ZF=1 → je（相等）。

rbx == rax：减法结果为0，ZF（零标志）置1。

为什么：je/jne只看ZF：相等走je分支。

把 if 画成两个入口和一个汇合点

考虑 `long absdiff(long x,long y){ if(x>y) return x-y; else return y-x; }`，假设差值均可表示。x 在 rdi，y 在 rsi：

```
    cmpq %rsi, %rdi
    jle .Lelse
    movq %rdi, %rax
    subq %rsi, %rax
    ret
.Lelse:
    movq %rsi, %rax
    subq %rdi, %rax
    ret
```

先取 x=7、y=4。`cmpq %rsi,%rdi` 计算 7−4 对状态位的影响，不把差 3 写入两个输入寄存器。结果非零、最高位 0、没有无符号借位也没有有符号溢出，所以 ZF=0、SF=0、CF=0、OF=0。ZF 记录是否为零，SF 记录最高位，CF/OF 分别记录无符号借位和有符号溢出；下一节再系统比较它们。

`jle` 的有符号条件是“ZF=1，或 SF 与 OF 不同”。当前两项都不满足，不跳转，接着将 7 复制到 rax，再减去 4，rax 得到 3，遇 ret 结束本函数。标签`.Lelse` 标的是另一条指令路线，本次没有执行。

再把输入改为 x=4、y=7，重新开始。cmp 的数学差为−3，在 64 位中可表示，所以 SF=1、OF=0，ZF=0；无符号 4−7 需要借位，CF=1。SF 与 OF 不同，jle 跳到`.Lelse`，rax 先取 7，再减 4，仍返回 3。两条路径的结果相同，但执行的 mov 和 sub 来自不同位置。x=y 时 ZF=1，也会走 else 并得到 0。

把每个基本块圈起来，就能看到块之间的两条边。基本块是从一个入口顺序执行、到转移点结束的一段指令；一个块内部通常没有需要从中间跳出的条件分支。

条件传送 cmov 可以避免某个控制流跳转。例如先把两个候选值分别算进 rax、rdx，再根据比较决定是否把 rdx 搬到 rax。它不能自动把所有 if 安全改写：若某个候选计算会除零、访问空指针或产生其他本

不该执行的副作用，提前同时计算两个分支会改变语义。选择“算哪个值”和选择“执行哪段有副作用的代码”需要分清。

5.1 四个条件码各回答什么问题

ZF 是零标志，结果所有位为 0 时置 1。SF 是符号标志，复制结果最高位。CF 是进位标志，在加法中反映无符号进位，在减法比较中可用于判断无符号借位。OF 是溢出标志，反映有符号结果是否超出对应位宽范围。

举八位加法例子，127+1 截断为 `10000000`：ZF=0，SF=1，OF=1，而无符号数学和 128 仍装得下，CF=0。255+1 截断为零：ZF=1，SF=0，CF=1；若按补码解释成 −1+1，结果正常，所以 OF=0。处理器保存的不是一句“比较结果为大于”，而是这些可组合使用的位。

`cmpq a,b` 计算 b−a 对条件码的影响，但不保存差值。AT&T 源在前，所以 `cmpq %rsi,%rdi` 对应比较 rdi 与 rsi。`testq a,b` 类似计算 b&a 的条件码影响，不保存按位与结果。`testq %rax,%rax` 常用来检查 rax 是否为零或是否为负，操作数自身仍保留。

add、sub 等通常会改条件码，lea 和 mov 通常不改。若 cmp 与 jcc 中间插入 add，跳转可能依据 add 的结果而非先前比较。因此反汇编时应从跳转往上找“最近一条真正定义相关条件码的指令”，不能一看到上方有 cmp 就直接认定它们配对。

5.2 有符号和无符号为什么要两套跳转

相等与不等只看位串是否一样，所以 je/jz 看 ZF=1，jne/jnz 看 ZF=0。有符号小于 jl 看 SF 与 OF 不同，有符号大于等于 jge 看两者相同；有符号大于 jg 还要求 ZF=0。无符号小于 jb 看 CF=1，无符号大于等于 jae 看 CF=0；无符号大于 ja 要求 CF=0 且 ZF=0。

为什么 jl 不只看 SF？因为减法可能发生有符号溢出，截断后最高位会给出反方向提示。例如八位比较 −128 与 1，数学差 −129 超出范围，截断成 +127，SF=0。此时 OF=1，SF≠OF，仍能正确识别前者更小。OF 对 SF 作了边界修正。

记忆时可以先把较常用条件写成一张表，再带一个正负边界例子验证。仅背 jg 是“大于”而漏掉“有符号”，会让处理无符号长度、地址或位掩码的代码出错。

5.3 算例一：同两组位串，两种大小关系

题目：八位 `a=0xFF`，`b=0x01`，执行相当于 b−a 的比较。无符号下 b<a 吗？有符号下 b<a 吗？

无符号解释中 a=255、b=1，b−a=−254，需要借位，故 CF=1，jb 会跳。补码解释中 a=−1、b=1，b−a=2，无有符号溢出，SF=0、OF=0，jl 不跳。相同的比较指令设置了同一组条件码，不同跳转按不同规则读这些位。

这也解释了上章 `-1 < 1u` 为什么危险：C 先做类型转换，决定采用无符号关系；你不能拿肉眼看到的负号推断最终跳转一定走“负数较小”的路线。

5.4 把布尔结果放进整数：setcc 后面为什么常有扩展

C 函数 `int less(long x,long y){return x<y;}`需要返回 0 或 1。汇编可以比较后用 setl 把条件写进 al。setl 只写一个字节，其余位保持原状；如果 rax 之前有其他数据，直接 ret 可能让整个返回整数包含垃圾高位。

可先 `xorl %eax,%eax` 清零，再 cmp、setl；也可 cmp、setl 后用 movzbl 把 al 零扩展到 eax。顺序很重要：如果先 cmp，再用 xor 清零，xor 会改条件码，setl 读到的就不再是原比较。movzbl 通常不改条件码，因此放在 set 之后不会影响已完成条件读取。

取 x=−2、y=3。cmp 计算 −5，有符号小于成立，al=1，零扩展后 eax=1。取 x=3、y=−2，小于不成立，al=0，扩展后 eax=0。即使比较前 rax 高位全为 1，两种正确清理方法仍给完整 0 或 1。这个例子把条件码与子寄存器规则接起来，正是两章交叉题容易考的位置。

5.5 算例二：逐轮还原一个 for 循环

下面这段程序从 0 开始，每轮加 2；若加完后的 x 超过 val，再额外加 3：

```
    movl $0, %eax
    movl $0, %edx
.Ltest:
    cmpl %edi, %edx
    jge .Lend
    addl $2, %eax
    cmpl %esi, %eax
    jle .Lnext
    addl $3, %eax
.Lnext:
    addl $1, %edx
    jmp .Ltest
.Lend:
    ret
```

edi=n，esi=val，edx 对应 i，eax 对应 x。第一次比较是 i−n，jge 在 i≥n 时退出，因此循环条件 i<n。第二次比较是 x−val，jle 跳过额外加 3，因此只有 x>val 才执行额外加法。

取 n=3、val=4。初始 i=0、x=0。第一轮 x 先到 2，2≤4，不额外加，i 到 1；第二轮 x 到 4，仍不额外加，i 到 2；第三轮 x 到 6，6>4，再到 9，i 到 3；回到测试发现 3≥3，返回 9。关键是条件检查使用“加完 2 之后”的 x，不能误写成先检查旧 x。

等价 C 可以写 `for(int i=0;i<n;i++){ x+=2; if(x>val) x+=3; }`。在本例小输入下不溢出。若要求对任意 int 输入成立，还要分析重复加法是否超出范围；机器指令行为与 C 有符号规则的差别仍在。

这段控制流可以把每次比较单独记下。条件码来自刚执行的 cmp；下表只列决定跳转所需的有符号关系，不依赖之后的 add 留下什么标志：

到达位置	i,x	本次 cmp 的数学差	跳不跳	随后怎样改变
顶部第 1 次	0,0	i−n=−3	jge 不跳	x 加 2 为 2
内部第 1 次	0,2	x−val=−2	jle 跳到 next	i 加 1 为 1
顶部第 2 次	1,2	1−3=−2	jge 不跳	x 加 2 为 4
内部第 2 次	1,4	4−4=0	jle 跳	i 变 2
顶部第 3 次	2,4	2−3=−1	jge 不跳	x 加 2 为 6
内部第 3 次	2,6	6−4=2	jle 不跳	额外加 3，x 变 9；i 变 3
顶部第 4 次	3,9	3−3=0	jge 跳出	rax 低 32 位为 9，返回

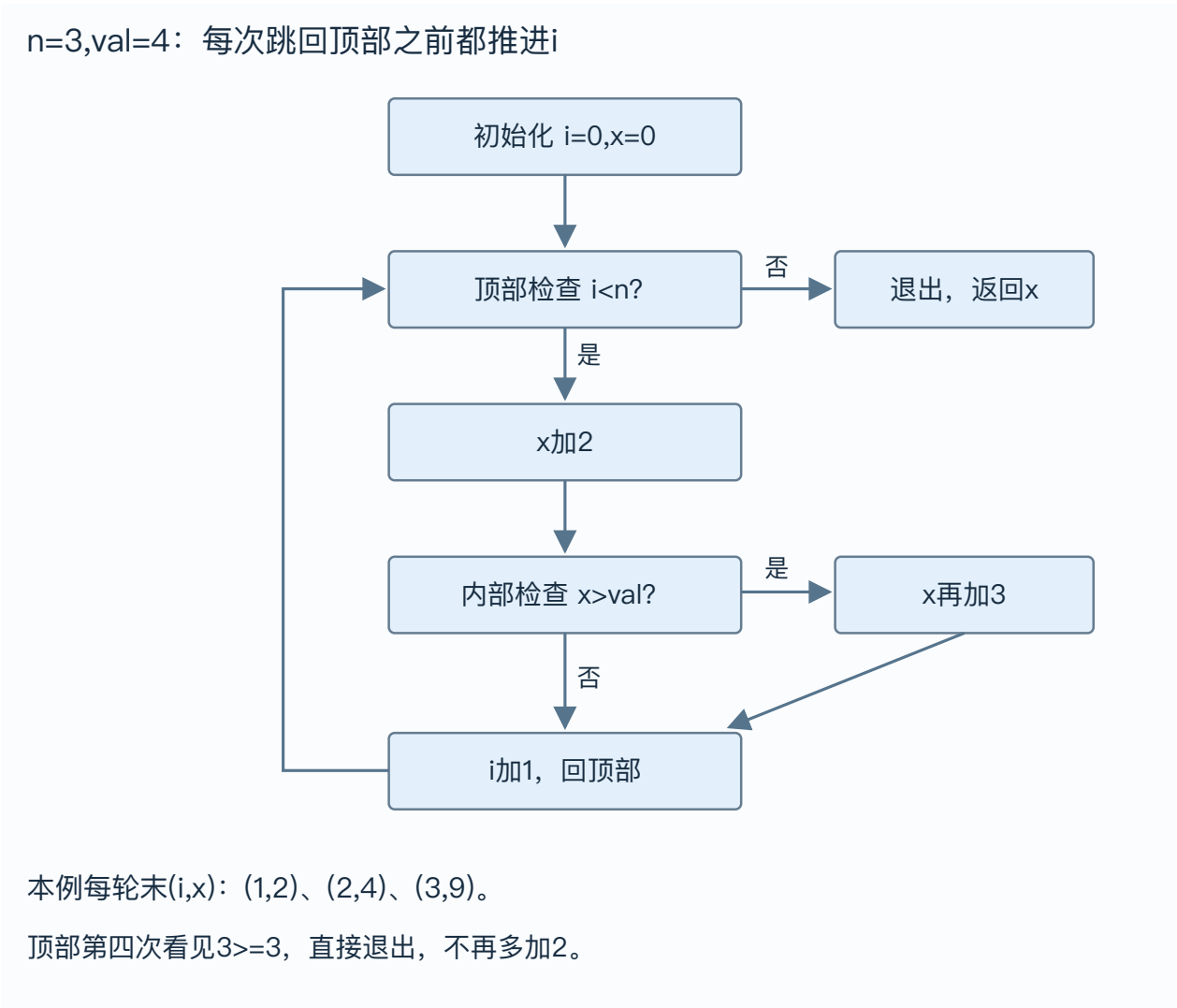


图 18　循环中的两个比较点与两条更新路线

图里每条回边都经过 i 加 1；只画“循环体”一个大方框会把内部条件的测试时间藏掉。对 `n=0`，第一条顶部检查便跳出，内部比较根本不会执行；这也是检查前测循环的最小输入。

5.6 while、do-while、for 的测试位置

while 的条件可以在第一轮之前挡住入口，因此初始条件为假时执行零轮。do-while 先执行循环体，至少一轮。for 把初始化、条件与更新写在同一行，机器代码仍要把这几项放到对应控制位置。

优化器可能使用“先跳到测试”“复制首次测试”等不同布局。反汇编的物理位置不等于源码书写顺序。你应看跳转关系：有没有不经过循环体就到出口的路径，回边从哪里指向哪里，每条路径上 i 在什么时候更新。

continue 也要按循环种类分析。for 的 continue 通常去执行更新部分再做测试；while 的 continue 去测试条件。如果在还原源码时把 for 更新遗漏，原本正常的循环就可能被误译为死循环。

5.7 通过跳转反推 break 与 continue

假设循环顶部检查 i<n，体中某个条件跳到统一出口，这对应 break 或其他提前返回路径；若跳到 i 递增块再回顶部，则更像 for 循环的 continue；若直接跳回顶部却没有 i 更新，可能是 while 的 continue，也可能是错误或由别处更新 i。不能只看标签叫.Lnext 就决定源码，它只是编译器生成名字。

取一个扫描数组、遇到负数跳过、遇到 0 结束的任务。i 应在跳过负数的路径也递增，否则会反复检查同一负元素。把每条回边前的 i 变化标出来，可以发现一种常见死循环。若循环变量是指针，则检查的是每条继续路径是否前进到下一元素。

练习：n=3，元素−2、5、0，目标跳过负数并在 0 结束，正确累加为 5。若 continue 绕过自写 while 尾部的 i++，第一项就使 i 永久为 0。这种错误与线程死锁不同：当前线程一直执行，却没有推进循环状态，应按控制流与不变量排查。

5.8 算例三：短路条件怎样变成两次跳转

题目：`for(i=n-2; i>=0 && i<val; --i) sum+=i;`。第一次条件应先排除 i<0，再判断 i≥val。可以使用：

```
    xorl %eax, %eax
    leaq -2(%rdi), %rdx
.Lcheck:
    testq %rdx, %rdx
    js .Ldone
    cmpq %rsi, %rdx
    jge .Ldone
    addq %rdx, %rax
    subq $1, %rdx
    jmp .Lcheck
.Ldone:
    ret
```

取 n=5、val=4，i 初始 3，满足两个条件，sum=3；接着 i=2、1、0，sum 依次 5、6、6；i 变−1 时，第一条条件就退出，不必再比较 val。若 n=7、val=4，i 初始 5，第二条条件第一次就失败，循环执行零轮，不会“等 i 减到 3 再进入”。循环条件不成立时，不会主动调整变量寻找一个能进入的值。

&& 的短路也保护诸如 `p!=NULL && *p>0` 的代码。机器码必须保留“p 为空就不读 *p”的语义。若看到条件传送前先无条件读内存，要检查它是否真的对应这样的受保护访问。

5.9 switch 的跳转表只是另一种选路

case 值密集时，编译器可能先检查下标范围，再按表中地址间接跳转。每项存 8 字节地址时，第 n 项位置是表首+8n；有些代码存 4 字节相对位移，不能把“跳转表一定八字节一项”当架构定律。case 稀疏时也可能生成比较树，不一定有表。

范围检查不能少。若把任意输入直接用作跳转表下标，就可能从表外读取地址。常见优化把减去最小 case 的结果当无符号数比较，一个条件同时挡住过小和过大的输入；理解它仍要回到模运算和无符号大小关系。

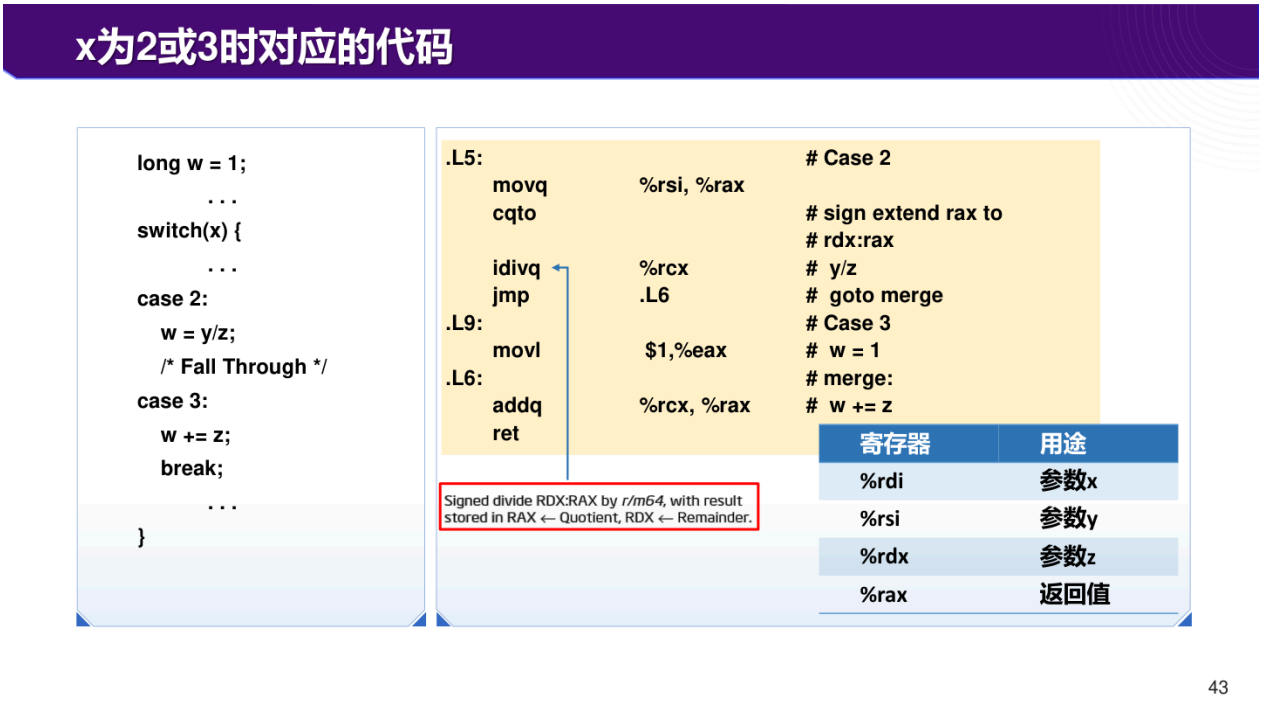


图 19　教师原页：case 2 落入共享代码，PDF 第 43 页。

原图左边 case 2 做 `w=y/z` 后没有 break，因此继续执行 case 3 的 `w+=z`。右边 `.L5` 把 y 搬入 rax，`cqto` 将 rax 符号扩展成 rdx:rax，`idivq %rcx` 以 rcx 中的 z 作除数，商在 rax、余数在 rdx。图右下仍把 z 的入参位置标为 rdx，说明进入除法前必须已把它复制到 rcx；`cqto` 会覆盖 rdx。不要只看该页局部就漏掉这个准备步骤。

`.L9` 是 case 3 自己的入口，它先把 w 初始化为 1；case 2 经 `jmp .L6` 直接到汇合点，绕过这次初始化，保留刚算好的商。因此两个 case 共享后半段，但入口不同。原图箭头与标签正是帮助你读出这种差别。

5.10 算例四：沿两个 case 各走一次

取 y=20、z=3。x=2 时，整数除法商 6、余数 2，随后共享代码加 z，返回 9。x=3 时，先令 w=1，再共享加 3，返回 4。若把 case 2 的跳转误写到 `.L9`，刚得到的 6 会被 1 覆盖，错误地返回 4。

还要检查除法前提。z=0 时除法不成立；64 位最小负数除以−1 也无法用有符号 64 位商表示。C 源码的合法输入约束与 x86 除法异常都要求你处理这些边界。不能仅靠共享代码看起来漂亮就忽略输入。

5.11 一段循环的证明，和几组测试各能说明什么

对求和循环，可以提出不变量：进入第 i 轮测试时，累加器等于前 i 项之和。初始化证明 i=0 成立；保持性证明加上第 i 项后，i 增加一，关系继续成立；退出条件把 i 约束为 n，于是结论对应目标。这个推理依赖有效地址、正确位宽、无不允许溢出等前提，必须与代码环境一起写。

测试则用具体输入找错。n=0 检查零轮路径，n=1 检查首元素，n=2 或 3 检查回边，正负混合检查有符号解释，接近上界检查范围。它们能证伪错误实现，但有限测试不能自动替代对所有合法 n 的推理。最好让测试与证明互补：证明指出必须保持的关系，测试专门打容易破坏它的边界。

例如把循环条件误写 i<=n，n=0 时仍会访问 a[0]，马上暴露零长度问题。若只测一个分配得比需要大的数组，错误访问可能没崩溃，结果又恰好加了零，就会漏检。对边界的观察应依对象有效范围，而不只依进程是否报错。

5.12 switch 范围检查为何常用一次无符号比较

设 x 的类型为 `int`，case 范围是 5 至 8，可以写 `unsigned int u = (unsigned int)x - 5u;`，再判断 `u <= 3u`。转换先于减法，减法在 `unsigned int` 中按模运算进行。x 处于 5、6、7、8 时，u 为 0、1、2、3；x 大于 8 时 u 大于 3；x 小于 5 时，结果落在较大的无符号值范围，也被排除。例如采用 32 位 unsigned int 时，x 为 4 得到 4294967295，x 为−1 得到 4294967290。这里 int 和 unsigned int 是一对对应类型，不能任意缩窄到更小的无符号类型，否则不同输入可能映射到相同小下标。

不要先写`(unsigned int)(x - 5)`再认为转换能保护减法：括号里的 x−5 先在 int 中计算，x 为 INT_MIN 时就可能有符号溢出，外面的转换来不及补救。上面的安全写法把转换移到了减法之前。

汇编里 sub 后用 ja 判断范围很常见。不要看到 ja 就断言原 C 变量必定声明 unsigned，它也可能是优化器把双向范围检查改写为无符号测试。还原源码时，应保留输入到分支的等价关系，源码类型还需结合其他使用推断。

表项也可能不直接存完整绝对地址，而存相对表基址的偏移。此时典型流程是按索引加载一个有符号偏移，扩展后加表基址，再间接跳转。若把偏移当完整地址，会跳到完全不对的位置；若忘记符号扩展，负偏移会变成很大正数。

5.13 分支预测只影响速度，不能替你改语义

处理器可能猜分支方向并投机执行后续工作，以减少等待。预测错时会撤销相关架构状态并重新走正确路线，通常付出时间代价。你在汇编中看到条件跳转，并不表示处理器每次都必须完全停住等比较完成；也不表示错误路径上的操作在语言可观察意义上被无条件执行了。

编译器用 cmov 可能减少难预测跳转，却可能增加必须计算的候选工作；可预测分支则可能很便宜。因此“去掉所有 if 就快”不成立。应先确保候选计算安全，保留程序语义，再在目标输入分布下测量。

练习：输入总是走同一个分支，与完全随机各半的分支，哪一个更可能容易预测？答案是前者通常更容易，但具体预测器与历史模式会影响结果。性能问题带环境，正确性判断则不能依赖预测通常猜对。

5.14 计数 1 位的循环：同一目标可以用不同工作量

最直接的 popcount 循环是每次取 x&1 加入结果，再把无符号 x 右移 1，直到 x 为 0。取 x=40，二进制 101000，依次观察最低位 0、0、0、1、0、1，共六轮，结果 2。循环轮数取决于最高 1 位位置，不是 1 位数量。

另一种方法每轮令 x=x&(x−1)，恰好去掉最低的那个 1，同时结果加一。x=40 时，x−1=39 即 100111，与 101000 按位与得到 100000=32；下一轮 32 与 31 按位与得到 0，只需两轮，结果仍 2。无符号 x=0 应先由 while 条件挡住，不执行无意义的循环体。

为什么每轮只清一个 1？设最低 1 在第 k 位，它以下全 0。减一后第 k 位变 0，更低各位变 1，更高位不变；与原 x 按位与，低位仍 0，第 k 位被清，高位保留。这个按位推导解释了机制，不只验证 40 一个输入。

两种算法的复杂度分析要注明宽度是否视为常数。固定 64 位机器上，两者轮数都有常数上限；若把输入位长当 n，第一种最坏 O(n)，第二种按置位数量计轮。课程里算法模型与实际机器位宽相遇时，必须说清 n 代表数值大小还是位数。

5.15 自己做，再看答案

练习一：`cmpq %rsi,%rdi; ja .Lbig` 在什么关系下跳？

答案：先形成 rdi−rsi 的条件码，ja 使用无符号大于，因此把两个 64 位位串当无符号值时 rdi>rsi 才跳。若要判断有符号 long 大于，通常应使用 jg。

练习二：初始 i=0，循环体先 i++，直到 i<3 为假才退出。若测试在体前，若测试在体后，最终 i 都是 3，是否说明两者等价？

答案：这个初值不足以区分。改成初始 i=5：体前测试的 while 执行零轮，仍为 5；体后测试的 do-while 至少执行一轮，变 6。检验控制流应挑能迫使分支走不同路线的输入。

练习三：条件传送为什么不能无条件把 `if(p) x=*p; else x=0;` 改成“先读*p 再决定是否取 0”？

答案：p 为空时源码不访问内存；改写后的加载仍会访问空地址，即使结果最后被丢弃，也可能先触发异常。保持返回值相同还不够，执行过程中允许发生的访存与副作用也要相同。

6 函数调用：参数怎么进去，返回时怎么找回原来的位置

主函数调用 `factorial(3)`，后者调用 `factorial(2)`，再调用 `factorial(1)`。每一层都把参数叫 n，但它们分别等于 3、2、1；最里层返回后，外面还记得要乘 2、再乘 3。程序需要保存这些尚未完成的工作，也需要保存“回去以后执行哪条指令”。运行栈正好适合这种最后进入、最先退出的顺序。

不过，C 的局部变量不一定每个都有一个固定栈格子。优化器可能把它放在寄存器，甚至证明它不需要真的存在。我们先用可见的栈帧模型学习调用，再说明哪些部分是调用约定要求，哪些只是编译器常用布局。

先看栈帧怎样随调用生长与消退：

调用栈：main→funcA→funcB的压栈弹栈 第 1 / 5 步

1. main运行中

调用栈：每次调用压入一帧（参数+局部变量+返回地址），返回弹出一帧



栈深1。

栈里只有main帧：局部i=9。

为什么：程序启动时main是栈底。

图 20　逐步演示：调用栈：main→funcA→funcB 的压栈弹栈

6.1 ## 算例一：一次调用的地址追踪

先只看一层调用。rsp 寄存器存当前栈顶地址，本例每个栈格为 8 字节。题目给定：call 前 rsp=0x8000；call 之后本来要执行的下一条指令在 0x401025；要进入的函数从 0x402000 开始。这三个地址分别描述栈、返回位置、被调用函数入口。

动作完成后	执行位置	rsp	这一步对栈或 rax 做什么
尚未 call	调用者即将发出 call	0x8000	rax 尚未规定
call 完成转入	0x402000	0x7FF8	从 0x7FF8 开始存八字节返回地址 0x401025
函数计算完毕，准备 ret	被调用函数的 ret 处	0x7FF8	假定计算结果 17 已放入 rax；栈顶仍存 0x401025
ret 完成	0x401025	0x8000	从栈顶取出执行位置，rax 仍为 17

call 必须记住回程，所以先腾八字节，把 0x401025 写进去，再转去 0x402000。ret 做相反的取回：从当前栈顶读取地址作为后续执行位置，栈顶恢复。17 是数学结果，0x401025 是继续执行的位置；它们通过不同通道交给调用者。

如果函数另外预留了 16 字节却没有收回，ret 面对的栈顶就不再是存 0x401025 的地方。即使 rax 中的 17 正确，也不能保证程序回到调用者。后面每一份调用表都同时追踪 rax 和 rsp，正是为了分别检查这两件事。

6.2 call 与 ret 每一步改什么

rsp 保存当前栈顶地址。常见 x86-64 栈向低地址增长。`pushq` 先令 rsp 减 8，再把八字节写到新栈顶；`popq` 先读当前栈顶，再令 rsp 加 8。两条操作的顺序不同，不要笼统写“栈动了一下”。

普通近调用 call 把下一条指令的地址压栈，再把指令执行位置转到目标函数。ret 从栈顶取出返回地址，令 rsp 加 8，并跳到该地址。返回值是另一个问题：它通常已放在 rax，ret 并不负责把数学结果搬进 rax。

正常 System V AMD64 函数入口满足 rsp+8 按 16 字节对齐。这是因为调用之前栈通常按 16 对齐，call 又压入八字节返回地址。被调用者若继续调用别人，要安排保存与局部空间，使下一次 call 前再次满足对齐要求。Windows x64 还有不同的参数寄存器和 shadow space，不能混用。

6.3 一次普通调用需要约定哪些事

调用者是发出 call 的函数，被调用者是接到 call 后开始运行的函数。两边要约定参数放哪、结果放哪、哪些寄存器可被改写、返回地址怎样保存、栈怎样对齐。这套约定叫调用约定，是 ABI 的一部分。ABI 是二进制接口规则，保证分别编译的代码能够合作。

本书采用的 System V AMD64 中，普通整数和指针参数的前六个通常依次在 rdi、rsi、rdx、rcx、r8、r9；更多这类参数通过栈传递。浮点参数使用另一套寄存器分类，结构体等聚合参数还要进一步按 ABI 分类，不能把这个简表套到所有函数。普通整数返回值通常在 rax。

调用者保存寄存器是“被调用者可以改，调用者若还要原值就自己先保存”。被调用者保存寄存器是“被调用者若使用并改写，返回前要恢复”。常见被调用者保存寄存器包括 rbx、rbp、r12 至 r15。保存责任并不是寄存器不能改，而是有人负责在约定时间把它还原。

6.4 算例二：带局部空间与保存寄存器的完整函数

假设函数收到 x，调用 helper(x)，然后把返回值与原 x 相加。x 在 rdi，而 helper 可能改写 rdi，所以必须保存原 x：

```
combine:
    pushq %rbx
    movq %rdi, %rbx
    call helper
    addq %rbx, %rax
    popq %rbx
    ret
```

函数入口 rsp 是“16 字节对齐地址减 8”。push rbx 后又减 8，正好恢复 16 字节对齐，可以安全进行下一次 call。rbx 原值被保存到栈，随后装原 x；helper 按约定保护 rbx，所以返回后它仍是 x。add 得到目标；pop 恢复调用者原 rbx；ret 再使用本函数返回地址。

取入口 rsp=`0x7FF8`，原 rbx=99，x=5。push 后 rsp=`0x7FF0`，该处保存 99；rbx 变 5。call helper 再把自己的返回地址压到 `0x7FE8`，helper 返回后 rsp 回 `0x7FF0`。若 helper 返回 10，add 后 rax=15。pop 恢复 rbx=99、rsp=`0x7FF8`，本函数 ret 后回到调用者，rsp=`0x8000`。这张地址表能同时核对保存责任与对齐。

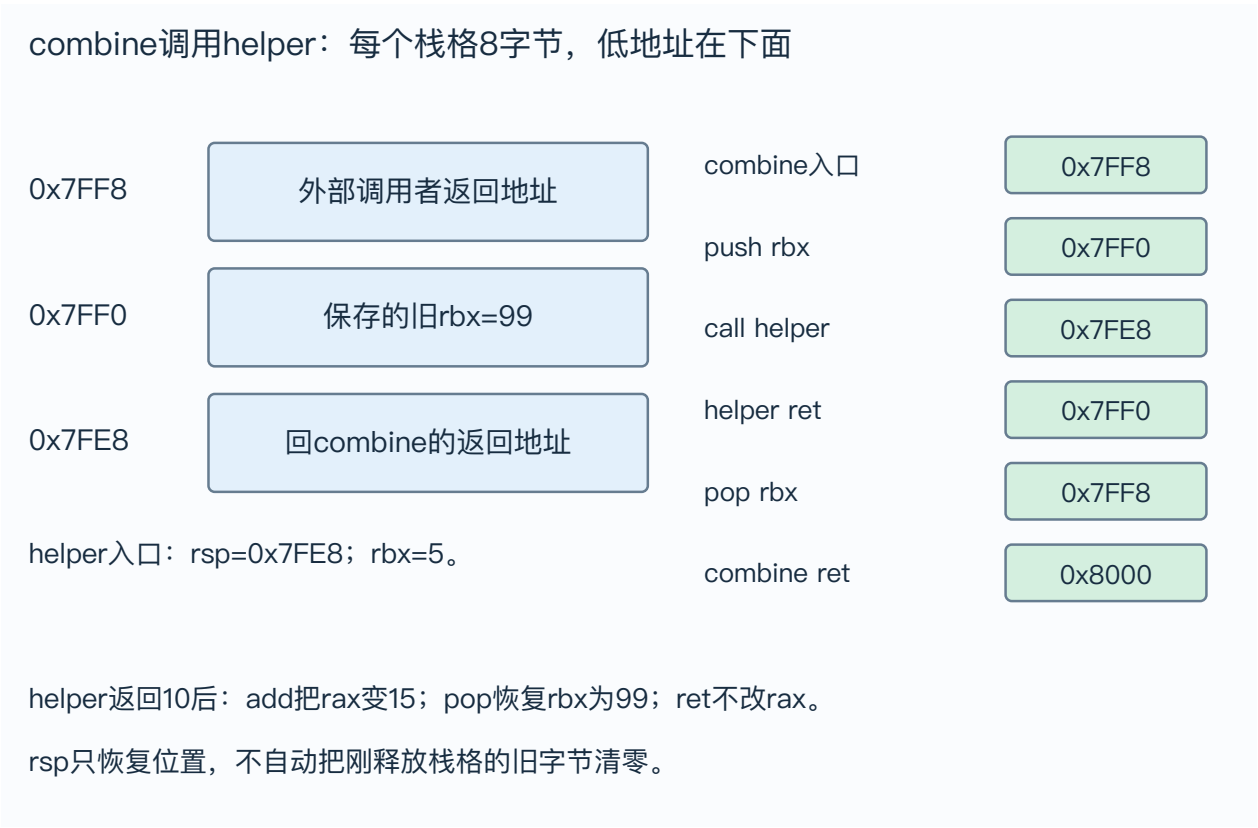


图 21　combine 调用前后，栈的低地址与高地址

每个横框恰为 8 字节。进入 combine 时，`0x7FF8` 已经保存调用者的返回地址；push 保存旧 rbx 到 `0x7FF0`，让下一次 call 前 rsp 按 16 对齐。helper 的返回地址在更低的 `0x7FE8`，不是覆盖上面的 99。helper ret 后先回到 `0x7FF0`，combine 再取回 99，最后才弹自己的返回地址。

若用小端观察保存的 99，最低地址字节是 `63`，其余七字节为 0；返回地址也是八字节地址值，但意义是“接着执行的位置”，不能把 99 当作返回地址。栈向低地址增长说的是 push 后地址数值变小，不是图必须画成某种上下方向。

6.5 栈帧怎样容纳一个尚未完成的函数

栈帧可以含局部数组、溢出到内存的临时值、保存的寄存器以及调用所需状态。常见教学代码先 `pushq %rbp`，再 `movq %rsp,%rbp`，令 rbp 成为这个函数的稳定参考位置，之后用负偏移访问局部数据。rsp 可能随临时压栈变化，rbp 通常暂时保持不变。

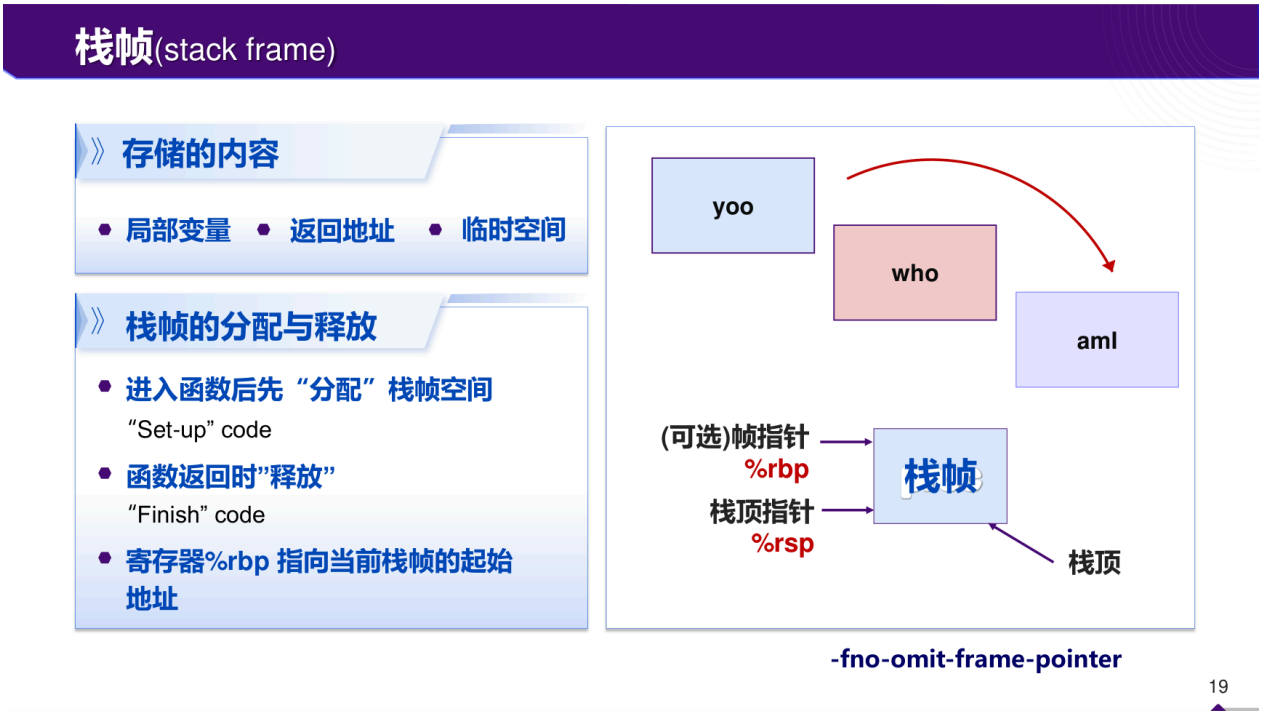


图 22　教师原页：栈帧布局，PDF 第 19 页。

原图上方的 yoo、who、aml 表示先后进入的函数。右下两个箭头分别标出可选帧指针 rbp 和当前栈顶 rsp。它在解释“同一函数活动中的一块状态”，并没有规定所有局部变量的固定顺序。图右下 `-fno-omit-frame-pointer` 是要求编译器保留帧指针的一类选项；优化编译时可能省略 rbp 帧指针，通过 rsp 偏移或调试展开信息恢复调用关系。

在常见带帧指针的函数入口，push 旧 rbp 后，`0(%rbp)` 是保存的旧 rbp，`8(%rbp)` 是返回地址。但这只适用于已经说明的序言布局。看到另一段代码没有这套序言，不能继续机械套同样偏移。

6.6 递归不是一段代码复制成许多份

代码通常只有一份，每次调用的执行状态不同。递归保存的 n、返回地址与局部值属于不同的活动实例。同名变量不是同一个存储实例，就像同一张表格可以填三份，每份里的“姓名”栏不同。

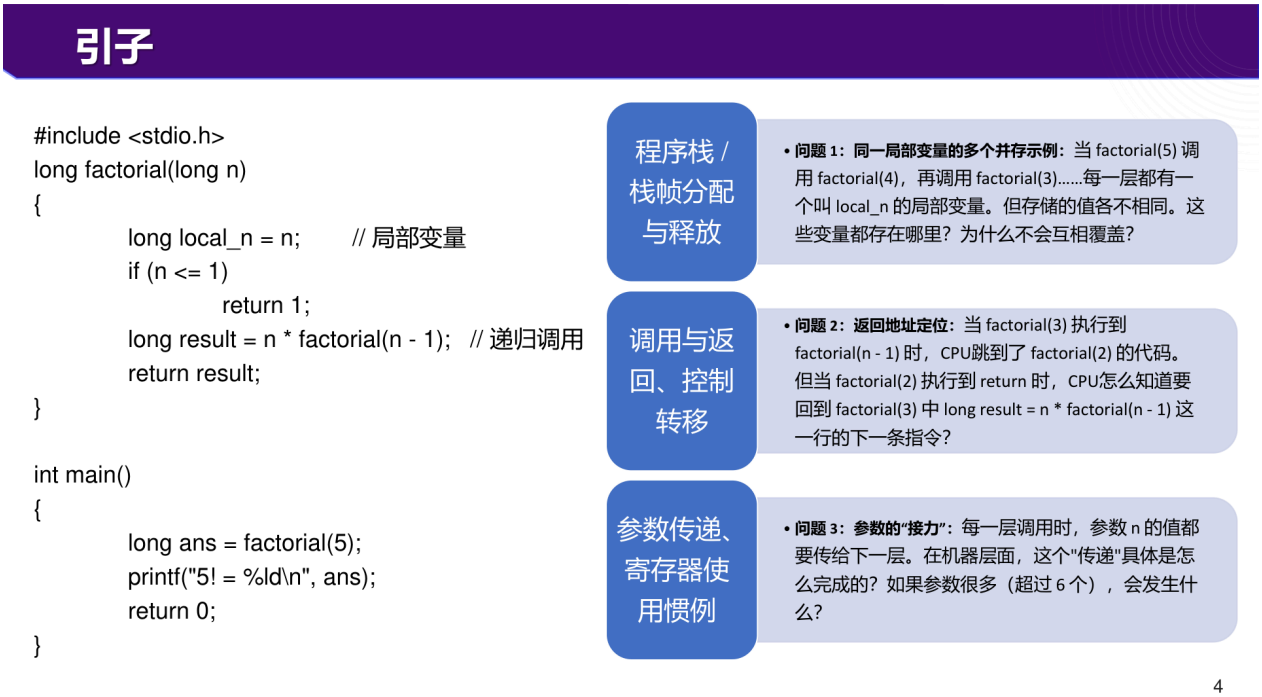


图 23　教师原页：递归阶乘提出的三个问题，PDF 第 4 页。

图左侧 factorial 的每一层需要保留自己的 n，因为递归返回后还要乘它。右上问这些同名变量为何不覆盖，答案是它们各有自己的活动状态，通常分布在不同栈帧或被正确保存的寄存器。右中问返回定位，答案是 call 保存返回地址。右下问参数接力，答案是调用约定安排寄存器或栈位置。把三个问题分别落到保存的值、地址和传参位置，就不会把“递归用了栈”当作无法展开的口号。

6.7 算例三：阶乘下去以后怎样一层层回来

考虑非负小 n 的阶乘：n≤1 返回 1，否则返回 n×fact(n−1)。fact(3) 先保留 3，调用 fact(2)；fact(2)保留 2，调用 fact(1)；fact(1)直接返回 1。此时 fact(2)才真正完成乘法 2×1=2，并返回 2；fact(3)再做 3×2=6。

可以把调用栈写成三行待办：最外层“等待 fact(2)，拿回来乘 3”；中间层“等待 fact(1)，拿回来乘 2”；最内层“直接交回 1”。返回时每消掉一行待办，就释放相应活动状态。向下展开 3 层，不意味着向下时已经把 3×2×1 一次算完；未完成的乘法在返回路上继续。

假设每层固定使用 16 字节辅助状态，最深同时存在 n 层，峰值辅助栈空间与 n 成正比。若一个递归算法先算左子树，返回后再算右子树，总调用次数可能很大，但峰值栈只取决于同时活着的调用深度。时间和栈空间必须分别数。

阶乘还受整数范围限制。64 位有符号 long 不是无限精度，n 太大时乘积溢出。写“递归公式正确”不等于“任何 n 都能由这个 C 函数正确计算”。基础条件也需说明：若允许负 n 而代码只检查 `n==0`，可能不断减小并最终出错。

6.8 算例四：第七个整数参数在哪里

假设函数只收七个普通 long 参数，无其他复杂分类。调用者把前六个装入六个参数寄存器，第七个放到栈参数区域。正常入口时 rsp 指向返回地址，因此在最简单布局中，第七个参数可从 `8(%rsp)` 读取。若函数随后 `pushq %rbp; movq %rsp,%rbp`，这个参数相对 rbp 变为 `16(%rbp)`。

偏移为什么增加 8？因为 push 又放入了一个保存值，rbp 被设在更低地址。不是第七个参数移动了，而是用于计数的参考点改变了。若函数再分配局部空间但保留 rbp，参数相对 rbp 的偏移仍不变；相对 rsp 则继续变化。

实际调用还需要考虑栈对齐、参数分类和可能的填充，学习时应把题目限定的参数类型写清。浮点参数与结构体参数不能只按“第几个名字”套这个结论。

6.9 x86–32 对照，以及栈上危险的两个来源

传统 32 位 cdecl 常把参数从右往左放到栈上，由调用者清理参数，结果在 eax。寄存器宽度、指针大小和参数偏移都与这里的 64 位例子不同。教师课件专门安排 32 位对照，就是提醒你先识别环境，再读偏移。函数名相同并不保证二进制调用方式相同。

栈越界是写入超出局部对象边界的位置，可能破坏相邻局部值、保存的寄存器或返回地址。返回局部变量的地址则是生命周期问题：函数退出后该对象已不存在，拿着旧地址继续访问不是“稍后可能被覆盖才算错”，而是已经违反对象生命周期规则。第 07 章会分别举例。

还有一个常见误会：栈空间释放通常只移动 rsp，不负责把旧字节清成零。调试器里还能看见旧值，不能证明对象仍然活着。内存内容暂时残留与语言层面的访问许可，是不同事情。

6.10 叶子函数、红区和尾调用：哪些栈动作可以省

不再调用其他函数的函数常叫叶子函数。若它只用少量寄存器，可能完全不调整 rsp，直接计算后 ret。这不意味着 call 没有保存返回地址，而是被调用函数无需额外局部栈空间。函数是否有栈帧，还要看你说的是逻辑活动记录还是显式分配的空间，术语应说清。

System V AMD64 在特定用户态约定中提供 rsp 下方 128 字节红区，某些函数可把临时数据放在那里而不显式下调 rsp。这不是所有平台都有的保证，内核代码、Windows x64 和特定编译选项可能不同。老师的基础题若没有涉及红区，不应为了套它去重解释普通 push/sub 布局；遇到优化叶子函数时再按 ABI 核对。

尾调用是函数最后一步直接调用另一个函数并把其结果作为自己的结果，不再有额外未完成工作。编译器在满足调用约定与优化条件时，可能恢复当前状态后直接跳到目标，省去新返回链的一层。普通阶乘

`n*fact(n-1)`在返回后还要乘 n，不是这种直接尾位置；带累加参数的版本才可能改写为尾递归。C 并不普遍保证尾调用优化，因此不能仅凭源码写成尾递归就保证常数栈空间。

6.11 调用者保存和被调用者保存各适合什么值

如果一个临时值只在两次 call 之间使用，放调用者保存寄存器无需额外负担；若它必须跨越许多 call 仍有效，放被调用者保存寄存器，函数入口保存一次、出口恢复一次，可能更省。反过来，被调用者若只做很短计算，也可以直接用易改寄存器而不建立保存序言。

例如函数 f 需要 x 跨过 g 与 h 两个调用。方案一把 x 放某个调用者保存寄存器，每次调用前后都存取；方案二把 x 放 rbx，f 入口保存旧 rbx，两个被调用函数按约定保持它，f 出口恢复。两者可以实现同样语义，成本取决于具体代码。调用约定规定责任边界，编译器在边界内选择安排。

初学读反汇编时，不必猜“为什么编译器偏爱 rbx”。先检查这个值是否跨 call，谁会覆盖它，哪里恢复。观察真实的数据生命范围，比把每个寄存器永久贴上“参数”或“局部变量”标签更可靠。

6.12 返回局部地址，与返回结构体值不同

`return &local;`把即将结束的对象地址交出去，访问会悬空。`return local_struct;`按值返回结构体，则语义是把成员值交给调用者，可通过寄存器或隐藏返回指针等 ABI 方式实现，不等于返回原局部对象的地址。看到机器代码中出现一个隐藏指针，也不能反推源码一定返回指针类型。

较大的结构体返回可能由调用者提供存储位置，被调用者把结果写入；这会影响显式参数寄存器的安排。因而“前六参规则”应一直带着普通整数/指针参数与分类条件，不能遇到任何函数都按源码从左往右机械配寄存器。

练习：函数返回一个含两个 long 的小结构体，是否一定放在栈上？答案是不一定，ABI 可能把符合分类的结果放在多个寄存器；具体按类型布局与 ABI 规则判断。先明确返回语义，再检查实现，不要用一种教学序言概括全部情况。

6.13 自己做，再看答案

练习一：一个函数 push 了 rbx、再 sub rsp,16 分配空间，返回前只 add rsp,16，漏了 pop，会怎样？

答案：局部空间已释放，但 rsp 仍指向保存的 rbx。ret 会把旧 rbx 内容当作返回地址弹出并跳转，真正返回地址还留在更高处，栈也少恢复 8 字节。必须同时恢复 rbx 并使 rsp 指回正确返回地址位置。

练习二：递归二分搜索每次只递归一个长度约减半的区间，峰值栈深度是多少？

答案：长度依次 n、n/2、n/4，直到常数，共约 $\log_2 n$ 层。每层固定辅助空间时，栈空间 O(log n)。如果写成循环，只保存当前左右边界，可以减少到常数辅助空间。这里讨论的是这套实现，并非所有名字叫“二分”的程序都同样用栈。

练习三：优化汇编没有 rbp，也没给局部整数分配内存，是否说明程序漏执行了局部变量？

答案：不一定。局部值可以一直在寄存器中，也可能已被常量传播消除。C 要求保持可观察行为，不要求每个源码变量都拥有固定内存格子。分析时追踪值如何产生和使用，不能只按源码名字寻找栈槽。

7 数组、结构体与内存安全：把对象边界画在地址旁边

a[i] 看起来只是取一个元素，机器最终要算出地址并读取字节。结构体里的 s.x 也是先找到对象起点，再加成员偏移。理解这两种布局以后，许多数组反汇编题只是带单位的乘加法；越界错误也会显现为“地址算到了对象外面”。

但一张数值地址表还不够判断 C 程序合法。对象有类型、有生命周期、有允许访问的范围。某地址现在确实映射着内存，也不表示你能通过任意类型的指针读取它。我们把数值地址、对象边界和生命周期一起写，才能避免“没崩，所以对了”的误判。

越界写在栈上会碰到谁：

数组越界：a[10]=7写到谁头上 第 1 / 3 步

1. 合法范围

int a[5] 与相邻变量 b 在栈上的布局：a[10]=7 会写到哪里?

a[0]	a[1]	a[2]	a[3]	a[4]	b
+0	+4	+8	+12	+16	+20

None

a有5个元素：合法下标0至4，占20字节。

为什么：编译器不做下标检查：a[i]就是取a首地址+4i。

一维数组连续，多维数组是数组里面放数组

int a[5] 是五个连续 int。设每个 int 为 4 字节，a 起点为 B，那么 a[i]地址为 B+4i。C 的 a[i] 定义与 *(a+i) 对应，所以数组下标缩放来自指针所指类型。

int m[3][4] 是“有三个元素的数组，每个元素又是四个 int 的数组”。内层一行占 16 字节，外层三行连续，故 m[i][j]地址为 B+16i+4j，即 B+4(4i+j)。这叫行优先布局：一行中的相邻列紧挨着。

将两种表示放在同一份数据上比较。矩阵 m 每行四个 int，三行分别为 10、11、12、13；20、21、22、23；30、31、32、33，基址 0x1000。读 m[1][2]先跳过一整行 16 字节到 0x1010，再跳过两个 int 共 8 字节到 0x1018，读取 22。0x1010 处存的是第二行第一个整数 20，不是一根“指向第二行”的指针。

另一种表示先准备三根 int 指针，放在从 0x5000 开始的指针表中，每根指针 8 字节：

指针表条目	该条目所在地址	条目保存的行起点	对应一行的整数
p[0]	0x5000	0x7000	10，11，12，13
p[1]	0x5008	0x9000	20，21，22，23
p[2]	0x5010	0xB000	30，31，32，33

若 p 的类型为 int **，它指向这张指针表。读 p[1][2]要先到 0x5008 读出地址 0x9000，再到 0x9000+2×4=0x9008 读出 22。两次读最终得到相同值，访问的对象却不同；三行也不必相邻。

这里两个星号对应两层对象：p 所指的是一个 int 指针，沿这个指针才找到 int。连续矩阵没有那张指针表，不能强转成 int **之后让处理器“猜到”行起点。int m[3][4]的首元素是一整行 int[4]，多数表达式中 m 转为指向这种行的指针，写作 int (*)[4]；括号把“指针”与“每行四个 int”结合。它加 1 跳过 16 字节，正好符合矩阵的下一行。



图 24　连续矩阵直接算地址，指针表先读行指针

7.1 算例一：从偏移反推二维数组列数

题目：每个 int 占 4 字节，二维数组 a 有固定 C 列；汇编先把 i 乘 12，再加 j，最后把结果乘 4 作为偏移。列数是多少?

一行含 C 个 int，跨 i 行跳过 iC 个元素，再走 j 列，元素序号 iC+j。题中为 12i+j，所以 C=12。若 i=2、j=3，序号 27，偏移 108 字节。假设 B=`0x1000`，地址为 `0x106C`。

核对可按行拆：先跨两行，每行 48 字节，共 96；再跨三列，共 12；合计 108。一个过程用公式，一个过程按行列数格子，二者一致。不要见到乘 4 就回答“4 列”，乘 4 在这里是 int 的字节大小。

若元素改为 double、每个 8 字节，同样 12 列时偏移会变成 8(12i+j)。数组列数是类型的一部分，元素大小与列数共同决定行跨度。

7.2 结构体成员之间为什么会有空白

处理器和 ABI 通常要求某类对象按一定边界对齐。对齐 a 字节，可以先理解为地址应为 a 的倍数。结构体要在不改变声明次序的前提下，为成员插入必要的填充字节。结构体总大小通常还要是其对齐量的倍数，保证数组中的下一个结构体也能正确对齐。

采用本书常见 LP64 布局，char 对齐 1，int 对齐 4，double 对齐 8。对于

```
struct Item {
    char tag;
    int count;
    double value;
};
```

tag 在偏移 0；随后偏移 1、2、3 是填充，使 count 从 4 开始；count 占 4 至 7；value 已能从 8 开始，占 8 至 15；总长 16，对齐 8。填充不是你声明的独立成员，不能假定里面自动为零，也不能把结构体直接写到网络上就获得稳定格式。

C 允许成员后有填充，但不允许在第一个成员前插入结构体内部填充。具体成员大小与对齐仍依赖实现。`sizeof`、`_Alignof`、`offsetof` 可以让程序询问实际环境，不要把教学算例中的数值当所有平台的规定。

7.3 算例二：调整顺序能省多少空间

题目：比较两个结构体：A 为 `char c; double d; int i;`，B 为 `double d; int i; char c;`，按 char1、int4、double8 的大小与对齐计算。

A 中 c 占 0，随后 1 至 7 填充；d 占 8 至 15；i 占 16 至 19；总长需补到 8 的倍数，因此 20 至 23 尾部填充，总 24。B 中 d 占 0 至 7，i 占 8 至 11，c 占 12，尾部 13 至 15 填充，总 16。

每个对象省 8 字节，一百万个对象理论上可省约 8 百万字节。但重排会改变 ABI 和文件布局，公开接口中的结构体不能随意调整。还要考虑访问模式和缓存：两个线程反复写同一缓存行中的不同成员，可能产生伪共享，未必越紧凑越快。先确认二进制兼容与语义，再做布局优化。

联合体 union 则让成员共享起点，大小足以容纳最大成员，并满足对齐。它和结构体的顺序叠放不同。通过非当前成员解读位表示还涉及 C 与 C++不同规则，不应把“占同一内存”推广成任意类型互转都可靠。观察对象字节可用 unsigned char；在浮点位串与整数之间保留表示通常可使用 memcpy。



图 25　结构体 A 和 B 的每个成员与填充区间

图上每一格为 1 字节，先认横轴偏移，再读成员。A 中 double 不是从偏移 1 开始，而是跳到下一个 8 的倍数 8；int 结束于 19 以后，还得补到 24，才保证第二个 A 对象里的 double 仍 8 字节对齐。B 的末尾 char 放在 12，补到 16 即可。成员重排省下的是这两组填充，不是把 double 从 8 字节压成 4 字节。

对别名也可以列两个小状态表：p 与 q 不同对象时，`*p=1` 后对象 P 为 1，`*q=2` 后 P 仍 1，所以返回 1；p、q 同指对象 M 时，第一写 M=1，第二写同一个 M=2，最终返回 2。两个形参名字不同，不保证两块内存不同。

7.4 对象的四类常见存储期

自动存储期的局部对象一般随所在块进入而产生、离开而结束，常见实现放栈上，但标准描述的是生命周期，未要求一定使用栈。静态存储期对象在整个程序运行期间存在，包括文件级变量与 static 局部变量。分配存储期对象来自 malloc 等分配接口，直到 free 结束。线程存储期对象每个线程各有实例。

“变量名字在这个函数可见”叫作用域；“对象还活着”叫生命周期。二者经常相关，却不是一个概念。你可以把堆对象地址传出创建它的函数，函数返回后对象仍活着；你也可以把局部变量地址传出去，但函数返回后对象已结束。

7.5 算例三：悬空指针为何有时还打印旧值

```
int *bad(void) {
    int x = 42;
    return &x;
}
```

返回时交出去的是 x 原位置的地址，不是把 x 复制到一块永久空间。函数结束后 x 生命周期结束，该指针不能用于访问原对象。调试时可能恰巧仍看到 42，因为栈字节尚未被覆盖；下一次函数调用可能把同一位置用于别的数据。两次运行输出不同只是症状，错误早在失去有效对象后访问时就存在。

安全选择取决于需求。若只要一个整数，直接返回 int 值。若要让调用者提供位置，则函数接收 `int *out`，写入由调用者保证有效的对象。若要创建长期存在的可变对象，可以分配并明确由谁 free。不要为了避免悬空指针就一律改成 static 局部变量，它会变成所有调用共享同一对象，在线程与重入场景下又带来新的问题。

7.6 数组边界不能靠相邻内存碰巧存在来放宽

`int a[3]` 可以形成 `a+3` 这个尾后指针，用于与遍历指针比较；不能解引用它。形成更远的 `a+4` 也已超出数组指针算术的正常允许范围。即使 a 后面恰巧摆着另一个 int 变量，`a[3]`也不能作为访问那个变量的可移植写法。

二维数组还有一个容易忽略的边界。`int a[2][3]`虽然六个 int 在布局上连续，`a[0][3]`仍越过第一行这个三元素数组的边界；不要用物理连续性替代类型边界。正规写法是 `a[1][0]`，或者设计一个本来就扁平的一维数组并按统一下标访问。

字符串需要给终止符留空间。`char s[3]="abc";`在 C 中可形成没有终止零的三字符数组，之后若当普通以零结尾字符串传给 printf 的 %s，可能继续读到边界外。`char s[]="abc";`则由初始化推导长度 4，包含终止符。复制函数的名字带 n 也不一定保证终止，必须查具体接口语义。

7.7 算例四：缓冲区与长度的双重检查

题目：分配 n 个 `struct Item`，为什么只写 `malloc(n*sizeof *p)`还可能不够?

如果 n 与元素大小的乘积在 size_t 中溢出，得到的分配字节数会比需要的小，后续按 n 个元素写入就越界。应在乘法之前检查 `n > SIZE_MAX/sizeof *p`。例如用一个仅有 8 位的玩具 size_t 模型，n=40，元素大小 8，数学字节数 320，模 256 后只剩 64，实际只够 8 个元素，循环却计划写 40 个。

检查乘积之后还要检查 malloc 返回值。成功分配也只得到未初始化的存储，不能假设所有成员为零；初始化时要尊重成员类型。使用 calloc 可请求按字节清零并让实现负责乘法溢出处理，但“全零位一定对应所有类型的零值”也不应无条件推广到任意 C 实现的所有对象表示。

如果外部长度来自有符号整数，先检查非负，再转换到 size_t；把−1 直接转换为 size_t 常会成为巨大值。长度、容量、已使用数量也要分开：容量 20 并不表示已有 20 个合法初始化元素。

7.8 画内存图时把权限也写上

推荐在每块对象旁写四项：起点、字节数、类型、生命结束位置。若是共享对象，再写哪些线程能访问、用什么锁保护。地址表只回答“字节在哪里”，这几条补充回答“此时允许怎样用”。

调试器可以查看内存，地址消毒器可发现许多越界和释放后使用，未定义行为检查器可发现一部分语言层面的错误。它们是发现具体执行问题的工具，没报错不能证明所有输入都安全。阅读源码时的对象边界分析仍然必要。

7.9 柔性数组成员怎样把头与变长数据放一起

有时对象需要固定头部，加一段长度运行时才知道的数据。C 的柔性数组成员允许结构体最后写一个不完整数组成员，例如 `struct Packet { size_t n; unsigned char data[]; };`。sizeof 只计固定部分及必要布局，不含实际正文，因此分配时要加正文长度并检查加法溢出。

采用题目给定固定头长 8 字节、正文长度 5 字节的简化布局，总共至少请求 13 字节；n 记录 5，data 合法元素为 0 至 4。不能因为 malloc 可能实际分到 16 字节，就擅自使用 data[5]至 data[7]。应用拥有的合法范围由请求和对象构造约定决定，不由分配器内部余量自动扩张。

这类布局还会涉及结构体尾填充与 offsetof(data)的细节，严谨实现应按实际类型规则计算，不机械把头长写死 8。它适合说明“一个连续分配里有固定和变长两部分”，但不会消除长度字段损坏的风险；每次根据 n 访存仍要确认 n 与实际分配一致。

7.10 指针别名让一个写入影响另一个读值

考虑 `int f(int *p,int *q){ *p=1; *q=2; return *p; }`。若 p 与 q 不同，返回 1；若指向同一对象，第二次写会把它改成 2，返回 2。优化器不能无条件把最后读取替成常数 1，除非它有合法依据排除别名。

因此读汇编时，一个看似多余的重新加载可能在保持别名语义。你也不能因为两个形参名字不同，就在纸面上给它们画两块互不关联内存。题目没说明时，应该分别分析可能相同与保证不同的情况。

restrict 是 C 中表达受限制别名承诺的一种机制，含义有具体规则，不是一个只为了加速的装饰。调用者违反承诺可能使程序失去保证。初学时先理解普通指针可能别名，再研究如何合法表达不别名，比先加关键字后猜效果更可靠。

7.11 未初始化和越界是两种不同错误

数组容量 10，程序只初始化前 3 项，却循环读取 10 项。每次地址都在分配范围内，但后 7 项没有按需求初始化，读取仍不提供你以为的合法业务数据，有些类型或表示还会涉及更严重语言问题。地址消毒器未必发现这类逻辑错误，因为没有越界。

反过来，越界位置恰好被别的对象初始化过，也不能因此允许读。边界检查回答对象是否属于本次访问，初始化检查回答属于范围的值是否已经准备好。应分别保存 capacity 和 size：capacity 是最多能容纳多少，size 是目前有效元素多少。

练习：动态向量容量 16、已有元素 5，能否遍历 0 至 15 输出？答案是不能按“已有元素”输出，除非其余元素也被设计为有效初始化数据；通常只遍历 0 至 4。扩容后容量变大，不会凭空产生更多业务元素。

7.12 自己做，再看答案

练习一：`struct S { short a; char b; int c; };`，short 大小对齐 2，char1，int4。各成员偏移和总长是多少？

答案：a 占 0 至 1，b 占 2，偏移 3 填充；c 从 4 开始占 4 至 7。最大对齐 4，总长 8，已是 4 的倍数，无需额外尾填充。若是数组 S[2]，第二个对象从偏移 8 开始。

练习二：一个函数 malloc 出 10 个 int，初始化后返回指针；另一个函数返回局部 int 数组首地址。为什么前者可以、后者不可以？

答案：malloc 对象的生命周期不随创建它的函数返回而结束，调用者可继续用并最终 free；局部自动数组在函数返回时结束生命周期，地址不能继续解引用。前者仍要检查分配成功和所有权，不能因为“在堆上”就永远不用释放。

练习三：`int m[2][5]`，起点 B，每个 int4 字节，求 m[1][4]的地址。若有人按 `int **`处理，第一次读取会发生什么误会？

答案：地址 B+4(1×5+4)=B+36。`int **`解读会先把某个位置的一组字节当作指针，而原对象那里存的是 int 数据，类型与布局均不符。不能把第一行中的整数拼成一个地址再继续访问。

8 编译与链接：函数写在另一个文件里，机器怎么找到它

main.c 里调用 sum，sum 的定义却在 sum.c。编译 main.c 时，工具可能还没看到 sum 的函数体，照样能生成 main.o。直到链接时，工具把“这里需要 sum”与“这里定义了 sum”对应起来，再修正调用位置。理解这条过程，你就能分清语法错误、未定义符号、重复定义和运行时找不到共享库分别发生在哪一层。

本章以常见 Linux 的 ELF 目标文件为例。macOS 使用 Mach-O，Windows 常用 PE/COFF，细节与命令输出不同。可以比较概念，但不要拿 ELF 的段名与重定位格式逐字套到所有系统。

8.1 算例一：两个源文件怎样接起来

把以下三个文本文件放进同一目录。sum.h 只声明接口，告诉两份源文件参数和返回值都是哪些类型：

```
#ifndef SUM_H
#define SUM_H
long sum(const long *a, long n);
#endif
```

sum.c 给出实现。它只在本题约定的 n 非负、a 有至少 n 项且部分和不溢出的输入上工作：

```
#include "sum.h"
long sum(const long *a, long n) {
    long total = 0;
    for (long i = 0; i < n; ++i)
        total += a[i];
    return total;
}
```

main.c 准备三个元素并调用这个接口：

```
#include <stdio.h>
#include "sum.h"
int main(void) {
    long a[3] = {2, 4, 8};
    printf("%ld\n", sum(a, 3));
    return 0;
}
```

在终端依次执行：

```
cc -std=c17 -Wall -Wextra -c main.c -o main.o
cc -std=c17 -Wall -Wextra -c sum.c -o sum.o
cc main.o sum.o -o app
./app
```

最终输出 14。前两个命令只处理各自的.c；main.c 知道 sum 的声明，不需要在这一阶段得到 sum 的函数体。第三个命令才把两者合起来。分别编译后，main.o 含 main 的定义、对 sum 和 printf 等符号的引用；sum.o 含 sum 的定义。链接器找到 sum 的入口，修正 main 中调用 sum 的位置；printf 则由相应标准库链接安排解决。程序运行时传 a 地址与长度 3，sum 返回 14，printf 按 long 格式输出。

若命令漏写 sum.o，main.o 里的 sum 引用无人提供，链接失败。若把 sum.c 中的函数改成文件级 static，那个定义只在 sum.c 内部有效，通常同样不能满足 main.o 的外部 sum 引用。若把 main 里的声明改为错误参数类型，符号名字可能仍对上，错误便延后到运行行为，不能指望“链接成功”兜住它。

8.2 一个编译命令背后的四个阶段

预处理展开 include、宏和条件编译，输出仍是 C 文本。编译阶段分析 C 的类型和语义，做优化，生成汇编或中间表示到汇编的结果。汇编器把指令与数据转换为可重定位目标文件，通常后缀.o。链接器把多个目标文件和库组合，完成符号解析与重定位，生成可执行文件或共享对象。

例如可以用 `cc -E main.c` 观察预处理结果，用 `cc -S main.c` 生成汇编，用 `cc -c main.c` 停在目标文件。`cc main.o sum.o -o app` 再链接。cc 在这里是驱动程序，它替你选择和调用后端工具；看到同一个命令名字，不表示所有阶段都由同一个内部模块完成。

错误发生的阶段影响排查方法。少一个分号通常在编译阶段报；声明了函数但没有提供定义，可能编译通过、链接报 undefined reference；共享库路径不对，则可能链接成功，到启动时才报加载失败。先定位阶段，再查对应输入，能少走很多弯路。

8.3 符号表：名字要对应哪块代码或数据

符号可以代表函数入口、全局变量等位置。目标文件记录自己定义了哪些符号、使用了哪些未定义符号。文件级 static 函数或变量通常具有内部链接，不应作为其他目标文件同名引用的候选定义。普通局部自动变量往往没有链接意义上的全局符号，优化后甚至不在内存中单独存在。

C 的声明告诉编译器“这个名字的类型是什么”，定义还负责真正提供对象或函数。头文件通常放声明，让多个.c 一致地知道接口。把一个普通全局对象定义直接放进头文件，再被两个.c 包含，就可能得到两个定义。头文件保护宏防止的是同一翻译单元重复包含，不会替不同.c 合并全局定义。

链接器通常不会替你做完整 C 类型检查。一个文件以 long 声明某外部对象，另一个以 int 定义，即使名字能对上，程序也可能按错误大小读写。应把声明集中到共同头文件，并让定义文件也包含它，尽早由编译器检查一致性。

8.4 算例二：相对调用位移怎么填

题目：某 x86-64 相对 call 位于地址 `0x400100`，该指令长度 5 字节，目标函数入口为 `0x400200`。位移是多少？

相对基准是下一条指令地址 `0x400105`，所以位移为 `0x400200−0x400105=0xFB`，十进制 251。执行时 CPU 把下一条地址加位移，得到目标入口，同时将下一条地址作为返回地址压栈。

如果错用 call 起始地址作基准，会算出 `0x100`，最终跳到 `0x400205`，偏了五字节。链接器的重定位记录正是告诉它“哪几个字节要填、依据哪个符号、采用哪种计算规则”。不同重定位种类的公式不同，不能把这个 call 公式套到任意绝对地址数据。

重定位之后，指令本身并不需要保存“sum”这个人类名字。它可以直接携带偏移。符号和调试信息常为了分析保留，也可以被剥离；程序仍能运行，因为运行所需的位置关系已经编码到指令或动态链接结构中。

把这个 call 的五个机器字节也列出来，能检查“符号地址”和“位移字段”是不是被混在一起：

地址区间	内容	它负责什么
0x400100	E8	近相对 call 操作码
0x400101 至 0x400104	FB 00 00 00	32 位小端位移+251
0x400105	下一条指令起点	计算位移的执行基准
0x400200	目标函数入口	最终跳到的位置



图 26　相对 call 从待修字段到最终字节与目标

若相应 ELF 重定位按 `S+A−P` 表达：S 为目标符号地址 `0x400200`，P 为位移字段起点 `0x400101`，这一写法的加数 A 为−4，便得到 `0x400200−4−0x400101=0xFB`。−4 把“字段起点”换成“字段结束后的下一条”。具体重定位类型按目标文件给定规则，不能看见相对引用就假定所有类型的 A 都为−4。本例位移落在有符号 32 位范围内；若目标距离超出范围，就需要其他调用安排，截断字段并不能保证仍跳到目标。

反向核对 CPU 执行：把字段解码成+251，取下一条 `0x400105` 加 `0xFB`，恰好 `0x400200`。若目标地址小于下一条指令地址，位移为负，字段仍按 32 位补码保存，不是加一个巨大无符号跳距。

8.5 目标文件里的节与运行时的段

ELF 节用于组织链接时的信息，例如.text 保存机器代码，.rodata 保存只读数据，.data 保存有初值的可写数据，.bss 描述需要零初始化的可写存储。符号表、重定位表等也有自己的节。节的名字帮助理解职责，具体文件可以有更多节，也有不完全同名的安排。

.bss 中的大量零通常不需要在磁盘文件中逐个存出来；文件记录运行时要多少空间，由加载器建立为零初始化的内存。因此一个定义了巨大静态零数组的程序，运行内存可能远大于可执行文件磁盘大小。malloc 得到未初始化存储的情况与此不同，不要把“系统某些新页看起来为零”当 malloc 的语言级承诺。

加载时更关心段。程序头描述哪些文件区域要映射到哪些虚拟地址，以及读、写、执行等权限。多个节可以归入一个加载段。节表便于链接和分析，程序头便于加载，不能看到两个名词都叫“块”就合并为同一东西。

8.6 静态库为什么有时换个顺序就能链接

静态库通常是多个目标文件的归档，不是把整座库无条件装进最终程序。传统链接器从左到右处理输入，遇到库时按当时尚未解决的符号需求提取成员。因此“先出现使用者，再出现提供者”常是静态库顺序的重要原则。

设 main.o 需要 a，libA.a 中的 a 又需要 b，libB.a 提供 b。常见命令顺序为 `main.o −lA −lB`。如果先写-lB，当时尚无 b 需求，链接器可能不提取其成员；之后 a 引入 b 需求时，未必回头自动重扫前面的库。实际现代工具支持分组重扫、全库纳入等选项，但你首先要理解普通扫描为什么会失败。

循环依赖库时，简单调换一个顺序可能不够，分组扫描或更合理的模块拆分才是办法。不要通过随意重复十遍库名碰碰运气。链接器诊断给出的未定义符号与引用对象，能指向真正的依赖边。

8.7 算例三：重复定义与 static 的区别

a.c 和 b.c 都写 `int count=1;`，两者都提供一个有初值的外部定义，链接器通常报重复定义。若两个文件都改为 `static int count=1;`，它们分别得到只在各自文件可见的独立对象，名字相同但不是共享计数器。

若本意是共享一个计数器，应该在一个.c 中定义一次，在共同头文件中写 `extern int count;`，其他.c 包含头文件。extern 声明让编译器知道类型，实际存储由那一个定义提供。若本意是每个模块拥有私有计数，static 是合理选择，但不能拿它只为消除错误提示而改变程序语义。

旧工具链对无初值暂定定义可能采用公共符号合并规则，较新工具链默认设置也可能变化。课程题常讲“强符号、弱符号、公共符号”的历史模型；实际项目不应依赖错误多重定义恰巧被合并。明确单一定义，比猜链接器版本可靠。

8.8 共享库、位置无关代码和动态装入

共享库可以在运行时映射进进程，不同程序使用同一库的代码页时有机会共享物理页。库在每个进程里的虚拟地址不一定相同，也不意味着它的可写全局数据在不同进程间自动共享。

位置无关代码尽量避免把固定绝对地址写死在指令中，配合相对寻址、全局偏移表 GOT 等结构访问外部数据。过程链接表 PLT 常参与外部函数调用。动态链接器负责建立映射、解析动态符号、完成必要的运行时修正；某些环境使用延迟绑定，某些采用立即绑定，具体要看链接选项和系统策略。

程序启动时找库的路径，也与编译时头文件搜索路径不同。`-I` 主要影响包含头文件，`-L` 主要影响链接时搜索库，运行时还有加载器的路径规则。因为编译能找到声明，不代表运行时一定找到实现。诊断时分开检查“声明来自哪里、链接选中哪座库、运行加载哪份库”。

8.9 算例四：库加载失败应该查什么

题目：编译链接都成功，把 app 复制到另一台相同架构 Linux 机器却提示缺少 libexample.so。你应先改 C 源码吗?

首先确认错误发生在动态加载阶段。可检查可执行文件记录的依赖库名字、目标机器是否安装兼容版本、运行时搜索路径与架构是否匹配。将头文件一起复制过去通常不能解决问题，因为头文件在编译阶段使用，运行时需要的是共享对象及其依赖。

若程序已启动，调用某函数时才报找不到动态符号，还要检查库版本是否提供该符号，以及符号版本或加载范围。若架构不同，文件即使同名也不能直接使用。修复应针对实际依赖，不要把网上某个同名二进制随意塞到系统目录。

8.10 头文件接口为什么也要被定义文件包含

假设头文件声明 `long transform(long);`，实现文件却没有包含头文件，自己定义成 `int transform(int);`。调用文件按 long 参数与 long 返回值生成代码，定义文件按 int 处理，符号名字仍可能一样，链接器只见到名字对上，不能替你恢复 C 类型一致性。

让实现文件也包含公共头文件，编译器便能在同一翻译单元看到声明和定义矛盾，尽早诊断。跨语言接口还需要注意 C++名字修饰、extern “C”、结构体布局等规则；把文件后缀从.c 改.cpp，不等于二进制接口保持不变。

这解释了一个看似繁琐的工程习惯：接口声明只有一个可靠来源，每个使用者和提供者都包含它。复制粘贴一份看起来相同的函数原型，未来某次修改后可能分叉，链接成功也不能证明调用正确。

8.11 动态链接查到同名函数，为什么仍可能不是预期那份

进程可加载多座共享对象，它们可能导出同名符号。动态符号解析有查找范围与顺序，某些平台还允许符号插入等行为。于是“系统里确实存在一个名为 foo 的符号”并不充分，还要确认当前引用会绑定到哪一个定义。

课程基础部分不需要记完所有加载器选项，但应知道排查方向：实际加载路径、库版本、导出符号、可见性与绑定方式。把多个不兼容版本放进同一搜索路径，可能让开发机偶然成功、部署机失败。记录实际依赖版本与可重复构建过程，比只存一条编译命令更稳。

共享库 ABI 兼容还涉及结构体大小、枚举或类型约定、函数签名和符号版本。源码能重新编译并不意味着旧二进制无需重编。头文件是源码接口的一部分，共享对象提供二进制实现，两者版本必须协调。

8.12 一道 bss 与文件大小的推理题

程序 A 定义 `static unsigned char buf[1000000];`，无显式非零初值；程序 B 在运行时 malloc 同样多字节。A 的可执行文件不必因此多出一百万个零字节，但运行时需要建立相应零初始化对象。B 直到分配路径执行时才请求存储，失败与对象生命周期也由动态路径决定。

若 A 的数组完全不被使用，优化与链接垃圾回收还可能移除它，所以实验时要确认对象真的保留。若想比较文件大小，须确认对象未被删除，否则会把删除效果误读为 bss 压缩。读工具输出时，把磁盘文件字节数、虚拟映射大小和实际驻留物理页数分开。

练习：把零数组改为一百万个明确非零字节的初值，文件大小是否通常更可能增大？答案是更可能，因为初始内容需要在文件或等效机制中表达；但压缩、链接优化与格式仍可影响具体大小。不要只凭源码数组长度推断最终磁盘精确字节数。

8.13 编译期替换接口：宏能替你改调用，也能带来误会

课程选节中的库插入可以从编译期方法理解：在编译某段源码时，用头文件宏把一个函数名重定向到包装函数。比如把普通分配调用导向一个记录请求大小的 my_malloc，包装函数再调用实际分配接口。这样可以统计分配，但必须避免包装函数自己也被同一个宏再次替换，造成无限递归。

宏在预处理阶段生效，只影响包含它之后的源码文本，不会自动改写一个已经编译好的第三方目标文件。想在链接期或运行期拦截，是其他层次的机制，具有不同工具和限制。把三种阶段分开，就知道为什么“我加了宏，但库内部调用没变”并不奇怪。

函数式宏还可能重复求值实参，例如不谨慎的平方宏传入 i++，行为与普通函数不同，甚至触及未定义的修改顺序。接口包装应尽量保持参数只求值一次、类型一致和错误语义不变。为了统计而改变 malloc 失败处理或把 errno 无意覆盖，就会使观察工具本身改变被观察程序。

8.14 一个重定位字段不是整个符号表

符号表负责告诉工具“这个名字的定义或引用是什么”；重定位项负责指出“输出文件哪个位置需要按哪种规则修正”。同一个函数符号可以被多个调用点引用，每个调用点各有修正位置；某个符号也可能仅用于调试或导出，并不对应你刚看到的那条 call。

设 main.o 里两处调用 sum，汇编时 sum 不在本文件中，工具可以为两处保留相应待修正字段。链接把 sum 放到最终地址后，两处 call 距离不同，因此位移数值也不同。不能把“sum 地址是 0x5000”直接填进两处相对位移字段，否则会把地址与位移混淆。

练习：两条相对 call 下一条地址分别 0x1005 和 0x2005，目标都 0x3000，它们位移一样吗？答案不一样，分别 0x1FFB 和 0x0FFB。目标符号相同，引用位置不同，重定位结果自然不同。这个例子正好把第 01 章的地址、第 04 章指令位置和本章链接责任连起来。

8.15 自己做，再看答案

练习一：`extern int x;`和 `int x=3;`各负责什么？将后一行放进被两个.c 包含的头文件，有什么风险？

答案：前者通常只是声明外部对象及其类型，不在此处提供带初值的定义；后者定义对象并给初值。放入两个翻译单元会各提供一个外部定义，通常导致多重定义。头文件保护宏不能跨翻译单元消除它们。

练习二：call 起点 `0x1000`，长度 5，目标 `0x0F00`，相对位移是多少？

答案：下一条地址 `0x1005`，位移 `0x0F00-0x1005=-0x105`，即−261。它需按该重定位字段的有符号编码保存。执行时 `0x1005-0x105=0x0F00`，核对无误。

练习三：两个进程加载同一个共享库，库里的普通全局变量是否就是同一个共享计数器？

答案：通常不是。代码页可能共享，但可写数据属于各自进程的地址空间；私有映射可能先共享再写时复制。若希望跨进程共享数据，要明确使用共享内存等机制，并安排同步，不能从“共享库”的名字推出可写对象共享。

9 异常与进程：程序为什么能停下来，又从原处接着跑

你在终端里启动程序，它并没有独占整台电脑。操作系统不断安排不同执行活动使用处理器，同时处理磁盘、时钟和网络带来的事件。有时是程序主动请求系统服务，有时是当前指令访问了不允许的位置，有时是外部设备通知处理器“数据到了”。正常顺序执行的指令流因此被打断，这一类机制统称为异常控制流的一部分。

本章先分清异常、进程、上下文切换，再用 fork 画出父子进程。看 fork 题时，最有效的工具不是把源码机械复制粘贴，而是画一棵进程树，为每个进程单独保存变量和下一条指令位置。

fork 之后进程树的生长：

fork：一次调用，两份进程 第 1 / 3 步

1. 初始：只有pid 1

fork：把当前进程复制一份，父子从同一行代码之后各自继续

```
pid 1
```

进程树只有根。

父进程pid=1运行中。

为什么：fork之前的代码只执行一次。

进程、程序文件和地址空间

程序文件是磁盘上的代码与数据描述；进程是运行时实例。你可以把同一可执行文件启动两次，得到不同进程 ID、不同执行进度和通常相互隔离的数据。进程地址空间是它可使用的虚拟地址集合及映射，下一章组中的虚拟内存会解释如何映射到物理页。

设同一个求和程序启动两次，记作进程 P 和 Q。它们的代码可以相同，但当前加到哪里不同。只列三项状态：P 暂时有 rax=7、rsp=0x8000、下一位置 0x401020；Q 有 rax=99、rsp=0xA000、下一位置 0x501040。这些地址只为演示，实际还有其他寄存器与映射状态。

阶段	CPU 当前执行谁	CPU 里的 rax/rsp/下一位置	被暂停者的状态留在哪里
先运行 P	P	7 / 0x8000 / 0x401020	Q 的 99 / 0xA000 / 0x501040 已由系统保存
系统接管切换	内核	执行内核所需的状态，本表不列	把 P 的 7 / 0x8000 / 0x401020 保存下来
恢复 Q	Q	99 / 0xA000 / 0x501040	P 的保存状态没有因 Q 使用 rax 而丢失
Q 执行一段后再切换	内核	先保存 Q 本次进度	假定 Q 已变为 100 / 0xA000 / 0x501044

阶段	CPU 当前执行谁	CPU 里的 rax/rsp/下一位置	被暂停者的状态留在哪里
恢复 P	P	7 / 0x8000 / 0x401020	Q 的新状态留待以后恢复

内核是操作系统中以受控权限管理这些资源的部分。它保存当前执行者继续所需的状态，再恢复另一执行者，这就是一次上下文切换。P 再次获得 CPU 时，不是从程序开头重做，也不会把 Q 留下的 100 当作自己的部分和。

上下文包括寄存器、程序计数位置、栈状态、地址空间关联等继续运行所需信息。普通函数调用在同一进程的执行路线中转入另一个函数；切换进程则会恢复另一个运行实例的状态。两者都出现“保存与恢复”，保存对象和目的不同。

在单核上，多个进程可以通过交错运行形成并发；在多核上，它们也可能真正同时执行。并发只要求执行时间段有重叠、彼此都尚未完成，不要求每一瞬间同时占着 CPU。后面分析竞态时，单核交错也足够产生问题。

9.1 普通函数调用与进入内核的区别

普通 call 遵循用户程序的调用约定，在当前允许权限下跳到另一段用户代码。系统调用则通过受控入口请求内核执行服务，例如打开文件或创建进程。用户程序不能仅靠把某个函数地址写进指令，就获得任意硬件权限。

处理器异常按成因和返回位置可作教学分类。中断通常来自当前指令流外部，例如时钟或设备事件；陷阱常用于有意进入系统服务或调试，处理后可从后续位置继续；故障由当前指令引起，处理成功后可能重新执行该指令，例如可修复缺页；终止类错误则可能无法安全继续。不同架构术语细节有所差别，重点是问清“因谁发生、能否修复、回到哪条指令”。

缺页不一定是程序错误。一个合法的虚拟页暂时没有驻留物理内存，内核可以准备页面，再重试原指令。访问根本没有合法映射或没有权限的位置，则可能使进程收到 SIGSEGV 等信号。缺页异常是处理器与内核之间的机制，信号是操作系统向进程报告事件的另一层接口，不要把两者写成完全同一个东西。

9.2 fork 返回两次，源码并没有从头重跑

fork 成功后，原进程成为父进程，新建子进程从 fork 返回后的同一逻辑位置继续。父进程得到子进程 ID 这个正数，子进程得到 0。失败时只有原进程，返回−1，没有子进程。通过返回值分支，父子可以执行不同工作。

普通私有内存开始时内容相同，但分别属于两个地址空间。父进程把自己的 x 改成 20，不会自动把子进程 x 也改成 20。Linux 常用写时复制实现：开始共享部分物理页，某方写入时再复制，保持私有内存的语义。实现上暂时共享物理页，不等于语言程序中的普通变量成为跨进程共享变量。

文件描述符则有一层不同关系：父子各有描述符表项，但对应表项可能指向同一个内核打开文件描述，从而共享文件偏移等状态。第 13 章会画这一层。回答“fork 共享什么”时，不能只说“全共享”或“全不共享”。

9.3 算例一：x 的两份副本

```c
int x = 10;
pid_t pid = fork();
if (pid == 0) {
    x += 1;
    /* 子进程看到 x=11 */
} else if (pid > 0) {
    x += 2;
    /* 父进程看到 x=12 */
} else {
    /* 处理失败 */
}
```

fork 之前一份 x=10。成功之后有两份内容同为 10 的私有 x。子进程自己的加法得到 11，父进程自己的加法得到 12。谁先执行不改变这两个结果，因为两边没有写同一个私有对象。

如果两边分别打印，输出行先后通常没有保证。你不能看到终端上“父先出现”，就认为 fork 保证父先运行。调度、缓冲和 I/O 都会影响可观察顺序。若题目明确父进程先 wait 子进程，则可增加一个实际顺序约束。

9.4 算例二：连续两次 fork 有几个进程

代码是两条不带条件的 fork，假定都成功：第一条之后有原父 P 与子 C1；第二条会由这两个进程各执行一次，P 再生 C2，C1 再生 C3，共四个进程。每个进程都从自己执行的 fork 之后继续。

可以按阶段画数量：开始 1 个，第一轮 2 个，第二轮 4 个。一般 k 条每个现存进程都必经且成功的 fork，最后有2^k个进程。但只要加入条件、提前退出、失败或 exec，这个公式就不能直接套。

例如 `if(fork()==0) fork();`，只有第一轮子进程继续执行第二次 fork，总共三个。看见两个 fork 字样就回答四个，漏掉的是控制流条件。分析时应把“哪个进程会经过这条指令”写在每个树节点旁。

用两条有名字的返回值把四个执行者分开。下面只是为图指定示意 PID，真实 PID 由系统分配：

```
pid_t r1 = fork();
pid_t r2 = fork();
/* 假定每个调用均成功，所有执行者都到达这里 */
```

最终进程	它由谁创建	r1 保留什么	r2 是什么	下一步执行哪里
P，示意 100	原进程	101	102	第二条 fork 之后
C1，示意 101	P 的第一次 fork	0	103	第二条 fork 之后
C2，示意 102	P 的第二次 fork	101（继承 P 当时的值）	0	第二条 fork 之后
C3，示意 103	C1 的第二次 fork	0（继承 C1 当时的值）	0	第二条 fork 之后

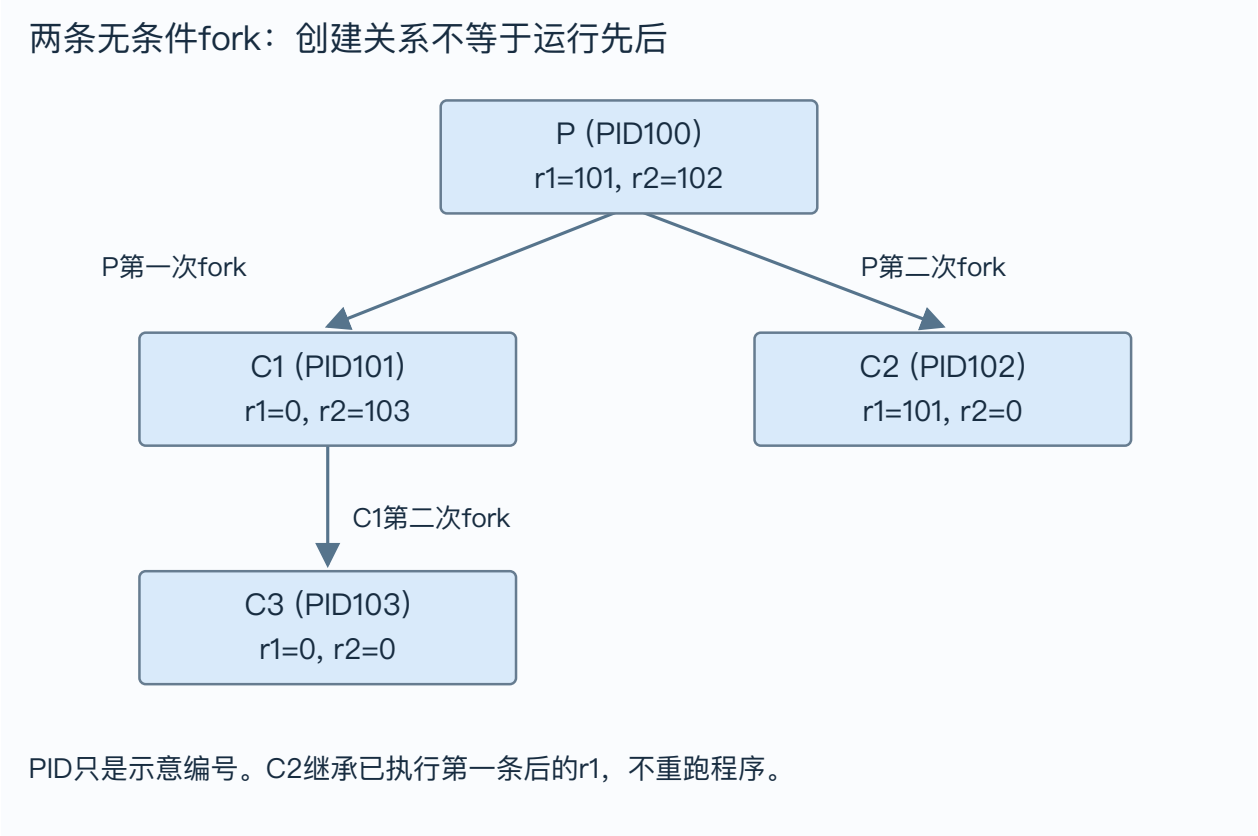


图 27　两次 fork 的父子树与返回值

C2 不会倒回第一条 fork 再产生孩子，它已继承第一条执行后的状态；C3 也不会重做第一条。树中的创建边表示谁生谁，不表示叶子之间有从左到右的运行顺序。若程序是第一次 fork 只由子分支继续 fork，就只保留 P→C1→C3 那部分，不会出现 C2。

9.5 exec：换程序，不是再生一个孩子

exec 系列在成功时用新程序替换当前进程的程序映像，当前进程 ID 通常保持。原来的后续 C 代码不会再执行；只有失败才返回错误。因此常见 shell 模式是父进程 fork，子进程准备重定向后 exec 目标程序，父进程等待或继续处理命令。

exec 需要准备参数数组，最后用空指针结束。环境变量也可由不同接口继承或明确提供。名字不同的 exec 接口在路径查找、参数传递方式等方面不同，使用前查实际原型。不要把参数数量、数组终止符和字符串本身的终止符混为一个问题。

多线程进程 fork 后，子进程只保留调用 fork 的那个线程，其他线程持有的锁状态却可能被继承到私有内存。子进程在 exec 之前可调用的安全函数因此受到严格限制。这是系统接口的真实边界，不应把单线程教学示例无条件放进复杂多线程程序。

9.6 wait 回收的是退出后的记录

子进程退出时，内核需要暂存退出状态，让父进程以后查询。已经退出但尚未被回收的子进程常叫僵尸进程，它没有继续执行用户代码，却占有必要的管理记录。父进程通过 wait 或 waitpid 取得状态并回收。父进程先退出还涉及孤儿进程由系统中的指定回收者接管等规则，具体按系统规定处理。

等待的状态不是一个可直接当退出码使用的普通整数。应使用 WIFEXITED、WEXITSTATUS 等宏解释：先确认正常退出，再取退出码；若被信号终止，要用对应宏。`waitpid` 还可能被信号打断，返回 −1 且 errno 为 EINTR，此时通常需要按程序策略重试。

9.7 算例三：让子输出先于父输出

想让子进程打印 C 并结束，父进程等待后打印 P。逻辑顺序是：fork 成功；子执行输出并退出；父对指定子 pid 调用 waitpid；等待确认结束后才输出 P。这里“子结束在父后续操作之前”由 wait 建立，不是靠 sleep 估计时间。

但输出缓冲仍需设计。若父进程在 fork 前有尚未刷出的 stdio 缓冲，父子都会继承那份用户态缓冲副本，可能各自再次刷出。可在 fork 之前明确 fflush 需要的流，或用适合场景的低层 write。在子进程 exec 失败的路径，常用_exit 结束以避免再次执行继承的 stdio 清理；这也需要正确报告错误。

完整真实代码必须检查 fork 与 waitpid 返回值。只在纸面写 `wait(NULL)`，却让失败返回与子进程分支混在一起，可能把“没有孩子”错误解读为“孩子已完成”。错误路径同样属于程序控制流。

9.8 算例四：为什么 hello 会打印两次

设程序先执行 `printf("hello");`，没有换行也没有 fflush；随后 fork 成功，父子都正常 return。在输出流此时确实保存未刷用户缓冲的条件下，fork 复制缓冲，父子退出各刷一次，可能看到两份 hello。

若 hello 在 fork 前已经刷出，则没有这份重复。终端通常行缓冲，重定向到文件时常采用不同缓冲策略，因此“我在终端没复现”不反驳缓冲复制原理。分析应先写出具体缓冲前提，不能把“fork 前打印必定重复”当无条件结论。

文件描述符共享的内核偏移与 stdio 复制的用户缓冲，是同时存在的两层状态。把它们各画一个盒子：一个在进程用户空间，fork 后两份；一个在内核打开文件描述中，可能由父子引用同一个。这张图能解释为什么输出内容和偏移顺序并不只取决于 C 变量。

9.9 非局部跳转与异常的区别

setjmp 保存某种恢复执行所需环境，longjmp 可回到仍有效的保存点，常用于跨越多层函数退出。它不等于普通 return，也不等于操作系统切换进程；使用它还要遵守局部对象、资源释放和信号掩码等规则。尤其不能跳回已经返回的函数活动。

本课程把这些内容放在异常控制流范围，是为了让你识别“下一条执行位置不只来自顺序、if 和 call”。遇到一种转移机制，始终追问保存了哪些状态、恢复了哪些状态、哪些资源需要手动清理。名字里都有“跳”，不代表它们有同样保证。

9.10 把 setjmp 和 longjmp 实际走一次

下面先在 main 仍活着的栈帧保存一个恢复点，再进入 f 与 g。g 不逐层 return，而是 longjmp 直接回到恢复点。setjmp 第一次正常执行返回 0；longjmp(env,7)之后，看起来像同一次 setjmp 现在返回 7，于是 switch 走 case7。

```c
#include <setjmp.h>
#include <stdio.h>
static jmp_buf env;
static void g(void) {
    puts("g: jump");
    longjmp(env, 7);
}
static void f(void) {
    puts("f: before g");
    g();
    puts("f: after g");
}
int main(void) {
    switch (setjmp(env)) {
    case 0:
        puts("main: first return is zero");
        f();
        break;
    case 7:
        puts("main: resumed with seven");
        break;
    default:
        return 1;
    }
    return 0;
}
```

执行输出依次是 main 第一次、f 调用前、g 跳转、main 恢复。f 中的 after g 不会出现，因为控制没有经 g 返回到 f。与异常控制转移类似，这里跨过了中间普通返回路线；但它仍在同一线程、同一进程中，不是创建或恢复另一个进程。

几个限制要跟例子一起记：不能在 main 已经返回后再跳回它的旧恢复点；不能把另一个线程保存的 env 当本线程恢复环境；跳转不会自动替你释放 malloc 对象、解锁或关闭途中取得的资源。setjmp 之后被修改过的某些非 volatile 自动局部变量，在 longjmp 回来后也不能继续假定保留修改后的值；本例没有依赖这类变量。

setjmp 作为宏还对调用所处表达式有规则，本例直接把它作为 switch 控制表达式，避免随手放进任意复杂计算。longjmp 第二参数为 0 时，恢复点得到的返回值会改成 1，以便与首次正常返回 0 区分。理解这些约束以后，才适合把它用于错误恢复，不能拿它当跨函数 goto 随意跳。

9.11 进程树要画父子关系，执行图要画先后关系

父子关系告诉你谁创建谁，不自动规定谁先执行 fork 之后的指令。执行图则记录由程序顺序、wait 等建立的约束。假设父打印 P，子打印 C，父未 wait，则 P 先或 C 先都可能；若父在打印 P 前 wait 子结束，子先完成的约束才成立。

输出由多个字节或多个调用组成时，还要问一次输出是否作为一个整体、底层对象是否允许交错。stdio 一行文本在你的 C 代码中是一条 printf，不等于它一定只对应一条 write。题目若采用“每次打印原子输出一个字符”的模型，可以按该模型枚举；真实系统需要明确缓冲与写入保证。

画图时，把每个进程内的先后用箭头连起来，再加入 wait 边。符合全部边的排列才是允许顺序。只背“fork 后顺序不确定”，既不能枚举答案，也不能看出哪些顺序其实被等待排除了。

9.12 退出状态怎样正确解释

子进程 `return 7` 通常经运行时转换为退出，父进程 waitpid 取得的是编码状态，不是直接整数 7。父先用 WIFEXITED 确认正常退出，再用 WEXITSTATUS 取得退出码。若孩子被信号终止，WIFSIGNALED 和 WTERMSIG 等宏才有相应意义。

假设父直接打印原始 status，看到 1792 而以为系统把 7 乘了 256 弄错。在常见编码里 7 可能位于特定位段，这个值并不意外；但正确程序不应手写移位猜所有平台编码，而使用规定宏。接口提供宏就是为了隔开编码细节。

退出码可表达范围也有限，不能拿它当任意 64 位计算结果传输渠道。若要父子交换大结果，应通过管道、共享内存、文件等合适机制，并定义格式与同步。进程结束记录负责报告状态，不代替通用数据通信。

9.13 fork 失败时为什么不能落入父子成功路径

fork 返回−1，表示没有新孩子。错误代码若只写 `if(pid) parent(); else child();`，会把−1 也当作真，直接进入“父已经有孩子”的逻辑，可能等待错误对象或记录非法 pid。应明确分成 pid<0、pid==0、pid>0 三类。

资源限制、内存压力或系统策略都可能使创建失败。教学算题可以注明“假定全部成功”以专注进程树；工程代码必须检查。写注释说明假设，比把失败路径悄悄删除更诚实，也让读者知道何时需要补上。

练习：父创建两个孩子，第一个成功、第二个失败，是否仍按四个进程分析？答案是否。必须按每个现存进程真正执行到的调用与每次结果画树。简单的2^k只适用于每轮每个进程都执行且成功的特定模型。

9.14 自己做，再看答案

练习一：fork 成功后，父子都执行 x++，初始私有 x 为 0。最终父子各看到多少？

答案：各为 1。它们各自修改自己的私有 x，不是两个线程共同对一个共享计数器加一。若希望跨进程共享同一计数器，必须使用明确共享映射等机制，并安排同步。

练习二：`fork(); if(fork()==0) puts("X");`，假定全成功，X 执行几次？

答案：第一条后两个进程。每个进程都执行第二条 fork，并各产生一个返回 0 的孩子，所以 puts 在两个新孩子中执行，共两次。四个进程的总数与打印次数不同，要看打印所在分支。

练习三：exec 成功后，紧接着写在源码里的 `puts("done")`是否执行？

答案：不会。当前程序映像已被替换。若这一行执行，通常意味着 exec 失败并返回，需要检查 errno，而不能把它当成“新程序执行完成后自动回来”。要等待新程序结束，应由另一个进程使用 wait 类接口。

接口细节可对照 Linux man-pages 的 fork 文档 和 execve 文档。

10 信号与竞态：通知来得太早，也会让程序等不到

子进程完成了，父进程想收到通知再继续。最直觉的代码是“检查标志，没有完成就 pause 等待信号”。这段代码可能永远睡下去：通知恰好在检查与睡眠之间到达，处理函数已经把标志设好，主程序却仍执行 pause，之后再没有新通知把它叫醒。

这里没有哪条加法算错，错在两件事之间留下了一个时间窗口。本章通过逐步安排信号到达时刻，把竞态从“偶尔抽风”变成可复现的执行顺序问题。

信号到达前后控制流的走向：

信号：断点、handler与返回 第 1 / 3 步

1. 正常顺序流

信号：异步事件打断控制流，执行handler后回到断点

main()顺序执行

尚无信号。

程序按指令顺序推进。

为什么：信号是异步的：任意两条指令之间都可能到达。

信号携带的主要是事件类别

信号是操作系统通知进程或线程发生某类事件的机制，例如终端中断、子进程状态变化、定时器到期。每种信号有编号与默认处理方式，可以是终止、忽略、停止等。程序可以对部分信号安装处理函数，也可屏蔽其递送。SIGKILL 和 SIGSTOP 不能被捕获、忽略或屏蔽。

产生、挂起、递送要分开。产生表示事件已发生；挂起表示有待处理的通知；递送表示系统真正采用某种动作。屏蔽通常推迟递送，不等于事件从未发生。解除屏蔽后，挂起信号可能很快被递送。

普通标准信号通常不按次数排成无限队列。同一类信号已经挂起，再来几次可能合并。因此收到一次 SIGCHLD，不代表只退出了一个孩子；处理时往往要循环 waitpid 直到当前没有可回收的孩子。实时信号有不同排队语义，但不能用它反推普通信号也逐个保存。

10.1 处理函数在主程序的什么位置插进来

可以把主程序执行序列想成 A、B、C 三段。信号可能在 A 与 B 之间递送，处理函数执行完再回到原执行流。程序员通常无法事先固定递送点。若主程序正在改一个复杂库结构，处理函数再次调用同一库的非重入操作，可能看到中间状态或产生死锁。

异步信号安全函数是接口规范明确允许在这种上下文使用的函数集合。printf、malloc 等一般不适合直接放进信号处理函数。想输出固定短消息时，可考虑符合要求的 write 并处理场景限制；想做复杂工作，更常见的设计是处理函数只记录一个简单标志，正常执行流稍后处理。

`volatile sig_atomic_t` 常用于单线程主程序与信号处理函数之间的简单标志。sig_atomic_t 提供适合信号场景的简单访问单位，volatile 避免某些不符合这种通信意图的编译器缓存处理。它不是任意线程间同步的万能类型，也不能用来替代多线程原子变量、锁和内存顺序。

10.2 算例一：检查后再 pause 的丢失唤醒

初始 done=0，处理函数收到目标信号后设 done=1。主程序写 `while(!done) pause();`。构造执行顺序：主程序读 done=0，决定进入循环体；信号此时到来，处理函数把 done 设为 1 并返回；主程序接着执行 pause；没有后续信号，程序睡眠。

为什么 while 下一轮不能救它？因为 pause 尚未返回，根本没机会执行下一次条件检查。这里的关键不是“标志变量不够快”，而是检查与进入等待不是一个不可分割动作。

用 sleep 代替 pause 只会让某些运行暂时绕过问题：它会超时醒来重查，但增加延迟，且可能掩盖其他时序错误。真正需要的机制是先屏蔽通知，检查状态，再原子地解除屏蔽并等待。

10.3 用 sigsuspend 关闭时间窗口

掩码是一组“目前推迟递送的信号”，不是完成标志。以 SIGUSR1 为目标，先区分三份集合：oldmask 保存进入这段代码前的集合；当前集合在保护区中额外包含 SIGUSR1；waitmask 从 oldmask 复制但移除 SIGUSR1，专门用于等待时允许目标通知进入。复制集合不会自动改变当前生效的集合，只有相应系统接口才会改变它。

操作	当前是否屏蔽 SIGUSR1	oldmask	waitmask
进入前，先取常见未屏蔽情形	否	尚未保存	尚未建立
屏蔽目标，同时保存旧集合	是	保存原来未屏蔽的集合	尚未建立
从 oldmask 复制并移除目标	仍是	不变	允许目标递送
sigsuspend 采用 waitmask 等待	等待期间否	不变	不变
处理函数结束，sigsuspend 返回	重新为是	不变	不变
最后恢复 oldmask	恢复原来未屏蔽	不变	不变

这几行中，“请求是否已经发生”由 pending 表示，“处理函数是否已经记录请求”由 requested 表示；它们都不等于某份掩码。旧集合原本就含目标时，waitmask 仍将目标移除，本次等待结束后再恢复调用者原本的屏蔽状态。

基本思路是：先保存旧信号掩码并屏蔽目标信号；在屏蔽期间检查 done；若未完成，调用 sigsuspend，用临时掩码原子地替换当前掩码并等待；信号处理完后，sigsuspend 返回，原先屏蔽掩码恢复，再检查 done；最终恢复调用前掩码。

原子地替换掩码并等待，意味着不会在“解除屏蔽”和“真正睡眠”之间再留下可丢通知的缝隙。如果目标信号已挂起，临时解除屏蔽时就能被递送，不会被忘掉。sigsuspend 通常被信号打断后以 −1 和 EINTR 返回，这在此设计中是正常唤醒路径，不应当作意外失败直接退出。

注意这是单线程教学方案。多线程程序中，每个线程有自己的信号掩码，通常使用 pthread_sigmask，并合理安排哪一个线程同步等待或处理信号。不能把进程级信号想成必然在某个指定线程里执行处理函数。

在紧接着的单线程 SIGUSR1 自测中，分别记：`blocked` 是目标信号当前是否被屏蔽，`pending` 是是否有未递送的目标通知，`requested` 是处理函数写的标志。先假定旧掩码原本没有屏蔽目标：

时刻	blocked	pending	requested	程序/系统现在做什么
屏蔽后	1	0	0	安装处理函数，尚无请求
raise 后	1	1	0	信号已经产生，不能递送，所以还没改标志
while 检查	1	1	0	看见 0，准备 sigsuspend

时刻	blocked	pending	requested	程序/系统现在做什么
原子采用 waitmask	0	1	0	解除屏蔽与等待不可插入丢失窗口
处理函数运行	1	0	1	挂起信号递送；系统暂时屏蔽正在处理的 SIGUSR1，处理函数置 1
sigsuspend 返回	1	0	1	恢复进入等待前的屏蔽状态
while 重查，最后恢复 oldmask	0	0	1	退出等待，在正常流做复杂工作

本例的 `sa_flags=0`，没有使用 `SA_NODEFER`。因此进入处理函数时，系统会把当前递送的 SIGUSR1 临时加入屏蔽集合，避免同一种信号立刻重入。处理函数返回时撤销这次临时添加；`sigsuspend` 结束等待、返回调用者时，再恢复调用它之前的掩码。本表中这份调用前掩码仍屏蔽 SIGUSR1，所以返回行的 `blocked` 也是 1，直到最后恢复 `oldmask` 才变 0。

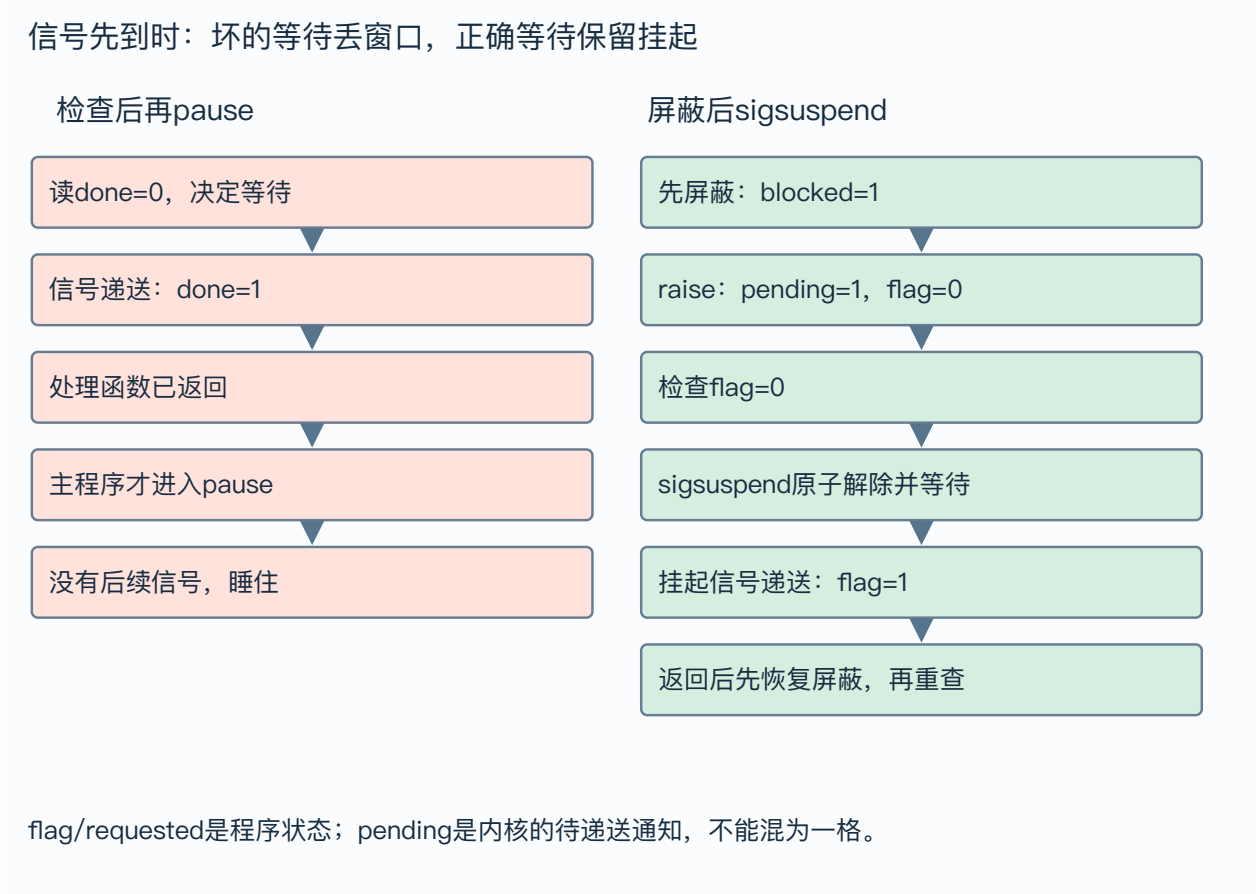


图 28　坏的检查再睡眠，与屏蔽后原子等待

若信号到得晚，程序先在 sigsuspend 中等待，后来递送同样设 1 并返回；若先到，pending 记录它。两种到达时机都能走到重新检查，才是修复的内容。无关信号只会唤醒一次，requested 仍 0 时继续等。旧掩码原本屏蔽目标的情况，等待临时允许它，退出仍恢复原屏蔽，不是把调用者环境清空。

10.4 把原子等待写成一段能核对的程序

下面给出完整单线程自测程序：它先屏蔽 SIGUSR1，再用 raise 给自己产生一个请求，然后执行刚才的等待流程。requested 对应坏例中的 done，只表示处理函数是否记录过请求。将代码存为 signal-demo.c，在支持 POSIX 接口的系统执行 `cc -std=c17 -Wall -Wextra signal-demo.c -o signal-demo` 及 `./signal-demo`，预期只输出一次“收到请求……”后结束。

```c
#define _POSIX_C_SOURCE 200809L
#include <signal.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>

static volatile sig_atomic_t requested = 0;
static void on_request(int signo) {
    (void)signo;
    requested = 1;
}
static void fail(const char *what) {
    perror(what);
    exit(EXIT_FAILURE);
}
int main(void) {
    sigset_t block, oldmask;
    struct sigaction sa = {0};
    sa.sa_handler = on_request;
    if (sigemptyset(&sa.sa_mask) == -1) fail("sigemptyset");
    if (sigemptyset(&block) == -1) fail("sigemptyset");
    if (sigaddset(&block, SIGUSR1) == -1) fail("sigaddset");
    if (sigprocmask(SIG_BLOCK, &block, &oldmask) == -1)
        fail("sigprocmask");
    if (sigaction(SIGUSR1, &sa, NULL) == -1) fail("sigaction");

    /* 自测：此时递送被屏蔽，通知先成为挂起状态。 */
    if (raise(SIGUSR1) != 0) fail("raise");
    sigset_t waitmask = oldmask;
    if (sigdelset(&waitmask, SIGUSR1) == -1) fail("sigdelset");
    while (!requested) {
        if (sigsuspend(&waitmask) == -1 && errno != EINTR)
            fail("sigsuspend");
    }
    if (sigprocmask(SIG_SETMASK, &oldmask, NULL) == -1)
        fail("sigprocmask restore");
    puts("收到请求；复杂工作在正常执行流中进行。");
    return 0;
}
```

顺序值得逐项核对：先屏蔽目标信号，再安装处理函数；raise 使信号产生，但当前仍屏蔽，所以标志尚未被处理函数设 1。waitmask 从旧掩码复制，再明确移除目标信号，保证等待时允许递送它，即使调用前恰好已屏蔽这一信号也能完成本次等待。sigsuspend 原子地采用 waitmask 并等待，挂起请求立即有机会被处理，函数返回后又恢复到进入前的屏蔽集合。

最后恢复 oldmask，而不是把所有信号一律放开。处理函数只写一个简单标志，没有 printf，没有分配内存。输出发生在正常执行流中。代码中的 raise 是为了给出可重复自测；实际应用若等外部请求，应去掉自测产生步骤，但保留完整等待结构。

这里还存在一个应用层选择：初始安装处理之前产生的信号如何处理，本例通过先屏蔽来保护后续安装窗口，但程序刚启动、尚未运行到屏蔽之前的外部信号不由这段代码承诺接收。若需要从进程创建最开始就保证信号策略，应由启动者或完整架构安排。局部代码能证明的范围，要和系统整体启动协议分开。

10.5 算例二：子进程结束得比登记还快

父进程 fork 后，准备把子 pid 放入作业表。子进程很短，可能在父登记前就退出；SIGCHLD 处理函数运行时，尝试从作业表删除这个 pid，却找不到。处理函数返回后，父进程才把一个已退出孩子加入作业表，留下假作业。

修复顺序应是：父在 fork 之前屏蔽 SIGCHLD；fork 成功后父把 pid 登记好；父恢复旧掩码，使挂起通知能递送；子进程在 exec 之前恢复继承来的旧掩码，避免新程序一直继承不该有的屏蔽。

这不要求父进程“跑得比孩子快”。即使孩子先结束，SIGCHLD 暂时挂起，父仍能先完成受保护的登记，然后再处理。同步的目的正是让正确性不依赖谁更快。还要处理 fork 失败，不能把−1 当有效 pid 加入表。

10.6 临界区不一定只有线程才有

一段代码在修改共享结构时，若信号处理函数也可能访问同一结构，就有异步重入问题。主程序修改链表一半时被打断，处理函数看到的指针可能尚未连好。可以在合适范围屏蔽相关信号保护主程序更新，但处理函数自身仍必须遵守异步信号安全限制。

把屏蔽范围做得过大也有代价：通知迟迟不能处理，响应变差。应只围住必须保持一致的更新操作，而不是从程序启动到结束都屏蔽。离开临界区时恢复原掩码，比无条件“全部解除屏蔽”更稳妥，因为调用者可能本来就屏蔽了一些信号。

保存 errno 同样重要。处理函数执行某些接口可能改变 errno，主程序恢复后若要解释原来的错误，会受到干扰。需要使用 errno 的处理函数常在入口保存旧值、出口恢复；这并不能让任意函数变得异步安全，只是额外保护错误状态。

10.7 算例三：一次 SIGCHLD 为什么要收两个孩子

假设两个孩子 C1、C2 几乎同时退出，而 SIGCHLD 在一段时间里处于屏蔽状态。解除屏蔽后只收到一次标准 SIGCHLD 通知。如果处理函数仅 waitpid 一次，可能只回收 C1，C2 仍留下退出记录，而未必再有新的独立通知来提醒。

因此典型逻辑是循环调用 `waitpid(-1,&status,WNOHANG)`：返回正 pid 则处理该孩子；返回 0 表示仍有孩子但当前没有新的可回收状态，停止；返回−1 要区分 EINTR、ECHILD 等情况。−1 与 0 不是同一含义，不能用一个笼统“没有了”忽略真正错误。

循环的作用不是猜信号来了几次，而是查询内核当前真实状态，直到清空已完成工作。信号只负责提醒“可能有事”，状态接口负责告诉你“现在有哪些事”。这套思路也适用于某些事件通知系统：唤醒次数和待处理任务数常不是一一对应。

10.8 竞态怎样用时间线证明

假设主程序更新 x，处理函数也更新 x。不要只说“它们同时执行会出错”，而应写出一个交错：主读旧值 0；信号到来；处理函数读 0、写 1；回主程序；主用自己早先读的 0 加 1、写 1；最终只加了一次。即使所有操作在单核上按顺序轮流发生，也能出现这个丢失更新。

对信号中的简单标志，设计“只设为 1，不做计数”常更容易正确。若事件次数本身重要，普通标准信号合并就已经使计数信息丢失，应选择具备明确队列或计数语义的机制，而不是用 volatile 修修补补。

10.9 算例四：kill(pid,0)到底做了什么

kill 名字看起来像“结束进程”，但接口实际是发送指定信号。信号为 0 时，不发送实际信号，主要执行存在性和权限检查。返回成功不能保证下一行那个进程仍存在，因为它可能马上退出；某个 pid 以后还可能被复用。

因此“先检查进程在不在，再执行操作”仍有检查与使用之间的时间窗口。系统编程中这种结构叫 TOCTOU，即检查时和使用时状态可能已不同。是否有更稳定的句柄或原子接口，需要看具体系统。理解这种窗口，比记住一个偶尔可行的检查命令更有用。

10.10 信号与硬件故障别混在一个层次

SIGSEGV 可能由非法内存访问引起，但不是所有收到 SIGSEGV 的场景都意味着某一条普通访存触发了同一种硬件故障；程序也可能显式发送某个信号。反过来，可修复缺页可能完全由内核处理，对用户程序透明，不一定递送信号。

处理严重程序错误的信号后继续执行，通常非常复杂，不能把“安装一个处理函数忽略它”当成修复内存越界。应找到非法访问及其根因。信号机制帮助通知和控制，不会自动把损坏的对象重新变得有效。

10.11 为什么处理函数只设标志，经常比直接干活好

假设 SIGINT 到来时，你想保存一份复杂统计报告。如果在处理函数里遍历链表、分配内存、格式化输出，信号可能恰好打断主程序对同一结构的更新，或打断库内部持锁操作。处理函数再进入同一路径，可能看到半成品或等待自己无法释放的锁。

更简单的架构是处理函数只做允许的简短动作，例如把请求标志设 1；正常主循环在安全点检查标志，取得必要锁后生成报告。这样复杂工作回到可控制上下文，也更容易测试。标志只是“有人请求过”，不表达请求次数；若每次请求都必须保存，应另设队列或计数机制。

主循环可能阻塞于 I/O，因而还需设计怎样被唤醒，例如采用合适的文件描述符通知或同步信号等待结构。只加标志却从不让主循环有机会检查，也不能完成响应。设计时沿“产生请求至通知至检查至处理至清除”画完整路径。

10.12 屏蔽与忽略为什么不同

屏蔽表示暂缓递送，解除后挂起通知可能继续处理；忽略表示对该类信号采用忽略动作。若本意是保护一个很短的状态更新，却把信号处置长期改为忽略，可能直接丢掉需要处理的信息。应使用掩码控制临界区，而不是临时胡乱安装忽略处理再猜能否恢复。

恢复旧掩码也比“清空全部掩码”重要。设调用函数前，外层已屏蔽 SIGUSR1；内部为保护作业表又屏蔽 SIGCHLD。退出内部临界区时，应该只恢复原状态，仍保留外层 SIGUSR1 屏蔽。无条件解除所有信号会破坏调用者的同步假设。

这是一种普遍的资源管理原则：改变调用环境前保存原值，结束后恢复你所接手的状态。类似地，修改文件描述符标志、终端模式或线程取消状态时，也应避免把“我觉得默认应该这样”覆盖调用者已有设置。

10.13 一个容易遗漏的重检查：唤醒不等于目标发生

sigsuspend 可能被另一个无关信号唤醒，条件变量也可能虚假唤醒。正确结构都围绕谓词循环：只要目标状态不成立，就继续等待。通知只是促使重新检查状态，不应直接替代状态本身。

例如同时装了 SIGCHLD 和 SIGUSR1 处理函数，父在等孩子，SIGUSR1 先到。sigsuspend 返回后，如果不看 done 而直接认定孩子已完成，就会继续使用未准备好的结果。while 重新检查能保持目标语义。你可以把这个原则带到事件循环，但每种等待接口仍有自己的原子性与清理规则。

练习：把 done 在等待前清零是否总安全？答案是否。若目标事件已发生、处理函数已设 1，主程序随后无条件清零就抹掉真实状态。标志何时重置应属于完整协议，不能为“让循环再等一次”随意清空。

10.14 自己做，再看答案

练习一：目标信号已被屏蔽，连续产生三次普通标准信号，解除屏蔽后一定调用处理函数三次吗？

答案：不一定。标准信号同类挂起通知通常可合并，不能靠调用次数精确还原事件次数。若事件是子进程退出，应循环查询可回收状态；若需要通用计数，使用明确支持计数或排队的通信方式。

练习二：主程序屏蔽 SIGCHLD 后 fork，父完成登记再解除。子直接 exec 而没有恢复旧掩码，可能有什么问题？

答案：子继承了屏蔽状态，exec 通常不自动清除该信号掩码，新程序可能继承本不期望的屏蔽。父保护自身作业表所用的临界区措施，不应无意改变新程序长期的信号策略。

练习三：为什么“在处理函数中 printf 调试一下”可能使原问题更难复现？

答案：printf 不是一般可依赖的异步信号安全操作，可能碰到主程序正在使用的内部缓冲或锁；额外 I/O 也会改变时序。它既可能引入新错误，又可能暂时掩盖旧竞态。调试输出应选择适合信号上下文的方式，并把复杂诊断移回正常执行流。

接口语义可核对 sigprocmask、sigsuspend 与 signal-safety。

11 虚拟内存：同一个地址，为什么能装着两份不同数据

两个进程都打印某个变量地址为 `0x400000`，内容却一个为 7、一个为 99，并不矛盾。程序通常看到的是虚拟地址。每个进程有自己的映射规则，同一个虚拟地址可以翻译到不同物理位置。虚拟内存把“程序如何给位置编号”和“数据实际放在物理内存哪里”分开，也为隔离、共享和按需装入提供基础。

学习地址翻译时，最容易混的是四组词：虚拟页号、页内偏移、物理页号、物理地址。本章会用小得能手算的地址模型，反复把它们写出来。真实 x86-64 系统的页表层级更复杂，课程引言明确多级页表不作默认要求；先把单级模型讲透，不拿层级名字堆满纸。

11.1 先把整段地址切成页号和页内位置

先按 256 字节一段给位置编号。第 0 页覆盖地址 0 至 255，第 1 页覆盖 256 至 511，第 2 页覆盖 512 至 767。页号回答“第几段”，页内偏移回答“从这一段起点再走几个字节”。例如地址 300=1×256+44，所以在第 1 页、页内偏移 44。页号 1 本身不是该页起点 256。

沿用本章的十六进制地址 0x12AB，它等于 4779。整除 256 得到 18，余数 171，即

$$4779 = 18 \times 256 + 171.$$

18 写成十六进制为 0x12，171 为 0xAB，因此它在虚拟页 0x12 的偏移 0xAB 处。256 为2^8，每跨一页恰好跨过 8 个二进制位的范围，所以同一件事也可以用位串表达：低 8 位是余数，高位是商。先有地址区间的分段，才有高低位的切分。

映射按整页建立。若虚拟页 0x12 对应物理页框 0x3A，虚拟起点 0x1200 就对应物理起点 0x3A00；本页第 171 个字节仍对应目标页内第 171 个字节。因而改变的是页号所选的页框，偏移保持：

$$\textbf{物理地址} = \textbf{物理页号} \times \textbf{页大小} + \textbf{原偏移}\ .$$

物理页框也叫页帧，是物理内存里容纳一页的位置。页表为虚拟页记录所对应的页框及访问属性，例如是否驻留、是否允许当前读写。实际字段依系统而异；后面做题会明确给出哪些状态，不能只凭有页号就断言访问被允许。

完整翻译的逐步演示：

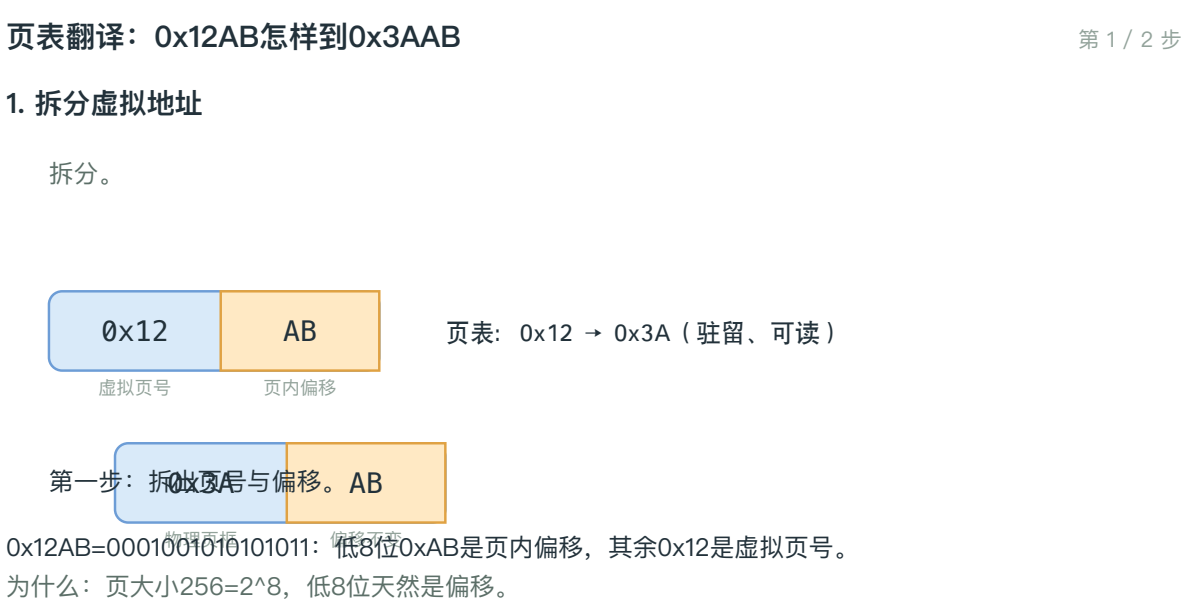


图 29　逐步演示：页表翻译：0x12AB 怎样到 0x3AAB

11.2 算例一：第一次完整翻译

题目：虚拟地址 16 位，页大小 256 字节。页表中虚拟页 `0x12` 映射到物理页 `0x3A`，且允许当前读操作。读虚拟地址 `0x12AB`，物理地址多少？

第一步，256=2^8，偏移 8 位。第二步，虚拟页号为高 8 位 `0x12`，偏移为低 8 位 `0xAB`。第三步，查该进程的页表得到物理页号 `0x3A`。第四步，拼接物理页起点 `0x3A00` 与偏移，得到 `0x3AAB`。

核对：`0x12AB` 和 `0x3AAB` 低 8 位完全相同。若算出 `0x3A12`，说明把页号和偏移位置混了；若算出 `0x3A+0xAB`，说明忘记页号需要先乘页大小。

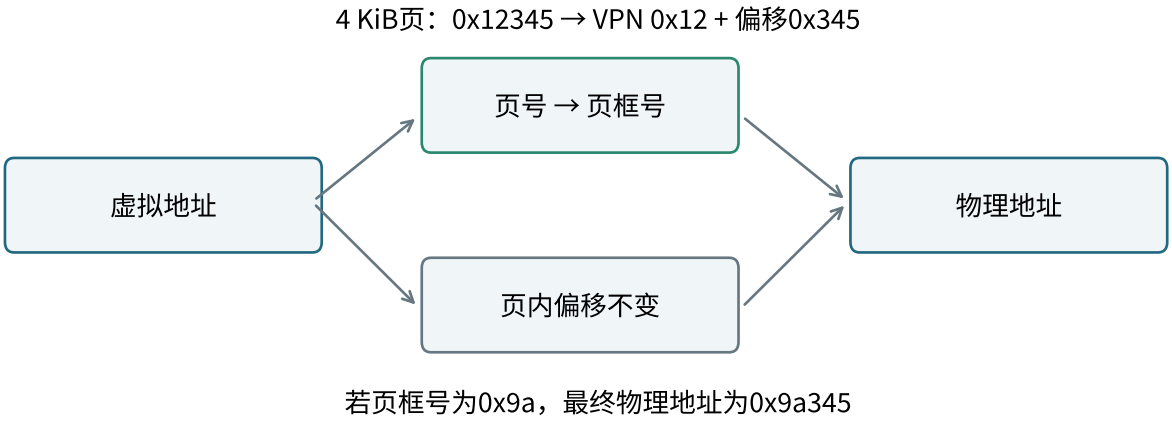


图 30　原创示意：虚拟地址按页号查表，再保留页内偏移。

这张图另用 4KiB 页，不是上一题的 256 字节页。图中 0x12345 拆为 VPN0x12 与偏移 0x345，页框号取 0x9A，得到物理地址 0x9A345；低 12 位保持不变。更换页大小时，必须重新确定偏移位数，不能沿用上一题的低 8 位。

读图时先沿虚拟地址的高位找到页表项，再沿映射箭头到物理页号；低位偏移另有一条不变通路。箭头表示查找关系，不表示 CPU 把整个页复制一次。普通每次访存做的是地址解释；只有缺页处理、写时复制等情况才可能另外搬运页面内容。

11.3 页表有效与权限允许是两次不同检查

某页表项表明页面不驻留，操作系统可能仍知道它来自哪份文件或交换区域，能够装入后继续执行。另一个地址可能根本不属于进程合法映射，无法通过正常缺页处理修复。两者都可能在硬件翻译时遇到异常，但后续处理不同。

权限检查也独立存在。页面已驻留，但只读，写入仍会触发保护相关异常。页面可读，不等于可执行；代码页可执行，也不应当然可写。用户态可访问权限与内核态访问权限也要区别。判断访问是否成功，不能只看“有没有物理页号”。

课本教学中有效位常简写“该页是否缓存到 DRAM”，真实操作系统的页表状态与内核辅助结构更复杂。做题按给定字段定义；解释实际系统时，不要把一个教学有效位当成唯一的合法性证明。

11.4 TLB 是翻译结果缓存，不是数据缓存

地址翻译后还要真正读取数据。如果每次都先访问内存中的页表，额外开销很大。TLB 缓存最近使用的部分虚拟页到物理页翻译。查到叫 TLB 命中，查不到叫 TLB 未命中。

TLB 未命中不等于缺页。TLB 里没有记录，页表里仍可能有一个已驻留且合法的映射，硬件或系统查页表并把结果补进 TLB 即可。缺页则是页表翻译或权限检查需要异常处理的另一种情况。数据缓存未命中又是第三层问题：地址翻译正确，但所需数据不在某级缓存，要去更低层取。

可以按门牌本、门牌查找缓存、屋内物品三个对象来分别想象，不过最后仍要回到实际名词：TLB 存翻译，页表存映射和属性，数据缓存存数据块。比喻只能帮助分开概念，不能代替字段与地址计算。

11.5 算例二：TLB 索引与标记怎样拆

先把 TLB 画成编号 0、1、2、3 的四个槽位，每槽只能保存一条翻译。每条需要记录有效位、虚拟页的标记和物理页号：槽号只负责定位，标记用来确认这条记录是不是要找的页。多个页会被分到同槽，因此定位之后还要比较标记。

题目：虚拟地址 16 位，页大小 256 字节，TLB 有 4 组，每组 1 项，按虚拟页号低 2 位索引。求地址 `0x2D83` 的页内偏移、TLB 组号和标记。

地址低 8 位为 `0x83`。虚拟页号 `0x2D` 的二进制为 `00101101`。低 2 位 `01` 给组号 1；余下高 6 位 `001011` 给标记 11。查 TLB 第 1 组时，必须比较保存的标记是否等于 11 并检查有效，不能仅凭索引相同就命中。

虚拟页 `0x29` 二进制 `00101001` 也落在第 1 组，但标记为 10。它可能替换掉 `0x2D`，这叫冲突。TLB 按页缓存，访问同一页里的不同字节不会产生不同 VPN 条目；访问很大数组时的翻译局部性与跨页方式有关。

实际处理器还可能带地址空间标识，使不同进程的同一虚拟页号可区分；上下文切换如何管理 TLB 依赖硬件与操作系统。教学模型若没给这些字段，就先按单地址空间计算，不必自行编造额外行为。

对第 1 组 TLB 给一个具体访问序列，假定页表里 VPN0x2D、0x29 都已驻留且可读，映到页框 0x19、0x20。每组一项、初始无效：

访问虚拟地址	VPN/偏移	组/标记	比较前第 1 组	结论与更新
0x2D83	2D / 83	1 / 0B	无效	未命中，查页表，存 tag0B⟶PPN19
0x2D84	2D / 84	1 / 0B	tag0B⟶19	命中；物理地址 0x1984
0x2983	29 / 83	1 / 0A	tag0B⟶19	不匹配，查页表，替换成 tag0A⟶20
0x2D83	2D / 83	1 / 0B	tag0A⟶20	再次未命中，补回 tag0B⟶19

四次都不必从磁盘装页。前后两个地址只差页内 1 字节时，翻译条目不变；改变到 VPN29 时组相同、标记不同，才发生替换。数据是否命中 L1 还需另查数据缓存，这张表没有偷偷假设数据一定已在 L1。

11.6 按需分页的一次时间线

程序第一次读某个合法但未驻留的页面。处理器翻译发现当前页表无法直接完成访问，触发异常，保存可恢复状态并进入内核。内核检查地址确属合法映射，找到数据来源，分配物理页框，必要时先选择其他页面回收；把文件或零初始化内容准备好，更新映射，再返回重试原来的访存指令。

程序看到的效果可能只是这次访问比平时慢很多。若它是首次访问零填充匿名页，系统不一定要从磁盘读一页；若文件内容已在页缓存，也未必产生实际磁盘 I/O。不能把所有缺页统一描述为“从硬盘读回来”。

若内存压力很大，页面反复被换入换出，程序花大量时间处理缺页而非做有效工作，称为抖动现象的一种。工作集是某段时间里经常访问的页面集合。程序的空间局部性、访问顺序与系统可用物理内存一起决定是否容易陷入这种状态。

11.7 算例三：缺页率很低，为什么仍可能很慢

题目：普通一次访问按简化模型耗时 100 纳秒，某类需要磁盘的缺页处理耗时 8 毫秒，缺页概率 $p=10^{-5}$。忽略其他层次，估计平均访问时间。

先统一单位，8 毫秒为 8,000,000 纳秒。平均时间约为$(1-p)\times 100 + p\times 8,000,000$纳秒，得到约 180 纳秒。缺页概率只有十万分之一，仍接近多花 80% 的时间，因为两种路径耗时差约八万倍。

这里是帮助理解数量级的玩具模型，不是当代每台机器的性能保证。缺页来源、存储设备、调度和 I/O 并发都会改变实际耗时。算式真正教的是：低概率事件若代价极大，平均性能仍可能被显著拖慢。估算时既看发生率，也看慢路径成本。

11.8 写时复制怎样支持 fork

fork 后父子私有页面可先映射到同一个物理页，并被标成受写保护的写时复制状态。双方只读时无须立即复制内容；某一方第一次写入，触发保护异常，内核确认这是允许通过复制解决的情况，分配新页、复制旧内容、更新该方映射并恢复写权限，再重试写入。

父子随后看到的修改各自独立，符合私有地址空间语义。不是所有只读保护异常都可用复制修复；必须由内核知道该页是写时复制候选。如果程序试图改真正的只读常量映射，不能凭“复制一份就好了”的想法认定系统应允许。

11.9 算例四：同虚拟地址的两份内容

父子页表起初都把 VPN5 指向物理页框 9，某字节内容 7。父写该字节为 20，触发写时复制后，父 VPN5 改指向页框 12，页框 12 对应字节为 20；子仍指向页框 9，对应字节保持 7。

两边打印的虚拟地址仍可能完全相同，因为 VPN 与页内偏移没变，变化在父页表的映射。若某页从一开始就是明确的共享映射，写入语义又不同；是否共享由映射类型决定，不由“现在碰巧指向同一物理页”单独决定。

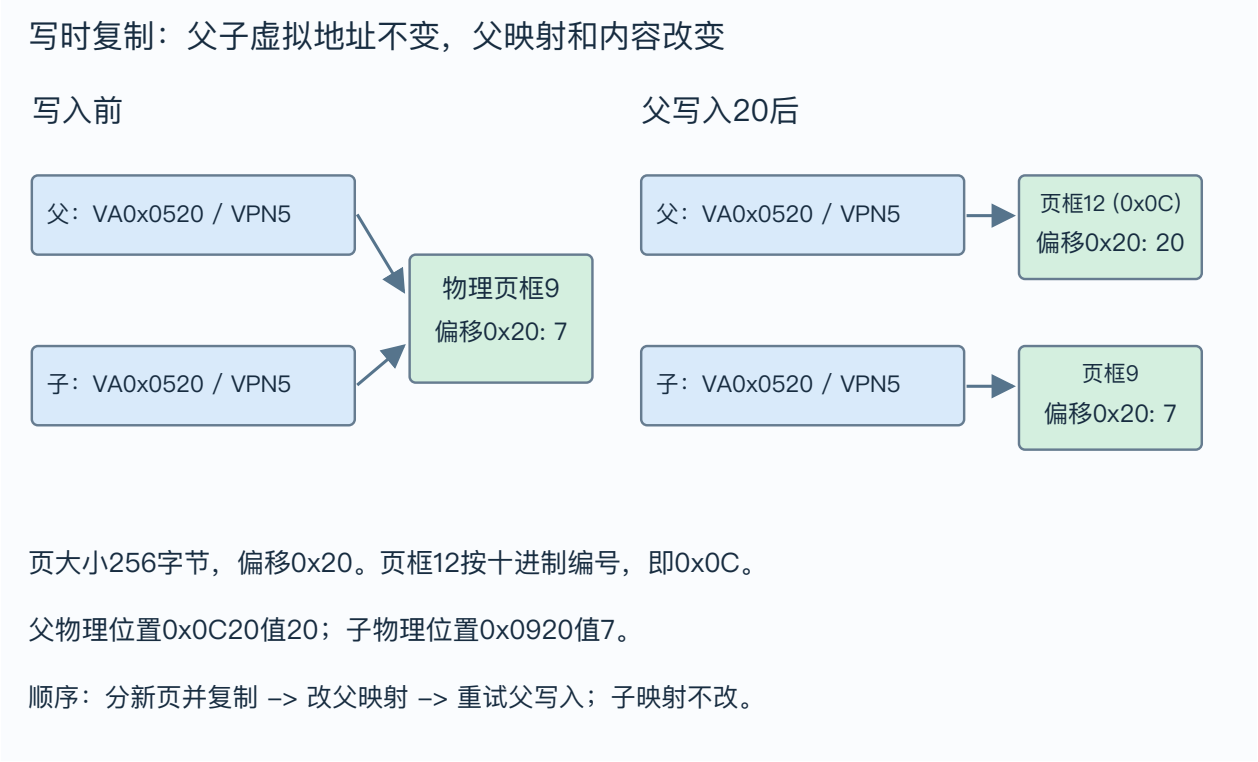


图 31　写时复制前后：相同虚拟页，父子物理映射分开

图中选定页内偏移 `0x20`，父子开始都读到 7。父写 20 时，并非直接把共享的旧页 9 改成 20：先得到新页 12 并复制，父的页表才改指向 12，最后重试写入。子仍沿旧映射到页 9。画图时若只改内容、不改父页表，就会错误地预测子也看到 20；若只改页表、不重试写，父新页仍只含复制来的 7。

11.10 mmap 与普通读文件的关系

内存映射可把文件的一段内容关联到虚拟地址范围，通过普通访存使用。它不保证映射一建立就把整个文件读进物理内存，常可按需准备页面。私有映射的修改通常不回写到原文件；共享映射提供不同的可见性和持久化行为，还要考虑同步接口与错误处理。

读文件 read 则把内容显式复制进调用者缓冲，返回实际读取字节数。两者是不同接口，各有适用场景，不应简单说 mmap 永远更快。映射建立、缺页、地址空间占用、随机访问方式和故障处理都会影响选择。课程先要求你画清映射关系，再讨论性能。

11.11 共享物理页不等于共享所有写入

两个虚拟地址可以映到同一物理页，不同进程也可以如此。但写入可见性还受映射类型、权限、写时复制与同步规则影响。共享代码页通常只读，多进程安全复用；私有数据页可在写时复制之前暂用同一页，写后分离；显式共享映射则允许按规定共享修改。

因此判断“是否共享”，应先问接口语义，再看当前映射。观察某一时刻物理页号相同，只说明当前实现复用了存储；它不必允许一方修改影响另一方。反过来，两个不同虚拟地址可以指同一共享内容，地址数字不同也不证明独立。

这层区分在 fork、共享库和 mmap 三处都会出现。把它们用一张统一表总结：虚拟地址属于谁，页表映到哪里，权限是什么，写入时是否复制，是否需同步。每列都回答一个具体问题，避免用一句“共享内存”包住所有差异。

11.12 地址空间很大，不等于驻留内存同样大

进程预留或映射一大片虚拟范围，未必立即占有相同数量物理页。按需分页可在首次访问时才准备内容。程序的虚拟大小、驻留集大小和实际物理内存压力，因此可能差很多。它们分别衡量映射规模、当前驻留页和系统资源状态。

若系统允许某些过量承诺策略，分配接口成功也不总能理解为“以后每一页都已稳稳放在物理内存里”。实际内存不足时如何处理依系统策略。应用仍应合理限制规模、处理可报告失败，并避免用一个进程内的成功返回推断整台机器无限可用。

本课程基础模型可以假设合法页面最终能被准备好，以练地址翻译；讨论真实程序稳定性时，就要把资源限制带回来。教学简化应说出边界，而不是藏起来让读者误以为所有系统都这样。

11.13 跨页读取怎样拆成两次检查

假设页大小 256 字节，读取一个 8 字节数，起点虚拟地址 0x12FC。它覆盖 0x12FC 至 0x1303，前 4 字节在页 0x12，后 4 字节在页 0x13。两页可映到不相邻物理页，必须分别满足映射与权限。

如果只查起始页有效，就断言整个 8 字节访问安全，会漏掉第二页。硬件如何完成跨页指令、异常时状态如何保持，要按架构规则；应用层最先应保证整个对象范围均合法。对文件映射末尾、缓冲区边界附近的宽加载尤其要留意。

练习：0x12F8 起读 8 字节是否跨页？答案是不跨，范围到 0x12FF；起点 0x12F9 则到 0x1300，跨到下一页。判断用“起点偏移+长度是否超过页大小”，记住长度包含起始字节，避免多加或少加 1。

11.14 自己做，再看答案

练习一：页大小 4KiB，虚拟地址 `0x12345`，页内偏移与 VPN 各是多少？若映射到物理页 `0xAB`，物理地址是多少？

答案：4KiB=2^{12}，低 12 位偏移 `0x345`，VPN=`0x12`。物理页起点 `0xAB000`，拼接得 `0xAB345`。低三位十六进制数字正好对应 12 位，所以此题可直接拆，但仍要先确认页大小。

练习二：TLB 未命中、页表有效且有读权限，是否一定要访问磁盘？

答案：不需要。只需按模型查页表取得翻译并填充 TLB，随后读数据。数据本身还可能命中缓存。TLB 未命中不是缺页；即使缺页，也未必需要磁盘。

练习三：只读代码页已在物理内存，用户程序写它失败。能否说“页已经有效，所以操作系统错了”？

答案：不能。驻留状态与访问权限是不同条件。页存在只说明能定位内容，写权限仍可禁止。应检查访存类型、页属性和程序为何尝试写代码，而不是只看物理页号。

12 动态分配：malloc 给了你一块内存，还给了你哪些责任

数组大小要等读入文件后才知道，函数结束后对象仍要存在，这时可以使用动态分配。malloc 按请求字节数提供一块可用存储；free 结束这次分配对象的使用。难点往往不在函数名，而在“谁拥有这块对象、长度是多少、哪些地方仍握着地址、何时最后一次使用”。
本章先从使用者角度写安全代码，再拆开分配器内部的块、空闲表、分割和合并。理解内部实现可以解释性能与碎片，但不能把某个教学分配器的布局当作所有 malloc 的固定规格。
malloc 切割与 free 合并的块级变化：

malloc/free：切割与合并 第 1 / 3 步

1. 堆的当前布局

隐式空闲链表：头部记录块大小与分配位；分配即切割、释放即合并



查找第一个足够大的空闲块。

已有三块分配、两块空闲（各4、8个double字）。
为什么：malloc(32字节)需要4字块：首个放得下的是末尾8字空闲块。

请求字节数，返回的不是初始化数组

`malloc(n)`请求 n 字节，不是 n 个 int。若要 n 个 int，常写 `int *p=malloc(n*sizeof *p);`。`sizeof *p` 由指针所指类型决定，避免类型改了而常量大小没改。使用前应检查乘法溢出与分配失败。
malloc 成功得到的内容未初始化，不能读来当已有合法值。你需要给每个将要读取的元素赋值，或者按需求使用其他初始化方式。返回地址满足接口规定的对齐保证，但超大对齐需求需要专用接口，不能认为任何自定义对齐都自动满足。
free 要求传入空指针，或仍有效、且来自兼容分配接口的原始分配起点。传入 p+1、栈数组地址或已经释放的指针都不合法。`free(NULL)`是允许的无操作。释放以后旧地址可能仍在变量里，但对象已失效；字节暂时没变，不会延长生命周期。

12.1 算例一：分配并初始化 n 个元素

设 n 来自可靠解析后的 size_t。先检查 `n>SIZE_MAX/sizeof *p` 则拒绝；再请求 n×sizeof*p 字节；对非零请求检查返回是否为空；最后在 `0<=i<n` 范围逐个初始化。用完调用 free，并在本作用域把 p 设为 NULL 可以减少误用，但不能自动清除别处保存的别名。
例如 n=3、sizeof*p=4，需要 12 字节。p 指向起点 B，p[0]、p[1]、p[2]起点分别 B、B+4、B+8。最后一个元素占 B+8 至 B+11。p+3 可作为尾后指针，不能读写；`free(p+1)`也不成立，因为它不是分配起点。
n=0 要单独说明接口策略。malloc(0)在 C17 下有实现规定行为，可能返回空指针或一个不可用于访问的独特指针，后者仍可 free。应用可以明确把零长度当空数组，避免写出依赖某一种返回形式的代码。

12.2 从头文件到 free，完整运行一次

先在同一个函数里完成整段生命期，不引入输出参数。n=3，每个 int 按本书平台占 4 字节，先请求 12 字节；分配成功以后才有可写的 p[0]到 p[2]，依次填 0、1、2，读取输出，再释放。这份程序把 n=0 规定为空数组，不请求存储。

```c
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    size_t n = 3;
    int *p = NULL;
    if (n > SIZE_MAX / sizeof *p) {
        fputs("request too large\n", stderr);
        return EXIT_FAILURE;
    }
    if (n != 0) {
        p = malloc(n * sizeof *p);
        if (p == NULL) {
            fputs("allocation failed\n", stderr);
            return EXIT_FAILURE;
        }
    }
    for (size_t i = 0; i < n; ++i)
        p[i] = (int)(i % 10);
    int status = EXIT_SUCCESS;
    for (size_t i = 0; i < n; ++i) {
        if (printf("%d\n", p[i]) < 0) {
            status = EXIT_FAILURE;
            break;
        }
    }
    free(p);
    if (fflush(stdout) == EOF) status = EXIT_FAILURE;
    return status;
}
```

将代码存为 alloc-demo.c，可用 `cc -std=c17 -Wall -Wextra alloc-demo.c -o alloc-demo` 编译。n 为 3 时输出三行 0、1、2。假设成功地址为 B，初始化后的三个对象从 B、B+4、B+8 开始；free(p)结束的是这次分配，p 变量不会因此自动变为 NULL。

把 n 改成 0，两个循环均零次，p 一直为 NULL，free(NULL)合法。把 n 改成 SIZE_MAX，本环境的 sizeof *p* 为 *4*，会在乘法前拒绝；不能先算 *n*4 再检查已经绕回的结果。sizeof *p 这里只询问类型大小，不实际沿 NULL 访问 int。

12.3 所有权：谁最后负责 free

所有权不是普通 C 类型系统自动替你保存的信息，而是程序设计约定。函数返回动态分配对象时，应说明调用者负责释放；函数只借用外部缓冲时，不应偷偷释放它。两个模块都认为“对方会释放”会泄漏，两个模块都认为“我应该释放”会重复释放。

可以在每个指针旁写“拥有”或“借用”。拥有指针负责最后释放，借用指针只在规定时间内使用，不延长对象生命。若数据结构中有多个别名，需要更具体的所有权策略，例如唯一拥有者、引用计数或外部生命周期管理。把每次 malloc 与某次 free 按文本数量配对不够，还要确认所有执行路径和所有别名关系。

把 p 设 NULL 只影响 p 这个变量。若 q 早已等于 p，free(p)后 q 仍是悬空指针。许多释放后使用错误正来自“我已经清空 p 了，所以安全”的误判。

12.4 让函数把新地址交回调用者

现在增加一个需求：分配工作交给 make_array，调用者 main 还要在函数返回后使用对象。若只把 main 中 p 的当前值传进去，函数修改自己的指针副本，main 的 p 不会自动改变。需要把“main 的 p 这个变量的地址”传给函数，让函数能写回它。

设 main 的 p 变量位于 0x5000，初值 NULL。调用 make_array(3,&p)时，参数 out 保存 0x5000。函数内另有局部指针 p，假设 malloc 成功后它保存 0x6000：

表达式	在这份状态里访问什么	含义
out	函数参数里保存的 0x5000	main 的指针变量的位置
*out	0x5000 处的指针对象	main 将要使用的地址
函数局部 p	函数自己的指针对象	malloc 给出的 0x6000
**out，成功写回之后	0x6000 处的第一个 int	数组第 0 项

*out=p 把 0x6000 写进 main 的指针变量，没有复制整个数组。函数返回后，函数局部 p 结束生命期，但分配对象仍在，main 保存的新地址仍可使用。int **中的两层星号分别对应指针对象和最终 int；sizeof **out 为单个 int 大小。

下面是可与前面头文件配合的函数定义，另需包含<stdbool.h>。out 必须指向一个仍存活、可写的 int*变量，且该槽位没有尚未释放的旧拥有对象；本例调用者的 p 尚未获得分配。函数不会替你释放被覆盖的旧地址。返回 false 代表请求过大或分配失败，返回 true 后由调用者最终 free：

```
static bool make_array(size_t n, int **out) {
    *out = NULL;
    if (n == 0) return true;
    if (n > SIZE_MAX / sizeof **out) return false;
    int *p = malloc(n * sizeof *p);
    if (p == NULL) return false;
    for (size_t i = 0; i < n; ++i)
        p[i] = (int)(i % 10);
    *out = p;
    return true;
}
```

调用者可写 int *p; if (!make_array(3,&p)) { /* 报错并退出 */ }，成功后按前一程序使用并 free(p)。函数开始把*out 置 NULL，因此失败不会留下一个假装成功的旧地址；零长度成功得到 NULL。给每个对象分开标生命期，比看哪个变量也叫 p 更可靠。

12.5 算例二：realloc 为什么不能直接覆盖原指针

错误模式是 p=realloc(p,new_size);。如果失败返回 NULL，原分配仍有效，但原地址被覆盖，程序失去了释放和继续使用它的入口。应先用临时指针接结果，成功后再更新 p。

例如原 p 指 B、容量 100 字节，请求 200 失败。临时变量 tmp=NULL，而 p 仍指 B，可选择保留旧对象、报告错误或释放。成功时，返回地址可能仍是 B，也可能是新位置 C；旧对象的前 min(旧大小,新大小)字节内容被保留，扩展部分未初始化。成功后旧指针及旧对象内部的地址都不能再当作原对象访问依据，即使数值碰巧相同也要遵循新对象关系。

如果你保存了 int *q=&p[2]，realloc 移动后 q 不会自动跟着更新。可保存元素下标 2，成功后重新计算 q=p+2。请求大小为 0 又有额外实现与版本规则，本书建议应用显式用 free 处理清空，避免把零大小 realloc 的历史差异藏进普通扩容路径。

12.6 分配器看到的是一串块

先看一个具体布局模型：块按 8 字节对齐，头部 8 字节；malloc 请求 13 字节载荷。若这个块从 0x1000 开始，先有 8 字节头部，再有 13 字节用户空间，共 21 字节，向上补到 24：

地址范围，含两端	用途	字节数
0x1000—0x1007	头部，记录本块大小和状态	8
0x1008—0x1014	本次请求的载荷	13
0x1015—0x1017	为块对齐留下的填充	3
0x1018	下一个块可能开始的位置	不属于本块

用户得到的 p 为 0x1008，不是块起点 0x1000。它可以按请求使用 13 字节，不能把填充当作额外申请到的空间。块总占 24 字节，13 字节请求之外的 11 字节由头部 8 和填充 3 组成。

教学分配器常把堆区域分成若干块。每块有头部记录块大小与分配状态，部分设计还有脚部。已分配块中包含用户载荷与可能的对齐填充；空闲块可存链接字段。用户拿到的 p 通常指载荷起点，不是整个块头。

内部碎片是已分配块内部无法交给用户有效使用的部分，例如对齐、最小块限制和管理开销。外部碎片是空闲总量不少，却分散成许多小块，无法满足一块大的连续请求。两者成因不同，不能只看“还有多少空闲字节”就判断能否成功分配。

这是用于手算的分配器，实际 malloc 的头部、对齐、最小块和开销可能完全不同。

12.7 算例三：首次适配、分割与剩余块

空闲表按地址有块 A=32、B=80、C=48 字节。请求最终块大小 40 字节。首次适配从头找，A 不够，B 够用，选择 B。若最小合法空闲块为 24 字节，可把 B 分成 40 字节已分配块与 40 字节空闲块。

若请求最终大小 64，B 剩 16 字节，小于最小合法空闲块，不能留下一个连头部与链接都装不下的小碎片；常见策略把整个 80 字节给这次请求，内部多出 16。最佳适配会寻找足够大的最小空闲块，此题 40 请求时可能选 48，是否可分割仍受最小块规则限制。

首次适配不保证碎片最少，最佳适配也不保证整体速度或未来碎片最优。请求序列未知，局部最合适不等于全局最合适。分离空闲表、按大小分类等设计试图改善查找与利用率，但每项都有管理成本。

12.8 再做一道完整空闲块序列题

采用块大小已包含管理开销的教学模型，起始有一个 128 字节空闲块，最小可分出的空闲块 24 字节，首次适配，立即合并。依次请求 A32、B40、C24，然后释放 B、释放 A，再请求 D64。请每一步画地址区间。

A 占 0 至 31，剩 32 至 127 共 96；B 占 32 至 71，剩 72 至 127 共 56；C 占 72 至 95，剩 96 至 127 共 32。释放 B 后，32 至 71 空闲 40，两邻 A 与 C 都在用，不能合并。释放 A 后，A 与 B 相邻同为空闲，合并成 0 至 71 共 72；C 仍占 72 至 95，后面 96 至 127 空闲 32。

D 需要 64，首次找到前面的 72。若分割，剩 8 小于最小空闲块 24，因此整个 72 交给 D，形成 8 字节额外内部余量。剩余独立空闲块仍为 32。若有人把前 72 与后 32 加起来当 104 连续可用，就漏掉中间仍活着的 C24。

现在释放 C，会与后 32 合成 56，但不能与 D 合并，因为 D 仍在用。再释放 D，三个相邻范围合并回 128。每次检查“实际地址是否相邻”和“是否当前空闲”，比只在链表里找前后节点更清楚。

这道题的块大小已经包含头部等管理开销，不能再额外加一次头部。题目前提写清，才能防止两套模型叠加。换成请求载荷大小的题，第一步应先算实际块大小再做同样流程。

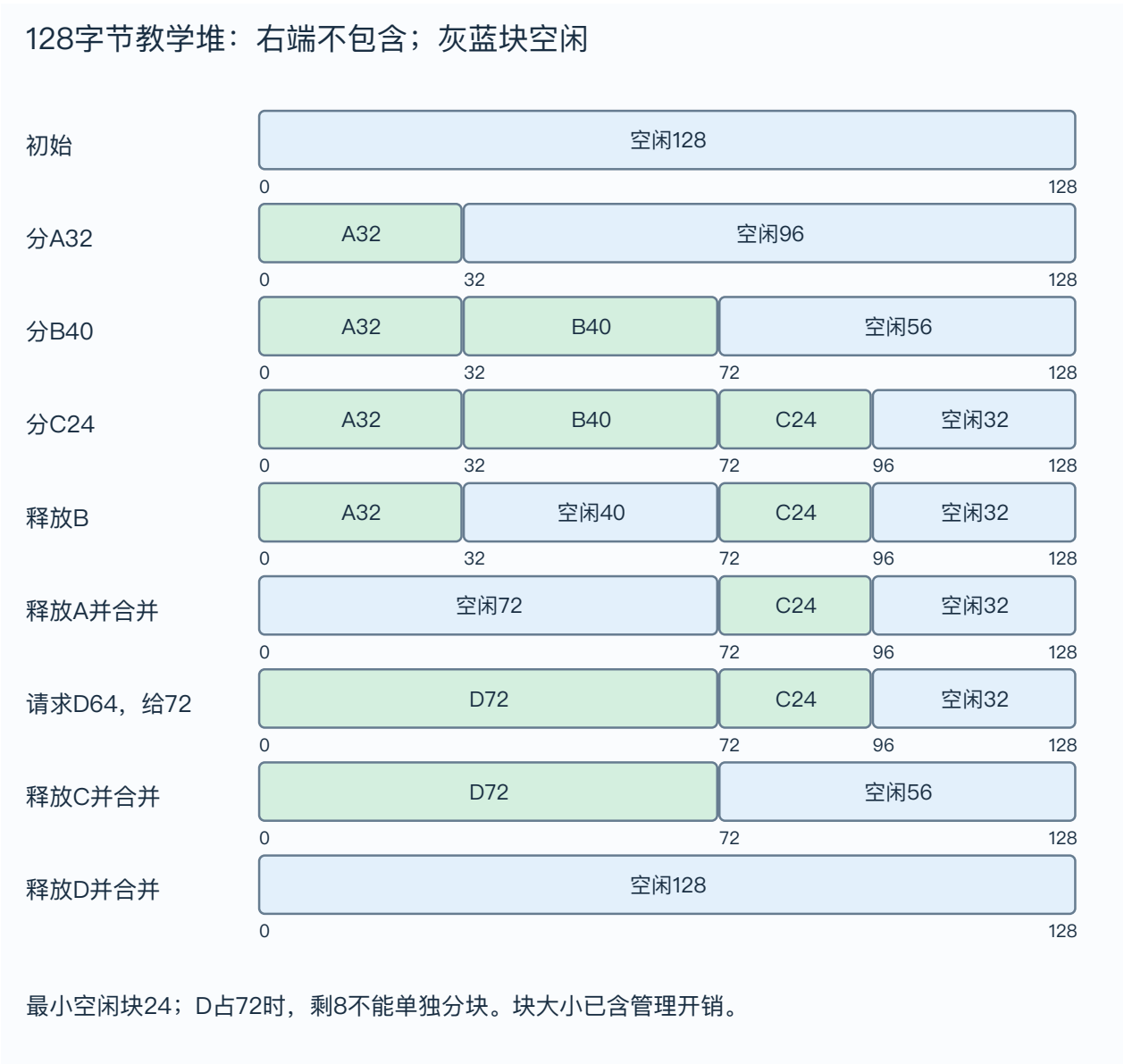


图 32　128 字节教学堆每次分配、释放、分割与合并

横轴是同一块堆的相对字节偏移，所有区间右端不包含。例如 B 占 `[32,72)`，正好 40 字节；这与正文用“32 至 71”描述同一范围。每一行总长度仍 128，颜色改变表示所有权/状态改变，不表示把不相邻地址凭空搬到一起。D 请求 64 却占 72 那一行，8 字节余量在 D 块内部，没有另画成一个非法小空闲块。

显式空闲表还需同时变化：释放 B 后空闲节点为 `[32,72)`、`[96,128)`；释放 A 后前者必须被 `[0,72)` 替代，不能两个都留下。D 占前 72 后，空闲表只剩 `[96,128)`；释放 C 后变成 `[72,128)`；释放 D 后才只剩 `[0,128)`。物理块图与链表节点图要描述同一批空闲范围。

12.9 释放后为什么要合并邻居

相邻空闲块可合并成更大的空闲块，减少外部碎片。合并要区分前邻、后邻分别已分配或空闲的四种情况。若有边界标记，可在常数时间定位相邻块大小与状态，但仍要正确维护空闲链表。按地址相邻与在空闲链表中相邻不是同一回事。

例如内存从低到高为已分配 A32、空闲 B48、已分配 C32、空闲 D64。释放 C 后，它可与左右空闲 B、D 合并为 144 字节块。若只把 C 加入空闲表不合并，空闲总量仍是 144，却最大单块只有 64，对 100 字节请求可能失败。

不能合并两个不相邻块，即使它们在空闲链表上碰巧前后相连。合并改变的是连续地址范围，链表顺序只是一种查找组织。

12.10 算例四：四种合并情况逐项检查

假设释放当前块 X。前后均已分配时，只把 X 标空闲并登记；前空闲、后已分配时，前块与 X 拼接，更新总大小；前已分配、后空闲时，X 与后块拼接；前后均空闲时，合成前+X+后一个块。

若使用显式空闲链表，合并之前要把参与的旧空闲节点从链表摘除，再插入合并后的新节点，或采用等价且保持一致性的更新顺序。否则链表可能同时保留指向块内部的旧节点，下一次分配会把同一内存交给两个请求。

验证可以画两张图：一张按实际地址排块，一张画空闲表指针。释放与合并后，确认每个空闲地址范围只出现一次，块不重叠，头脚一致，前后指针互相对应。只核对总字节数，会漏掉严重的链表损坏。

12.11 泄漏、越界、重复释放、释放后使用怎样区分

泄漏是对象仍被分配器视为使用中，但程序已无法按设计释放或不再需要却长期不释放。越界是访问对象范围外的字节。重复释放是同一分配对象生命周期结束后，再次把旧地址交给 free。释放后使用是对象已结束，还通过旧指针读写。

一个错误可引发其他症状。例如越界写坏下一块头部，当前写入时未立即崩溃，下一次 free 才被分配器检查发现。崩溃栈显示在 free，并不说明 free 调用本身最先错；要沿对象与块边界回查谁先改坏管理信息。

垃圾回收与引用计数是另外的自动管理思路，各有成本和边界。课程中的 C 显式分配要求你自己表达所有权。不能通过“进程结束系统会回收”替代长期运行程序的内存管理，服务可能在结束前就耗尽资源。

12.12 峰值利用率与吞吐量，分配器常要取舍

课程实验可能同时衡量有效载荷占堆大小的比例，以及单位时间能处理多少请求。把每次请求都细拆到最小，可能省空间却增加查找与管理；保留大小类别和快速缓存，可能提高速度却暂时留住更多空闲内存。设计不是只优化一个数字。

计算利用率时，应明确分子是某一时刻用户有效载荷峰值，还是整段请求累计量。后者把已经释放的历史请求重复算入，可能得到超过 100% 的荒谬结果。分母也要按实验定义，是实际扩展的堆字节数，还是某个时刻驻留内存，二者不能混用。

玩具序列：先分配 100 字节 A，再分配 100 字节 B，释放 A，再分配 80 字节 C。累计请求 280 不代表同时需要 280；峰值用户载荷是 200。若分配器始终占用 256 字节堆，按这一简化口径峰值利用率为 200/256，而非 280/256。时间线能把“累计”与“同时活着”分开。

12.13 空闲表损坏为什么会延迟爆发

显式空闲表常把前后链接存进空闲块载荷。用户对象释放后，这些旧载荷字节可能立即变成分配器管理字段。若程序继续通过旧指针写“我自己的数组”，其实是在改空闲链表。下一次 malloc 遍历时才崩溃，错误位置远离最初释放后使用。

这解释了为什么 free 后打印旧对象做调试也不可靠。对象已不属于你，分配器可以把内容改作任何合法内部用途。把 free 后的地址当作仍可读快照，会与实现争用同一存储语义。

调试器中的内存断点有时能找出谁改坏某字节，消毒器也能标出释放后访问。若错误只在高负载下出现，还要排查跨线程所有权：一个线程释放对象时，另一个是否仍借用。分配器内部线程安全不等于应用对象生命周期自动安全。

12.14 扩容策略为何影响总复制量

动态数组每次只增加一个元素的容量，插入第 k 项时可能复制前 k−1 项，总复制量约 1+2+…+(n−1)，为二次增长。若容量每次翻倍，复制规模约 1、2、4、…，总量小于 2n，因此平均每次插入的复制开销可以为常数数量级。

这是摊还分析，不保证每次插入都快；碰到扩容的那次仍可能复制很多项。翻倍也会留下空余容量，空间与时间有取舍。实际增长因子、峰值内存和 realloc 是否能原地扩展都会影响表现。要从请求序列和对象需求选择策略，不能只凭“malloc 很快”忽视频繁移动。

练习：容量从 4 翻到 8 时已有 4 项，新插入第 5 项，realloc 成功移动到新地址。旧元素指针是否自动有效？答案是否；应重新从新基址与下标计算。摊还时间结论不会消除旧地址失效的问题。

12.15 自己做，再看答案

练习一：总空闲 160 字节，由不相邻的 40、40、80 三块构成。能否满足最终块大小 100 的连续请求？

答案：按这个分配模型不能，因为最大连续空闲块只有 80。总量够而单块不够是外部碎片。不能把不相邻的 40 与 80 简单拼成一段地址，除非使用不同的虚拟映射或应用级分段结构，而那已不是本题分配操作。

练习二：p 与 q 指向同一分配对象，free(p)后将 p=NULL，q 还能用吗？

答案：不能，q 仍指向已结束的对象。p 置空只改一个指针变量，不会遍历程序清除所有别名。应由所有权设计保证借用者在释放前停止使用。

练习三：请求 17 字节，头部 8 字节，对齐 8 字节，最小块 24 字节，实际块至少多大？

答案：载荷加头部 25，向上对齐到 32；32 也大于最小块 24，所以取 32。总开销 15 字节，其中 8 头部、7 对齐填充。不能先把 17 对齐到 24 后忘记头部，虽然此题碰巧再加 8 也得 32，步骤必须与给定布局一致。

13 输入输出：文件描述符、偏移和缓冲到底分几层

你打开同一个文件两次，两个读取位置可能互不影响；把一个描述符 dup 一下，两个名字却通常共用同一读取位置。fork 又会复制用户态 stdio 缓冲，同时让相应文件描述符引用同一个内核打开文件描述。看起来矛盾的现象，画出三层对象以后就清楚了。

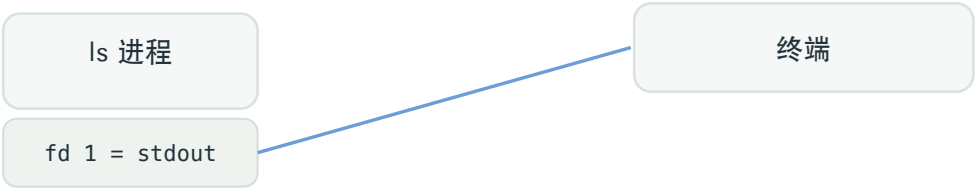
本章先讲底层 read/write，再讲共享偏移、重定向与 stdio。所有 I/O 例子都要检查实际返回值。请求读 100 字节，不等于这次真的得到 100 字节；读到短数据也不自动等于文件结束。

shell 执行 ls > out.txt 时 fd 表的变化：

重定向：fd 1指终端还是文件 第 1 / 2 步

1. 默认：stdout接终端

重定向：同一程序，stdout指向终端还是文件由fd表决定



输出上屏。

进程启动时fd 0/1/2默认指向终端。

为什么：printf写fd 1，屏幕上看得见。

先打开一个文件，读出两个字节

准备一个普通文件 data.bin，内容恰为六字节 ABCDEF。下面先分析一次成功路径：进程原有标准输入、输出、错误三个描述符，open 返回尚未使用的编号 3。编号 3 没有装着文件的六个字符，它用于在进程的表中找到这一次打开的状态。

动作完成后	进程用的编号	这次打开记录的偏移	用户缓冲中哪些字节有效
`open("data.bin",O_RDONLY)`成功	3	0	尚未读取
`read(3,buf,2)`本次返回 2	3	2	buf[0]='A'，buf[1]='B'
`read(3,buf2,1)`本次返回 1	3	3	另一个缓冲 buf2[0]=C，原 buf 仍为 AB
`close(3)`成功	3 不再供本次打开使用	撤销这个引用	先前复制到 buf 的 AB 仍在

第二行同时做了两件相关的事：把读到的字节复制到用户缓冲，并推进打开状态中的偏移。buf 是普通内存；关闭文件引用不会擦除已经读入的字符。O_RDONLY 表示以只读方式打开，本例不修改文件。

这里明确假定本次 read 实际返回 2，不是声称每次请求 2 都必然得到 2。真正的程序要检查 open 的−1 和 read 的返回值，接下来分别处理短读、结束和错误。等一次打开的过程清楚之后，再给同一文件增加第二次 open 和 dup，观察哪些位置会共享。

13.1 read 返回值怎样解释

`read(fd,buf,n)`成功时返回实际字节数 r，满足 0≤r≤n。r>0 意味着这次确实得到 r 字节；r=0 通常表示在对应读语义下到达文件结束；r=−1 表示失败，要查看 errno。n=0 等边界另有规定，不能用零长度调用的返回值推断整个文件为空。

短读可能由管道、终端、网络或剩余文件长度等原因产生。若要读满一个固定长度记录，就循环累计，直到读够、遇到真正 EOF 或发生应处理的错误。被信号打断时的 EINTR 通常可按策略重试；非阻塞描述符上的 EAGAIN 表示当前暂时不可用，需要等待合适事件，而非马上无限忙循环。

write 也可能短写，只写出前 r 字节。下一轮应从 buf+r 继续，长度减 r；重发整个缓冲会重复数据，忽略尾部则丢数据。返回类型 ssize_t 能表示−1，不能一开始就转成 size_t，否则−1 会变成巨大无符号数。

13.2 算例一：三次短读拼一个记录

需要读 10 字节。第一次 read 请求 10，返回 4，已填 buf[0..3]；第二次请求剩余 6，返回 3，填 buf[4..6]；第三次请求剩余 3，返回 3，填 buf[7..9]，累计完成 10。

若第二次返回 0，则只有 4 字节有效，记录不完整，应按协议报告截断或保留部分数据等待其他上层处理，不能把 buf 剩余旧内容当成新数据。若第二次返回−1 且 EINTR，则第一次的 4 字节仍在，不要把累计量清零重读整个记录。

安全读满循环的核心状态只有两个：已完成 done 和剩余 n−done。每次成功 r>0 后令 done+=r；EOF 返回 done；真实错误返回错误并说明接口是否保留部分进度。实际函数应明确区分“提前 EOF 得到部分数据”和“调用失败”，不能靠一个模糊布尔值丢掉信息。

把 10 字节记录假定为 `ABCDEFGHIJ`，两种数量不要混：`done` 是已经得到的字节数，`r` 是本次新得到的字节数。

调用前 done	传给 read 的指针/长度	本次 r	新增有效位置	更新后 done
0	buf+0，10	4	0 至 3：ABCD	4
4	buf+4，6	3	4 至 6：EFG	7
7	buf+7，3	3	7 至 9：HIJ	10

若在第二行先遇 EINTR，`done` 仍为 4，下一次仍传 `buf+4,6`，不能退回 buf 开头。返回 0 则是“只有 ABCD 有效”，不能把未填的 6 格拿去解析记录。写出 `ABCDE` 时同理，实际返回 2、2、1 就依次推进指针 0、2、4：

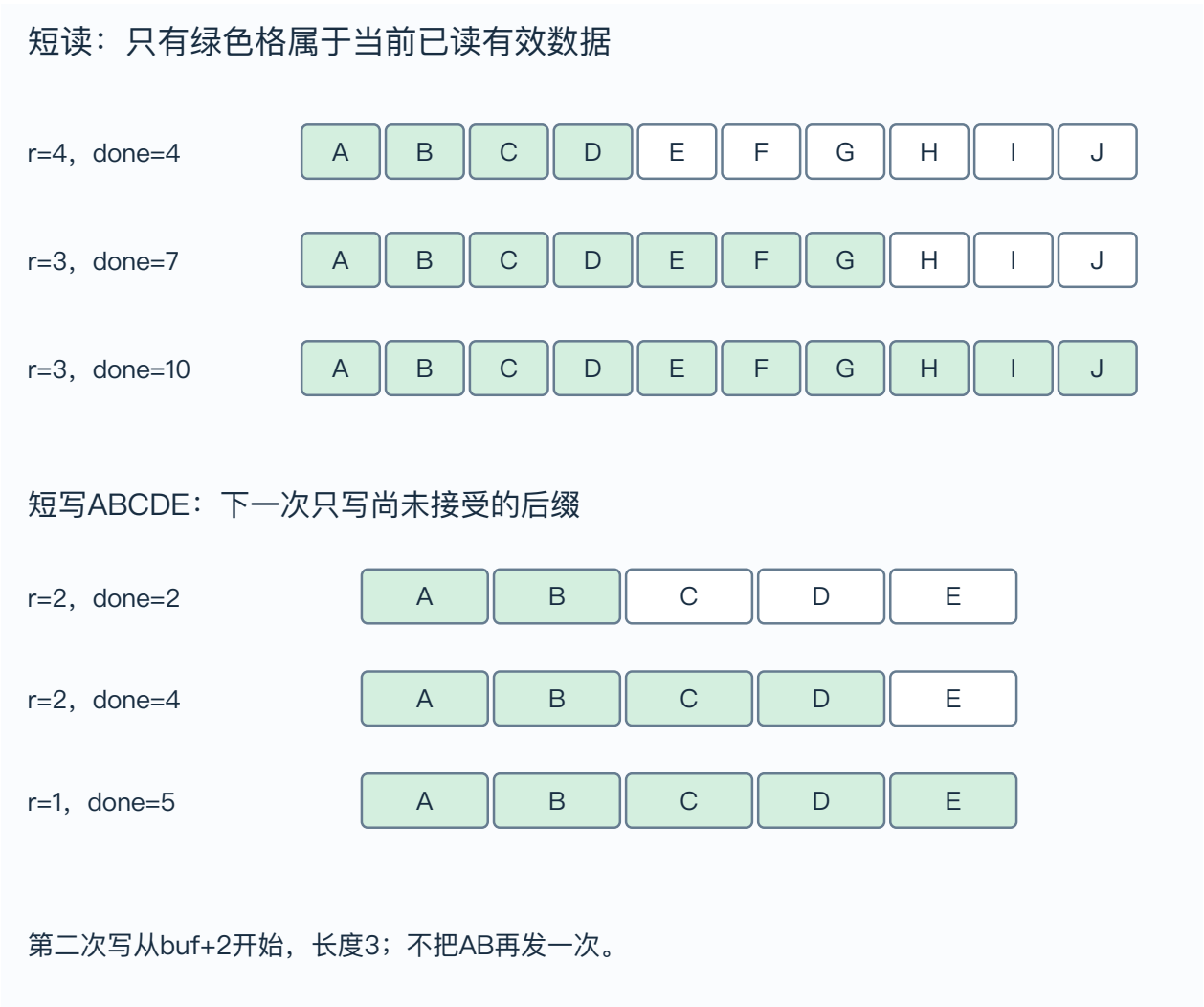


图 33　短读累计和短写剩余区间

13.3 算例二：短写时从哪个字节继续

要写字符串 ABCDE，共 5 字节。第一次 write 返回 2，说明 AB 已经被接受；第二次应传指针 buf+2、长度 3，即 CDE。如果第二次也返回 2，接受 CD；第三次写 E。

若程序第二次仍写 ABCDE，最终可能得到 ABABCDE；若程序把返回 2 当“成功了”直接结束，则只写 AB。正确循环不关心一次调用是否“看起来成功”，只根据实际计数更新。若返回 0 但尚有待写数据，要避免死循环并按接口场景处理异常停滞。

对于管道或网络，还要考虑对端关闭造成的错误与 SIGPIPE。返回检查和信号策略需要配合，不能把所有失败统一重试。对永久错误无限重试既不会修好问题，还可能把 CPU 占满。

13.4 可靠读写的可运行核心

以下两个函数只面向阻塞式字节流教学使用。调用者要求的 n 不得超过 SSIZE_MAX，使成功字节数能装进 ssize_t。read_exact_or_eof 在 EOF 时返回已得到字节数，调用者必须与 n 比较；write_all 要么完成 n 字节，要么返回-1，但失败前可能已有部分字节写出，不能无条件把整个请求重新发送。

```
#define _POSIX_C_SOURCE 200809L
#include <unistd.h>
#include <errno.h>
#include <stdint.h>
#include <limits.h>

ssize_t read_exact_or_eof(int fd, void *buf, size_t n) {
```

```
    if (n > (size_t)SSIZE_MAX) { errno = EINVAL; return -1; }
    unsigned char *p = buf;
    size_t done = 0;
    while (done < n) {
        ssize_t r = read(fd, p + done, n - done);
        if (r > 0) { done += (size_t)r; continue; }
        if (r == 0) break;
        if (errno == EINTR) continue;
        return -1;
    }
    return (ssize_t)done;
}

ssize_t write_all(int fd, const void *buf, size_t n) {
    if (n > (size_t)SSIZE_MAX) { errno = EINVAL; return -1; }
    const unsigned char *p = buf;
    size_t done = 0;
    while (done < n) {
        ssize_t r = write(fd, p + done, n - done);
        if (r > 0) { done += (size_t)r; continue; }
        if (r == 0) { errno = EIO; return -1; }
        if (errno == EINTR) continue;
        return -1;
    }
    return (ssize_t)done;
}
```

为什么把 buf 转成 unsigned char？因为加 *done* 应该按字节移动；*void* 没有标准 C 指针算术，int*则会每次跳 sizeof(int)。为什么 r 先保持 ssize_t？因为−1 是错误标记，转成无符号再比较会误认巨量成功。为什么 done 只在 r>0 时增加？因为失败与 EOF 都没有本次新字节可计。

这两个函数没有把 EAGAIN 当 EINTR 重试。非阻塞对象应结合就绪通知与更完整状态机，立即忙重试会占满 CPU。也没有解决写入的消息原子性与断点重传语义：write_all 保证正常完成时总量正确，失败时外部可能已看到前缀，协议若重试要知道如何避免重复。接口名字里的 all 不是事务保证。

自测可先用临时普通文件写入 ABCDE，重置偏移后读取 5 字节核对，再请求 8 字节应只返回 5。还可用管道人为分两批写入，检查读满循环是否保留第一批。不要只测一次 read 恰好返回 n 的幸运路径，否则循环体里最重要的累计逻辑根本没有被检验。

13.5 算例三：同文件两次 open 与 dup

文件内容为 ABCDEF。fd1=open 得到打开文件描述 O1，偏移 0；fd2=open 同一路径得到 O2，偏移 0。从 fd1 读两字节得到 AB，O1 偏移 2；从 fd2 读一字节得到 A，O2 偏移 1，二者互不影响。

现在 fd3=dup(fd1)，fd1 和 fd3 都引用 O1，当前偏移 2。从 fd3 读两字节得到 CD，O1 偏移 4；再从 fd1 读一字节得到 E。若误以为 dup 创建独立偏移，就会错误预测 fd1 仍读 C。

三个描述符可能都指同一个文件内容，但共享层次不同。判断题中不要只画文件名，把 O1、O2 两个内核对象明确画开。fork 后相应描述符共享偏移，也按同一原则追踪。

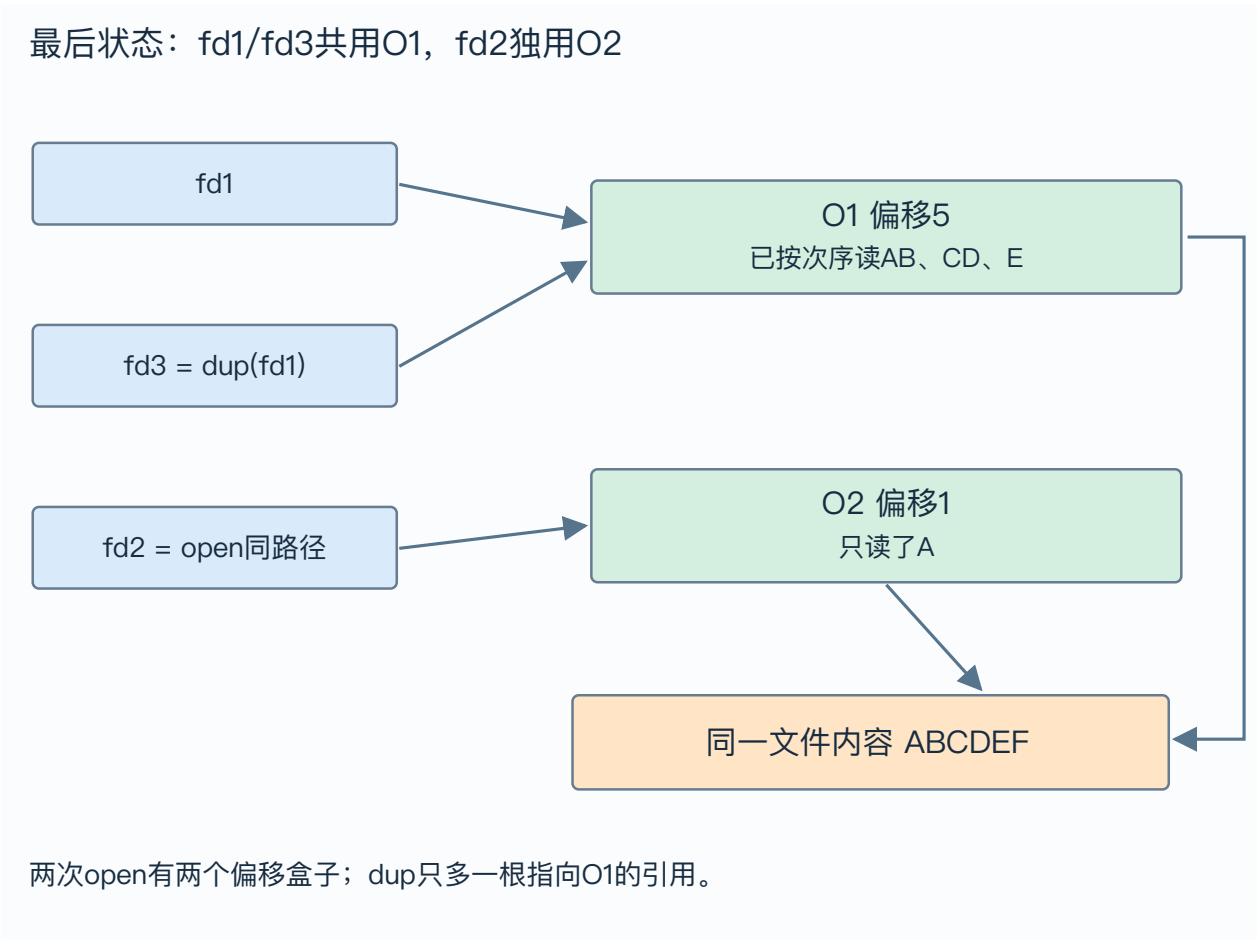


图 34　两个 open 创建不同偏移，dup 只增加同一对象的引用

箭头要读到内核对象一层再停止：fd1 与 fd3 共用 O1 的偏移，fd2 有独立 O2；两对象最终仍读同一文件内容 ABCDEF。上图不是“三份文件副本”。close(fd3)只删 fd3 那根引用，fd1 仍可沿 O1 继续，不能因编号不同或相同路径就猜共享关系。

13.6 三层对象先分开

第一层是进程自己的文件描述符表，描述符是小整数，例如标准输入 0、标准输出 1、标准错误 2。表项引用内核中的打开文件描述。第二层的打开文件描述记录当前文件偏移、打开状态标志等；多个描述符可能指向同一个它。第三层是底层文件对象及文件系统元数据，描述实际文件内容与属性。

同一个文件可被多次 open，产生不同打开文件描述，因而各有偏移。dup 复制一个描述符引用，通常指向同一个打开文件描述，因而共享偏移。fork 后子进程继承的相应描述符也引用同一打开文件描述。close 一个描述符只是撤掉该引用，不一定立即关闭所有别人仍在使用的引用。

用户态的 FILE*与缓冲再放在图上方，是 C 标准 I/O 库维护的一层，不能和整数描述符混成同一个对象。一个库流内部可能关联描述符，但 printf 先写用户缓冲，write 直接请求内核，两者的可见时机不同。

13.7 偏移与文件长度不是同一个量

文件偏移是下一次按该打开文件描述读写的位置。文件长度是文件目前有多少字节。lseek 可改变支持寻址对象的偏移，不一定立即读取或写入数据；管道等对象不支持普通文件式任意寻址。

把偏移移到文件末尾之外再写，可能形成空洞文件，中间逻辑上读为零，但文件系统未必分配同等物理存储。磁盘占用与逻辑文件长度因此可能不同。这个现象与.bss 在磁盘上不必逐字存零的理由也不同，应分别按文件系统与加载器机制理解。

并发追加不能简单写成“先 lseek 到末尾，再 write”，因为两步之间别的执行者可能也追加。O_APPEND 等标志提供更合适的追加语义，但跨多次 write 的整条逻辑记录是否不可交错，仍需另作设计，不能把“每次写前在末尾”当无限大的事务。

13.8 重定向不是改 printf 的代码

shell 可在 exec 之前把描述符 1 重新指向某个文件，之后程序仍执行 printf，底层使用的标准输出却已换了目的。dup2 常用于把一个已打开描述符复制到指定编号。

例如打开 output.txt 得到 fd=3，调用 dup2(3,1)，让 1 引用同一个打开文件描述，再 close(3)，描述符 1 仍有效，因为它保留了引用。若误以为 close(3)把文件彻底关闭，就无法解释重定向为何继续工作。

重定向后 FILE*缓冲策略可能与终端不同。把输出从终端改到文件会改变可见时机，不表示程序突然丢失数据。需要在关键点检查 fflush 或关闭结果，但不要每输出一个字符都无条件刷缓冲，性能和接口意义都要考虑。

13.9 算例四：fork 共享偏移，不共享普通缓冲变量

先 open 文件 ABCDEF，偏移 0，随后 fork 成功。父先 read 两字节得到 AB，偏移变 2；子随后 read 两字节得到 CD，偏移变 4。这个顺序是假定的调度之一，真实情况下谁先读不保证，但对应描述符指向同一个打开文件描述，所以共同推进偏移。

若父子在 fork 之前已分别把文件内容读进自己的用户数组，那么 fork 后数组只是普通私有内存副本，修改不互相影响。若 fork 前还有 stdio 未刷缓冲，双方也各得一份缓冲副本。三个场景同时出现时，必须标出每项状态属于用户空间还是内核对象。

要让不同线程或进程按明确位置读取，可研究 pread 等位置式接口，它不按普通共享文件偏移的方式推进。但接口是否适用于目标对象、并发修改文件如何处理，仍要明说。

13.10 stdio 为什么快，为什么也容易让人误判

每次系统调用有固定成本，printf、fread 等库接口常通过用户缓冲积累或预取一批数据，减少底层调用次数。行缓冲常在换行时刷出，完全缓冲常按块刷出，无缓冲则尽快传递；具体流初始策略依环境和实现规定。

`fflush` 把库缓冲中待输出的数据交给底层，不等于保证数据已经永久落到物理存储介质。要讨论断电后是否持久，需要文件系统与 fsync 等更低层保证。读者若把“终端看到了”“内核接收了”“磁盘持久化了”当同一步，会错误判断日志与数据库安全性。

混用 stdio 与 read/write 操作同一底层文件时，库可能已经预读了一批数据，其用户态逻辑位置与内核偏移不再是你直觉中的同一数。除非明确处理同步规则，最好为同一数据流选择一致接口，避免一部分代码以为只读了一字节，另一部分发现内核偏移已前进一整块。

FILE*是库管理的流对象指针，通常由 fopen 返回；它不是整数文件描述符，正常用 fclose 结束该流。fread 从流读取若干“项”，每项多少字节由调用者另一个参数指定。这与 read 只用一个字节数参数不同。

取一个内容为 `00 00 00 02` 的四字节二进制头，下面两种调用分别从文件起点独立开始，且都假定四字节完整读到：

调用	每项大小	请求项数	成功返回值	实际接收字节
`fread(bytes,1,4,fp)`	1 字节	4	4	4
`fread(&n,sizeof n,1,fp)`，n 为 uint32_t	4 字节	1	1	4

第一种把字节保存在 unsigned char 数组里，可显式解码；第二种直接改写 n 的对象表示，仍受主机端序影响。返回 1 不是只读了一字节，而是完整读到一个四字节项。若文件只剩两字节，第一种可返回 2，第二种没有完整项，返回 0，即使对象的一部分已被改写，也不能据此使用 n 的值。

读不满时还应通过 feof、ferror 区分结束与错误。fread 的项数语义不改变文件格式：格式规定大端，完整读到本机 uint32_t 也不等于已经正确解码。19 章用这项区别定位一个具体错误。

13.11 为什么二进制协议不能直接写结构体

结构体可能有填充，整数有端序，类型大小和对齐依赖 ABI。直接 `write(fd,&s,sizeof s)`只能在严格受控的同布局约定下使用，不自动构成可移植格式。填充字节还可能包含未初始化内容，带来信息泄露。

应明确每个字段宽度与编码，逐字节序列化，或者使用规定格式的库。读入时也检查总长度与字段范围。第 01 章的 read_be32 就是一个小例子：它从四个明确字节构成一个 32 位值，不依赖主机结构体布局。

13.12 读行与读字节：换行也占空间

假设输入是三个字符 A、B、C，随后换行。在常见单字节编码中共 4 字节。若缓冲容量 4，想把整行读成 C 字符串，还要终止零，共需 5 字节。给定容量的读行接口通常至多保存容量减 1 个字符并添加终止符；若没读到换行，可能只是缓冲装满，不表示一整行结束。

处理超长行时，应明确策略：扩容、分段处理，或拒绝并消费本行剩余内容。若只读前一段，下一次把剩余片段误当新记录，就会使整份输入错位。解析器既要知道缓冲大小，也要知道记录边界，二者不能混用。

输入还可能含零字节。底层 read 返回 r 字节不保证它们是以零结尾的文本，直接用 printf(“%s”,buf)可能越过有效部分。若用于文本，预留一字节并在 buf[r]写终止符，且考虑内部零是否允许；若是二进制，则应按明确长度输出或处理，不能用 strlen 重新猜长度。

13.13 管道的结束条件为何取决于所有写端引用

管道是一种字节流通信对象。读端在没有数据但仍有写端可能产生数据时，阻塞读可以等待；当所有写端引用都已关闭，且剩余数据耗尽时，读端才看到 EOF。fork 后不需要的写端若忘记关闭，会使某个读者一直等不到结束。

例如父子各继承管道读写两端，父只想读、子只想写。父应关闭自己不使用的写端，子关闭自己不使用的读端；子写完关闭写端，父读尽后才能得到 EOF。如果父一直留着写端，内核仍认为存在写者，父可能在等一个永远不会到来的新字节。

这不是“close 越多越安全”，而是每个进程只保留自己确实需要的引用。先画出 fork 后四个描述符表项，再逐个划掉不使用的端点，能看出哪个引用阻止了 EOF。管道没有消息边界，写一次不保证读一次对应同样长度，上层仍需协议或分隔符。

13.14 多线程写日志，完整记录怎样保持在一起

若一条日志由三次 write 拼成：时间、内容、换行，两个线程可能交错成时间 A、时间 B、内容 A、换行 B 等。单次接口调用的某些原子保证不自动覆盖多次调用组合。可以在应用层用同一锁保护整条记录，或先在私有缓冲构造再按目标对象的保证写出。

管道对于不超过规定大小的写入有特定原子性约束，普通文件、终端、套接字又有各自规则。不要从某一种对象推导“write 永远完整且不交错”。短写处理仍必需；重试是否会被其他写者插入，取决于你如何保护整个记录。

练习：一个读管道的父进程一直得不到 EOF，子已退出，最先该查什么？答案是查父自身或其他继承者是否还持有管道写端。进程退出会关闭其引用，但其他表项仍可保持写端存在；结束条件是引用全部消失，不是某个特定孩子退出。

13.15 文件元数据：stat 看的是属性，不是正文

文件不只有内容，还带类型、权限、长度、时间等属性，称元数据。stat 按路径取得相关属性，fstat 按已经打开的描述符取得属性，lstat 在路径最后一项是符号链接时报告链接本身而不是跟到目标。它们回答的是“这个对象是什么”，不会把文件正文读进你的缓冲。

属性通常放进 struct stat。st_mode 同时包含文件类型与权限相关位，不能直接拿整个整数与一个自写常量比较；用 S_ISREG、S_ISDIR 等宏判断普通文件或目录。普通文件的 st_size 通常是逻辑字节长度，不等于磁盘实际占块量，也不保证下一次读取前长度不变。时间字段也要按接口类型处理，不把它们一概当成 int 打印。

举例：一个普通文件逻辑长 1000 字节，进程当前打开偏移为 200。stat 仍报告长度 1000，不是剩余 800；剩余量是你结合偏移与长度另算的，而且并发截断或追加可能马上改变它。lseek 改变当前打开文件描述的偏移，通常不会直接改变 stat 报告的文件长度；实际写入才可能扩展文件。

权限位中的 r、w、x 分别表示读、写、执行等权限，但对象类型不同，含义也不同。目录的 x 主要表示能否按名字搜索或穿过它，r 主要涉及读取目录项，w 涉及修改目录内容，并非“把目录当文本覆盖”。删除文件通常要满足父目录相关权限等条件，不能只看目标文件是否可写。真实权限判断还包括身份、组、ACL 和系统策略，基础位图不概括全部规则。

13.16 符号链接把名字再指向另一个名字

假设目录里 target 是 100 字节普通文件，shortcut 是指向 target 的符号链接。stat(“shortcut”)通常跟到目标，报告普通文件属性；lstat(“shortcut”)报告符号链接本身，其长度等字段具有链接对象的含义。目标被删除后，shortcut 名字仍可能存在，但跟随它的 stat 或 open 失败，称悬空链接。

这与硬链接不同。硬链接是在目录中增加对同一个底层文件对象的另一名字，改一个名字不自动删除另一个。符号链接保存的是一段目标路径，可能指向不存在的位置，也可能跨不同文件系统。初学时先画“目录名字到对象”与“链接内容再解释成路径”两条不同关系，不要都叫“快捷方式”就认为语义相同。

路径也不是打开对象的永久身份。一个线程先 stat 路径确认类型，另一方随后替换目录项，前者再 open 同一路径可能得到另一个对象。这是检查时与使用时之间的竞态。按需求先安全打开，再用 fstat 核对已打开对象，可以避免一部分路径替换窗口；若需要禁止跟随链接或限制目录边界，还要采用具体系统提供的合适接口。单次 fstat 也不保证文件内容此后永不变化。

13.17 目录项不是文件内容列表自动递归展开

目录保存名字到文件系统对象的关联。opendir 打开一个目录流，readdir 逐个取得目录项，closedir 关闭。目录项中的 d_name 是名字，不是完整路径，也不自动包含该文件的所有元数据。某些系统给 d_type，但它可能未知或不便可移植依赖；要可靠知道类型，应使用合适属性接口。

readdir 返回空指针时，可能到结尾，也可能出错。一个清楚的循环在每次调用前把 errno 置 0；返回空时，errno 仍 0 表示正常结束，非 0 表示错误。不能在整段循环一开始只清一次 errno，因为中间的其他操作可能留下错误状态，使最后一次正常 EOF 被误认失败。

返回的目录项存储可能在下次 readdir 时被复用，若要长期保留名字，应自己复制；不要把每次返回的内部指针简单存进数组，最后才统一读取。读取顺序也不保证按字母或创建时间排序。想按名字展示，先收集再明确排序。

13.18 算例五：列出一个目录，并正确处理结束和错误

下面完整程序接受一个目录路径，逐个输出名字。它只列当前一层，包含接口返回的“.”与“..”，没有声称递归统计全部文件。某些名字可含换行或控制字符，所以这是学习接口的直接文本输出；若做可机器解析的清单，应另选明确转义或长度编码格式。

```
#define _POSIX_C_SOURCE 200809L
#include <dirent.h>
#include <errno.h>
#include <stdio.h>

int main(int argc, char **argv) {
    const char *path = argc > 1 ? argv[1] : ".";
    DIR *dir = opendir(path);
    if (dir == NULL) {
        perror("opendir");
        return 1;
    }
    int status = 0;
    for (;;) {
        errno = 0;
        struct dirent *entry = readdir(dir);
        if (entry == NULL) {
            if (errno != 0) {
                perror("readdir");
                status = 1;
```

```
            }
            break;
        }
        if (printf("%s\n", entry->d_name) < 0) {
            perror("printf");
            status = 1;
            break;
        }
    }
    if (closedir(dir) == -1) {
        perror("closedir");
        status = 1;
    }
    if (fflush(stdout) == EOF) {
        perror("fflush");
        status = 1;
    }
    return status;
}
```

逐步走一个只有 alpha、beta 两个普通文件的目录。opendir 成功建立目录流；readdir 可能先给“.”、再给“..”、再给两个名字，但顺序不是程序可依赖保证。每取得一项，当前就打印其名字，不跨下一次调用保存内部指针。最后一次返回 NULL 且 errno 为 0，正常退出循环；仍要 closedir 释放目录流，检查 stdout 最后刷新是否失败。

若给不存在的路径，opendir 失败，程序报告并返回 1，不进入循环。若中途目录读取错误，当前 errno 被 perror 解释，仍尝试 closedir，再以非零状态结束。若只把 while(readdir(...))写完就返回 0，会把读取错误与正常读完混在一起。

目录正被别的进程修改时，枚举结果不必是一张某个单一时刻完整不变的快照。递归归档或同步程序若要求完整性，需要设计枚举、打开、校验与重试的协议，不能把“循环到 EOF”夸成证明整个变化中的目录从未漏项。课程目录接口与工程快照保证是两个层次。

13.19 为任务选择接口：高层与低层各有适用场景

按格式读写少量文本，stdio 的缓冲与格式化往往方便；处理明确字节协议、描述符重定向、管道或网络，需要理解 read/write 与返回计数；查询类型长度用 stat 类接口，不应为了知道长度把整个文件先读一遍；遍历名字用目录流，而不是把目录路径当普通文件内容硬读。

选择由所需语义决定。高层接口没有免除错误检查，低层接口也没有自动更快或更安全。对于同一数据流，混用前应知道各自缓冲与偏移关系；对于外部输入，始终明确字节数、字段格式和对象范围。完成 I/O 的程序最后还要判断输出是否真正成功，不能只因主循环结束就忽略刷新或关闭阶段的失败。

13.20 自己做，再看答案

练习一：文件 XYZ，fd1 和 fd2 由 dup 关联。fd1 读一字节后，fd2 读一字节，分别得到什么？

答案：在题目给定顺序且无其他 I/O 时，fd1 得 X，偏移变 1；fd2 得 Y，偏移变 2。描述符编号不同不代表偏移独立。

练习二：read 请求 100 返回 20，能否立即认定 EOF？

答案：不能。只知道本次得到 20 字节，可能是短读。若需要继续，下一次再读；返回 0 才按对应接口的 EOF 语义处理。切勿读取缓冲后 80 字节的旧内容当成本次数据。

练习三：fflush 成功是否意味着拔电后数据必然保留？

答案：不能。它处理用户态库缓冲向底层传递，持久化还涉及内核、文件系统、设备缓存与相应同步保证。先明确你要保证哪一层“完成”，再选接口。

返回值细节可对照 read、write 与 dup。

14 线程与并发：同一个变量被两个人动过以后

单线程代码 `counter++`很容易理解：把计数加一。两个线程各做一遍，你可能自然期待加二。但源代码的一行通常不是处理器的一步；它可能要先读旧值、在寄存器里加一、再写回。两条执行流交错时，后写回的一方可能覆盖先前更新。在 C 里，普通共享变量上的这种未同步冲突访问还构成数据竞争，程序不能再按普通顺序交错模型获得标准保证。

本章把两件事一起讲：用具体交错帮助理解问题，再用语言层面的同步规则判断什么程序才合法。不能看懂一次“丢失更新”以后，就以为真实编译器必须只产生这一种坏结果。

14.1 一个进程里的线程共享什么

同一进程中的线程通常共享地址空间，包括全局数据和动态分配对象，也共享许多进程资源。每个线程有自己的寄存器状态和执行位置，通常有自己的栈；线程局部存储则给每个线程单独实例。

“每个线程自己的栈”不等于其他线程硬件上不能访问它。若一个线程把自己局部变量地址交出去，其他线程可以通过该地址尝试访问；必须保证对象仍存活，并按共享访问规则同步。栈是分配和管理方式，不是自动建立的隔离墙。

线程结束与进程结束也不同。一个线程从线程入口返回，通常结束该线程；进程退出会结束整个进程中的线程。主线程若过早 return 导致进程结束，其他线程可能没来得及完成工作。使用 join 等机制等待，并检查创建是否成功。

14.2 先创建一个真实线程，再把参数交给它

先让一个新线程算“编号加 10”，主线程等它结束后读取结果。参数对象有两个字段：index 由主线程在创建前写好，result 由工作线程写；主线程在 join 成功之后才读 result。对象先建在主线程中，直到 join 结束仍存活。

`pthread_create(&tid,NULL,work,&arg)`中，tid 用来接收线程标识；NULL 表示默认属性；work 是新线程要进入的函数；&arg 是传给它的数据地址。创建成功后，主线程从调用之后继续，新线程则从 work 入口开始。它们先后谁运行不由源码的行序统一规定。

work 的参数类型为 void，表示通用对象地址；函数把它转回实际的 *struct Arg* 后访问字段。函数名 work 在这里提供入口地址，不是先在主线程执行 work 再把计算结果传过去。

阶段	主线程的位置	工作线程的位置	arg.index / arg.result
创建前	写好参数	尚不存在	0 / 0
创建成功后的一种次序	准备调用 join	运行 work，将 result 设为 10	0 / 10
另一种允许次序	已在 join 里等待	稍后才运行 work	先 0 / 0，再 0 / 10
join 成功返回	可以读取 result	已经结束	0 / 10

一个线程的规则确定后，扩成三份独立参数。完整程序在支持 POSIX 线程的系统上可用 `cc -std=c17 -Wall -Wextra -pthread thread-demo.c -o thread-demo` 编译：

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Arg { int index; int result; };

static void *work(void *raw) {
    struct Arg *arg = raw;
    arg->result = arg->index + 10;
    return NULL;
}
```

```c
int main(void) {
    struct Arg args[3];
    pthread_t tids[3];
    size_t made = 0;
    int status = EXIT_SUCCESS;
    for (size_t i = 0; i < 3; ++i) {
        args[i].index = (int)i;
        args[i].result = 0;
        int rc = pthread_create(&tids[i], NULL, work, &args[i]);
        if (rc != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(rc));
            status = EXIT_FAILURE;
            break;
        }
        ++made;
    }
    for (size_t i = 0; i < made; ++i) {
        int rc = pthread_join(tids[i], NULL);
        if (rc != 0) {
            fprintf(stderr, "pthread_join: %s\n", strerror(rc));
            exit(EXIT_FAILURE);
        }
    }
    for (size_t i = 0; i < made; ++i) {
        if (printf("%d -> %d\n", args[i].index, args[i].result) < 0)
            status = EXIT_FAILURE;
    }
    if (fflush(stdout) == EOF) status = EXIT_FAILURE;
    return status;
}
```

正常输出依次为 0 -> 10、1 -> 11、2 -> 12，因为输出由主线程在全部 join 之后按下标进行；这不代表三个工作线程按编号顺序运行。三个工作线程写不同的 result 对象，主线程创建前写 index，join 后才读 result，没有同时争写同一字段。

pthread 接口直接返回错误号，因此用 strerror(rc)解释，不把 errno 当作这里的返回错误。如果第几个线程创建失败，只 join 已经创建的 made 个，不读取未创建线程的结果。main 在等待结束前保持 args 存活；若 join 自身失败，本例报告后终止进程，不假装已经确认工作完成。

14.3 创建线程时参数必须活得够久

前面的程序传入三份 args。现在改变一件事：每次都传循环变量的同一个地址，结果会发生什么？

pthread_create 把入口函数与一个 void*参数交给新线程。新线程可能在调用者下一行之前就开始，也可能过一会儿才运行。把循环变量 i 的地址反复传给多个线程，很可能让它们都读取同一个不断变化的对象，而非创建时的独立编号。

可为每个线程准备独立参数对象，例如数组 args[k]，在创建前填好，直到所有相关线程读完之前保持有效。若父函数返回，局部 args 生命周期结束，再访问就悬空。线程参数设计同时涉及“每份数据是否独立”和“对象是否仍活着”两项检查。

14.4 算例一：三个线程为什么都可能打印 3

循环 k=0、1、2 创建三个线程，每次传 &k，循环完成时 k 变 3。若新线程直到此后才读取，就都可能看到 3。若创建者仍在修改 k 而线程未同步读取，普通 C 对象还存在数据竞争，不能把“都打印 3”当完整语义描述。

修复为 args[0]=0、args[1]=1、args[2]=2，每个线程接收对应元素地址，创建前完成写入，后续不再修改，join 全部线程后再销毁数组。现在各线程读自己的稳定参数，执行顺序仍不固定，但编号含义不再依赖调度时机。

若仅把 k 在循环内复制到一个新的块局部变量，再传其地址，也不一定安全：该局部对象可能在本轮结束后就消失，下轮还可能复用同一存储。必须按对象生命周期分析，不能只看源码变量名换了。

14.5 数据竞争与竞态条件有什么区别

前面的线程参数例已经安排了两种顺序：先准备某份参数再创建使用它的线程，工作结束后 join 再读结果。这里的“先后”来自程序和同步接口，而不靠观察哪个线程似乎跑得快。语言把可用于推理内存效果的这类关系称为 happens-before，中文常称先发生关系。

数据竞争是语言模型中更精确的冲突访问问题：不同线程访问同一内存位置，至少一次是写，至少一次访问为非原子，并且这两次冲突访问之间没有所需的先发生关系（happens-before）。C 中这样的数据竞争导致未定义行为。两个合法原子访问本身不因缺少 release/acquire 配对就成为数据竞争，但多个原子步骤仍可能有下面的逻辑竞态。竞态条件则更宽，指正确性依赖不受控制的执行顺序；即使每次访问都是原子操作，几步组合仍可能有逻辑竞态。

例如银行余额采用原子变量，两个线程都“先读余额足够，再各扣款”。每个读写都可能合法，但两个检查都看到旧余额，组合操作仍可能允许透支。你需要使“检查与扣款”作为一个受保护事务，或使用正确的原子比较交换循环，不是只把类型加一个 atomic 就万事大吉。

volatile 也不提供普通线程间互斥或完整内存同步。它用于特定访问语义，并不能把读改写变成不可分割操作。锁、原子操作与条件变量各解决不同问题，后面分别展开。

14.6 算例二：用合法的原子读写看清丢失更新

为了让每步结果有明确的语言保证，本例用 C11 原子变量，不拿有数据竞争的普通 int 预测结果。初始 x=5，A 和 B 各执行同样三行：

```
int local = atomic_load(&x);  // 读到自己的局部变量
local = local + 1;            // 只改自己的局部变量
atomic_store(&x, local);      // 写回共享原子对象
```

其中 x 声明为 `_Atomic int`，代码需包含 `<stdatomic.h>`。`atomic_load` 和 `atomic_store` 分别保证一次读、一次写的原子性；局部变量由各自线程拥有。把两线程的局部副本分别命名为 a 和 b，便于在图里区分。默认原子操作采用顺序一致内存序，本例不需要先掌握更弱的内存序。

依次执行的动作	A 的局部 a	B 的局部 b	共享 x
初始	未读	未读	5
A：load	5	未读	5
B：load	5	5	5
A：局部加 1	6	5	5
B：局部加 1	6	6	5
A：store	6	6	6
B：store	6	6	6

下面保留同一交错，分别显示共享 x 与两份局部副本。格子下方的 0、1 仅为示意槽位编号；A、B 的局部变量不是同一个共享数组。

两次原子读写为什么仍会丢失更新

1. 初值5，两个线程各加1

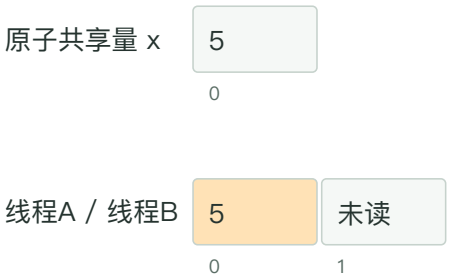


每个线程的程序都是：原子load、局部加1、原子store。
原因：每一次load/store各自不可分割，但三步合起来仍可被另一个线程插入。

图 35　逐步演示：两次原子读写为什么仍会丢失更新

两次原子读写为什么仍会丢失更新

2. A读取共享量



A的局部副本a←x=5。
原因：读取不会给共享量加锁；此后A只在自己的局部副本上计算。

图 36　两次原子读写为什么仍会丢失更新，第 2 步

两次原子读写为什么仍会丢失更新

3. B也读取共享量



B的局部副本b←x=5。
原因：A还未写回，所以B读到同一个初值。

图 37　两次原子读写为什么仍会丢失更新，第 3 步

两次原子读写为什么仍会丢失更新

4. A局部加1



a←a+1=6，共享x仍是5。
原因：寄存器或局部变量的改变没有自动写回共享对象。

图 38　两次原子读写为什么仍会丢失更新，第 4 步

两次原子读写为什么仍会丢失更新

5. B局部加1



b←b+1=6，共享x仍是5。
原因：B独立计算，尚不知道A也打算存入6。

图 39　两次原子读写为什么仍会丢失更新，第 5 步

两次原子读写为什么仍会丢失更新

6. A原子写回



x.store(a)，共享x变成6。
原因：写回这一步不可分割，但写入的是此前算好的局部结果。

图 40　两次原子读写为什么仍会丢失更新，第 6 步

两次原子读写为什么仍会丢失更新

7. B原子写回



x.store(b)，共享x仍为6。
原因：B用6覆盖6，没有把现有共享值再加1，所以丢失一次业务更新。

图 41　两次原子读写为什么仍会丢失更新，第 7 步

两次原子读写为什么仍会丢失更新

8. 与原子fetch_add作对照



如果两线程各执行一次fetch_add(1)，最终才保证7。
原因：本图故意使用原子变量避免C/C++数据竞争的未定义行为；普通int版本不能靠此图预测标准语义。

图 42　两次原子读写为什么仍会丢失更新，第 8 步

A 写回之后，B 并未重新读取 6；B 的局部副本早已算成 6，所以它只是又写一次 6。两次合法的原子 store 没有把业务上的“两次加 1”合成不可分割操作。这条交错完全可以发生在单核上，关键是步骤的先后，而非两个线程是否真的同时占用两个核。

若让 A 三行全部完成后 B 才开始，B 会读到 6，随后写 7。两种次序都遵守每个线程内部的代码顺序，却得到不同业务结果。需要无条件完成两次加 1 时，可让每个线程执行一次 `atomic_fetch_add(&x,1)`；这个接口把“读取当前值、加 1、写回”作为一个原子读改写操作，两次操作合计得到 7。也可用同一把互斥锁保护三行，让另一线程在整段结束后再进入。

普通非原子 int 上的未同步冲突会触发数据竞争和未定义行为，不能把上表的 6 或 7 当成它全部允许的结果。分清这一点后，图里的具体交错既能解释丢失更新，又不会误导你把 C 程序当作只能逐行交错的解释器。

这条交错的 C11 核对程序把表中顺序明确写成语句，可用 `cc -std=c11 原子交错.c -o 原子交错`编译运行，分别得到 6 和 7。它验证的是已经指定的这一条轨迹；真正开两个线程时，谁先读、谁先写仍由实际调度决定。

14.7 join 建立等待，也帮助管理生命周期

pthread_join 等待指定可连接线程结束，并可取回线程返回的 void*结果。返回指针不能指向线程已经结束的局部对象；可返回由外部拥有且仍存活的对象，或动态分配对象并说明释放责任。

成功 join 还提供相应同步保证，使等待者能在正确条件下看到线程完成前的效果。不能用 `sleep(1)`替代 join：线程可能因负载、I/O 或调度超过一秒，睡够时间不证明工作完成。sleep 是一段时间的暂停，join 是对某个执行者结束状态的等待。

分离线程结束后由系统自动回收线程相关管理资源，不能再用普通 join 回收；这不等于应用自己的堆对象自动释放。线程生命周期与对象所有权仍要分别安排。

14.8 算例三：并行求和怎样避免热点计数器

数组有 8 个数 1 至 8，两个线程各算一半。T1 在私有局部 sum1 中算 1+2+3+4=10；T2 在 sum2 中算 5+6+7+8=26；各自把结果写到独立结果槽；主线程 join 两者后汇总为 36。

如果每处理一个元素都抢同一把锁更新全局总和，虽可正确，但锁成为热点，线程大部分时间争用它。先做局部累加，再减少合并次数，通常更有利。正确性来自各段覆盖不重叠、无遗漏，以及合并发生在所有部分完成之后；性能改善来自减少同步操作，而不是“线程越多必然越快”。

还要考虑整数溢出与负载均衡。每段长度相同不保证工作量相同，例如某元素触发大量计算。结果槽若落同一缓存行，频繁更新仍可能伪共享；把每个线程的临时累加放寄存器、最后写一次可缓解这种热点。

14.9 原子性、可见性、顺序性分别问什么

原子性问一个操作能否被其他执行者观察到中间一半。可见性问某方写入何时按规则对另一方可用。顺序性问两个操作之间是否建立了所需先后关系。它们相关，但不是一个词可以包办。

设生产者先填 data，再设 ready。消费者看到 ready 后读 data。若两者只是普通变量且没有同步，既可能数据竞争，也不能凭源码顺序假定跨线程发布成立。采用锁可把数据与标志一起保护；采用 C 原子时，可用 release 写 ready 与 acquire 读 ready 建立所需关系，前提是消费者确实读到对应发布值，且对象无其他未同步冲突。

对只需累加独立计数、不用该计数发布其他数据的场景，较弱原子顺序可能足够。但初学时应先写出需要保证的关系，再选择顺序；不要为了“更快”先把所有操作改成 relaxed，之后才猜是否正确。

线程间的“先发生关系”不只是墙上时钟谁早谁晚，而是语言与同步操作建立的、可用于推理内存效果的关系。生产者先写 data=42，再以 release 方式发布 ready=1；消费者以 acquire 读取并确实观察到这次发布的 1，随后读 data，才能据该关系推理看到已发布的数据。

如果消费者读到的仍是初始 ready=0，就不能把这次 acquire 当作已经接上生产者发布。若另一个线程后续无同步再改 data，先前一次正确发布也不会保护所有未来写。要把“哪次写、哪次读、哪段数据”具体连起来，不能把 release/acquire 当两个随处放的保险符。

使用默认较强顺序的原子操作可以简化部分排序推理，但仍不替你维护业务不变量。锁内检查、修改和解锁形成的结构，对初学者往往更容易审查；只有在需求和性能证据支持时，再引入更复杂原子协议。

14.10 算例四：原子计数也救不了两步检查

库存原子变量 stock=1。两个线程都先 load，均读到 1，均决定可以买，然后各自 fetch_sub(1)。两次扣减都原子，最终 stock=−1，卖出两件。

错误在“检查到扣减”的组合没有成为一个不可分割决策。可以用互斥锁把检查与扣减圈在一起，或用 compare_exchange 循环，只有把某个已知正库存成功改为减一后的值，才认定购买成功。后者循环失败时要重新检查更新后的库存，不能无限按照最初看到的 1 继续。

这与第 10 章检查 done 后 pause 的竞态有共同结构：两步之间的状态可能被别人改变。但修复机制不同，不能拿信号屏蔽代替线程锁，或拿原子计数替代条件等待。

14.11 比较交换怎样把库存检查与减一合成一次决策

CAS 是 compare-and-swap 的常用简称；C11 的 compare_exchange 接口把“当前值是否等于我以为的旧值，若是就改成新值”作为原子操作。这里用 strong 版本，失败表示当前值不匹配；weak 版本还可能出现允许的伪失败，通常也放在正确循环中使用。

库存初始为 1，两个线程都先读到 1，分别存在自己的 expected 局部变量。T1 尝试把原子 stock 从 expected 的 1 改为 0，比较成功，库存变 0，T1 取得商品。T2 随后也带 expected=1 尝试，实际 stock 已 0，比较失败，不扣库存；接口还把实际 0 写回 T2 的 expected。T2 重新检查 expected>0 为假，退出并报告未取得商品。

时刻	原子库存	T1 的 expected	T2 的 expected	业务结果
双方读完	1	1	1	尚未出售
T1 比较交换成功	0	1	1	T1 出售一件
T2 比较交换失败	0	1	0	T2 不得出售
T2 重查退出	0	1	0	总共只出售一件

关键在失败后 expected 是一个输出参数，会被更新，不是永远保留最早的 1。若程序忽略失败仍发货，或失败后用旧判断直接扣款，原竞态仍在。每次成功 CAS 才是“这次购买完成”的唯一状态转换点；看到一次正库存只是尝试的起点，不是已经买到的证据。

若其他线程同时补货，失败后读回的 expected 可能又大于 0，循环可根据新值重新尝试减一。若接口允许一看到 0 就失败返回，则并不承诺等待未来补货；需要等库存的任务应另用条件等待协议。原子比较交换保证一次决策不可分割，不自动提供业务上的排队和等待。

14.12 可重入与线程安全也要分开

一个接口线程安全，可能内部用锁保护共享状态，但不适合在信号处理函数里重入，因为信号可能打断正持锁的线程，处理函数再等同一把锁就死锁。可重入强调在尚未完成一次调用时再次进入能否正确工作，讨论信号与递归等上下文时尤为重要。

返回指向内部 static 缓冲的函数，也可能让两个调用者拿到同一块会被后续调用覆盖的数据。接口文档若要求调用者提供缓冲，常是把隐式共享状态显式化。判断安全性要看共享对象与调用约束，不能只看函数名后有没有_r 之类后缀。

14.13 join 与线程返回值的对象归属

线程入口返回 void*，它只是一个地址值，不自动拷贝指向的结构体。若返回局部结构体地址，线程一结束对象就失效，join 取到地址也救不回。可让调用者提供一个结果槽，线程写完后返回 NULL，调用者 join 后读取；也可由线程分配结果，约定 join 者释放。

后一种还要处理线程创建、分配和 join 失败。若线程分配成功但没人收结果，可能泄漏；若多个等待者或其他模块都 free 同一结果，可能重复释放。线程接口和动态分配结合时，每个地址仍要有清楚拥有者。取回的指针是否为空，也不能独立判断线程是否成功。应用可以约定 NULL 表示成功无结果，或表示失败；应定义明确结果结构或错误通道，不要靠随意把小整数强转指针在所有平台传错误码。

14.14 工作拆分：不只是平均分段

有 n 个任务与 k 个线程，简单分段可令第 t 段边界为 floor(tn/k)至 floor((t+1)n/k)，覆盖全部下标且不重叠。但 tn 乘法本身也可能溢出，真实实现需采用安全计算或限制规模。数学分段公式正确，不免除机器整数范围检查。

若任务成本不同，动态工作队列可能更均衡，但引入队列同步；静态分段开销低，却可能让某个线程拖后腿。评估总完成时间时看最慢分段，不是平均线程耗时。并行算法的结构正确与负载是否均衡，也应分别分析。

练习：8 个任务，前 4 个各耗时 10 单位，后 4 个各 1 单位，两个线程按连续一半分段，理想总完成多久？答案是 max(40,4)=40；若能合理均分重任务与轻任务，可能各约 22。并行度相同，分配方式影响利用率。

14.15 自己做，再看答案

练习一：每个线程有自己的栈，所以把局部数组地址给另一线程一定安全，对吗？

答案：不对。地址可以被其他线程使用，但必须保证数组生命周期覆盖所有访问，并按共享读写情况同步。创建线程的函数先返回，会使数组失效；同时无同步写同一元素还可能数据竞争。

练习二：两个线程只读取创建前已完成初始化、随后不再修改的数组，是否必须为每次读加锁？

答案：在正确创建与发布关系下通常不需要，因为不存在并发冲突写。同步需求来自访问关系，不是“用了线程就所有变量都上锁”。仍需保证数组活到线程使用完。

练习三：程序使用 atomic_flag 或原子整数以后，就一定没有竞态条件吗？

答案：不能。单次操作可以原子，多个操作组成的业务条件仍可能被交错破坏。库存例子中每次 load 和 fetch_sub 都原子，却仍透支。应把完整不变量及其保护范围写出来。

15 同步：什么时候锁门，什么时候等人，怎样避免大家都等着

“加个锁就好了”只解决了问题的一小部分。锁要保护哪几个字段？等待队列为空时能否一直拿着锁？两个对象都要锁时先拿哪把？通知发生在检查前怎么办？这些具体选择决定程序会正确工作，还是偶尔丢数据、一直睡眠，甚至所有线程互相等待。

本章围绕一个有界队列展开。队列最多放 N 个元素，生产者放入，消费者取出。我们分别用互斥锁加条件变量、计数信号量来表达它，让每个同步对象负责什么一目了然。

15.1 算例一：容量 3 的队列走一轮

先不加入并发，确定队列本身怎样保存数据。buffer 有 3 格，下标 0、1、2。head 指下一待取位置，tail 指下一待写位置，count 记有效元素数；索引前进到 2 以后，再加 1 按 3 取余就回到 0。

动作完成后	buffer[0],buffer[1],buffer[2]	head	tail	count	逻辑队列，从待取到最后
初始化	未用，未用，未用	0	0	0	空
放 A	A，未用，未用	0	1	1	A
放 B	A，B，未用	0	2	2	A，B
取 A	旧 A，B，未用	1	2	1	B
放 C	旧 A，B，C	1	0	2	B，C
放 D	D，B，C	1	1	3	B，C，D

取 A 只需要读出该值、推进 head、减少 count，不必把格子清零。表中的“旧 A”表示字节仍可能在那，但它已不属于队列。后来写 D 覆盖旧 A，并不会丢掉一个尚未取出的元素。

现在 head 与 tail 都为 1，队列却是满的。仅用 head==tail 不能区分满与空，本模型用 count==3 判断满、count==0 判断空。如果另选预留一空格的实现，容量与判据都要一起改变，不能混用两套规则。

后面会有生产者负责放入、消费者负责取出。队列满时，再放 E 要等消费者先取 B；但等待期间必须允许消费者修改上面这些状态。这正是锁和条件变量接下来需要解决的任务。

15.2 互斥锁保护的是一条不变量

不变量是在指定观察点应一直成立的关系。例如队列满足 `0<=count<=N`，head 指向下一待取元素，tail 指向下一待写位置，三个字段必须对应同一队列状态。若只锁 count 而不锁 head、tail，其他线程仍可能看到互相不一致的组合。

临界区是访问受保护共享状态的一段操作。线程成功取得同一把锁后，其他遵守规则的线程不能同时进入相应临界区。锁不会神奇地扫描内存阻止所有未加锁的代码；只要某条路径绕过锁，保护就可能失效。

锁还有同步效果：前一个持锁者在解锁之前的相关写入，通过后续成功加锁建立所需可见关系。初学时可把状态统一放在同一锁下，先证清正确，再考虑缩短临界区和拆分锁。多个字段是否能分开保护，应由不变量决定，不由变量名字随意划分。

15.3 条件变量等的是状态，不保存一份永久通知

消费者发现 count=0，当前没东西可取。它需要等待生产者改变状态，但等待时又必须让生产者有机会取得锁。条件等待接口会原子地释放关联锁并进入等待，返回前重新取得锁。

等待时应写 while 而不是 if：`while(count==0) wait(...)`。原因包括允许虚假唤醒、多个等待者竞争，以及被通知后到真正重新持锁之间状态可能又被别人改变。醒来只意味着“值得重新检查”，不意味着“队列一定还有元素”。

条件变量通知不是一个自动累加的任务计数。若通知时没有等待者，通常不会储存一个以后可消费的通知。真正持久存在的事实是受锁保护的 count 等状态。先在同一锁下检查谓词，再用正确等待机制睡眠，才能避开检查与入睡间的窗口。

15.4 算例二：两个消费者为何必须 while 重查

队列初始空，C1、C2 都在等待。生产者放入一个元素，使 count=1，并通知。假设两个消费者都因广播或其他允许的唤醒而返回等待。C1 先重新拿到锁，取走唯一元素，count=0，解锁。C2 随后拿到锁，若只用 if 而不重查，会错误地从空队列取；while 则再次看到 0 并继续等待。

即使只 signal 一次，使用 while 仍是标准设计习惯，因为返回后的谓词是否成立必须以当前状态为准。具体实现可能只唤醒一个，但不能把这当作“从检查到使用期间无人改变”的保证。

条件变量与锁配合使用，而不是拿条件变量替代锁。修改 count、判断 count、更新队列索引都需要约定保护；signal 只负责唤醒，没有替你完成这些访问的互斥。

把两个消费者唤醒后的竞争写成一个确定交错，`L` 为同一队列互斥锁。条件等待返回之前必须先重新取得 L：

时刻	谁持 L	count	C1 的位置	C2 的位置
开始	无人	0	等 not_empty，已释放 L	也在等，已释放 L
生产者入一个项	生产者	1	可被通知	可被广播唤醒
生产者解锁，C1 抢到	C1	1	wait 返回，重查为非空	即使被唤醒，仍在等 L
C1 取走并解锁	无人	0	本次 get 完成	准备取得 L
C2 取得 L	C2	0	已在别处	wait 返回，while 发现又空
C2 再次 wait	无人	0	无关	原子释放 L，再等下一次

两消费者被唤醒：先取得锁者可能已取走唯一元素

发生什么	锁归谁	count	C1	C2
都在等	无	0	等待	等待
放入并广播	生产者	1	可唤醒	可唤醒
C1先取得锁	C1	1	wait返回	等锁
C1取出并解锁	无	0	完成get	等锁
C2终于取得锁	C2	0	已离开	while重查为空
C2再次wait	无	0	无关	释放锁，再等待

通知不预留元素；返回条件等待前，要重新取得同一把锁。

图 43　条件等待的解锁、唤醒、重新争锁与谓词重查

“被唤醒”与“已拿到锁继续执行”之间可以隔着其他线程的一整段临界区。if 只记得过去那次 count=0，不能保护现在取元素所需的 count>0；while 在每次真正持锁后读当前状态。

15.5 用锁与两个条件变量写队列逻辑

生产者拿锁，while(count==N)等待 not_full；有空位后写 buffer[tail]，tail 按 N 取模前进，count 加一；通知 not_empty，再解锁。消费者拿锁，while(count==0)等待 not_empty；取 buffer[head]，head 前进，count 减一；通知 not_full，再解锁。

通知放在锁内还是特定条件下放在锁外，需要结合接口和谓词生命周期设计。本章统一采用修改状态与通知都在同一锁管理范围内的易读形式。不要因为某个简单测试中锁外通知也能运行，就省略状态保护。

15.6 同步协议还要安排关闭与退出

有界队列服务结束时，消费者可能仍在等 not_empty。若生产者直接退出且不再放数据，消费者可能永久等待。需要显式关闭状态 closed，并在同一锁下修改后唤醒相关等待者。消费者条件变为“队列空且未关闭就等”；醒来后若空且已关闭，退出，否则继续取剩余元素。

关闭不等于立刻丢掉已有元素。可以设计排空后退出，也可以设计取消剩余任务，两者都应明确。信号量版本也要安排结束许可或哨兵，不能随意多 post 几次让计数与内容脱节。协议必须覆盖从初始化、正常运行到停止清理全过程。

练习：一个消费者在 while(count==0)里等，主线程只把 closed 设 1 却没有通知，会怎样？答案是它可能没有机会醒来检查 closed。改变使等待条件可能结束的状态后，需要按协议唤醒；谓词也应把 closed 纳入，否则醒来仍会再次等。

15.7 一个包含关闭流程的完整队列例子

下面代码用容量 3 的队列连接主线程与一个消费者。主线程依次放入 1 至 10，再关闭队列；消费者取到排空后退出，把总和写回调用者提供的结果槽。检查 pthread 返回值时使用其返回的错误编号，而不是误以为它一定通过 errno 报告。

```
#include <pthread.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

enum { CAP = 3 };
static struct {
    int data[CAP];
    size_t head, tail, count;
    bool closed;
    pthread_mutex_t lock;
    pthread_cond_t not_empty, not_full;
} q = {
    .lock = PTHREAD_MUTEX_INITIALIZER,
    .not_empty = PTHREAD_COND_INITIALIZER,
    .not_full = PTHREAD_COND_INITIALIZER
};

static void check(int err) {
    if (err != 0) {
        fprintf(stderr, "pthread: %s\n", strerror(err));
        exit(EXIT_FAILURE);
    }
}
static bool put(int value) {
    check(pthread_mutex_lock(&q.lock));
    while (q.count == CAP && !q.closed)
        check(pthread_cond_wait(&q.not_full, &q.lock));
    if (q.closed) {
        check(pthread_mutex_unlock(&q.lock));
        return false;
    }
    q.data[q.tail] = value;
    q.tail = (q.tail + 1) % CAP;
    ++q.count;
    check(pthread_cond_signal(&q.not_empty));
    check(pthread_mutex_unlock(&q.lock));
    return true;
}
static bool get(int *out) {
    check(pthread_mutex_lock(&q.lock));
    while (q.count == 0 && !q.closed)
        check(pthread_cond_wait(&q.not_empty, &q.lock));
    if (q.count == 0) {
        check(pthread_mutex_unlock(&q.lock));
        return false;
    }
    *out = q.data[q.head];
    q.head = (q.head + 1) % CAP;
    --q.count;
    check(pthread_cond_signal(&q.not_full));
    check(pthread_mutex_unlock(&q.lock));
    return true;
}
```

```
static void close_queue(void) {
    check(pthread_mutex_lock(&q.lock));
    q.closed = true;
    check(pthread_cond_broadcast(&q.not_empty));
    check(pthread_cond_broadcast(&q.not_full));
    check(pthread_mutex_unlock(&q.lock));
}
static void *consume(void *arg) {
    long *result = arg;
    long total = 0;
    int value;
    while (get(&value)) total += value;
    *result = total;
    return NULL;
}
int main(void) {
    pthread_t thread;
    long result = 0;
    check(pthread_create(&thread, NULL, consume, &result));
    for (int i = 1; i <= 10; ++i)
        if (!put(i)) exit(EXIT_FAILURE);
    close_queue();
    check(pthread_join(thread, NULL));
    printf("sum=%ld\n", result);
    check(pthread_cond_destroy(&q.not_empty));
    check(pthread_cond_destroy(&q.not_full));
    check(pthread_mutex_destroy(&q.lock));
    return 0;
}
```

逐段解释状态：全局静态对象中未显式指定的普通标量初始为 0，所以 head、tail、count 为 0，closed 为 false。三种 pthread 对象使用各自初始化宏，而不是假设“全零位就是合法锁”。put 等待条件为“满且未关闭”，get 等待条件为“空且未关闭”，关闭能使双方从等待中醒来。

get 在关闭后仍优先取剩余元素，因此实现“排空后退出”。若 count 为 0 且 closed 为真，get 返回 false，消费者结束。close_queue 广播两种条件，是因为关闭会让等满队列的生产者与等空队列的消费者都需要重新检查，而不仅是提醒一个消费者有物品。

result 在 main 栈上，main 直到 join 以后才继续销毁同步对象并返回，因此对象活得够久。消费者只在结束前写一次 result，main 只在成功 join 以后读，存在所需同步关系。这里数值 1 至 10 的和为 55，远小于 C 标准要求 long 至少能表示的上限 2147483647；泛化到任意任务时还要规定范围。

关闭后还有线程在使用队列时，不能提前 destroy 锁或条件变量。本例只有一个消费者且已 join，所以清理时没有剩余使用者。若扩成多个消费者，要等待所有创建成功的线程；若支持线程取消，还需清理处理保证 cond_wait 等路径不遗留锁和状态。本例没有启用取消协议，不应把这段小程序宣称为所有生产环境需求的完整库。

15.8 信号量保存的是可用许可数量

计数信号量有非负许可计数。等待操作取得一份许可，若当前没有则阻塞；发布操作增加许可并可能唤醒等待者。教材常用 P/V 或 wait/post 表示。它可表达“还有多少空槽”“已有多少物品”，与仅表达互斥的锁职责不同。

容量 N 的队列可用 empty 初始 N、full 初始 0、mutex 保护具体队列字段。生产者先 wait(empty)，再锁 mutex 写入，解锁后 post(full)。消费者先 wait(full)，再锁 mutex 取出，解锁后 post(empty)。每一步改变的对象不同：许可说明能否进行，锁保证具体修改不相互冲突。

信号量不一定要求由取得许可的同一线程发布，因此适合跨线程生产消费；互斥锁通常有所有权要求，不能把“别人可以替你 unlock”当通用功能。二元信号量虽然计数只有 0/1，也不因此与所有互斥锁语义完全相同。

信号量许可还可能处于“已预订、未发布”的途中。容量 1 初始 `empty=1,full=0`，生产者 wait(empty)后变 `0,0`，但还没写数据；写入后队列物理上有 1 项，full 仍 0；post(full)后才变 `0,1`。消费者 wait(full)后又是 `0,0`，这时它拿到了取走许可；真正取走并 post(empty)后回 `1,0`。

因此不能在任意机器指令之间都要求 `empty+full=N`。还要加上已经预订而尚未完成转换的在途许可。只有选在完整操作完成等规定观察点，简单计数关系才成立。许可约束与队列 count 不是随时一模一样的变量，读同步算法要问当前是哪一个阶段。

15.9 算例三：为什么不能拿着 mutex 等空槽

队列已满，empty=0。错误生产者先锁 mutex，再 wait(empty)，于是拿着 mutex 睡眠。消费者本来可以取走元素并 post(empty)，但它取元素前也要锁 mutex，被生产者挡住。生产者等空槽，消费者等锁，双方都无法推进。

正确顺序是生产者先等 empty，拿到许可后再锁 mutex；等待资源时不占着别人释放该资源所必需的锁。消费者也相应先等 full 再锁 mutex。这个顺序不是为了“好看”，而是避免一条具体的循环等待。

使用许可后如果线程被取消或中途失败，还要恢复相应计数或完成清理。否则 empty 减了却没有真正放入物品，许可会逐渐丢失。教学算法说明正常路径，工程实现还须设计取消、异常与错误路径。

15.10 死锁的四个条件怎样落到实例上

经典死锁分析常列四项：资源互斥使用；线程持有一些资源时继续等待；资源不能随意被外力抢走；等待关系形成环。它们帮助你找破坏环路的方法，不是背四个词就结束。

两线程转账，T1 先锁账户 A 再锁 B，T2 先锁 B 再锁 A。T1 持 A 等 B，T2 持 B 等 A，形成环。规定所有线程按账户稳定编号从小到大锁，可以破坏循环等待。排序依据必须全局一致，不能每个线程都把“自己的源账户”当第一。

用 trylock 失败后释放已持锁并重试，可能避免某种死锁，但若大家同步重复失败，也可能活锁，即线程都在动作却没有有效进展。饥饿则是某线程长期拿不到资源，其他线程仍可能不断完成。死锁、活锁、饥饿是不同问题。

15.11 算例四：转账的锁顺序与一致性

账户 A 余额 100，B 余额 50，T1 从 A 转 10 到 B，T2 从 B 转 20 到 A。两线程都按编号 A 后 B 加锁。T1 先取得两锁时，检查 A 够钱，写 A=90、B=60，释放；T2 随后取得两锁，写 B=40、A=110。总额始终在每次完整事务观察点为 150。

若只锁扣款账户，另一个线程可能看到扣款已发生、入账未完成的中间状态。是否允许看到它，取决于业务不变量。若查询总额要求一致快照，也要采用与写者协调的策略，不能只让转账线程彼此加锁，而查询线程绕过保护。

如果源与目标恰好同一账户，盲目按两账户分别 lock 会对同一非递归锁加锁两次，可能自锁死。应先处理同一对象情况，或者只获取每把不同的锁一次。边界输入经常暴露锁策略中未写明的假设。

15.12 读写锁、屏障与同步成本

读写锁允许多个读者同时进入，但写者需要独占，适合读多写少且临界区有足够工作的场景。写者公平性与递归规则依实现和配置，不能默认永不饥饿。升级读锁到写锁也不是一般可直接完成的安全动作。

屏障让一组参与者都到达某阶段后再继续。它适合分阶段并行，例如每个线程完成当前迭代的一段数组更新，再一起进入下一轮。参与者数设置错、某线程错误路径提前退出，都可能让其他线程永久等待。

同步本身有成本：锁争用、线程阻塞唤醒、缓存行迁移都会花时间。正确之后应减少无意义共享、缩短持锁时间、批量处理，但不能为了减少锁次数拆开必须原子保持的不变量。性能优化需要保住原来的正确性证明。

15.13 读者优先和写者优先各会让谁等

读写场景中，若只要仍有读者就允许新读者继续进入，连续到来的读者可能使写者长期等不到清空。若写者一来就阻挡全部新读者，写者延迟可能降低，但读者在写压力下又可能长等。公平性是接口与实现策略，不是“用了读写锁”就自动解决。

设计时先问需求：最大写延迟是否有限、读吞吐是否最重要、临界区够不够长抵消读写锁成本。小到只有一次整数读取的临界区，复杂锁管理可能比普通互斥锁更贵。没有测量前，不应因为名字更高级就替换所有锁。

升级锁尤其要谨慎。两个线程各持读锁，均想升级为写锁，若都等待对方释放读锁，可能形成无法推进的状态。某些接口不提供原子升级，应释放读锁、取得写锁后重新检查数据，因为释放期间状态可能改变。重新检查的原则又回到了本章最开始的谓词。

15.14 资源顺序不只包括 mutex

死锁等待图可以包含信号量许可、线程结束、I/O 完成和资源池名额。线程 A 持锁 L 并 join 线程 B，而 B 退出前需要 L，就形成 A 等 B、B 等 L 的环。即使所有 mutex 获取都遵守单一顺序，这种混合等待仍可能死锁。

另一个例子：线程池全部工作线程都在等待各自提交到同一池里的子任务，而没有空闲工作线程执行子任务。没有第二把 mutex，也可能系统不再推进。排查时应问“完成这件等待所需的工作由谁做，它又在等谁”，把依赖图画完整。

因此持锁调用未知回调、等待外部事件或 join 都要小心。未知代码可能再次请求当前锁，或在不可控时间内不返回。能先复制必要状态、解锁后再做外部工作时，可降低耦合，但必须证明复制快照与后续提交仍保持不变量。

15.15 自己做，再看答案

练习一：条件变量 wait 返回后为什么还要重新看 count？

答案：返回只表示重新获得了锁并值得检查，状态可能因其他消费者变化，也可能有虚假唤醒。使用 while 保持“条件不满足就等”，才能让每次真正取元素前都确认非空。

练习二：队列容量 5，初始 empty=5、full=0。成功放两件、取一件后，两个信号量计数应怎样？

答案：按没有并行在途操作的稳定时刻，队列有 1 件，full=1、empty=4。每次放将一份 empty 转为 full，每次取反过来。若观察在某线程已拿许可、尚未完成入队的中途，简单 full+empty=N 不一定作为瞬时恒等式成立，应把在途许可也计入。

练习三：所有线程都按同一全局锁序获取锁，是否就证明程序没有任何并发错误？

答案：只能帮助排除由反向获取这些锁形成的一类循环等待。仍可能有遗漏加锁、条件等待错误、数据生命周期问题、饥饿或其他资源死锁。每个不变量和等待条件都要单独检查。

16 性能与缓存：同样读一个数组，为什么顺序不同就差很多

两个程序做相同次数加法，却可能相差数倍。原因之一是处理器等待数据的时间不同。寄存器、各级缓存、主存和存储设备构成不同容量与延迟的层次，数据靠近处理器时更快，但空间更少。缓存利用程序常有的局部性，把最近或附近要用的数据保留在较快层。

本章是理解系统性能的补充，不把所有处理器实现都塞进课程要求。我们用简单缓存模型手算命中，再用访问次序与依赖链解释优化。速度结论必须带模型和测量条件，不能看见一句“O(n)”就断言两个程序耗时相同。

16.1 时间局部性与空间局部性

时间局部性是刚用过的数据很快又用；例如循环反复更新同一个 sum。空间局部性是用了某位置后，很快访问附近位置；例如顺序扫数组。缓存通常按一整块而非一个字节传输，块也叫缓存行，读 a[i]时可能顺带把后面几个元素带来。

容量大不等于速度快。更靠近 CPU 的小缓存通常延迟低，远层容量大却更慢。一次访问可能先查 L1，未命中查 L2，再查更远层。平均耗时取决于各层命中率与未命中代价，不是把所有延迟简单相加到每次访问上。

写入后的下层状态将在第一条读访问路线完成后单独追踪；读命中规则本身还没有决定写入策略。

16.2 算例一：32 字节小缓存逐次访问

题目：直接映射缓存，4 组，每块 8 字节，初始全空，访问字节地址 0、4、8、0、32、0。

这台缓存只有编号 0 至 3 的四个槽，每槽保存 8 个连续字节。地址先除以 8 得到内存块号，除法余数是块内位置；块号再对 4 取余选择槽位。例如块 0 和块 4 都只能放槽 0，槽里另外保存标记区分它们。用位数表示同一件事，B=8 给块内偏移 3 位，S=4 给索引 2 位。地址 0 属于内存块 0，组 0，标记 0，首次未命中，装入 0 至 7。地址 4 仍在同一块，命中。地址 8 属于块 1，组 1，标记 0，首次未命中。再次 0 命中。地址 32 属于块 4，组号 4 模 4=0，标记为 1，替换组 0 原块 0，因此未命中。最后再访问 0，组 0 里标记已是 1 而非 0，又未命中。六次共 2 次命中、4 次未命中。虽然缓存总容量 32 字节，0 和 32 仍互相冲突，因为它们被限制到同一组。

三类常见未命中可从这个例子延伸：第一次见某块是强制未命中；总工作集超出容量可能有容量未命中；即使总容量尚有余量，同组冲突仍可能造成冲突未命中。分类需与参照模型配合，不能仅凭一次替换就随意归类。

把“查组”和“比标记”写成每次都执行的动作。对本题，块号为 `address//8`，组号为 `block%4`，标记为 `block//4`：

地址	块号/偏移	组/标记	访问前该组	命中?	访问后该组
0	0/0	0/0	无效	否	有效，tag0，数据 0 至 7
4	0/4	0/0	有效，tag0	是	不变
8	1/0	1/0	无效	否	有效，tag0，数据 8 至 15
0	0/0	0/0	有效，tag0	是	不变
32	4/0	0/1	有效，tag0	否	改 tag1，数据 32 至 39
0	0/0	0/0	有效，tag1	否	改回 tag0，数据 0 至 7



图 44　直接映射缓存的四组与同组替换

命中不仅要求组号对，还要求有效位和标记都对。块内偏移只决定命中块里的哪一个字节，不会让地址 4 另占一条缓存行。只看 0 与 32″差 32 字节、刚好总容量“会给直觉，算出同组不同标记才给出实际替换机制。

16.3 地址怎样分成标记、组索引与块内偏移

设块大小 $B=2^b$ 字节，有 $S=2^s$ 组，每组 E 行。地址低 b 位是块内偏移，再上 s 位是组索引，剩余高位是标记。组索引决定到哪一组查，标记用于区分映到同组的不同内存块；有效位说明该行是否装着可用块。直接映射缓存每组只有一行。组相联每组有多行，需在组内找匹配标记，未命中时按替换策略选行。全相联只有一组，多行可容纳任意块。容量公式 $S\times E\times B$ 通常只计算数据容量，不含标记与状态位管理开销。

16.4 写入后，缓存和主存可能暂时不同

在上面六次读结束后，组 0 再次缓存地址 0 至 7 的块，标记 0。补充给定地址 0 的原字节为 7，现在处理器要把它写成 42。下面两种策略分别从同一份“缓存和主存都为 7”的状态开始。

动作	写回缓存中的地址 0	写回模型的主存地址 0	dirty 位
写前	7	7	0
写命中，把新值 42 写入缓存	42	7	1
再读地址 0	返回 42	仍为 7	1
读地址 32，组 0 需要换块	先把旧块写给下层	变为 42	旧块写回后，新块 dirty=0

dirty 意为“缓存副本已改过，下层还未得到新值”。写回策略允许暂缓向下层写；替换脏块时，必须先把旧数据交给下层，不能直接丢掉。这里讨论缓存与主存的模型，不把它等同磁盘持久化。

如果采用直写，写命中时就把 42 同时传给下层，主存不等待替换才更新。是否配写缓冲、什么时候对外完成，实际实现还会有细节；教学模型先明确每次命中都按题设向下层写。

另一个独立问题是“写的块根本不在缓存里”。仍从组 0 保存块 0 的原状态开始，写地址 64，其块号 8 也映到组 0：

- 写分配：先把 64 至 71 这一块取入缓存，再改地址 64。若配写回，缓存字节变新值并标 dirty，下层可稍后更新。
- 不写分配：这次写直接交下层，不为它换入该块；组 0 仍可保留原块 0。常见配合直写的模型会这样处理。

写回/直写决定已缓存数据怎样向下层传播，写分配/不写分配决定写未命中是否先取块，它们回答不同问题。模拟一道题之前，先把题目指定的两项策略写下来；上面的常见组合不是对所有机器的强制规定。

16.5 算例二：按行与按列遍历二维数组

假设 int4 字节，缓存行 64 字节，数组每行恰好 16 个 int。按行读取时，第一次访问某行可能取入 64 字节，随后同一行 15 个元素可利用该缓存行。若缓存足够避免其他干扰，每 16 个元素约一次强制取块。

按列读取时，相邻访问跳过整行 64 字节。若数组很多行，且缓存无法同时保留所有行，读完第一列后再读第二列，先前行可能早已被替换，重复取同样缓存行。数学上仍访问每个元素一次，实际下层传输量却可能更大。

这不是“列循环永远慢”。若矩阵很小全在缓存里、数据原本按列存放、编译器做了变换，或硬件预取与并行掩盖部分代价，结果会变化。正确结论是让最内层沿实际连续存储方向走，通常更有空间局部性，应在目标环境测量确认。

16.6 分块为什么有用

矩阵乘法等算法会重复使用行与列。把数据分成能在缓存中暂留的小块，可以在一块被替换之前完成更多相关计算。分块改变的是复用距离：两次使用同一数据之间经过的其他数据量减少。

选择块大小不能只按“缓存总容量除元素大小”计算，因为还要同时容纳多个矩阵块、程序其他数据和组相联冲突。太小会增加循环与边界处理开销，太大又装不下。先用模型理解，再通过测量选择合理参数，比背一个固定 32 或 64 更可靠。

16.7 算例三：Amdahl 定律怎样限制加速

某程序原耗时 100 秒，其中 80 秒可用 4 个处理单元理想并行，剩 20 秒必须串行。理想新时间为 20+80/4=40 秒，加速比 100/40=2.5，不是 4。

一般可并行比例 p、并行倍数 k，理想加速为$1/((1-p)+p/k)$。当 k 趋向无穷，最大加速仍受 1/(1−p)限制。本例最高 5 倍。实际还要增加线程创建、同步、数据传输与负载不均，常比理想更慢。

这也适用于优化一个热点函数。只占总时间 1% 的代码，即使优化到零，也不可能使整体快一倍。先测量主要时间花在哪里，再选择优化对象，能避免花很多精力精简不重要的循环。

16.8 指令数相同，依赖链也会不同

处理器可在一定条件下重叠执行独立操作。连续 `sum+=a[i]` 让每次加法依赖上一次 sum 的结果，形成依赖链。用多个独立累加器处理不同元素，最后合并，可能增加并行机会。

但浮点数不满足精确结合律，拆累加器会改变加法分组，可能改变结果末位。整数有符号溢出与无符号模运算规则也要先说明。允许怎样的数值差异由任务决定，不能为了测速默认所有重排都合法。

编译器可能已经做循环展开、向量化和别名分析。源码少一行不保证机器指令少，机器指令少也不保证更快。查看生成代码和实际计时，能帮助区分“我觉得快”与“目标机器真的快”。

16.9 算例四：伪共享让独立计数器互相拖累

两个线程 T1、T2 各自只写自己的 counter1、counter2，逻辑上没有同一个变量的数据竞争。但若两者位于同一缓存行，不同核写入时可能不断争取该行的独占所有权，让缓存行在核之间来回迁移。

这叫伪共享，因为程序变量不同，硬件一致性管理单位却是整行。可减少写入频率、使用线程局部累加最后合并，或按实际缓存行布局隔开热点。简单给结构体塞很大填充可能浪费内存并降低别处局部性，应根据访问与测量判断。

区分真共享也很重要：若大家确实写同一个全局计数器，即使填充周围空白，也不能消除对同一位置的竞争。先弄清共享对象，再选择布局措施。

16.10 怎样做一个可信的性能实验

明确输入规模、编译选项、架构和被测代码版本。避免把一次程序启动、文件下载或首次缺页的时间误当内层循环速度。重复测量，记录分布或中位数，观察异常波动；同时检查结果正确，避免编译器把未使用的计算全部删掉，得到一个“极快但没做事”的程序。

热缓存与冷缓存实验回答不同问题。若实际任务每次只处理一个从未读过的大文件，专门预热到全部缓存的成绩可能不相关。若真实任务反复访问同一小表，冷启动延迟也不能代表稳定吞吐。测量场景应与需求对应。

吞吐量是单位时间完成多少工作，延迟是某个任务从开始到结束多长。多线程提高总吞吐，也可能让单个请求等待更久。报告性能时说清指标，不能用一个“更快”覆盖所有体验。

16.11 平均访问时间再算一层

设 L1 命中耗时 1 纳秒，未命中概率 5%，L1 未命中后到下一层取得数据额外耗时 10 纳秒。简化平均时间为 1+0.05×10=1.5 纳秒。这里 10 是额外惩罚，不是把整个访问总耗时再无条件加一遍。题目若给的是未命中总耗时 11，则应写 0.95×1+0.05×11，同样 1.5。

再加入下一层未命中，必须说清概率是总体概率还是在上层未命中的条件概率。假设 L2 在被访问时又有 10% 未命中，需要额外 100 纳秒，平均可写 1+0.05×(10+0.1×100)=2 纳秒。若误把 10% 当所有请求中的比例，就会把慢路径算重。

这种小模型用于培养概率与成本意识，真实 CPU 还有重叠访问、预取、队列和并行访存等机制，未必能用单请求串行式精确预测吞吐。算式的每项应标清“什么时候付这笔时间”，你就能检查模型是否自洽。

16.12 CPE 怎样从测量中读出来

处理数组时可用每元素周期数 CPE 估计稳定段的成本。测多个长度 n，把周期近似写成 T(n)=固定开销 +CPE×n，斜率反映每增加一个元素多花多少周期，截距反映函数调用、设置循环等成本。

小 n 时固定开销占比大，直接用一次总时间除 n 可能高估稳定 CPE；大 n 跨缓存容量后，斜率可能变化，不能用一条直线覆盖所有范围。应按数据层次和输入区间解释曲线，观察折点是否对应缓存、内存带宽或分支行为变化。

例如 T(100)=500 周期，T(200)=800 周期，简化线性斜率(800−500)/(200−100)=3 周期/元素，截距 200 周期。直接 500/100=5 并非稳定斜率。两点只能给一个粗估，更多数据和重复测量才能检查线性假设。

16.13 指针追逐与顺序数组不是同样的 n 次读

链表下一个节点地址要等当前节点被读取后才知道，形成地址依赖链。数组顺序访问的未来地址更容易提前计算，硬件可预取或并行发出多个请求。即使两者都访问 n 个元素，内存等待的重叠能力不同。

节点若分散在许多页，还会增加缓存和 TLB 压力；紧凑数组有更好空间局部性。但链表支持某些已知节点位置的局部插删，不一定每项任务都该改成数组。数据结构选择要结合操作需求、对象大小和访问模式，不能单用一个渐近时间界判性能。

练习：一个算法每步必须先读出下一地址，另一个算法可同时知道接下来 16 个地址，为什么后者可能吞吐更高？答案是它可能重叠多个独立访存、让预取工作；前者被数据依赖串住。总字节数相同，不保证等待能同样被隐藏。

16.14 一次访问究竟搬了多少数据

假设每条缓存行 64 字节，一个 int4 字节。程序只访问大型数组中每 16 个元素的第一个元素，逻辑上每次读取 4 字节，却可能为每次访问取入一整行 64 字节，有效载荷比例仅 1/16。若后面不会使用同一行的其他元素，空间局部性没有被利用。

与之相比，顺序读取 16 个连续 int，第一次取行之后可利用同一行其余元素，64 字节传输带来 64 字节有效数据。两段循环做同样次数的加法，或都标成线性复杂度，并不能看出传输效率差别。要把访问步长换算成字节，再与缓存行大小比较。

如果数据布局是“结构体数组”，只处理某一个很小成员时，可能反复把其他成员一起搬来；“数组的结构体”式分离字段可能提高某种遍历效率。但当任务总要一起读取一个对象的所有字段，紧凑结构体也很自然。布局选择来自主要访问，不是存在一种名字永远更优。

16.15 带宽上限也能做粗估

假设每处理一个元素都必须从主存读取 8 字节，目标内存有效带宽按简化模型为 40GB/s，忽略写回和其他开销，吞吐上限约为$40 \times 10^9 / 8 = 5 \times 10^9$元素每秒。分子是每秒能传输的字节数，分母是处理一个元素必需传输的字节数，相除才得到每秒最多能处理多少元素。若每元素实际要读两份 8 字节并写一份 8 字节，基本传输量至少 24 字节，上限应按 24 重算，写分配还可能增加流量。

这个上限不保证程序能达到，只说明超过它的某个测量若仍声称数据来自主存，就需要检查单位、缓存命中或编译器是否删了工作。GB 与 GiB 也要区分，位每秒与字节每秒更不能混。性能估算的第一步仍是第 01 章学过的单位核对。

16.16 自己做，再看答案

练习一：缓存 8 组、每组 2 行、每行 32 字节，数据容量多少？地址块内偏移和组索引分别几位？

答案：容量 8×2×32=512 字节。32=2^5，偏移 5 位；8=2^3，索引 3 位。两路相联不增加索引位数，而是每组可比较两份标记。

练习二：程序只有一半可并行，即使无限多处理器，理想加速上限是多少？

答案：串行一半仍需原时间 50%，另一半理想趋零，总时间至少原来一半，上限 2 倍。实际同步等开销还会降低收益。

练习三：两个独立字段无数据竞争，能否据此证明它们不会有缓存争用？

答案：不能。硬件按缓存行跟踪一致性，不同字段可能落在同一行形成伪共享。语言正确性和硬件性能是两层问题，应分别检查。

17 综合走读：沿着一个小程序，把前面的知识连起来

单章例子常把环境准备得很干净：给定寄存器，给定地址，只问一条指令。真正的程序需要同时考虑数据类型、布局、调用、分配、I/O 和并发。本章用几段小程序串起来，每次明确输入与环境，再追踪从源代码到可观察结果的路径。

不要企图一眼看穿所有层。先找当前问题最先可能出错的位置：是输入解析就错，还是整数范围不够，还是地址有效但生命周期结束，还是两个线程把状态改乱。沿着实际发生顺序检查，比把所有名词挨个写一遍更有用。

17.1 例一：从数组求和到机器状态

问题：在 LP64、long 为 8 字节、x86-64 System V 环境下，函数 `long sum(const long *a,long n)`求 n 个元素之和。限定 n 非负，a 指向至少 n 个有效元素，所有部分和均不溢出 long。若 a={4,−2,7}、n=3，结果多少，循环汇编如何走？

先看 C 逻辑：sum 初始 0，i 从 0 到 n−1，每轮加 a[i]。部分和依次 4、2、9，结果 9。输入约束中的“所有部分和可表示”很重要，最终结果在范围内并不排除中间先溢出再相消。

先把清零和递增对应到已经学过的位操作：xor 把相同两位异或为 0，因此 xorq %rax,%rax 将 rax 全部 64 位清零；incq %rdx 把 rdx 加 1。其余地址、cmp 和跳转沿用 04、05 的规则。教学汇编为：

```
sum:
    xorq %rax, %rax
    xorq %rdx, %rdx
.Ltest:
    cmpq %rsi, %rdx
    jge .Ldone
    addq (%rdi,%rdx,8), %rax
    incq %rdx
    jmp .Ltest
.Ldone:
    ret
```

rdi 保存数组首地址 B，rsi 保存 n，rdx 保存 i，rax 保存部分和。第一次比较 0−3，jge 不跳；地址 B+0，加载 4，加到 rax；i 变 1。第二轮地址 B+8，加载−2；第三轮地址 B+16，加载 7。i 变 3 后，比较 3−3，ZF=1，jge 退出。

不变量可以写成：“每次进入.Ltest 时，i 在 0 至 n 之间，rax 等于前 i 个元素的和。”初始 i=0，空和为 0；每轮把第 i 个元素加入，再让 i 加一，关系保持；退出时 i=n，rax 就是全部 n 个元素和。这样不仅算了一个输入，也解释了为什么循环结构实现目标。

若 n=0，第一次测试就退出，允许 a 为空吗？要看函数是否真的不解引用以及接口约定。此代码零轮不访存，C 版本若没有其他指针运算，可设计允许空指针与零长度组合；不能自动把这个结论推广到函数里还先做了 a+n 或其他操作的版本。写接口时明确零长度规则。

现在把数组基址固定为 `0x1000`，不把 B 继续留成抽象字母。内存中三个 long 占`[0x1000,0x1018)`，每项 8 字节：

循环轮次	cmp 时 rdx/rax	jge	这次内存地址/读值	add 后 rax	inc 后 rdx
1	0/0	不跳	0x1000，4	4	1
2	1/4	不跳	0x1008，−2	2	2
3	2/2	不跳	0x1010，7	9	3
退出测试	3/9	跳	没有访存	9	3



图 45　数组字节位置、每轮有效地址和累加器

第二行读入的−2 是八字节补码 `0xFFFFFFFFFFFFFFFE`。按完整 64 位有符号意义相加 4+(−2)=2，与只把一个字节 FE 当 254 不同。最后一次 cmp 结果为 0，jge 决定到 ret；ret 自己还要从栈恢复调用者指令位置，9 则已经在 rax，不由 ret 另算。

17.2 例二：把字节文件读成数组之前，要先决定格式

现在真正接起一次读取任务。规定文件前 4 字节是大端无符号计数 n，随后正好有 n 个四字节大端无符号数；本应用至多接收一百万项，额外尾部字节算格式错误。只要总和，不需要把全部元素留下，因此使用一个四字节缓冲反复读。

沿同一份输入走完整条路线：

```
00 00 00 03 | 00 00 00 04 | 00 00 00 02 | 00 00 00 07
  计数3         第0项4         第1项2         第2项7
```

刚完成的读取	四字节缓冲	i 与累计和	下一步要做什么
头部	00 00 00 03	n=3，total=0	确认 n 未超过应用上限
第 0 项	00 00 00 04	i=0，加得 4	还需两项
第 1 项	00 00 00 02	i=1，加得 6	还需一项
第 2 项	00 00 00 07	i=2，加得 13	项数已满足，再查文件结束
结束检查	read 返回 0	总和仍 13	输出 13，关闭描述符

每个四字节字段都先读完整，再解码。read4 内部若第一次只得到 1 字节，done 变 1，下一次仍在同一个字段内，从 b+1 继续请求 3 字节；不能把这次短读当作新元素。字段未齐就遇到 EOF，应报格式截断，不把旧缓冲字节混进去。下面程序把这些出口与正常路径同时写出：

```c
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <fcntl.h>
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* 1: four bytes; 0: EOF before any byte; -2: partial field; -1: error. */
static int read4(int fd, unsigned char bytes[4]) {
    size_t done = 0;
    while (done < 4) {
        ssize_t r = read(fd, bytes + done, 4 - done);
        if (r < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (r == 0) return done == 0 ? 0 : -2;
        done += (size_t)r;
    }
    return 1;
}
static uint32_t be32(const unsigned char b[4]) {
    return ((uint32_t)b[0] << 24) | ((uint32_t)b[1] << 16)
         | ((uint32_t)b[2] << 8) | (uint32_t)b[3];
}
static int reject(int fd, const char *message) {
    fputs(message, stderr);
    if (close(fd) == -1) perror("close");
    return EXIT_FAILURE;
}
int main(int argc, char **argv) {
    if (argc != 2) {
        fputs("usage: record-sum file\n", stderr);
        return EXIT_FAILURE;
    }
    int fd = open(argv[1], O_RDONLY);
    if (fd == -1) { perror("open"); return EXIT_FAILURE; }
    unsigned char b[4];
    if (read4(fd, b) != 1)
        return reject(fd, "missing or incomplete header\n");
    uint32_t n = be32(b);
    if (n > 1000000u)
        return reject(fd, "too many elements\n");
    uint64_t total = 0;
    for (uint32_t i = 0; i < n; ++i) {
        if (read4(fd, b) != 1)
            return reject(fd, "missing or incomplete element\n");
        total += be32(b);
    }
    unsigned char extra;
    ssize_t r;
    do { r = read(fd, &extra, 1); } while (r < 0 && errno == EINTR);
    if (r < 0) return reject(fd, "cannot check file end\n");
    if (r > 0) return reject(fd, "unexpected trailing data\n");
    int ok = printf("%" PRIu64 "\n", total) >= 0;
    if (close(fd) == -1) { perror("close"); ok = 0; }
    if (fflush(stdout) == EOF) ok = 0;
    return ok ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

完整源文件可在支持 POSIX 的系统用 `cc -std=c17 -Wall -Wextra cs-aha-record-sum.c -o record-sum` 编译，运行时给文件路径。PRIu64 来自 inttypes.h，为 uint64_t 提供匹配的输出格式，不假定它必定等同 unsigned long。

read4 只负责固定四字节字段，返回 1 表示完整、0 表示字段开始就结束、−2 表示只拿到一部分、−1 表示真实读取错误；主循环只接受 1。be32 中先转 uint32_t 再移位，避免按窄类型或有符号提升误读。reject 集中关闭已经打开的描述符，然后返回失败，不会继续使用不完整数据。

累计类型选 uint64_t 有数值依据：即使把应用上限放宽到 n 最大为$2^{32}-1$，每项最大也为$2^{32}-1$，总和至多$(2^{32}-1)^2 < 2^{64}$。本例一百万项的限制更小，所以总和不溢出。若修改格式或允许持续无限输入，需要重算上界，不能只凭“64 位”断言安全。

文件字段固定四字节，本机 long 却在本书环境中占八字节，因此不能直接 fread 成 long 数组；直接读成 uint32_t 也不会自动改变端序。若把头部 00 00 00 03 在小端主机按本机整数解释，会得到 0x03000000，而不是 3。这里先按格式解码，再决定计数、范围和资源需求。

17.3 例三：程序从源文件到输出的一条完整链

沿刚才的 record-sum 继续看运行过程。源文件中的头文件让编译器知道 open、read、close 和 printf 等接口；编译器为 main、read4、be32 生成目标机器代码，汇编器把代码与必要记录放入目标文件，链接器再解析程序对系统库的外部引用。把程序拆成多个.c 时，也按 08 的声明、定义和重定位规则相接。

程序启动时，加载器建立可执行代码、静态数据等映射，必要时由动态链接器处理共享库。main 取得命令行中的文件路径，open 成功后得到一个描述符。每次 read 请求内核把文件字节放进 b，返回的 r 决定本次有效部分；read4 在用户代码中累计 done，不是让内核猜这一字段共有四字节。

be32 只在四字节完整时读取 b 并计算整数，total 随后累加解码后的值。n 和 total 分别是计数与数值，不能把字节偏移、记录项数、累计和写进同一列状态。局部缓冲及累计变量可能在栈或寄存器中，具体分配由编译器决定；本程序不需要为全部 n 项 malloc 一份数组，峰值额外缓冲保持固定。

普通访存仍经过 11 的翻译和权限规则，缓存只影响数据取得的路径与成本，不会替代码补格式校验。数据齐全、尾部验证通过后，printf 将数值按匹配格式转成字符；库可能先缓冲，fflush 再把待输出内容交给底层。close 撤销输入文件引用，不清空局部 b；main 结束才结束这次自动对象的生命。

这条路线给出不同失败的先后关系：输入文件根本打不开，就不会开始字段解码；头部不足四字节，就不能根据 n 安排循环；端序读错，会在资源需求和循环次数之前先污染 n；输出失败则发生在计算完成之后。先找第一条不满足条件的操作，再研究更晚的症状。

17.4 例四：把求和分给两个线程，正确性分几层检查

问题：数组长度 8，内容 1 至 8，两线程各负责一半，主线程取总和。先规定数组在创建前初始化、整个计算期间只读，参数对象与结果槽在 join 全部结束前持续存在。这样输入读取没有并发写冲突。

参数分别记录 `begin=0,end=4` 与 `begin=4,end=8`，区间采用左闭右开，即包含 begin、不含 end。两个区间不重叠，并集正好覆盖 0 至 7，没有漏掉中点也没重复。T1 局部和 10，T2 局部和 26，各写自己结果槽；主线程在两个 join 成功后读结果并加得 36。

若线程创建第二个失败，主线程不能假装两个都存在而直接 join 未初始化的线程句柄。它可以等待已创建的第一个，并在当前线程计算剩余段，或报告失败并按设计清理。错误路径的参数和数组生命周期仍要保证。

若把每项结果都更新同一个全局 total，必须同步，性能也可能被锁热点限制。独立局部和减少共享，但不能忽视数值范围；对浮点求和，分组变化可能改变舍入结果。并行化至少要检查覆盖、生命周期、同步与数值语义，缺哪层都不能用“线程跑完了”证明正确。

17.5 例五：一个看起来只错一行的接口设计

函数接收 `char *buf,int len`，内部做 `malloc(len+1)`再复制 len 字节并加终止符。乍看似乎考虑了字符串结尾。若 len<0，转换到 size_t 可能成巨大值；若 len 接近 INT_MAX，`len+1` 先在 int 里有符号溢出；若源 buf 实际不足 len 字节，即使目标分配够也会越界读。

改进从约定开始：长度用能表达对象大小的 size_t；要求源缓冲至少有 len 字节；检查 len==SIZE_MAX 时无法加一；分配 len+1 并检查返回；复制后写 target[len]='\0'；明确调用者释放。若目标是文本，还要考虑内部是否允许零字节、编码是否合法；若只是二进制数据，附加零并不使它自动成为可安全按字符串处理的文本。

这个例子说明"加一个检查"往往不够。要把所有操作按发生顺序写：参数解释、长度运算、分配、读取、写入、生命周期。检查必须在危险运算之前，并覆盖读取端和写入端。

17.6 做综合题时怎样写纸面答案

开头列环境：位宽、端序、ABI、类型、有无并发。接着列输入对象与有效范围，再写关键状态表。对于分支题，给出能覆盖不同出口的测试；对于并发题，给出导致问题的具体交错；对于公式计算，写中间单位与范围。

最后核对三个边界：空输入、最小或最大合法值、与别名或共享有关的特殊关系。它们不保证穷尽所有错误，却经常能发现"一般情况算对了"的漏洞。答案若依赖题目未给出的条件，应明确写假设，不要悄悄把未知当零或把失败当不存在。

17.7 例六：一步步检查一个可扩容读入器

问题：读入未知长度文件，最终返回全部字节及长度。初始容量为 4，已用 0。每次在剩余空间读数据，空间耗尽就扩容。输入共有 10 字节，假设每次 read 恰好填满可用部分。

先分配 4，读 4，used=4、capacity=4；扩到 8，保留原 4，再读 4，used=8；扩到 16，再读最多 8，实际读 2，used=10；继续读得到 0 才确认 EOF。短读 2 本身不是必须结束的证据，所以仍要做下一次读取或按已知协议判断。

扩容前检查 capacity 翻倍是否超出 SIZE_MAX，realloc 使用临时指针，失败时仍能释放旧对象。若 used<=capacity 不变量被破坏，`capacity-used` 会无符号下溢，所以每次更新都应保持这一关系。若文件是文本，还要为终止零预留空间；若是二进制，返回明确长度，不依赖零字节。

这个程序同时使用动态分配、I/O、整数边界和对象生命周期。检查清单按执行顺序展开：容量运算是否安全，分配是否成功，读取返回值是否正确处理，已用量是否只加实际读到字节，最终由谁释放。每一条都有明确失败输入，不是泛泛的"注意安全"。

17.8 例七：一份输出顺序题怎样列约束

父 fork 一个孩子，子依次输出 A、B 后退出；父先输出 C，再 wait 孩子，最后输出 D。假设每个字母输出作为一个原子事件，无缓冲复制问题。约束是子内 A 在 B 前，父内 C 在 D 前，wait 使 B 在 D 前；C 与 A、B 之间没有额外顺序。

允许的整体顺序有 CABD、ACBD、ABCD。D 必须最后，因为 C 与 B 均在它前；A 必须在 B 前。写出 ACBD 这类顺序时，要按字符真正合并，不能漏事件。若父把 wait 移到 C 之前，则只剩 ABCD。

真实 printf 多字节文本可能需要额外输出原子性与缓冲假设，但纸面题可明确采用本模型。这样你知道哪些是程序证明出来的边，哪些是为了聚焦调度而采用的简化条件。

17.9 例八：先找不变量，再选同步对象

设计一个后台任务队列，约定已提交数 submitted、已完成数 finished、队列中的任务和正在处理任务共同满足数量关系。若只把两个计数分别设原子，队列插入与计数增加仍可能被观察在不同时间，查询者可能拿到不一致快照。

最易审查的第一版是用同一锁保护队列与相关计数，在同一临界区完成提交或完成状态更新；消费者等"队列非空或已关闭"的谓词。待正确以后，若查询很频繁再研究快照或分离计数，但不能无依据拆掉关系。

练习：主线程看到 finished==submitted 是否一定能销毁所有任务对象？答案要看这两个值的发布与读取关系、是否仍有人可继续提交、是否还存在借用者。单个数值相等不自动证明系统已静止；需要关闭提交并等待所有工作和借用结束的完整协议。

17.10 自己做，再看答案

练习一：求和函数最终结果为 5，是否足以证明有符号 long 所有中间计算都没溢出？

答案：不足。可能先累加到超过 LONG_MAX，后面又加负数使数学最终和回到 5，但 C 的溢出已经发生。需要保证每个部分和可表示，或采用明确的检测、更宽精度、任意精度等设计。

练习二：把长度 n 划分为两个左闭右开区间，能否用`[0,n/2]`与`[n/2,n)`？

答案：前一个写成右端包含，会把中点算两次。统一左闭右开应是`[0,n/2)`与`[n/2,n)`。取 n=5，前段 0、1，后段 2、3、4，覆盖正好一次。

练习三：程序对同一个文件先 fread 再 read，发现第二次像跳过了一段。第一怀疑应是什么？

答案：先检查 stdio 是否预读到用户缓冲，导致底层共享偏移已前移。不要立刻归咎于磁盘丢字节。把 FILE* 缓冲、描述符、打开文件描述的偏移分别画出来，再检查混用接口的同步规则。

18 机器表示工坊：从题面条件一步步算到答案

本章用整数、浮点、寻址和控制流题目检验前面的机制。每题先固定表示规则和输入，再给出中间状态与理由。第一次可以沿过程演算，第二次换一个能触发不同分支的输入，检查同一方法是否仍成立。

算题前统一四个约定：位宽必须固定；十六进制每位对应四位二进制；浮点特殊值按题目给定格式；汇编按 AT&T 源在前、目的在后。最常见的错并不复杂，往往是中途忘了其中一个约定。

18.1 工坊一：限定三个 1，怎么找最大与最小补码

问题：八位补码，位串恰含三个 1、五个 0，求最大值、最小值、绝对值最小值对应位串。

最大值先争取符号位 0，这样结果非负。剩余三个 1 放在最高的正权重位置 64、32、16，得到 `01110000`，值 112。把一个 1 从较高正权位移到较低位只会减少数值，因此这确是符号 0 组中的最大；符号 1 组全为负，不可能超过它。

最小值要让符号位 1，先获得−128；剩余两个 1 放在最小正权重 1、2，得到 `10000011`，值−125。若把它们放高位，会抵消更多负值，反而变大。

绝对值最小要比较正负两类的最近零候选。符号 0 时，三个 1 放 1、2、4，得到 `00000111`，值 7。符号 1 时，为了靠近零，应把剩余两个 1 放 64、32，得到 `11100000`，值−32。两者绝对值 7 与 32，故答案是 `00000111`。注意“最小值”与“绝对值最小”是两个优化目标，不能用同一位串回答。

18.2 工坊二：十六进制补码减法，不靠猜符号

问题：16 位 X=`0x7FF0`，Y=`0x0030`，求机器加法结果，判断有符号与无符号是否溢出。

数学和为 `0x8020`，小于 `0x10000`，所以没有无符号进位。X 与 Y 按补码都是正数，和位串最高位却为 1，发生有符号溢出。若按补码解释结果，`0x8020-0x10000=-32736`，这与数学和 32800 不同。

再做 X−Y：`0x7FC0`，数学值 32704，可表示，没发生有符号溢出。不要仅因为结果接近符号边界就认定危险；应检查实际数学范围。对 C 源代码中的有符号加法，第一种输入不可当作定义良好的截断计算，而汇编位串题仍可明确问低 16 位是什么。

18.3 工坊三：两种 FP8，分别把范围与精度算出来

采用 IEEE 风格，E4M3 有 4 指数位、3 小数位，bias=7；E5M2 有 5 指数位、2 小数位，bias=15。E4M3 最大次正规位串为 `0|0000|111`，值7×2^{-9}。E5M2 最大正常位串为 `0|11110|11`，有效数 1.75，实际指数 15，值 57344。

最小正次正规值分别为 E4M3 的2^{-9}和 E5M2 的2^{-16}。指数位更多的 E5M2 能把非零值延伸到更小数量级，也覆盖更大的有限值；E4M3 在相同正常数量级里有更多有效位、间距更细。

例如区间[1,2)内，E4M3 可表示 1、1.125、1.25……1.875，共 8 格；E5M2 为 1、1.25、1.5、1.75，共 4 格。固定 8 位都只有 256 种位串，浮点没有凭空增加编码数量，只把它们按不均匀间距分配，并留一些给零、无穷和 NaN。问“定点可表示数的个数多还是浮点多”时，还必须明确特殊编码算不算数值、正负零算几个数，不能只比较范围就回答个数。

作业中的梯度是训练时用于调整参数的导数值，激活值是网络计算的中间数值。本题只需把它们视为需要保存的实数，不要求先学完神经网络。数值可能特别大或特别小时，先比较可表示范围；范围稳定时，再比较同一数量级中的间距。于是大范围梯度倾向选 E5M2，范围稳定、希望保留更多有效位的激活倾向选 E4M3。真正的无穷不可能由任何有限格式恢复为有限准确值，换格式只能改善尚在有限范围内的表示余量。

INT8 保存的则是整数码，怎样用它表示实数？先给定一个已经校准的正比例因子 s。本例取 s=0.02，并明确采用−127 到 127 的对称码域，−128 留不用；这是一种对称量化约定，不把它当作所有部署唯一的编码。整数码 q 解码为近似值 q×s，所以范围为−2.54 至 2.54，相邻间距 0.02。

保存实数 x 时，先算 x/s，再按本例规定的就近取偶舍入为整数；超出码域时限制到最近端点。例如 x=0.50 得到码 25，解码仍为 0.50；x=1.23 得到 61.5，按取偶选 62，解码为 1.24，误差+0.01；x=3 超出范围，码限制到 127，只能还原 2.54，误差就显著增大。限制到端点不等于绕回成负数。

因此“校准比例因子”的工作，是让预期数值范围与这个固定网格相配。已完成训练、范围已估计、任务允许该量化误差时，INT8 配缩放是一种合理部署选择。它没有浮点那种逐个数改变指数的能力；E5M2 和 E4M3 也各有范围与精度限制。题目给出的场景决定优先照顾哪一项，不能离开数值范围、误差容许和具体格式无条件排名。

18.4 工坊四：阶码全零时别减 bias

问题：1 符号、5 指数、2 小数格式，位串 `0|00000|10`，求数值。

指数全零，必须用次正规规则，实际指数 E=1−15=−14；小数域 F=2，所以有效数为 2/4=1/2，值2^{-15}。如果误用 E=0−15，再误加隐藏 1，得到的值既不符合次正规定义，也无法与相邻格点连续衔接。

核对最小步长为2^{-16}，当前 F=2，所以正好两个步长，即2^{-15}。用“域公式”和“步长乘整数”两条路线检查，可以迅速发现把隐藏 1 放错的错误。

18.5 工坊五：三条 lea 的系数核对

要求计算 32x+4y+2z+w，四参在 rdi、rsi、rdx、rcx。候选指令：

```
leaq (%rsi,%rdi,8), %rax
leaq (%rcx,%rdx,2), %rdx
leaq (%rdx,%rax,4), %rax
```

第一步产生 8x+y；第二步用旧 rdx 中的 z 产生 w+2z，覆盖 rdx；第三步把第二步结果加四倍第一步结果，得到目标。特别检查第二条使用 rdx 旧值，第三条使用 rdx 新值，不能在一张静态代数表里把它们都写成 z。

取四组基向量输入可分别验证系数：只令 x=1 其余 0，返回 32；只令 y=1 返回 4；只令 z=1 返回 2；只令 w=1 返回 1。再加混合输入检查寄存器覆盖是否破坏关系。随机测试能发现错，但这段线性表达式的符号展开更直接地解释为什么正确。

18.6 工坊六：条件传送选哪一个候选值

函数先算 hi=2x+y、lo=x+2y，再在 x>y 时返回 hi，否则 lo。令 rax 先放 hi，rdx 放 lo，比较 `cmpq %rsi,%rdi`。我们需要在 x≤y 时把 lo 送进 rax，所以条件传送为 `cmovle %rdx,%rax`。

取 x=5、y=2：hi12、lo9，比较大于，传送不发生，返回 12。取 x=2、y=5：hi9、lo12，比较小于，传送发生，返回 12。取 x=y=3：两候选都为 9。因此本函数里 cmovle 与 cmovl 都能给相同可观察返回值，后者在相等时保留 hi 也无妨。不能要求机器无条件保留源码 if 的分支选择；编译器只需在合法输入上保持可观察行为。若另一个函数在相等时两个候选值不同，才可用相等输入区分 l 与 le 是否实现所需语义。

若 hi、lo 的计算可能溢出或有副作用，需要回到源程序规则。这里题目中的候选都明确先算，条件传送只替换选择阶段；若原源码只允许执行其中一个危险分支，就不能照抄这个技巧。

18.7 工坊七：递归保存了什么，别只数 call

函数 `f(n)` 在 n≤1 返回 1，否则算 `n+f(n−2)`。取 n=5。下行状态为 f5 等待“回来加 5”、f3 等待“回来加 3”、f1 直接 1。回程 f3=4，f5=9。最大同时三层活动，总调用也是三次；把递归改成每层调用两次后，这两个数量可能明显分开。

若用 rbx 保存当前 n，函数必须先保存调用者原 rbx，并在返回前恢复。调用 f(n−2)前还需满足栈对齐。只写“把 n 放 rbx，call，再 add”却不保存 rbx，破坏调用约定，即使本次返回值 9，也可能使上层后续计算出错。

18.8 工坊八：从结构体偏移认出访问对象

常见 LP64 下，`struct R{char c; long x; int y;};`。c 偏移 0，x 需 8 字节对齐，从 8 占到 15；y 从 16 占到 19；总长向 8 对齐为 24。若数组 r 首地址 B，`r[i].y` 地址为 B+24i+16。

x86 寻址比例只支持 1、2、4、8，24i 可以先算 3i，再乘 8。例如 lea 产生 i+2i 到临时寄存器，最终以比例 8 访问。看到两次 lea 不等于数组有两层；它可能只是在凑一个 24 字节结构体步长。

若某位同学把结构体总长算 20，会在 i=0 时仍访问正确 y，但 i=1 起就错位。测试不能只测首元素，至少选第二个与最后一个合法元素，才能检验步长。

18.9 工坊九：读懂两个位置都有测试的 scan

这里的 scan 一开始令 sum=0、i=0；循环顶部检查 i<n，不满足就结束；只有 i 为偶数时才把 i 加入 sum；然后 i 加一；循环底部检查 sum<limit，满足才回顶部。这个控制流不等于在入口同时检查 `i<n && sum<limit`，因为底部的 limit 测试发生在一轮工作之后。

等价而易读的结构是：先检查顶部 n 条件，执行偶数累加和 i 递增，再在 sum>=limit 时 break。取 n=10、limit=6，i0 累加 0，sum0；i1 不加；i2 加 2，sum2；i3 不加；i4 加 4，sum6；底部比较不再小于 6，返回 6。实际被加下标为 0、2、4。

边界 limit=0 很有辨别力：若 n>0，原汇编仍先执行 i=0 这一轮，再在底部退出。虽然本例 sum 仍 0，看上去与零轮输出同值，副作用或其他初值改动可能区分它们。还原 C 时保持测试位置，不能仅凭最终数值一样就擅自前移条件。

顶部 `cmpq %rdi,%rdx; jge` 是有符号比较 i 与 n；底部 `cmpq %rsi,%rax; jl` 是有符号比较 sum 与 limit。test 最低位用 jne 跳过奇数累加，与数值是否负是两回事；本题 i 从 0 递增，在合法小输入里无需引入负下标。

把刚才的动作写成 C，先做顶部测试，再做一轮工作，最后处理底部测试。只讨论 n 和 limit 及累加结果均在可表示范围内的小输入：

```
long scan(long n, long limit) {
    long sum = 0, i = 0;
    while (i < n) {
        if ((i & 1) == 0) sum += i;
        ++i;
        if (sum >= limit) break;
    }
    return sum;
}
```

在 limit=0、n>0 时，它仍进入一次循环，执行 i=0 的偶数累加及 i 递增，才走 break；把 sum<limit 塞进入口条件会改掉这条执行路线。源码表达的测试位置要与汇编的两次比较一致，不能只因最后都返回 0 就认为控制流相同。

18.10 工坊十：为有缺口的 switch 填完整表

先给出完整函数，后面的返回值都由它算出。x 是要参与运算的数，n 决定走哪个分支；本例只代入不会发生有符号溢出的小整数。

```
long sw(long x, long n) {
    long result = x;
    switch (n) {
    case 0: result = x + 5; break;
    case 2:
    case 3: result = x + x; break;
    case 4: result = x - 2; break;
    default: result = 1;
    }
    return result;
}
```

case 为 0、2、3、4，1 走 default。范围检查先比较 n 与 4，使用无符号 ja 排除大于 4 的位串，也把负 long 当成很大无符号数而排除。表中必须有五项，按 0 至 4 顺序：L0、Ldef、L23、L23、L4。case2 与 3 共用同一目标，case1 虽无显式分支仍必须占一个表格位置。

n	查表编号	目标标签	实际计算，取 x=7	返回值
0	0	L0	7+5	12
1	1	Ldef	直接写入 1	1
2	2	L23	7+7	14

n	查表编号	目标标签	实际计算，取 x=7	返回值
3	3	L23	7+7	14
4	4	L4	7−2	5
−1 或 5	不查表	Ldef	范围检查直接跳到默认分支	1

例如 n=3 时，先确认它的无符号值不超过 4，再算表首地址加 3*8 字节，读取第 3 号槽保存的 L23 地址并跳过去。L23 用 `lea (%rdi,%rdi),%rax` 算 x+x；这里 `lea` 只计算数值，不读取地址 x+x 处的内存。rax 此时已经是 14，ret 恢复调用者的后续执行位置。标签名的 23 表示 case2 与 3 共用它，不能拿 23 当表下标。



图 46　教师原页：跳转表的构建与访问，PDF 第 39 页。

这张原图对应的是另一个 0 至 6 示例，但布局方法相同。右侧每条.quad 分配八字节目标地址；左侧间接跳转先计算表首+8×下标，再读取该位置保存的目标地址，最后跳转。它不是跳到“表项本身的地址”去执行数据。0 与 4 都指 L8，5 与 6 都指 L7，说明不同 case 可共享代码。

本工坊取 x=7。n0 返回 12；n1 走 default 返回 1；n2 与 n3 返回 14；n4 返回 5；n=−1 或 5 被范围检查挡到 default。选 n1 能发现漏填缺口，选 n−1 能发现只作有符号上界检查而遗漏下界，选 n4 能发现表长少一项。

18.11 工坊十一：把老师的递归位计数完整走一遍

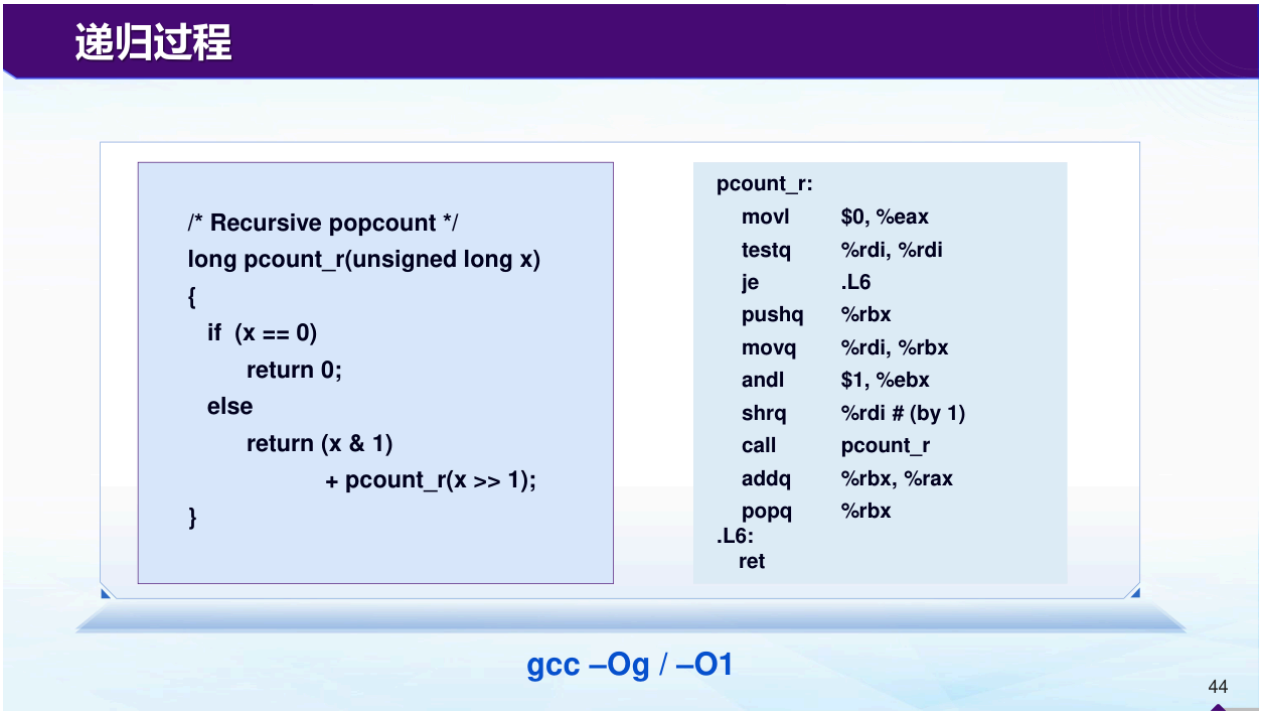


图 47　教师原页：递归 popcount，PDF 第 44 页。

图左每次取 x&1，统计当前最低位；再对无符号 x 逻辑右移 1，统计剩余高位。右侧先把返回值初始化 0，x 为 0 就直接 ret；非零时保存 rbx，把最低位留在 ebx 中，右移 rdi 后递归；回程把 rbx 加到 rax，恢复 rbx 再 ret。

取 x=13，即 1101_2。各层当前位依次 1、0、1、1，剩余 x 依次 6、3、1、0。最深 0 返回 0；x1 层得 1；x3 层得 2；x6 层加 0 仍 2；x13 层加 1 得 3。总共四个 1 位候选位置，但其中一个位为 0，所以结果 3。栈最深含 x13、6、3、1、0 五次活动；非零层都需要保存跨递归的低位。

图中 `andl $1,%ebx` 写 32 位子寄存器，会清高 32 位，rbx 最终只为 0 或 1，适合随后 64 位 add。源参数是 unsigned long，`shrq` 用逻辑右移，高位补 0。若误把类型改为有符号负数并用算术右移，可能永远保留最高 1 位，不再按同样方式到 0，终止分析就变了。

同一段汇编也可以直接逐行读，标签 `.Lzero` 是零输入的共同返回点：

```
pcount_r:
    movl $0, %eax
    testq %rdi, %rdi
    je .Lzero
    pushq %rbx
    movq %rdi, %rbx
    andl $1, %ebx
    shrq $1, %rdi
    call pcount_r
    addq %rbx, %rax
    popq %rbx
.Lzero:
    ret
```

`movl $0,%eax` 先把全 rax 清 0；test 只设标志；x 为 0 走共同 ret，不执行 push。非零层先保存旧 rbx，再把当前最低位留在 rbx；右移只改变下一层参数 rdi。call 回来后 rax 是高位部分的计数，add 才补上当前层的那一位，pop 再恢复上一层留着的位。不能把 pop 提前到 add 前，否则会拿上一层的低位代替本层低位。

把该教师递归例子的每个保存地址具体化。设最外层入口 `rsp=0x7FF8`，原 `rbx=99`，各非零层执行 push rbx 后再 call，每次共让下一层入口降低 16 字节：

当前 x	入口 rsp	push 保存在哪/保存什么	本层 rbx 留的低位	右移后给下一层
13	0x7FF8	0x7FF0 保存 99	1	6
6	0x7FE8	0x7FE0 保存 1	0	3
3	0x7FD8	0x7FD0 保存 0	1	1
1	0x7FC8	0x7FC0 保存 1	1	0
0	0x7FB8	基础情况不 push	保持调用者的 1	直接返回 rax=0

x=0 的 ret 先取 `0x7FB8` 处返回地址，使 rsp 回到 `0x7FC0`。此后逐层：

回到哪层	下层返回 rax	加本层 rbx 后 rax	pop 恢复 rbx	ret 之后 rsp
x=1	0	1	1（还给 x=3 层）	0x7FD0
x=3	1	2	0（还给 x=6 层）	0x7FE0
x=6	2	2	1（还给 x=13 层）	0x7FF0
x=13	2	3	99（还给外部调用者）	0x8000



图 48　递归位计数的待加低位与返回值

表里保存的 99 与每层 0/1 的作用不同：99 是外部调用者原有的值，必须最终还回去；0/1 是当前层跨递归仍要用的本地数据。基础层不需要额外 push，不能机械给五层都减同样 16 字节。每一非零层 call 前 rsp 按 16 对齐，返回后 pop 与 ret 也各恢复各自的 8 字节。

18.12 工坊十二：为什么 jl 要看 SF 异或 OF

把比较看成 a−b。若没有有符号溢出，OF=0，结果最高位 SF 就能表示差是否负，因此 SF 异或 OF 等于 SF。若发生溢出，截断结果的符号与数学差相反，需要翻转 SF；OF=1 恰好完成翻转。

用八位边界各走一次。a=−128、b=1，数学差−129，小于 0 但不能表示；低八位为 127，SF=0，OF=1，异或为 1，正确判小于。a=127、b=−1，数学差 128，大于 0 但不能表示；低八位为−128，SF=1、OF=1，异或 0，正确判不小于。a=2、b=5，差−3 无溢出，SF1、OF0，判小于；a=5、b=2 则 SF0、OF0，判不小于。这四组覆盖“无溢出正负差”和“有溢出两种方向”。对于相等，差 0，ZF=1，SF=OF=0，jl 不跳；jle 再把 ZF 纳入，才覆盖小于或等于。用条件码组合表达数学关系，需要关照截断边界，不能只背字母。

18.13 工坊十三：32 位栈帧偏移与 64 位别混

传统 32 位 cdecl 教学序言为 pushl ebp、movl esp,ebp。入口时栈顶已有 4 字节返回地址；push 旧 ebp 后，0(ebp)为旧 ebp，4(ebp)为返回地址，8(ebp)为第一个栈参数，12(ebp)为第二个。这与 64 位带帧指针例子的 8 字节步长不同。

若函数先 sub esp,16 分配局部空间，局部对象可位于 ebp 负偏移，参数仍在正偏移。恢复时可用等价于 esp=ebp、再 pop ebp 的序列释放局部空间并恢复参考点，最后 ret 弹 4 字节返回地址。调用者还可能负责清理它先前压入的参数，不能让被调用者和调用者重复清同一段。

取入口 esp=0x8000，push 旧 ebp 后 esp=0x7FFC，ebp 设此处。返回地址在 0x8000，第一个参数在 0x8004，相对 ebp 分别 4 和 8，核对一致。这里故意选入口 0x8000 便于算术，不用它暗示真实 32 位 ABI 所有调用前的对齐细节都已规定。

练习：有人看到 `8(%ebp)`就说“是 64 位返回地址”，哪里错？答案是寄存器名 ebp 已提示 32 位上下文，偏移还取决于建立帧的方法。在刚给定 cdecl 序言里，8(ebp)是第一个参数，不是返回地址。分析必须先识别架构与序言。

18.14 自己做，再看答案

练习一：8 位补码恰含一个 1，最小值与绝对值最小值分别是什么？

答案：最小值把唯一 1 放符号位，`10000000` 为−128；绝对值最小把它放最低正权位，`00000001` 为 1。因为没有全零候选，绝对值不可能为 0。

练习二：E5M2 格式 `0|01111|01` 表示多少？同数量级相邻正常数间距多少？

答案：q15、bias15，E0；小数域 1，有效数 1+1/4=1.25。区间[1,2)的间距为$2^{0-2} = 1/4$。与次正规最小步长不能混用，指数变化时正常数间距也会变化。

练习三：结构体大小 24，数组元素 i 地址为 B+24i。若 i=3、B=`0x1000`，成员偏移 16，最终地址多少？

答案：24×3+16=88，十六进制 `0x58`，得到 `0x1058`。先用十进制算字节数再转换十六进制即可；不要把十进制 88 直接当作十六进制 `0x88` 加上去。

19 故障定位：先找第一步错在哪里，再修最后看到的症状

程序在 free 里崩溃，可能是十分钟前越界写坏了分配器信息；输出重复，可能是 fork 前缓冲没刷；线程偶尔卡住，可能不是计算慢，而是两把锁等成了环。最后报错的位置只是你发现问题的地方，不一定是第一个做错的地方。

本章用几份具体的故障记录练习定位。每一份先列已知事实，再提出能区分原因的检查，最后解释修复为什么有效。不要一下同时改十个地方；否则程序恢复时，你仍不知道哪条假设对，下一次类似故障也无从下手。

19.1 故障一：读文件时数组大小突然变成几千万

已知：文件头四字节是 `00 00 00 02`，作者说只有两个元素；程序在小端主机上把四字节直接 fread 到 uint32_t 变量 n，打印 n=33554432，随后 malloc 很大。

第一步确认实际读到了四字节，排除短读导致未初始化混入。检查的必须是实际调用形式：若代码为 `fread(&n,sizeof n,1,fp)`，本题 sizeof n=4，完整成功时返回 1；若改成 `fread(bytes,1,4,fp)`，完整成功时返回 4。前者的 1 是一项，不是一字节；未完整读到一项时不能继续使用 n。第二步记录内存字节与字段格式：文件定义大端，而主机按小端直接读数。`00 00 00 02` 按小端无符号整数是 `0x02000000`，十进制 33554432，恰与症状一致。第三步改成显式大端解码，得到 2。

修复后仍应保留应用长度上限与乘法溢出检查。端序正确不代表外部文件永远可信，恶意或损坏头部仍可能声明很大 n。修复根因与防御异常输入是两个互补工作，不能因为“这次是端序”就删掉所有边界检查。

如果打印得到一个完全不对应这四字节的值，检查就应继续：fread 返回项数是否被误当字节数，变量类型是否匹配，printf 格式是否正确，是否有其他线程改写。好的假设应能预测具体数值，不只给一个听起来合理的标签。

19.2 故障二：程序到 free 才报堆损坏

已知：分配 10 个 int，每次 4 字节，循环写 `for(i=0;i<=10;i++) p[i]=i;`，随后 free(p)报错。分配范围为 40 字节，合法下标 0 至 9。i=10 时访问 p+40 起的 4 字节，已经在对象之外。

越界可能写中下一块管理信息，也可能暂时没碰到敏感位置。free 检查块结构时才发现异常，因而崩溃栈在 free。修复应把循环边界改成 i<10，并检查所有使用同一长度的路径；把 free 注释掉只是让损坏暂时不被检查，还造成泄漏。

用地址消毒器运行可帮助把诊断前移到越界写那一行。再测试长度 0、1、10 以及最后一个元素访问，确认边界规则一致。若代码还有读取外部长度、扩容或下标乘法，也要检查溢出，不能只改这个常量循环就宣布所有内存问题解决。

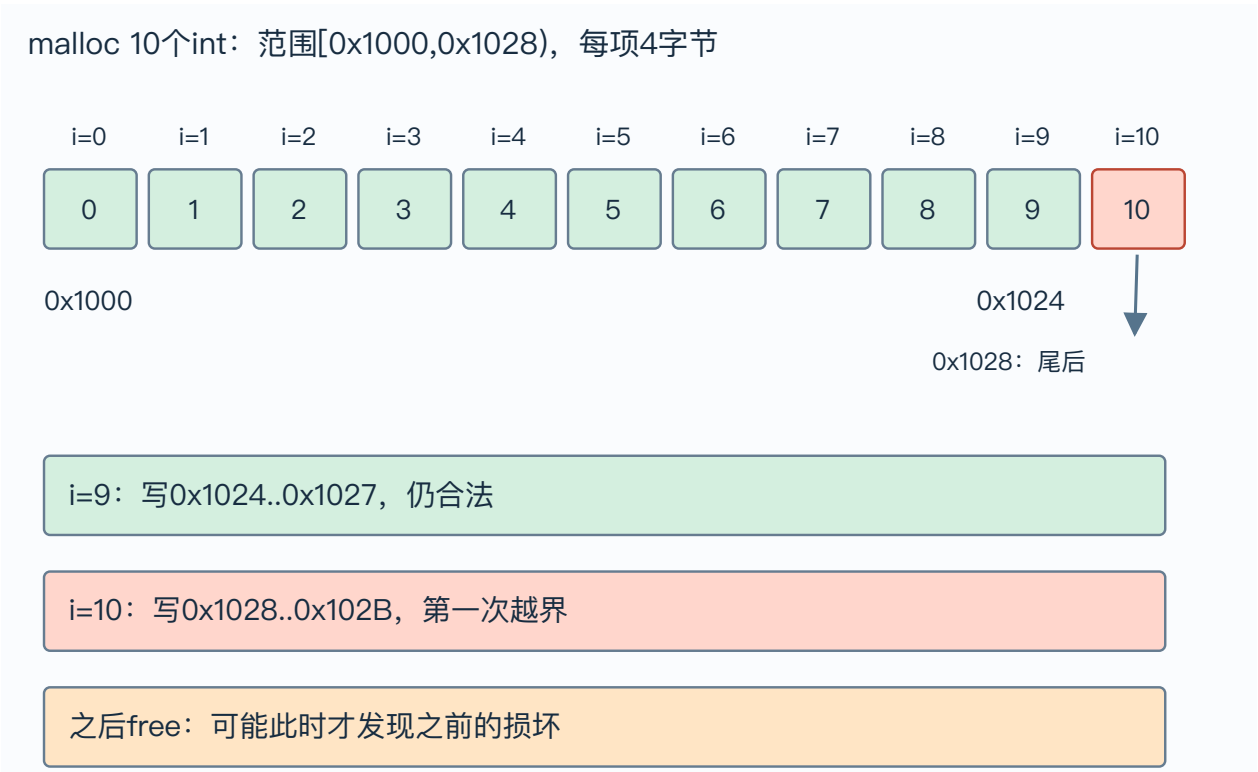


图 49　第一次非法写，与稍后发现损坏的位置

若分配起点假设 `0x1000`，40 字节范围是`[0x1000,0x1028)`。`p[9]`从 `0x1024` 写到 `0x1027`，恰是最后合法四字节；`p[10]`从 `0x1028` 写到 `0x102B`，第一字节就已越界。不需要知道外面是不是下一块头部，才能认定这次写错。

把调试时间线分开：分配成功只证明拿到 40 字节；i=0 到 9 每次在范围内；i=10 第一次破坏前置条件；free 在更晚时发现异常。跳过 free 改变的是“何时被看见”，不改变 i=10 已经发生的非法写。

可以用下面这份故意含错的最小程序，把首次非法写与后续 free 分开观察。它不是正确用法：

```c
#include <stdlib.h>

int main(void) {
    int *p = malloc(10 * sizeof *p);
    if (p == NULL) return EXIT_FAILURE;
    for (size_t i = 0; i <= 10; ++i)
        p[i] = (int)i; /* deliberate error: i==10 is out of bounds */
    free(p);
    return EXIT_SUCCESS;
}
```

在支持 AddressSanitizer 的 Clang 或 GCC 环境，把它保存为 heap-error.c，执行：

```
cc -std=c17 -O0 -g -fsanitize=address -fno-omit-frame-pointer heap-error.c -o heap-error
./heap-error
```

这里预期的是工具报告 heap-buffer-overflow，并指向 i=10 时的写入；工具可能在到达 free 之前就终止程序。这不与原故障“到 free 才发现”矛盾：原运行与加了检查的运行，在不同时间发现同一越界。把条件改成 i<10 后再用同一输入与工具运行，应不再报告这次越界；它只验证这段程序和本次执行，不证明其他代码都安全。

19.3 故障三：递归结果正确，返回后调用者却错了

已知：汇编函数使用 rbx 保存 n，递归返回值对，但上层另一个变量偶尔变化。检查函数是否保存与恢复 rbx。System V AMD64 规定 rbx 为被调用者保存；直接覆盖后不恢复，会破坏调用者依赖的旧值。

构造测试：调用者先令 rbx=99，调用函数，回来后检查 rbx 仍为 99。若变成某层 n，故障与约定违规一致。修复为入口 push rbx、使用、出口 pop rbx，并重新核对栈对齐和每条返回路径。只修正常出口、漏掉基础条件直接 ret，仍可能有问题。

返回值、寄存器保存和栈恢复是三个独立检查项。某函数在小测试中返回 6，不足以证明它是可组合的 ABI 合规函数。别人调用它以后仍能继续正确工作，才是接口约定的目的。

19.4 故障四：fork 之后日志重复

已知：程序在 fork 之前 printf 一段无换行日志，fork 后父子都正常退出，重定向到文件时重复出现。先检查 fork 前是否存在尚未刷出的 stdio 用户缓冲。若有，父子各复制一份，退出各刷一次，症状就能解释。

可在 fork 之前 fflush 相关输出流，或者重新设计日志输出位置，让只有预期的一方产生那段记录。子进程 exec 失败路径使用适合场景的_exit，避免误做继承缓冲的常规退出清理；错误报告本身也要遵守 fork 后环境限制。

不要通过“父子都 sleep 一下”修复，时间延迟不会使两份用户缓冲自动合并。也不要把重复归咎于共享文件偏移：共享偏移决定写到哪里，而重复文本来自两份待刷数据，层次不同。

19.5 故障五：父进程偶尔永远等孩子

已知：SIGCHLD 处理函数把 done 设 1，主程序写 `if(!done) pause();`。把信号安排在检查之后、pause 之前，就能产生处理函数已执行但主程序仍随后睡眠的路径。将 if 改 while 仍可能卡在第一次 pause 内。

修复使用屏蔽目标信号、检查谓词、sigsuspend 原子临时解除屏蔽并等待的模式。若还维护作业表，在 fork 之前就屏蔽，父登记完成后恢复，子在 exec 前恢复旧掩码。处理函数必须循环回收当前可回收孩子，而不是假定每个 SIGCHLD 只对应一个退出。

验证不靠“跑一百次没卡”一句话。应检查修复后的逻辑：信号早来会挂起，进入 sigsuspend 时能递送；信号晚来会唤醒等待；虚假或无关信号唤醒后会重新检查谓词。把所有时序窗口逐一封住，才说明机制修对了。

19.6 故障六：有界队列明明有消费者，生产者却卡住

已知：生产者先 lock(mutex)，再 wait(empty)。队列满时 empty=0，生产者持锁阻塞。消费者需要 mutex 才能取走一个元素并 post(empty)，因此被挡住。等待图是生产者等空槽、消费者等锁，形成不能推进的依赖。

修复为先 wait(empty)取得空槽许可，再 lock 修改队列，完成后 unlock 和 post(full)。消费者先 wait(full)，再 lock 取，完成后 post(empty)。对取消或错误路径也要归还或正确转移许可，不能让在途操作永久占住名额。

测试容量 1 尤其有用：每次放入马上满，每次取出马上空，能频繁触发等待路径。但容量 1 通过不证明所有容量正确，还应核对环形索引取模与满空条件。同步与数据结构两个层次都要验证。

19.7 故障七：两个线程各累加一百万次，结果不足两百万

若 counter 是普通共享整数且无锁，首先是 C 数据竞争，不能只用某个观测结果推断硬件执行细节。教学状态表可以展示两线程都读旧值再写同一个新值，解释丢失更新。但优化编译器还可能合并循环等，真实结果并不受那个简单表完整限制。

修复可以在同一互斥锁下更新，或使用合适的原子读改写。若只是最终统计，采用每线程局部累计、join 后合并通常更减少争用。不要只加 volatile：它不能提供原子读改写和线程同步。

检查正确性后再测性能。加锁版本慢，可能是每次只做一次加法却付出大量争用成本；这不是锁“不正确”，而是共享粒度不合理。重新设计局部状态，比随意移除锁更有用。

19.8 故障八：TLB 未命中率上升，磁盘却没有忙

有人看见 TLB 未命中，立刻推断大量缺页。检查页表与缺页统计后，发现页面都在内存，只有翻译缓存装不下。TLB 未命中需要补翻译，不等于从磁盘调页。访问跨很多页时，地址翻译局部性可能差，即使数据已在物理内存。

这时可以研究访问顺序、工作集和页大小等因素，但调整大页等系统配置属于更进一步工程选择，需要考虑内存浪费、权限与部署约束。先用统计区分翻译问题、数据缓存问题与真正 I/O，避免对着错误层次优化。

19.9 故障九：只改循环顺序，浮点结果最后几位不同

如果求和顺序变化，舍入路径也变化，有限浮点不满足精确结合律。这种差异可能符合任务容许误差，也可能使数值算法不稳定，需要按领域需求判断。先确认数据与算法步骤确实相同，再比较误差量级、绝对与相对阈值。

不要为了让测试“全绿”直接把误差容限改很大。应该先估算输入规模与舍入累积，考虑更高精度、稳定求和或更适合的算法。若差异来自数组越界，任何浮点误差解释都只是掩盖真正 bug，因此内存与线程检查仍要先过。

19.10 调试记录写成什么样，下一次才用得上

记录最小复现输入、环境假设、预期、实际、第一条被证伪的假设、最终根因和修复后的边界检查。不要只写“重启好了”或“加锁解决”。具体到“生产者在持 mutex 时等待 empty，消费者因此无法释放空槽”，未来才能识别同样结构。

每次修复尽量增加一个能触发原错误的验证。越界修复测最后合法元素和第一个非法元素；时序问题画早来、晚来两条路径；ABI 修复检查被保存寄存器；格式解码用非对称字节值，避免全零或 `11 11 11 11` 这类无法区分端序的输入。

19.11 故障十：只给缓存加大容量，程序还是冲突严重

已知一个直接映射缓存按组索引映射，程序交替访问两块相隔缓存总容量整数倍的数组位置。两者可能反复落同一组，即使其他组空着，仍互相替换。先按地址拆出块号、组号和标记，检查是否真是冲突，不要只看缓存总容量。

可通过改变布局或步长、分块访问、采用不同数据结构减少冲突；实际硬件相联度、预取和替换也会影响。若工作集总体已超过所有缓存，改一点对齐未必解决容量问题。诊断需要把强制、容量、冲突等成因与具体访问序列对应。

修复验证应记录总访问量、命中或未命中计数，以及输出正确性。只看时间略变快，可能是 CPU 频率或系统负载变化；只看模拟命中率改善，也不保证真实瓶颈就在缓存。模型与测量要互相核对。

19.12 故障十一：数据都用原子了，库存仍变负

两个线程都 load 到 stock=1，都判断能买，再各 fetch_sub。每一步原子仍允许两份检查都基于同一个旧值。根因是“条件检查与修改”不组成单一决策，而非变量读写撕裂。

先用锁把检查和减一放同一临界区，能直接保持 stock>=0 的不变量。若要用比较交换，则只有成功把某个正旧值改为旧值减一的线程获得商品，失败线程重新读取并重新检查。错误地在 CAS 失败后仍发商品，会保留业务竞态。

这类问题在检查账户余额、领取唯一任务、初始化一次性资源中都会出现。不要把某个代码模板直接复制到所有业务；先说清“哪个状态转换代表成功，谁能观察，失败后要重试还是放弃”。原子工具只是实现状态转换的手段。

19.13 故障十二：能访问地址，却仍是未定义行为

已知程序把 int 数组强制转成 double*读取，地址也对齐，页表和权限都正常。硬件可能确实读到八字节，但 C 对象类型访问规则并不因此允许这样解释。地址可访问、对齐合格、类型合法是不同条件。

需要从位表示得到另一类型时，研究明确支持的表示复制方式，例如 memcpy 到合适对象，并理解目标表示是否有效。需要解析外部格式时，更应按字段宽度、端序逐项解码。把指针强转当“万能解码器”，可能在优化以后出现与调试版不同的结果。

调试器能随意以某种格式展示内存，不代表同样的 C 表达式在程序中合法。工具显示是观察方式，源语言规则约束程序如何执行。判断题应分别回答硬件会不会访存异常，以及语言是否定义行为，不能用一个“能跑”替两题作答。

19.14 把修复分成机制与证据两栏

机制写“为什么从此不再允许原坏状态”，例如统一锁序消除那条等待环、读满循环只推进实际字节数、对象所有者延长到 join 之后。证据写“怎样核对实现确实遵守机制”，例如边界用例、消毒器、寄存器状态检查、允许交错分析。

测试通过说明已检查输入上没再观察到原错误，证明说明在假设模型下哪些路径被排除，两者都要写适用范围。不要把一次程序运行叫作完整证明，也不要只给抽象证明而不检查代码某条返回路径漏释放。工程可信度来自二者对应。

练习：修复释放后使用时，把指针改成全局变量是否有效？答案通常无效。指针变量活得更久，不会让已释放对象复活；你延长的是地址容器的生命周期，真正需要保证的是所指对象在最后借用结束前不被释放。

19.15 故障十三：把尾后指针当作最后一个元素

已知循环写 `for(p=a;p<=a+n;++p) use(*p);`。尾后指针 a+n 可以合法形成并参与这种范围内比较，但不可以解引用。循环最后多执行一次，访问第 n 个下标，合法最后下标应为 n−1。正确遍历通常使用 p<a+n，且要保证 a 与长度关系本身合法。

n=0 是最小复现：错误版本仍尝试读一次 a，正确版本零次。若程序中 a 可以为空，还应检查是否形成 a+n 这样的表达式符合所采用接口和语言规则，不能靠“加零应该没事”自动放宽指针约束。零长度的约定应明确设计。

同类错误在文件偏移区间、数组切片、分块任务中反复出现。统一采用左闭右开区间有利于把长度写成 end−begin，但读者仍要理解定义，不能只背符号。每次把起点、长度、最后合法位置各写一遍，边界会很直观。

19.16 自己做，再看答案

练习一：为了测端序，选择 `0x00000000`、`0x11111111`、`0x12345678` 哪个最有辨别力？

答案：`0x12345678` 各字节不同，倒序后明显变化。前两个每个字节相同，大端小端观察都一样，测试通过也无法证明解码规则正确。

练习二：程序加 sleep 后死锁不再出现，能否算修复？

答案：不能仅据此判断。sleep 改变执行时机，可能降低某条坏交错的概率，却不消除循环等待或检查窗口。必须解释同步关系怎样保证任何允许的先后都能正确推进。

练习三：free 处报错，最有价值的下一步是只删除 free，还是检查之前对该对象及邻近对象的边界写入？

答案：应检查首次非法写入与所有权。删除 free 可能掩盖检查并引入泄漏。可以借助消毒器、块边界图和最小复现，让诊断靠近真正根因。

本课程资料、章节范围与图源

教师资料对应

本地资料	主要对应章节	读的时候关注什么
第 01 讲-课程引言.pdf	00、08 至 17、19	整套系统的关系，以及 CS:APP 选节范围
2.1-整数.pdf	01、02、18	字节序、位权、补码、类型转换、溢出
2.2-浮点数.pdf	03、18	指数偏置、正常与次正规、特殊值、舍入
3.1-C 语言-1.pdf	04、07、18	寄存器、操作数、寻址、mov 与 lea、基本算术
第 05 讲-C 语言的机器表示-2.pdf	05、18	条件码、setcc、条件传送、循环、跳转表
第 06 讲-C 语言的机器表示-3.pdf	06、18	call/ret、栈帧、保存责任、递归位计数、32 位对照
HW1/hw1.pdf	02、03、18	限定置位数量、16 位运算、FP8、BF16/FP16、乘常数
HW2/题目与材料/作业题目.pdf	04 至 06、18	三条指令加权和、循环还原、条件选择、跳转表、SF 与 OF、scan
实验 01-AttackLab 题目与材料	07、19 的内存边界与控制状态分析	理解局部对象越界为何可能破坏保存状态；不收录私人提交

本地已收到的详细教师课件到第 06 讲。08 至 15 章的后续系统内容按课程引言列出的 CS:APP 选节组织，用于预习与复习；不能把它们当作已经收到的本学期逐页讲稿。第 16 章缓存与性能为补充，条件变量完整队列、部分原子发布例子也用于解释机制，具体考试要求以教师通知为准。

引言指定的参考范围包括 CS:APP 第 2、3、7、8、9、10、12 章的选节。第 9 章多级页表不作默认要求；分配器以 9.9.12 之前的引导与实验范围为主。正文使用单级页表玩具模型把页号与偏移讲清，不要求先记所有实际硬件层级。

原图准确页码

以下图片均直接渲染教师 PDF 原页，保留原始版面与箭头。页码是 PDF 阅读器中的页序，从 1 开始。

本书图片	原始文件	PDF 页码	本书使用处
老师原图-整数.png	2.1-整数.pdf	7	01：按地址读取两行字节序
老师原图-浮点.png	2.2-浮点数.pdf	13	03：指数全零、隐藏位与渐进下溢
老师原图-switch.png	第 05 讲-C 语言的机器表示-2.pdf	43	05：共享代码、落入与除法寄存器
老师原图-调用栈.png	第 06 讲-C 语言的机器表示-3.pdf	4	06：递归实例、返回位置、参数接力
老师原图-栈帧布局.png	第 06 讲-C 语言的机器表示-3.pdf	19	06：可选 rbp、rsp 与活动状态

本书图片	原始文件	PDF 页码	本书使用处
老师原图-跳转表.png	第 05 讲-C 语言的机器表示-2.pdf	39	18：表项地址与跳转目标地址分开
老师原图-递归位计数.png	第 06 讲-C 语言的机器表示-3.pdf	44	18：保存最低位、逻辑右移与回程累加

`本书示意-浮点格点.svg`、`本书示意-地址翻译.svg` 为补充示意，分别帮助观察可表示数的间距和地址拆分，不属于教师原图。正文保留了每张图的读法说明，图片不能代替适用条件与计算过程。

原生文字便于核对数值、指令和页码，渲染页便于核对箭头、布局、表项与公式。所引用的 7 张教师原页已逐张视觉核对；本册不声称对全部原资料每一页都完成了逐字审校。

接口和平台限制

整数与汇编主线按 x86-64、AT&T 语法、System V AMD64、常见 LP64 环境说明。32 位 cdecl 只在明确标出的对照中使用。C 边界主要采用 C17 口径，FP8 按题目给定的 IEEE 风格规则，不能无条件套用其他 E4M3 变体的特殊值定义。POSIX 程序可以在支持相应接口的系统验证，但 Linux、macOS 与 Windows 的二进制格式、调用约定和系统接口并不相同。

完整队列和信号示例展示一个有限功能场景，不包含所有取消、进程崩溃恢复或服务关闭策略。教学分配器的头部、最小块与合并规则按题目设定，实际 malloc 实现可能不同。性能算例里的纳秒、容量和带宽都是题目参数，不是所有机器的实测规格。

需要核对原始定义时，可用以下接口资料：

- CS:APP 第三版官方配套页：章节背景、实验和勘误入口。
- OSTEP 并发入门原章：对照线程私有状态与共享量逐指令交错的讲解。第 14 章用自编原子 load/store 例子说明组合竞态，另行明确普通 C 数据竞争的语言边界。
- fork、execve、waitpid：创建、替换与回收进程。
- sigprocmask、sigsuspend、signal-safety：掩码、原子等待与处理函数限制。
- read、write、dup：实际计数、错误与描述符引用。
- malloc/free/realloc、mmap：分配生命周期、映射与权限；页面中的 Linux 实现行为应与 C/POSIX 通用规则区分。
- pthread_cond_wait 的 POSIX 说明：等待时释放与重取锁，以及谓词重检查。

新增完整示例的范围

14 章的三线程示例采用 POSIX 线程，工作函数分别修改独立参数对象，主线程 join 后输出；它是解释创建、参数生命与同步的自编示例。17 章的 record-sum 程序使用 POSIX read 解析固定四字节大端格式，并明确应用项数上限、截断与尾部策略。19 章的 heap-error 是故意含越界的诊断输入，用于观察 AddressSanitizer 前移报错位置，不能作为正确分配用法。

07 章 cs-aha-07-addresspaths.svg 为本书自绘，比较 3×4 连续矩阵与三条行指针，两者仅在示例数据上相同；地址均为手算位置。01 的 16 位字段、16 的缓存写策略也是明确限定模型的自编预备例，不伪称教师原题。18 章的 FP8 及 INT8 选择仍对应 HW1 第 5 题，对称量化示例显式取码域−127 至 127、固定比例因子，不代表所有部署的唯一约定。

看内存、指令和系统状态时怎样查术语

本页供查到陌生词后回到对应过程。定义尽量短；若需要计算或执行顺序，按下面章节位置阅读，不把一串术语作为进入正文前必须背过的清单。C 语言、x86-64 和操作系统约束不同层面的行为，同一词要放回当时的对象与规则中。

地址、内容与大小

地址指一个存储位置，内容是这个位置保存的位。一个对象可以占多个字节，它的地址通常指起始字节。`&x` 取 x 的地址，`*p` 沿 p 保存的地址访问一个由 p 类型指定的对象。`sizeof x` 问占多少字节，不问 x 的值多大。

一个字节在本书平台上为 8 位；一个十六进制数字对应 4 位，所以打印一个字节通常用两位十六进制。`0x` 是数值写法的前缀，不占额外的存储字节。小端表示最低权重字节位于最低地址；同一字节内部的位权没有倒过来。

对象的生命期是它作为某个对象存在的那段时间。地址数值还在，并不能证明原对象仍存在。例如函数返回后，指向其局部变量的指针不会自动被清空，但原来的自动对象已经结束生命期。地址合法、权限允许、类型规则满足、对象仍存活，是需要分别检查的条件。

指令正在读什么、写什么

寄存器是处理器中的小容量存储位置。图中的 `rax`、`rsp` 等是寄存器名字，不是 C 变量名。按本书 x86-64 约定，`rsp` 保存当前栈顶地址；`rip` 指示执行位置。程序在寄存器中保存的位也需要按指令解释，不能从“这个数像地址”就断言它一定要被解引用。

AT&T 汇编通常把源写在前、目的写在后。`movq %rax,%rbx` 把 rax 的 64 位值复制到 rbx，rax 不因复制而清空。括号通常表示访存，例如`(%rax)`访问 rax 保存的地址。`lea` 则计算地址形式的算术结果，不执行那次内存读取，具体例子见第 04 章。

条件码是前一条相关指令留下的状态位。ZF 表示结果是否为零，SF 反映结果最高位，CF 与无符号进借位有关，OF 表示有符号溢出。根据有符号或无符号比较选择相应跳转条件，不能只凭 SF 猜两个有符号数的大小。第 05 章把每个条件与具体位运算连起来。

栈上的数据和返回路径

栈顶是当前最先能取出的位置。本书 x86-64 栈向低地址增长，压入 8 字节通常先让 rsp 减 8，再写入；弹出则从 rsp 处读取，再让 rsp 加 8。画图时需看清高低地址标记，不能用纸上的“上”“下”替代地址关系。

返回地址指调用完成后继续执行的位置，不是函数的结果。函数返回值通常放在约定的寄存器里。栈帧是某次调用保存参数、局部对象或寄存器等状态的一组区域，不保证每个优化后的函数都有相同的帧指针布局。具体布局要按实际指令走，第 06 章和第 18 章有逐步表。

程序、进程与线程

可执行文件是存放代码和数据等信息的文件。进程是一个正在执行的程序实例，有自己的地址空间和系统管理状态。`fork` 创建一个子进程；父子从同一调用之后继续，但得到不同返回值。`exec` 在当前进程中替换程序映像；它不是再创建一个新的子进程。

线程是进程中的一条执行流。同一进程中的线程共享许多对象，但各自有执行位置、寄存器状态，通常还有各自的栈。传出去的局部对象地址可能被其他线程访问，所以“在线程自己的栈上”不等于硬件隔离。

竞态条件表示业务结果依赖没有控制好的执行次序。C 的数据竞争指不同线程对同一内存位置有冲突访问，至少一次是写、至少一次为非原子，而且二者之间没有所需的 happens-before 关系；它导致未定义行为。两个合法原子访问本身不因缺少 release/acquire 配对就自动成为数据竞争，但多步组合仍可能有业务竞态。第 14 章用合法的原子读写演示组合竞态，把它与普通 int 上的数据竞争区分开。`volatile` 不能代替互斥锁或原子同步。

页、缓存行与文件描述符

虚拟页是虚拟地址空间的一块固定大小区域；物理页框是物理内存中的对应容器。页号要经过映射，页内偏移保持。TLB 缓存页表翻译结果，它和保存数据内容的缓存不是同一个东西。缓存行是缓存装入或替换数据的一块单位；页大小、缓存行大小都必须用题目或平台给定值，不能混成“一个块”。

文件描述符是进程使用的小整数编号，例如常见的 0、1、2。它引用系统维护的打开状态；两个描述符可以引用同一打开文件描述，并因此共享文件偏移。偏移表示下一次顺序读写的位置，不等于文件描述符编号。`dup` 和 `fork` 后共享关系见第 13 章的图。

一次 `read` 或 `write` 返回的正数是本次实际处理的字节数，可能小于请求值。读取返回 0 通常表示到达流的结束；返回−1 表示失败，要按 errno 判断。字符数与字节数不能互换，尤其是 UTF-8 文本。系统调用的确切约定随接口与平台说明核对。

数值格式与 C 规则

位权是该位置为 1 时对数值贡献多少；02 章的补码只把最高位赋负权。符号扩展复制原符号位，使较宽补码仍表示同一个负数；零扩展在高位补 0，保持无符号值。截断只保留低位，但 C 的有符号超范围转换和普通有符号溢出还要分别按语言规则判断。

实际指数 E 是科学计数中的幂；指数编码 q 是保存到位域中的无符号整数；正常数用 E=q−bias。小数域 F 是一个整数，除以2^f才成为 f 位小数的贡献。次正规数使用 0 开头的有效数及 E=1−bias，不套正常数的隐藏 1。完整编码、边界和舍入均见 03。

有定义行为表示语言给出可依赖语义；实现定义表示实现须选择并说明行为；未定义行为表示标准不再要求提供结果保证。C17 普通有符号溢出属于后一类，无符号算术则按类型模数约化。“机器留下了某个位串”与“C 源程序保证这个结果”不能合并判断。

调用、等待和 I/O 的回查位置

需要解决的问题	回到哪里	应跟踪的对象
整数怎样拆字节，指针到底改谁	01、02	数值位权、字节地址、指针对象与所指对象
mov 取内容还是 lea 算地址	04	寄存器旧值、有效地址、读取宽度
一次比较为什么跳到这个标签	05	cmp 的顺序、ZF/SF/CF/OF、下一执行位置
函数如何回到调用者	06、18 递归位计数	rsp、返回地址、保存寄存器、rax 结果
连续矩阵与指针表有何不同	07	先算行地址还是先读行指针
另一个文件里的函数怎样接上	08	声明、定义、未解决符号、待修字段
进程暂停后原值放在哪里	09	CPU 当前状态与被保存的进程状态
信号发生了，为什么标志还没变	10	pending、当前掩码、处理函数、requested
同一个虚拟地址为何可指向不同内容	11	页表、页框号、页内偏移、访问权限
malloc 地址由谁最终释放	12	拥有者、借用者、对象生命期和各指针副本

需要解决的问题	回到哪里	应跟踪的对象
read 短读或 fread 返回 1 意味什么	13、19 故障一	请求字节/项数、实际计数、缓冲有效区
线程从哪开始，参数活到何时	14	入口函数、参数地址、每线程状态与 join
通知后为什么还不能立即取队列	15	谓词、持锁者、等待者和重新争锁
写完缓存为何主存可能仍是旧值	16	缓存副本、dirty 位、写策略与替换

happens-before 表示语言与同步接口建立的先后关系，不只是观察到墙钟时间谁先结束。成功 join、正确的锁交接、读到对应发布值的 release/acquire 等可以在各自条件下建立关系。14 章用实际共享对象解释，不能把任意两次原子访问都画成发布链。